JAVA INTERNALS · A DEEP READ

Java 底层原理

精读 · 28 个机制的来龙去脉

每一章都从「线上出了这个错」开始，
拆到源码，再回到「下次怎么判断」

覆盖 JDK 8–17 · Spring 5 · Netty 4

目　录

01 序：为什么要读底层 3

02 一、这本书的读法 4

03 二、20 个自测问题 5

04 三、一个提醒 6

05 第1章　内存模型：并发的一切从这里开始 7

06 第2章　happens-before：JMM 的官方定义 11

07 第3章　AQS 骨架：一个 int 和一个双向队列 15

08 第4章　公平与非公平：差在那一行判断 20

09 第5章　共享模式：CountDownLatch 和 Semaphore 怎么复用同一套队列 25

10 第6章　读写锁：读读不互斥是怎么做到的 29

11 第7章　Phaser：比 CountDownLatch 更灵活的分阶段同步 33

12 第8章　线程池：ctl 这一个 int 装了什么 37

13 第9章　线程池：任务从提交到执行的全过程 41

14 第10章　BlockingQueue：七种队列差在哪 45

15 第11章　RateLimiter：令牌桶的两种实现 49

16 第12章　HashMap：从 JDK 7 到 8 的那次改造 53

17 第13章　ConcurrentHashMap：从分段锁到 CAS + synchronized 57

18 第14章　字符串常量池：intern 到底做了什么 62

19 第15章　HotSpot 内存管理：堆、栈、方法区到底怎么分的 66

20 第16章　Metaspace 与堆外内存：元数据搬出堆之后 69

21 第17章　G1：把堆切成 Region 之后 72

22 第18章　ZGC：染色指针与读屏障 75

23 第19章　NIO 三件套：Buffer、Channel、Selector 78

24 第20章　非阻塞 IO 与异步 IO：差的不只是一个字母 82

25 第21章　Tomcat 的 NIO：Acceptor、Poller、Worker 85

26 第22章　Netty 启动：Reactor 线程模型是怎么搭起来的 89

27 第23章　Netty 内核：Pipeline、ByteBuf 与内存池 94

28 第24章　IOC 容器：refresh() 的十二步 99

29 第25章　AOP 的演进：从 AspectJ 到动态代理 106

30 第26章　Spring AOP 源码：代理是什么时候造出来的 110

31 第27章　AspectJ：编译期织入与 @Aspect 的真相 115

32 第28章　Sentinel：责任链与滑动窗口 120

33 结语：怎么自己读源码 125

序：为什么要读底层

大部分 Java 开发者都经历过同一个阶段：会用，但不知道为什么。

`HashMap` 用得很熟，可被问到"为什么扩容是 2 倍"就答不上来；`synchronized` 天天写，但说不清它和 `volatile` 到底差在哪；线程池参数配了一堆，线上却出现过任务堆积、`OOM`、甚至关不掉；`@Transactional` 加在方法上，某天发现事务根本没生效。

这些问题的共同点是：**API 层面看不出答案，答案在下面一层。**

这本书就是把"下面一层"拆开给你看。全书 28 章，覆盖五个层面：

层面	讲什么	对应章节
并发	内存模型、happens-before、AQS、读写锁、Phaser	1–7
并发工具	线程池、阻塞队列、限流器	8–11
数据结构	HashMap、ConcurrentHashMap、字符串常量池	12–14
虚拟机	内存布局、Metaspace、G1、ZGC	15–18
IO 与框架	NIO、非阻塞/异步、Tomcat、Netty、Spring IOC/AOP、Sentinel	19–28

> 说明：这本书的素材来自你收集的 27 篇源码分析文章。我没有照搬，而是按"为什么需要 → 它是什么 → 怎么实现的 → 哪里会错"重新组织成 28 章，补上了章节之间的衔接和判断标准。原文里大段贴源码的部分做了精简，只保留能说明机制的关键片段。

一、这本书的读法

不要从头读到尾。 28 章里真正需要"按顺序"的只有第 1–2 章（内存模型是所有并发的地基），其余都可以按需跳读。

三种读法：

第一种：按故障查。 线上遇到问题，先看下面这张"症状 → 章节"对照表，直接跳过去。

症状	先看
加了 `volatile` 还是出错	第1、2章
锁住了但还是有并发问题	第3、4章
读多写少，`synchronized` 性能不够	第6章
线程池任务堆积 / 关不掉 / `OOM`	第8、9、10章
接口被刷爆，要限流	第11、28章
`HashMap` 并发下死循环（JDK 7）	第12、13章
`String` 的 `==` 结果和预期不一致	第14章
`OutOfMemoryError: Metaspace`	第16章
GC 停顿太长	第17、18章
一万连接把线程打满	第19、20、21章
Netty 内存泄漏 / 业务阻塞 EventLoop	第22、23章
循环依赖 / `@PostConstruct` 里拿不到对象	第24章
`@Transactional` 自调用失效	第25、26章
Sentinel 规则推了不生效	第28章

第二种：按层读。 一个层面一个层面往下推。并发那 7 章连起来读，能建立完整的"从内存模型到同步器"的认知链；JVM 那 4 章连起来读，能理解"对象从分配到回收"的完整生命周期。

第三种：当手册。 每章都是自包含的，有结构表、有关键代码、有"容易误解的地方"。忘了某个机制，翻到那一章看表和代码就行。

二、20 个自测问题

读完这本书，下面这些问题应该能脱口而出。答不上来的，就是该回去补的章节。

并发基础

- **Q1** `volatile` 保证了什么，没保证什么？为什么它能保证可见性却不能保证原子性？
- **Q2** `happens-before` 的八条规则里，哪一条是"兜底"的？
- **Q3** 双重检查锁为什么必须给实例加 `volatile`？

同步器

- **Q4** AQS 的 `state` 在 `ReentrantLock`、`Semaphore`、`CountDownLatch` 里分别是什么意思？
- **Q5** 公平锁和非公平锁的代码差在哪一行？为什么非公平吞吐更高？
- **Q6** `Condition` 的等待队列和 AQS 的同步队列是什么关系？
- **Q7** 读写锁为什么用高 16 位和低 16 位表示两个锁？为什么最大只能 65535？
- **Q8** 锁降级可以做，锁升级为什么不行？

线程池

- **Q9** `ctl` 这一个 `int` 里装了什么？为什么要合成一个字段而不是两个？
- **Q10** 线程池为什么"先入队，队列满了才扩容"？这个顺序带来了什么坑？
- **Q11** `newFixedThreadPool` 为什么可能 `OOM`？
- **Q12** `shutdown` 和 `shutdownNow` 差在哪三点？

集合

- **Q13** `HashMap` 的容量为什么必须是 2 的幂？`(n-1) & hash` 比 `%` 好在哪？
- **Q14** JDK 8 扩容时为什么不需要重新计算 hash？
- **Q15** JDK 7 的 `HashMap` 并发扩容为什么会成环？JDK 8 怎么避免的？
- **Q16** `ConcurrentHashMap` 的 `get` 为什么可以不加锁？

虚拟机与 IO

- **Q17** 永久代为什么被 Metaspace 取代？Metaspace 用的是哪块内存？
- **Q18** G1 的 Region 化和 CMS 的分代有什么本质区别？
- **Q19** `selector.select()` 返回 0 时应该做什么？为什么 key 用完必须 `remove`？
- **Q20** Netty 的 `ByteBuf` 为什么要自己管引用计数，而不是交给 GC？

三、一个提醒

读源码最容易掉的坑，是**把"能背出流程"当成"懂了"**。

比如你能背出 `refresh()` 的十二步，但如果问你"第三步 `prepareBeanFactory` 里注册的 `ApplicationContextAwareProcessor` 是干什么的、不注册会怎样"，答不上来，那还是没懂。

所以每章的"**容易误解的地方**"那一节，比正文更值得反复看。那些地方是绝大多数人以为自己懂了、实际上理解偏了的位置。

> 判断自己有没有读懂一个机制，只有一个标准：**能不能说出"如果没有它，会出什么具体的错"。** 说不出，就是还没懂。

第1章　内存模型：并发的一切从这里开始

一句话：JMM 用主内存/工作内存抽象掉硬件差异，只对同步过的读写承诺结果。

为什么需要它：

先看一段谁都会写的代码。

```
static boolean stop = false;

// 线程 A
new Thread(() -> {
    while (!stop) {
        // 干活
    }
}).start();

// 线程 B，一秒钟后
stop = true;
```

在单核机器、解释执行模式下，这段代码能正常退出。换到一台多核服务器，让 C2 把循环编译到极致，A 线程可能永远不退出。更诡异的是，你在循环体里加一行 `System.out.println`，它又好了——因为 `println` 内部有 `synchronized`，顺带让 `stop` 重新从主内存读了一次。

"加一行日志就好了"这类现象说明的不是 JVM 有 bug，而是这段代码从一开始就没有确定的语义，之前只是碰巧跑对了。并发程序的可预测性不是自然属性，是被规范授予的。

这一章要解决的不是"怎么开线程"，而是一个判定问题：给定一段多线程代码，你能不能事先判断它有没有唯一的执行结果；`volatile`、`synchronized`、`final` 这三个关键字各自买到了什么、明确没买到什么。判定清楚了，后面所有并发工具——`AQS`、`ConcurrentHashMap`、`ThreadPoolExecutor`——的源码才读得动，因为它们每一个的注释里都在引用同一套规则。

核心结构：

概念	作用
主内存（main memory）	存放所有实例字段、静态字段、数组元素；线程之间唯一的交换场所
工作内存（working memory）	每个线程私有的抽象，寄存器、L1/L2、store buffer、编译器临时值的统称，不是一块真实内存
重排序（reordering）	编译器、CPU、内存系统三层机制，共同破坏"代码顺序 = 执行顺序"这个假设
数据依赖	单线程内重排序的边界：有依赖不能换，无依赖可以随便换
内存屏障（memory barrier）	禁止屏障两侧特定类型的重排，是 JMM 语义落到指令上的手段
volatile	写具 release 语义、读具 acquire 语义；不提供互斥，不提供复合操作原子性
synchronized	monitorenter 等价 acquire、monitorexit 等价 release；额外提供互斥
final	构造方法退出时 freeze，禁止 final 字段的写被重排到引用发布之后

关键代码：

四个变量，两个线程，各两行，能跑出一个"不可能"的结果。

```
// x = y = a = b = 0，两个线程同时起跑
// Thread 1                Thread 2
a = 1;                     b = 1;
x = b;                     y = a;
// 直觉：x 和 y 至少有一个是 1
// 实测：跑几万次能抓到 (x, y) = (0, 0)
```

要得到 `x == 0 && y == 0`，必须让两个线程各自的第二行都跑到第一行前面。三条独立路径都能做到，下面拆解里展开。

第二个例子是这三条路径的实战后果——双重检查锁（Double-Checked Locking，DCL）。

```
public class Singleton {
    private static Singleton instance;   // 少一个 volatile，整个类就是错的
    private int v;

    private Singleton() { this.v = 3; }

    public static Singleton getInstance() {
        if (instance == null) {                    // 第一次检查，无锁
            synchronized (Singleton.class) {
                if (instance == null) {            // 第二次检查
                    instance = new Singleton();    // 这一行在字节码里是三步
                }
            }
        }
        return instance;
    }
}
```

`instance = new Singleton()` 展开后：

```
new           #2   // 1. 分配内存，所有字段先填 0/null
dup
invokespecial #3   // 2. 执行 <init>，写入 v = 3
putstatic     #4   // 3. 把引用写进 instance
```

第 2 步和第 3 步之间没有数据依赖。JIT 完全可以把 `putstatic` 提到 `invokespecial` 之前——于是别的线程在第 1 步检查时看到 `instance != null`，拿到一个 `v == 0` 的半成品。

拆解：

一、重排序有三个来源，只有一个和 CPU 乱序有关

回到那个四行程序。要出现 `(0, 0)`，需要 `x = b` 在 `b = 1` 之前读到值，同时 `y = a` 在 `a = 1` 之前读到值。三条路径：

编译器重排。`a = 1` 和 `x = b` 之间没有数据依赖，也没有同步约束，交换它们不改变单线程的可观察结果。需要澄清一点：`javac` 几乎不做这种重排，它基本是逐条翻译 AST。真正干这事的是 JIT 的 C2，在 SSA 形式的中间表示上做指令调度，把访存指令往前提以掩盖延迟。

CPU 乱序执行。即使字节码原封不动，处理器也可以重排。x86 是 TSO（Total Store Order）模型，只允许一种重排：store 之后的 load 可以被提前。而这里的 `a = 1` 是 store、`x = b` 是 load，正好踩在 TSO 唯一的那个口子上。ARM 更宽松，四种组合里三种都能换。

内存系统"看起来"乱序。这一条最容易被忽略，因为它不是重排。指令确实按顺序发射了，`a = 1` 也确实执行了，但它落在 core0 的 store buffer 里，还没合并进缓存行；core1 读 `a` 时读的是自己缓存里的旧副本，拿到 0。顺序没变，可见性没了。

JLS 第 17.4 节把这三者统称为 compiler——原文写的是 "we refer to all of these as compiler"，还特意说明不翻译成"编译器"。这个用词是刻意的：后面写规则时，规范不需要区分到底是谁干的，只关心"结果是否合法"。

这里有一条红线：**单线程语义不可违反**。任何重排都不能改变单线程内可观察的结果。所以重排永远只在"跨线程观察"时才暴露——这也是为什么单线程调试永远复现不出来。

JIT 判定"能不能换"的依据是数据依赖和别名分析。`a = 1; x = b;` 里，前者写的是变量 `a`，后者读的是变量 `b`，两者不构成依赖，交换合法；如果写成 `a = 1; x = a;`，就构成写后读依赖（RAW），不能换。问题在于别名分析只能做到"保守地证明不别名"，证明不了的时候就不优化。而跨线程的共享变量恰恰是最难证明的那一类——两个线程持有同一个对象的引用，编译器在编译其中一个方法时根本看不到另一个线程在干什么。于是它只能按"我看不到就当我独占"的假设来优化，这个假设在单线程里永远成立，在多线程里永远可能不成立。

二、可见性不是多核的锅，是私有缓存加写缓冲的锅

一个常见的错误归因：多核导致可见性问题。不对。如果所有核心共享同一份缓存，就不存在可见性问题。问题的根源是每个核心有自己的一级缓存、二级缓存，以及更隐蔽的 store buffer。

MESI 协议保证了缓存行级别的一致性：core0 写了某行，core1 对应的行会被标记为 Invalid。听起来已经够了，但 store buffer 打破了它——写操作先进入本核的 store buffer，CPU 不等它落到缓存就继续往下执行；此时别的核看到的还是旧值。失效消息（invalidation）也有传递延迟，读操作可能命中一条"已经该失效但还没处理完"的行。

JMM 不去描述 MESI、store buffer、invalidation queue，它换了一套完全不同的词：所有共享变量放在**主内存**，每个线程有自己的**工作内存**，线程读写共享变量必须先经过工作内存。

工作内存不是真实内存。它是寄存器、L1/L2、store buffer、编译器优化出来的临时变量的统称。规范选用这个抽象，是因为它必须在所有平台上都成立——包括只有一个核、压根没有缓存的嵌入式 JVM。抽象越粗，覆盖面越广。

这个抽象的收益是：把"硬件细节"换成了"读写规则"。你不需要知道 x86 的 TSO 和 ARM 的弱内存模型差在哪，只需要知道"工作内存里的副本什么时候失效"。

代价是：抽象是单向的。规范只保证它白纸黑字写下来的东西，没写的一律不保证。这也解释了一个常见困惑——为什么 `volatile` 的语义在 JLS 里读起来那么绕，因为它描述的不是"做了什么"，而是"允许你观察到什么"。

那为什么不干脆采用连续一致性（sequential consistency）？那样所有线程看到同一个全局顺序，每个操作原子且立即可见，问题一次性解决。答案是成本：连续一致性直接禁止了编译器和 CPU 的绝大部分优化。一旦 `p.x = 3` 执行，后续任何位置对该字段的读都必须立刻看到 3，JIT 不能再把值缓存在寄存器里，CPU 不能再用 store buffer，连循环里的不变量提升都做不了。这不是"慢一点"，是把现代处理器的性能模型整个废掉。JMM 的选择是：默认放开，只在同步点收口。这就是为什么并发正确性必须由程序员用同步手段主动"购买"，而不是默认赠送。

三、synchronized 和 volatile 是同一套语义的两个档位

先把两个关键字的对应关系摆出来：

- 进入 `synchronized`（monitorenter）≈ volatile 读
- 退出 `synchronized`（monitorexit）≈ volatile 写

具体行为：线程拿到监视器锁进入临界区时，它工作内存中该锁保护的共享变量副本失效，后续读必须回主内存取；退出时，把进入之前和临界区内对共享变量的所有写刷回主内存。

注意这里的不对称：**进入时不刷，退出时才刷**。所以"`synchronized` 保证可见性"的准确表述是"解锁 happens-before 后续对同一监视器的加锁"，是"退出 → 进入"这一对配对，不是"进入 → 退出"。如果你在临界区里等一个别的线程写的值，等不到，因为进入时并不会把别人的写拉过来——它只是让你的副本失效，你重新读的时候读到的可能是主内存里尚未更新的旧值。

两者的唯一区别是互斥。`synchronized` 有，`volatile` 没有。所以 `volatile` 便宜：它不涉及操作系统互斥量，不需要在用户态和内核态之间切换，x86 上编译成一条带 lock 前缀的指令（形如 `lock addl $0x0,(%rsp)`，用一个空操作触发缓存行锁定和 store buffer 刷出），ARM 上是一条 `dmb ish`。

第二件事是禁止重排序，而 `volatile` 的禁止范围比大多数人以为的宽得多。它不只是"两个 volatile 操作之间不能换"，而是"volatile 操作和它前后的普通读写也不能随便换"。落地成四条屏障：

```
volatile 写之前  → StoreStore：前面的普通写必须先落地
volatile 写之后  → StoreLoad ：这个写不能被后面的读越过
volatile 读之后  → LoadLoad + LoadStore：后面的读写不能被提前
volatile 读之前  → 无需屏障（acquire 只约束后面）
```

这四条屏障直接推出一个常被当作"结论"来背的推论：线程 A 写一个 volatile 变量，线程 B 随后读它，那么 A 在这个写之前的所有写，对 B 都可见。推导过程是：A 的普通写 →(程序顺序)→ A 的 volatile 写 →(synchronizes-with)→ B 的 volatile 读 →(程序顺序)→ B 的普通读。四次传递，链条闭合。它不是一条独立规则，是屏障语义加传递性的算术结果。

这里必须把原子性单独拎出来说，因为它是和可见性正交的第二个轴，混淆这两者是并发 bug 的主要来源。JLS 17.7 规定：对 non-volatile 的 `long` 和 `double` 的写，允许拆成两个相邻的 32 位写。一个线程写高 32 位、另一个线程写低 32 位，第三个线程就可能读到一个从未存在过的组合值——这就是字分裂（word tearing）的近亲。`volatile long` 和 `volatile double` 被强制要求原子，因为 volatile 语义本身就蕴含"这一次写要么全发生要么全不发生"。引用类型的读写在任何位宽上都是原子的，32 位机器上也是如此，这一点常被误以为需要 64 位才成立。

规范对此的措辞是"鼓励"（encourage）JVM 实现避免拆分 64 位写，不是"要求"。实际上 64 位 HotSpot 上 `long` 写本来就是原子的，但这是实现选择，不是语言承诺。把它当承诺写进代码，换一个 JVM 或换一个平台就可能出问题。原子性解决"读到的值是不是完整"，可见性解决"读到的值是不是新的"，两件事没有任何蕴含关系。

四、DCL 坏在构造函数不是原子的

回到字节码那三步。第 2 步 `invokespecial` 执行构造方法，第 3 步 `putstatic` 发布引用。两者之间没有数据依赖，所以第 3 步可以被提到第 2 步之前；即使不被提，第 2 步里的 `v = 3` 也可能还留在 store buffer 里没刷出去。

于是线程 B 在第 1 步检查时读到 `instance != null`，跳过了锁，直接返回一个 `v == 0` 的对象。这就是"部分构造"（partially constructed object）。

加 `volatile` 为什么能修好：`putstatic` 变成 volatile 写，它的 release 语义要求之前的所有写——包括 `<init>` 里的 `v = 3`——必须先完成。构造没结束，引用就发不出去。

有人试过绕开 `volatile`，用 `synchronized` 的可见性来凑：

```
public static Singleton getInstance() {
    if (instance == null) {
        Singleton temp;
        synchronized (Singleton.class) {
            temp = instance;
            if (temp == null) {
                synchronized (Singleton.class) {   // 内嵌一层
                    temp = new Singleton();
                }
                instance = temp;                   // 在锁外
            }
        }
    }
    return instance;
}
```

思路是：内层锁退出时，`temp` 一定已经完整构造。这个前提没错。错在下一步——规范只保证"解锁之前的代码在解锁之前执行完"，没有任何规则禁止"解锁之后的代码被提前到解锁之前执行"。`instance = temp` 在锁外，完全可以被提到内层锁里面去，重排问题原封不动地复现。

最后一个例外：如果 `Singleton` 的所有字段都是 `final`，不加 `volatile` 也是安全的。原因是 final 字段在构造方法退出时会执行 freeze 操作，freeze 之前的写不能被重排到引用发布之后。但前提很硬——`this` 不能在构造期间逸出：注册监听器、塞进静态集合、传给别的线程，任何一个都会让 freeze 的保证失效。

规范给过一个对照实验，把这件事说得很干净：

```
class FinalFieldExample {
    final int x;
    int y;
    static FinalFieldExample f;

    FinalFieldExample() { x = 3; y = 4; }

    static void writer() { f = new FinalFieldExample(); }

    static void reader() {
        if (f != null) {
            int i = f.x;   // 必然是 3
            int j = f.y;   // 可能是 0
        }
    }
}
```

同一个构造方法里写的两个字段，一个能保证、一个不能，差别只在 `final`。因为 `x = 3` 发生在 freeze 之前，而 `f` 的赋值（在 `writer` 里，构造完成之后）发生在 freeze 之后，freeze 把它们钉死在两侧。`y = 4` 没有 freeze 保护，它可以被延迟到引用发布之后才落地，读者看到 0。

顺带说一句 `final` 的实际用途比 DCL 大得多：`String` 的 `value`、`offset`、`count` 全是 final，`String` 的不可变性正是靠这个在数据竞争环境下也成立——即使恶意代码在无同步的情况下把 `String` 引用传来传去，`substring` 出来的对象也不会出现 `offset` 一半新一半旧的撕裂状态。这是 Java 安全模型的一块基石，不是语法糖。

容易误解的地方：

1. 很多人以为 `volatile` 是"开关缓存、强制每次读写主内存"，其实是 acquire/release 语义加内存屏障；实现上靠一条 lock 前缀指令或 `mfence`，而不是真的每次都走到 DRAM。
2. 很多人以为 `synchronized` 进入时会把之前的写刷进主内存，其实只有退出时才刷；进入时做的是让本地副本失效，方向相反。
3. 很多人以为 `volatile` 可以替代 `synchronized`，其实它没有互斥性，`i++` 这种读-改-写照样丢更新，因为三条字节码指令之间可以被插入。
4. 很多人以为 64 位 JVM 上 `long` 写天然原子就没问题了，其实原子性和可见性是两个正交的问题，原子只保证你不会读到半个值，不保证你读到新值。

记住这一句：JMM 从不承诺你的代码按写的顺序执行，它只承诺在有同步关系的地方，你读到的值是对的。

第2章　happens-before：JMM 的官方定义

一句话：happens-before 是可见性的偏序，不是时间先后；有链才可见。

为什么需要它：

上一章最后留下一个问题：凭什么说"线程 A 在 volatile 写之前的所有写，对线程 B 可见"？这句话在直觉上成立，但要写进规范、要让 Oracle 和 Azul 和 GraalVM 三家实现都认，就必须有一个可判定的形式化定义。

先看一个具体现场。

```
初始 A = B = 0
Thread 1:  B = 1;   r2 = A;
Thread 2:  A = 2;   r1 = B;
```

直觉推理是这样的：如果指令 1 先执行，那 `r2 = A` 时 A 还没被写，`r2 == 0`；如果指令 3 先执行，那 `r1 = B` 时 B 还没被写，`r1 == 0`。所以 `r1 == 1 && r2 == 2` 不可能同时成立。

这个推理看起来无懈可击，但它有一个隐藏前提：**存在一个所有线程共享的、统一的执行顺序**。而 JMM 明确不提供这个前提。实际运行中，`r1 == 1 && r2 == 2` 是可以出现的——只要两个线程各自的写都进了 store buffer，各自的读都从本地缓存拿。

问题来了：如果连"先读还是先写"都无法确定，你怎么判断一段代码对不对？靠经验、靠跑测试、靠加日志？都不行。JMM 必须给出一套**只看代码就能判定**的规则。

这套规则就是 happens-before。它不描述时间，它描述可见性；它不是"谁先跑"，而是"谁的结果必须被谁看见"。这一章要做的事，是把这套规则从定义推到用法，包括它明确不管的那些地方。

核心结构：

概念	作用
线程间操作（inter-thread action）	四元组 `<t, k, v, u>`：线程、类型、变量或监视器、唯一标识；只收录能被其他线程观察到的动作
程序顺序（program order, po）	单线程内所有线程间操作的全序，等于代码书写顺序
同步顺序（synchronization order, so）	所有同步操作构成的全序，且在每个线程内与 po 一致；是跨线程唯一的锚
synchronizes-with（sw）	so 里那些"释放 → 获取"的配对：解锁→加锁、volatile 写→读、start→首操作、末操作→join 返回
happens-before（hb）	`(po ∪ sw)+`，po 与 sw 的并集取传递闭包
冲突（conflicting）	两个线程访问同一变量，且至少一个是写
数据竞争（data race）	冲突且两个操作之间没有 hb 关系
正确同步（correctly synchronized）	所有冲突访问都被 hb 排序，即无数据竞争
连续一致性（sequential consistency）	存在一个全序，所有操作原子且立即可见，且与程序顺序一致

关键代码：

JLS 17.4.5-1 给的例子，同一段代码，两个都合法的执行。

```
初始 A == B == 0
Thread 1:  B = 1;  r2 = A;
Thread 2:  A = 2;  r1 = B;
```

执行一，符合直觉：

```
B = 1;
A = 2;
r2 = A;   // 读到 0（此时 A 已被写成 2，但这次读看到的是更早的写）
r1 = B;   // 读到 1
```

执行二，反直觉但完全合法：

```
r2 = A;   // 读到 2 —— 读发生在写之前，却看到了写的结果
r1 = B;   // 读到 1
B = 1;
A = 2;
```

第二个执行里，两个读都"看到了未来"。这在时间上荒谬，在 hb 一致性上无懈可击：两个线程之间没有任何 hb 边，读只要不违反 hb 约束，就可以看见任何一个写。

拆解：

一、两层结构：synchronizes-with 是锚，hb 是它的传递闭包

JLS 17.4 里其实有三个不同的东西，初读极易混为一谈。

program order 是单线程内所有线程间操作的全序。它就是代码顺序，仅此而已。注意它是个"名义"顺序：po 上有序，不代表执行时有序，重排随时可以发生。

synchronization order 是所有同步操作构成的一个全序，并且要求在每个线程内部，这个顺序与 po 一致。同步操作是一份封闭清单：volatile 读、volatile 写、加锁、解锁、线程的第一个和最后一个操作、线程启动、检测线程结束、写入默认零值、中断。

happens-before 则是前两者的合成。

关键洞察在这里：so 只覆盖同步操作，普通读写在 so 里根本没有位置。所以光有 so 管不了普通变量。必须有一个东西把普通操作挂到同步操作上——这就是 po。挂上去之后再用传递性串起来，才得到跨线程的可见性保证。

写成公式：

```
hb = (po ∪ sw)+
```

其中 sw（synchronizes-with）是 so 里那些方向明确的配对。JLS 用四条规则描述 hb，但等价于上面这个闭包：

1. po ⊆ hb —— 同线程内，代码在前则 hb 在前
2. sw ⊆ hb —— 同步配对
3. 传递性 —— hb(x,y) 且 hb(y,z) 则 hb(x,z)
4. 构造方法最后一行 hb finalize 第一行 —— 单独追加，不在闭包里

理解"hb 是闭包"这件事的价值在于：后面所有推论都不用背，可以算。比如上一章那个 volatile 推论，就是 po、sw、po 三段接起来的。再比如 `join` 之后能读到子线程的结果，是"子线程末操作 →(sw)→ join 返回 →(po)→ 后续读"。链条清楚，结论就稳。

顺带把"操作"这个词的边界划清楚，否则后面读规范会一直卡。JLS 只收录**线程间操作**，也就是"能被另一个线程检测到或影响到"的动作，用一个四元组 `<t, k, v, u>` 描述：哪个线程、什么类型、作用在哪个变量或监视器、以及一个唯一标识。四类：普通读、普通写；同步操作（volatile 读写、加解锁、线程首末操作、start、检测结束、默认零值写、中断）；外部操作（native 调用，结果受 JVM 之外的环境控制）；线程分歧操作（一个只跑无限空循环、不碰任何内存的线程产生的动作）。

只收这四类，是因为规范压根不打算管线程内部的事——把两个局部变量相加存进第三个，这种操作别的线程永远看不见，讨论可见性没有意义。线程内部的行为由"线程内语义"保证：你可以一行一行读代码预测结果，唯一不确定的是"这次读到底读到了哪个写"。这个分工很关键，它把一个大问题切成两半：单线程内的语义交给语言，跨线程的可见性交给内存模型。

外部操作和分歧操作的存在感很低，但各有各的用处。外部操作不能被重排，因为 JIT 不知道 native 方法体里干了什么——`jni(); foo = 42;` 这两句顺序必须保住，哪怕 `foo` 和 `jni` 看起来毫无关系。分歧操作则是为了让规范能描述"一个死循环线程把整台机器的核心占满，导致其他线程全部饿死"这种非终止执行：一旦线程执行了分歧操作，它后面就跟着无穷多个分歧操作，规范借此把"永远跑不完"形式化。

二、七条落地规则，每条都在堵一个具体的洞

把闭包展开，得到的是程序员实际会用的那几条。逐条看它堵的是什么。

同一线程内，代码在前则 hb 在前。这条最弱，也最容易误读：它不禁止重排，只是说"重排之后的可观察结果必须和没重排一致"。它堵的是"我明明先赋值后使用"这种单线程直觉——单线程内确实成立，但跨线程就只剩可见性，不剩顺序。

对监视器的解锁 hb 后续对该监视器的加锁。堵的是临界区之间丢更新。线程 A 在临界区里写完 `count` 退出，线程 B 进来必须看到新值。

对 volatile 变量 v 的写 hb 后续对 v 的读。堵的是标志位不生效，也就是上一章开头那个 `while (!stop)` 死循环。

线程 start() hb 该线程的第一个操作。堵的是传参不可见——`new Thread(r)` 之前在 `r` 里设置的字段，新线程必须能看到，否则每个线程启动都得自己再同步一次。

线程 T1 的所有操作 hb T2 从 `T1.join()` 返回之后的任何操作。堵的是"join 完了结果还是旧的"。注意这条比 start 那条强：start 只保证第一个操作之前，join 保证全部操作之后。

对每个字段写入默认值（0/false/null）hb 于任何线程对它的访问。这条最反直觉：它说零值初始化在概念上发生在程序启动那一刻，而不是对象分配时。堵的是"对象还没构造完就读到随机垃圾值"——Java 保证你至少能读到零值，不会读到上一轮回收留下的脏数据。

T1 中断 T2 hb 任何线程检测到 T2 被中断。堵的是中断状态丢失：`interrupt()` 设置了标志，另一个线程调 `isInterrupted()` 必须能看到。

这七条覆盖了 Java 里所有合法的跨线程通信方式。反过来说，如果你用的通信手段不落在这七条里，你就没有 hb 保证——包括 `Thread.sleep`、`Thread.yield`、写日志、`System.out`。JLS 17.3 专门写了一句：`sleep` 和 `yield` 不具备任何同步语义，虚拟机不要求在调用前后刷出或重读缓存。所以"睡一会儿等等它"从来不是同步手段，只是让 bug 更难复现。

三、hb 不是时间序，读可以看见"未来"的写

这是整章最容易错、也最值钱的一点。

hb(x, y) 的官方表述是"x 对 y 可见"，不是"x 先于 y 执行"。JLS 原文写得很直白：两个操作之间存在 happens-before 关系，并不蕴含它们在实现上必须按这个顺序发生；只要重排后的结果与某个合法执行一致，这个实现就不非法。

前半句还好接受。真正颠覆直觉的是反方向：**没有 hb 关系的两个操作，读可以读到执行顺序上更晚的写**。前面那个"执行二"就是实例——`r2 = A` 在 `A = 2` 之前执行，却读到了 2。

为什么规范要允许这种明显荒谬的事？因为要禁止它，JMM 就必须知道全局时间顺序，那就等于要求所有实现做全局同步、给每个内存操作打时间戳。所有优化立刻归零。JMM 的选择是：只约束有因果关系的地方，没因果关系的地方一律放开。

这个选择的直接后果，是一条必须刻进肌肉记忆的结论：**无同步的多线程代码，你连"先读后写"都保证不了，更别谈"先写后读"**。你能确定的只有一件事——如果两个访问之间没有 hb 链，那它们之间什么都没有。

还有一个例外值得单独记：final 字段的 freeze 规则在规范里明确标注了 "This happens-before ordering does not transitively close with other happens-before orderings"——final 的可见性保证是一条独立的边，不参与传递闭包。这是 JLS 里唯一一处主动切断传递性的地方。目的是让 JIT 能放心地把 final 字段的值缓存在寄存器里，不必因为别处的 hb 链就重新加载。

那么"读到哪个写"到底怎么判定？规范给了两条充要条件。读操作 r 能看到写操作 w，当且仅当：

1. r 不 hb 于 w（即不成立 hb(r, w)），也就是这个写没有在因果上被读甩在后面；
2. 不存在第三个写 w'，使得 hb(w, w') 且 hb(w', r)，也就是没有别的写插在中间把它盖掉。

这两条只用了"不 hb"这种否定式，没有用任何正向的时序要求——这正是"读可以看见未来的写"能成立的机制。只要没有 hb 链明确禁止，读就能看见。规范把这个性质叫 happens-before 一致性（happens-before consistency）。

但只有 hb 一致性还不够，因为它会放过循环论证。举例：一个线程写 `x = 1` 然后读 `y` 得到 1，另一个线程写 `y = 1` 然后读 `x` 得到 1——两边都能"自圆其说"地解释对方的值，形成一个没有起点的因果闭环。为了排除这类执行，JLS 17.4.8 追加了因果性要求（causality requirements），要求每个读看到的值都能通过一条有限的、从初始状态出发的因果链解释。这套形式化极其晦涩，实践中不必推演，但要知道它的存在：hb 一致性是必要条件，因果性是第二个筛子，两者合起来才定义"合法执行"。

四、数据竞争、正确同步，以及那个"逃生舱"

定义是层层递进的：

冲突：两个线程访问同一个变量，至少一个是写。注意"至少一个是写"——两个读永远不冲突。

数据竞争：冲突，且两个操作之间没有 hb 关系。有 hb 关系的冲突访问是合法的，那叫同步，不叫竞争。

正确同步：程序里所有冲突访问都被 hb 排序，也就是没有数据竞争。

然后是最重要的一条定理（JLS 17.4.5）：**如果程序是正确同步的，那么它的所有执行都表现为连续一致（sequential consistency）**。

这条定理的价值不在于它说了什么，而在于它给了程序员一个逃生舱：你不需要知道任何重排细节，不需要知道 x86 和 ARM 差在哪，不需要读汇编，只需要证明"所有共享可变状态都被同一把锁或同一个 volatile 保护"，就能推出结果可预测。

但边界必须说清楚，否则会滥用：

第一，它只管内存访问的可见性，不管业务逻辑。正确同步的程序照样可以算错账。它保证的是"你读到的是某个真实发生过的写"，不是"你读到的是你想要的那个写"。

第二，它要求"所有"冲突访问都有 hb 关系。漏掉一个字段，不是那个字段坏掉，是整个程序不再享有连续一致保证。这是一票否决制，不是逐条计分。

第三，连续一致是"表现出"，不是"实现上"。JIT 照样重排，只是重排后的结果必须和某个连续一致的执行无法区分。所以你在调试器里看到的执行顺序，未必是真实顺序。

一票否决这一点值得举个例子。假设一个对象有 `a`、`b`、`c` 三个字段，用 `synchronized` 保护 `a` 和 `b`，`c` 漏了、裸着被两个线程读写。这个程序不是"`a`、`b` 安全、`c` 不安全"，而是整体不再享有连续一致保证——因为定理的前提是"所有冲突访问都被 hb 排序"，缺一条就不成立。更麻烦的是，缺了之后你甚至不能用"其他字段是对的"来推理，因为编译器的优化决策是全局的：它看到 `c` 没被同步保护，反而更有理由把相关的读提到临界区外面去。这也解释了为什么"大部分时候没问题"的代码会在一次 JIT 重新编译后突然出错——优化边界变了，原来碰巧成立的执行路径没了。

最后还有一层区分，规范写得很晦涩但概念上必须分清：**happens-before 一致性**弱于连续一致性。17.4.5-1 里 `r1 == 0 && r2 == 0` 那个结果，是 hb 一致的（每个读都能找到一个不被 hb 禁止的写），但不是连续一致的（不存在一个全序能解释它）。JMM 把 hb 一致性当作必要条件，再叠加 17.4.8 的因果性要求（causality requirements）来筛掉循环论证的执行。这块原文极难读，实践中记住结论即可：hb 一致是底线，正确同步才拿到连续一致。

容易误解的地方：

1. 很多人以为 happens-before 是"先发生"的时间关系，其实是可见性偏序；重排之后仍然可以满足 hb，只要读到的值合法。
2. 很多人以为 volatile 写 hb volatile 读意味着值"立刻"可见，其实那是语义保证而非时序保证——读必须看到该写或更晚的写，但中间隔多久没有约束。
3. 很多人以为没有数据竞争就等于程序逻辑正确，其实它只保证表现出连续一致性，业务层的竞态、丢失更新、ABA 一个都不会少。
4. 很多人以为 `synchronized` 能防住所有重排，其实它只约束进出临界区的那一对屏障，临界区外的普通读写照样可以被重排——DCL 里内嵌 `synchronized` 的失败方案就是这么翻车的。

记住这一句：happens-before 不描述时间，它描述"你能看见什么"；只要有一条 hb 链从写连到读，读就必须看见那次写。

第3章　AQS 骨架：一个 int 和一个双向队列

一句话：一个 volatile int 记锁状态，一条双向队列排线程，park/unpark 负责挂起与唤醒。

为什么需要它：先看一个几乎所有人都写过的错误实现。想自己造一把锁，最直觉的写法是自旋 CAS：

```
public void lock() {
    while (!UNSAFE.compareAndSwapInt(this, stateOffset, 0, 1)) {
        // 空转，什么都不做
    }
}
```

单核两线程时它工作得很好，所以本地测不出问题。上到 32 核、200 个线程争同一把锁时，问题立刻暴露：抢不到的那 199 个线程全在 CPU 上烧指令，真正的业务线程分不到时间片，吞吐随并发数上升反而下降。改成 `Thread.sleep(1)` 让出 CPU，又换来三个新问题：唤醒延迟被固定成毫秒级、唤醒顺序完全随机、无法实现 `tryLock(timeout)`。再退一步用 `Object.wait/notify`，`notify` 只能随机唤醒一个、不能指定唤醒谁、不能在等待前判断条件是否已满足，更没法表达"许可数"这种非互斥语义。

推演一下"如果没有 AQS"：`ReentrantLock` 要写一遍排队、`Semaphore` 要写一遍、`CountDownLatch`、`ReentrantReadWriteLock`、`ThreadPoolExecutor` 的 `mainLock`、`FutureTask` 各自都要写一遍。而这条路径上每一处都是坑——CAS 失败后的重试、节点取消后的链表清理、`park` 与 `unpark` 的时序、中断语义、重入计数。Doug Lea 的选择是把这套"抢不到怎么办"的机制全部抽出来做成模板，只留一个钩子给子类：**state 怎么解释、此刻能不能拿**。这就是 AQS 的全部设计意图——它不实现锁，它实现排队。

本章要回答四件事：一个 int 凭什么够用、`waitStatus` 的四个值到底描述谁、队列为什么必须是双向的、`acquire` 的三步各自在防什么。

核心结构：

概念	作用
`volatile int state`	同步状态。AQS 只负责存和 CAS，语义完全由子类定义
`exclusiveOwnerThread`	独占模式下当前持锁线程，仅用于重入判定与 unlock 校验
`head` / `tail`	队列首尾。head 是哨兵，永远不持有线程
`Node.thread`	被包装的线程；head 上恒为 null
`Node.prev` / `next`	双向链表指针。`prev` 可靠，`next` 是"尽力而为"的
`Node.waitStatus`	0 / CANCELLED(1) / SIGNAL(-1) / CONDITION(-2) / PROPAGATE(-3)
`Node.nextWaiter`	条件队列的单向指针，与 `next` 完全无关
`tryAcquire(int)`	子类钩子：唯一一次"试试能不能拿"的机会，失败返回 false
`addWaiter(Node)`	把当前线程包装成节点，走快路径入队，失败退到 `enq` 自旋
`acquireQueued(Node,int)`	队列中的主循环：判断是否轮到自己 → 决定是否 park → 醒来重试
`shouldParkAfterFailedAcquire`	把前驱置为 SIGNAL，返回"现在能不能安全 park"
`parkAndCheckInterrupt`	`LockSupport.park` 的唯一调用点，返回时清中断标记
`release(int)` / `unparkSuccessor`	释放状态并唤醒 head 之后第一个未取消的节点

关键代码：

```
// AQS.acquire：三步走，失败才排队
public final void acquire(int arg) {
    if (!tryAcquire(arg) &&                       // 第一步：先试。成功则整个方法结束，
                                                  // 队列都不用建，这是绝大多数情况
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))  // 第二步入队，第三步在队里等
        selfInterrupt();                          // 排队期间被中断过，这里只补标记，不抛异常
}

// 子类钩子。AQS 里它是空实现，直接抛 UnsupportedOperationException
protected boolean tryAcquire(int arg) {
    throw new UnsupportedOperationException();
}
```

```
final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {                                // 死循环，唯一的出口是抢到锁
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {   // 只有前驱是 head 才有资格试
                setHead(node);                    // 自己变成新哨兵：thread=null, prev=null
                p.next = null;                    // 断开旧哨兵，help GC
                failed = false;
                return interrupted;
            }
            if (shouldParkAfterFailedAcquire(p, node) &&  // 问：现在能安全挂起吗
                parkAndCheckInterrupt())          // 能，才真的挂起
                interrupted = true;               // 醒来只知道"被中断过"，继续抢
        }
    } finally {
        if (failed) cancelAcquire(node);          // tryAcquire 抛异常时兜底，标记取消
    }
}

private static boolean shouldParkAfterFailedAcquire(Node pred, Node node) {
    int ws = pred.waitStatus;
    if (ws == Node.SIGNAL)
        return true;                              // 前驱已经承诺唤醒我，可以放心 park
    if (ws > 0) {                                 // 前驱取消了，往前找一个活的
        do {
            node.prev = pred = pred.prev;         // 顺手把取消节点从链上摘掉
        } while (pred.waitStatus > 0);
        pred.next = node;
    } else {
        // 前驱是 0（新建节点都是 0），CAS 成 SIGNAL 表示"我可能要你唤醒"
        // 注意这里返回 false，调用方会再转一圈 for 循环
        compareAndSetWaitStatus(pred, ws, Node.SIGNAL);
    }
    return false;                                 // 第一次进来几乎必然返回 false
}

private void unparkSuccessor(Node node) {         // 入参是 head
    int ws = node.waitStatus;
    if (ws < 0)
        compareAndSetWaitStatus(node, ws, 0);     // 唤醒责任已履行，状态清零
    Node s = node.next;
    if (s == null || s.waitStatus > 0) {          // next 不可靠：可能为 null，可能已取消
        s = null;
        for (Node t = tail; t != null && t != node; t = t.prev)
            if (t.waitStatus <= 0) s = t;         // 从尾部倒着找，找到最靠前的活节点
    }
    if (s != null) LockSupport.unpark(s.thread);
}
```

拆解：

一、state：一个 int，四种语义

很多人把 `state` 直接等同于"锁计数"，这只在 `ReentrantLock` 里成立。AQS 自己从不读 state 的含义，它只提供三个 `protected` 方法：`getState()`、`setState()`、`compareAndSetState()`。同一个字段在不同子类里是完全不同的东西：

- `ReentrantLock`：0 空闲，1 被占，n 表示重入 n 层。
- `Semaphore`：剩余许可数，`acquire` 是减，`release` 是加。
- `CountDownLatch`：还剩几次 `countDown`，减到 0 就永久放行。
- `ReentrantReadWriteLock`：高 16 位存读锁持有数，低 16 位存写锁重入数，一次 CAS 同时改两个计数。

为什么是 `int` 而不是 `boolean` 或 `long`？`boolean` 表达不了重入和许可数；`long` 在 32 位 JVM 上 CAS 不保证原子（要靠 `cmpxchg8b`，性能和可移植性都更差），而 32 位对这个场景绰绰有余——`ReentrantReadWriteLock` 把 16 位给读、16 位给写，65535 的上限已经远超任何合理用法。

独占模式下 state 的转移只有四种：`0→1` 加锁、`1→2` 重入、`2→1` 释放一层、`1→0` 真正释放并唤醒后继。这个闭环解释了 `ReentrantLock` 最经典的两个 bug：`lock()` 两次只 `unlock()` 一次，state 停在 1，队列里所有线程永久 park；`unlock()` 比 `lock()` 多一次，`tryRelease` 里 `Thread.currentThread() != getExclusiveOwnerThread()` 直接抛 `IllegalMonitorStateException`。`lock/unlock` 必须严格配对不是编码风格问题，是 state 能不能归零的问题。

还有一个容易被忽略的点：`setState` 是普通写，不是 CAS。为什么重入路径可以不用 CAS？因为能走到 `else if (current == getExclusiveOwnerThread())` 的只有持锁线程本身，独占语义保证同一时刻只有一个线程在写 state，不存在竞争。而 `0→1` 那一步必须 CAS，因为可能有一堆线程同时看到 0。判断"要不要 CAS"的标准不是"这个字段可不可变"，而是"此刻有没有别的线程可能同时写它"。

state 的 `volatile` 还承担了内存语义。`unlock` 里的 `setState(0)` 是一次 volatile 写，`lock` 里 `compareAndSetState(0, 1)` 是 volatile 读加写，前者 happens-before 后者，于是 A 在临界区里的所有普通写，对随后拿到锁的 B 全部可见。这就是"锁提供可见性"的底层机制——不是 JVM 对锁有什么特殊照顾，是 state 这个 volatile 字段的读写顺序在起作用。理解了这一点，就明白为什么自定义同步器时，所有状态变更都必须走 `setState` / `compareAndSetState`，而不能自己另开一个普通字段：另开字段就断了这条 happens-before 链，锁还在，可见性没了。

二、Node 与 waitStatus：SIGNAL 描述的是别人

`waitStatus` 有五个取值，其中只有一个是正数，这个设计不是偶然：

- `CANCELLED = 1`：节点已放弃。**唯一大于 0 的值**，所以全源码里判断"取消"统一写成 `ws > 0`，一次比较搞定，不需要 `==` 判断。
- `SIGNAL = -1`：**当前节点的后继**处于（或即将进入）park 状态，当前节点释放时必须 unpark 它。
- `CONDITION = -2`：节点在条件队列里等着，只在 `ConditionObject` 内部有效，进同步队列前必须被 CAS 改回 0。
- `PROPAGATE = -3`：共享模式下唤醒需要向后传播，独占模式见不到。
- `0`：初始值。`new Node()` 不设任何状态，所以每个刚入队的节点都是 0。

最反直觉的一点：**SIGNAL 描述的不是自己，是后继**。新节点入队时自己是 0，是它的**前驱**被改成 -1。所以一条正常队列里，除了最后一个节点，几乎每个节点的 `waitStatus` 都是 -1——因为它的后继替它设了。换个说法：每个节点入队后做的第一件事，是回头告诉前面那个人"我要睡了，你释放的时候记得叫我"。

这个约定解决了两个问题。第一，唤醒有了明确的责任人——不是"随便找个线程叫醒"，而是"前驱释放时叫醒它的直接后继"，责任链是局部的，不需要全局锁。第二，`release` 里可以廉价地判断"要不要叫"：`if (h != null && h.waitStatus != 0)`，head 的 ws 是 0 说明后面没人等着（或已经叫过了），直接跳过 `unparkSuccessor`，省掉一次系统调用。

把 `CANCELLED` 定成 `1` 而不是 `-4`，是个刻意的编码技巧：所有"还需要被唤醒"的状态都是负数（-1/-2/-3），所有"已经放弃"的状态是正数，于是全源码里只需要 `ws > 0` 和 `ws < 0` 两种比较，不用枚举。代价是可读性——第一次读的人很难从 `if (ws > 0)` 一眼看出这是在判断"节点已取消"，而不是"状态数值偏大"。

三、队列：为什么双向，为什么 head 是哨兵

AQS 的队列是 CLH 锁的变体。原始 CLH 是单向链表加自旋：每个线程在自己的前驱节点上 spin，前驱释放就自然观察到。AQS 做了两处改造——把自旋换成 `LockSupport.park`（避免空转），把单向换成双向。

改双向的直接原因是 `unparkSuccessor`。唤醒时需要"找到 head 之后第一个没取消的节点"，如果只有 `next`，一旦中间有节点取消就得靠遍历删除，而删除在无锁链表里非常难写。有了 `prev`，就可以从 `tail` 倒着走，跳过所有 `ws > 0` 的节点，一行循环解决。

更精妙的是入队的三个步骤：

```
node.prev = pred;                   // ① 先挂 prev
if (compareAndSetTail(pred, node)) { // ② 再 CAS 抢 tail
    pred.next = node;               // ③ 最后连 next
    return node;
}
```

这三步不是原子的。假设线程 X 刚做完 ②、还没做 ③，此时另一个线程从 `head` 正向遍历，走不到 X——因为 `pred.next` 还是 null。但从 `tail` 反向遍历一定能走到 X，因为 `node.prev` 在 ② 之前就挂好了。**这就是 `prev` 可靠、`next` 不可靠的根源**，也是 `unparkSuccessor` 里那段"倒着找"的代码存在的原因，不是防御性编程，是必须。

`head` 是哨兵，不是持锁线程。这一点值得单独强调：`head.thread` 恒为 null，`head.prev` 恒为 null。它代表的是"上一个持锁者留下的位置"。真正的持锁线程根本不在队列里——它在 `tryAcquire` 成功那一刻就返回了，压根没入过队。所以严格说，队列是 `head.next` 开始的那一段，说"队列长度"时不该把 head 算进去。`setHead` 的语义也因此很清楚：我刚拿到锁，我把自己变成新的墓碑，旧墓碑可以回收了。

`enq` 里有两个细节常被误读。第一，`compareAndSetHead(new Node())` 之后紧跟的 `tail = head` 是普通写而不是 CAS，因为能进这个分支就说明刚才读到 `t == null`，而 head 刚被自己 CAS 成功，别的线程要么卡在同一个 CAS 上重试、要么还停在读 tail 为 null 的那一步，没人能同时写 tail。第二，这个分支**不 return**，只是把地基打好，必须回到循环顶部走 else 分支才真正把节点挂上去。很多人第一次读 `enq` 时会疑惑"设完 tail 怎么不返回"，答案就在这个 for 循环的形状里：初始化和入队是两轮循环，不是一轮。

还有一点值得对照：原始 CLH 里每个线程都在自己前驱的状态上自旋，天然不存在丢失唤醒的问题，因为自旋本身就是持续观察。AQS 把自旋换成 park，就必须引入 SIGNAL 这个显式协议——"我要睡了，你释放时叫我"——来替代持续观察。换句话说，`waitStatus` 里的 -1 不是设计上的优雅，而是阻塞的代价。

四、acquire 三步与一次完整推演

把上面所有零件装起来，走一遍两个线程的完整时序。先记住一个前提：`acquireQueued` 里 `p == head && tryAcquire(arg)` 这个条件，`p == head` 是必要条件。原因不是"只有队头才该拿锁"这种朴素的公平观，而是 head 是队列里唯一确定已经让出锁的节点——它曾经持锁，现在把位置腾出来了。其他位置的前驱可能还攥着锁，去试只是浪费一次 CAS。所以这个判断首先是性能优化，其次才顺带产生了顺序性。

线程 A 调 `lock()`：`tryAcquire` 里 `state == 0`，CAS `0→1` 成功，`setExclusiveOwnerThread(A)`，返回 true。`acquire` 的 `&&` 短路，整个方法结束。此刻 `head == tail == null`，队列根本没被创建。

线程 B 调 `lock()`：`tryAcquire` 失败（state 是 1 且 owner 不是 B）→ `addWaiter` 发现 `tail == null`，走 `enq`。`enq` 第一轮 CAS 建出哨兵 `new Node()`（ws=0，thread=null），`tail = head`，**注意这里不 return**，第二轮才把 B 挂上去。所以哪怕只有一个线程排队，队列里也是两个节点：一个空哨兵 + B。

`acquireQueued(B)` 第一轮：`p = head`，`p == head` 成立，再试一次 `tryAcquire`（万一 A 恰好在这几十纳秒里释放了呢），失败。进 `shouldParkAfterFailedAcquire(head, B)`：`head.ws == 0`，走 else 分支，CAS `0→-1`，**返回 false**。

第二轮：`p == head`，`tryAcquire` 仍失败；`shouldPark` 这次看到 `head.ws == -1`，返回 true；`parkAndCheckInterrupt` 里 `LockSupport.park(this)`，B 停住。

第一轮为什么不直接 park？因为从"决定要睡"到"真的睡下"之间有一个窗口，如果在这个窗口里 A 释放了锁并调用了 `unpark`，而 B 还没 park，就可能出现"叫早了"。`LockSupport` 的许可机制能兜住这个竞态（`unpark` 先于 `park` 会留下一个许可，后续 `park` 立即返回），但 Doug Lea 仍然选择先设 SIGNAL、再重试一次、确认抢不到才 park。这是一次典型的 double-check：把"可能已经轮到我"的情况在 park 之前消化掉，减少一次无意义的上下文切换。

A 调 `unlock()`：`tryRelease` 把 state 减到 0，owner 置 null，返回 true。`release` 里 `h = head`，`h != null && h.ws == -1` 成立，调 `unparkSuccessor(head)`：先把 `head.ws` CAS 成 0（唤醒责任已履行），`s = head.next = B`，`ws <= 0`，直接 `LockSupport.unpark(B.thread)`。

B 醒来：从 `parkAndCheckInterrupt` 返回 `Thread.interrupted()` = false，`interrupted` 保持 false，回到 for 顶部。`p = head`（还是那个哨兵），`tryAcquire` 这次成功，state `0→1`，owner = B。`setHead(B)`：`head = B`，`B.thread = null`，`B.prev = null`；`p.next = null` 断开旧哨兵。返回 false，`acquire` 结束，不触发 `selfInterrupt`。

此刻队列回到 `head == tail`，B 成了新哨兵，等下一个线程 C 从 `addWaiter` 的快路径挂上来。整个循环闭合。

最后补一个容易被跳过的收尾：`cancelAcquire`。它只在 `acquireQueued` 的 `finally` 里、且 `failed == true` 时被调用，典型场景是子类 `tryAcquire` 抛了异常（比如重入计数溢出）。做的事是：`node.thread = null` 断掉线程引用，`node.waitStatus = CANCELLED` 打上死亡标记，然后按位置分三种善后。自己是 tail，就把 tail 回退到前驱并断开 next；自己不是 tail 但前驱还活着，就把前驱的 next 直接指向自己的后继，让后面的人跳过自己；前驱是 head 或前驱也死了，就直接 `unparkSuccessor(node)` 把唤醒责任往下传。

关键认知是：取消的节点**不会立刻从链表上摘掉**，只是被标记。真正的物理摘除发生在别的线程的路径上——`shouldParkAfterFailedAcquire` 里那段 `do-while`，或者 `unparkSuccessor` 的反向遍历。这是无锁链表的典型做法：标记删除加惰性清理。如果在取消路径上直接做指针手术，就要处理"我正在摘自己，别人正在读我的 next"这种竞态，代价远高于让后来者顺手清理。

容易误解的地方：

- **以为 head 是当前持锁线程**。不是。`head.thread` 永远是 null。持锁线程在 `setHead` 之前根本不在队列里，在 `setHead` 之后又主动把自己的 thread 清空了。把 head 理解成"上一个持锁者的墓碑"比理解成"持锁者"准确得多。
- **以为 `waitStatus` 描述自己的状态、以为入队就立刻 park**。`SIGNAL` 描述的是后继，看到某个节点 ws=-1，正确解读是"它后面有人睡着，它释放时得负责叫醒"。正因为新节点自己的 ws 是 0，第一次进 `shouldParkAfterFailedAcquire` 必然返回 false——代码只是把前驱改成 -1 就退出了，真正 park 发生在第二轮循环。这个"多转一圈"是刻意设计的，不是低效。
- **以为 `unparkSuccessor` 直接取 `node.next`**。`next` 只是快路径。入队三步非原子，`next` 可能还是 null；节点取消后 `next` 也可能指向已取消的节点。所以 `s == null || s.waitStatus > 0` 时必须从 `tail` 反向遍历。这个分支在低并发下几乎不会走到，所以很多人读源码时直接跳过了。
- **以为 `acquire` 会响应中断**。不会。`parkAndCheckInterrupt` 里的 `Thread.interrupted()` 会**清除**中断标记，`acquireQueued` 只是把这件事记在 `interrupted` 变量里，一路返回到 `acquire`，最后用 `selfInterrupt()` 把标记补回去。所以 `lock()` 期间被中断，行为是"继续抢锁，抢到之后中断标记还在"，异常一个都不抛。想要可中断，得用 `lockInterruptibly()`，那条路径在 `parkAndCheckInterrupt` 返回 true 时直接 `throw new InterruptedException()`，`finally` 里 `cancelAcquire` 把节点标成 CANCELLED。

记住这一句：AQS 从不实现"锁"，它只实现"排队"；state 的含义和 tryAcquire 的判定，永远是子类的事。

第4章　公平与非公平：差在那一行判断

一句话：两者只差两次 CAS 时机：非公平在入队前多抢一次，换来吞吐与饥饿。

为什么需要它：一个真实场景。线上有个任务调度线程持锁约 200ms 做批量计算，后面排着 12 个工作线程。监控显示：改成公平锁之后，P99 延迟反而从 210ms 涨到 380ms，吞吐掉了三成。团队里有人说"公平锁当然慢"，但说不出慢在哪。另一个场景正好相反：某线程在日志里连续出现 `park` → `unpark` → `park` 的循环，等了 3 秒还没拿到锁，而锁的持有时间只有 2ms——它被新来的线程反复插队，饿死了。

这两个现象来自同一行代码。`ReentrantLock` 的公平与非公平，实现上的差异只有两处，加起来不到十行，但它们决定了锁在"空闲的那几十微秒"里归谁。不理解这两处，就无法解释上面两种截然相反的表现，也无法在"吞吐"和"确定性"之间做有依据的取舍。

本章还要解决另一半问题：`Condition`。很多人把它当成 `wait/notify` 的语法糖，于是写出"signal 之后线程立刻继续执行"的错误假设。实际上 `await` 涉及**两条完全不同的队列**，一次 `signal` 只是一次搬家，不是一次唤醒。这两件事必须一起讲，因为 `ConditionObject` 的节点就是从同步队列那套 `Node` 复用的。

核心结构：

概念	作用
`NonfairSync.lock()`	上来先 CAS 抢一次，成功直接返回，连 `acquire` 都不进
`FairSync.lock()`	直接 `acquire(1)`，没有任何抢跑
`nonfairTryAcquire`	不看队列，state 为 0 就 CAS。`tryLock()` 也走它
`hasQueuedPredecessors`	公平锁的守门员：判断"我前面是否有人"，是则不许抢
`ConditionObject.firstWaiter/lastWaiter`	条件队列首尾，单向链表，与 `head/tail` 无关
`Node.nextWaiter`	条件队列指针，与 `next` 是两个独立维度
`addConditionWaiter`	包装成 ws=CONDITION 的节点，挂到条件队列尾
`fullyRelease`	一次性释放全部重入层数，返回 `savedState`
`isOnSyncQueue`	判断节点是否已被搬到同步队列，是 `await` 自旋的退出条件
`transferForSignal`	把节点从条件队列搬到同步队列尾，必要时直接 unpark
`savedState`	记住 await 前的重入次数，唤醒后原样恢复

关键代码：

```
// 非公平：进门先抢，抢不到才走正常流程
static final class NonfairSync extends Sync {
    final void lock() {
        if (compareAndSetState(0, 1))              // 差异点一：不看队列，直接插队
            setExclusiveOwnerThread(Thread.currentThread());
        else
            acquire(1);                            // 失败了才和公平锁走同一条路
    }
    protected final boolean tryAcquire(int acquires) {
        return nonfairTryAcquire(acquires);        // 差异点二：这里也不看队列
    }
}

final boolean nonfairTryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (compareAndSetState(0, acquires)) {     // 没有 hasQueuedPredecessors
            setExclusiveOwnerThread(current);
            return true;
        }
    } else if (current == getExclusiveOwnerThread()) {
        setState(c + acquires);                    // 重入：这里不看队列，公平锁也不看
        return true;
    }
    return false;
}

// 公平：唯一的差别就是多了这个判断
protected final boolean tryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        if (!hasQueuedPredecessors() &&            // 差异点：前面有人就闭嘴排队
            compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    } else if (current == getExclusiveOwnerThread()) {
        setState(c + acquires);
        return true;
    }
    return false;
}
```

```
public final void await() throws InterruptedException {
    if (Thread.interrupted()) throw new InterruptedException();
    Node node = addConditionWaiter();              // ① 进条件队列，ws = CONDITION(-2)
    int savedState = fullyRelease(node);           // ② 释放全部重入层数，state 归 0
    int interruptMode = 0;
    while (!isOnSyncQueue(node)) {                 // ③ 只要还在条件队列就睡
        LockSupport.park(this);                    //    被 signal 搬走后这里返回
        if ((interruptMode = checkInterruptWhileWaiting(node)) != 0) break;
    }
    if (acquireQueued(node, savedState) && interruptMode != THROW_IE)
        interruptMode = REINTERRUPT;               // ④ 在同步队列里重新抢锁，并恢复重入次数
    if (node.nextWaiter != null) unlinkCancelledWaiters();
    if (interruptMode != 0) reportInterruptAfterWait(interruptMode);
}

private void doSignal(Node first) {
    do {
        if ((firstWaiter = first.nextWaiter) == null) lastWaiter = null;
        first.nextWaiter = null;                   // 断开条件队列，防止重复转移
    } while (!transferForSignal(first) && (first = firstWaiter) != null);
}

final boolean transferForSignal(Node node) {
    if (!compareAndSetWaitStatus(node, Node.CONDITION, 0))
        return false;                              // CAS 失败说明节点已取消，跳过它
    Node p = enq(node);                            // 搬进同步队列尾，返回新前驱
    int ws = p.waitStatus;
    if (ws > 0 || !compareAndSetWaitStatus(p, ws, Node.SIGNAL))
        LockSupport.unpark(node.thread);           // 前驱死了或设不了 SIGNAL，只能当场叫醒
    return true;                                   // 正常情况：不 unpark，只是搬了个家
}
```

拆解：

一、两处差异，精确到行

差异一在 `lock()` 入口。非公平锁第一件事就是 `compareAndSetState(0, 1)`，这一步发生在任何排队逻辑之前，成功了连 `acquire` 都不进。公平锁没有这一步，直接 `acquire(1)`。

差异二在 `tryAcquire` 内部。两者都判断 `state == 0`，但公平锁多一个 `hasQueuedPredecessors()`：只要队列里有人排在我前面，即使锁此刻空闲，我也不许抢，乖乖去 `addWaiter`。非公平锁没有这个判断，看到 0 就 CAS。

`hasQueuedPredecessors` 的实现值得逐字读：

```
public final boolean hasQueuedPredecessors() {
    Node t = tail;
    Node h = head;
    Node s;
    return h != t &&
        ((s = h.next) == null || s.thread != Thread.currentThread());
}
```

三种情况。`h == t` 表示队列空或只剩哨兵，返回 false，可以抢。`h != t` 但 `h.next == null`，说明有线程 CAS `tail` 成功了却还没写 `head.next`——也就是上一章说的入队三步非原子的那个窗口——此时**保守地返回 true**，让当前线程去排队。`h.next.thread == currentThread`，说明队头就是自己，返回 false。注意最后这个分支：公平锁判断的不是"队列非空"，而是"**我前面有没有人**"。队列里只有自己时，`hasQueuedPredecessors` 返回 false，照样可以抢。

重入路径两者完全一致，都不看队列。这一点很关键：公平锁的"公平"不覆盖重入，因为重入的是已经持锁的线程，它本来就不该排队。

还有个细节：非公平锁其实有**两次**插队机会。第一次在 `lock()` 入口那次 CAS，第二次在 `acquire → tryAcquire → nonfairTryAcquire`。这两次几乎在同一时刻发生，中间只隔了一次失败分支，所以第二次成功率不高，但它覆盖了一个真实场景：第一次 CAS 失败是因为锁正被占用，而在这几纳秒里持有者刚好释放了，第二次就能无缝接上，完全不需要入队。公平锁把这两次机会都关掉了——它连第一次 CAS 都不做，直接进 `acquire`，然后被 `hasQueuedPredecessors` 挡在门外。

二、为什么非公平吞吐更高

从释放锁到被唤醒的线程真正执行，中间有一条固定成本链：`unpark` → 线程进入就绪队列 → 调度器选中 → 上下文切换 → 从 `park` 返回 → 重新 `tryAcquire`。在主流服务器上，这条链是几十微秒量级。

这几十微秒里，`state` 是 0，锁是空闲的。非公平锁允许一个**正在 CPU 上运行**的新线程直接 CAS 拿走它——这个线程不需要被唤醒、不需要切换、不需要恢复上下文，它的栈是热的。相当于把锁传递的空窗期填满了。公平锁则严格禁止这种填充：即使锁空闲，只要队列里有人，新线程也必须入队，然后 park，然后等着被唤醒。锁从"释放"到"再次被持有"之间必然插入一次完整的上下文切换。

第二个原因是减少无效唤醒。非公平下，队列里的线程被唤醒后常常发现锁又被别人抢走了，只能重新 park——一次锁传递引发两次上下文切换，还多一次 `shouldParkAfterFailedAcquire` 的 CAS。公平锁减少了这种"唤醒即失败"的概率。

代价是饥饿。极端情况下，一个排在队头的线程可能连续被 N 个新来的线程插队，等待时间无上界。而且非公平锁的延迟抖动更大：快的时候几微秒，慢的时候可能几毫秒。

但要说清一个常见误解：**非公平不等于无序，更不等于"新线程总能赢"**。它只是"允许"插队，不"保证"插队。竞争激烈时新线程的 CAS 也大概率失败，最终还是要入队。真正被插队的窗口，只在锁刚释放、且新线程恰好在这个瞬间到达时才打开。反过来说，如果临界区很长、锁几乎从不空闲，那公平与非公平的实际表现差别很小——插队窗口根本不存在。

还有一笔隐性成本常被忽略：公平锁每次 `tryAcquire` 都要经 `hasQueuedPredecessors` 读 `head`、`tail`、`head.next` 三个 volatile 字段。高竞争下 `head` 被频繁改写（每次 `setHead` 都写一次），这条 cache line 会在多个核之间来回弹跳，读它的代价远高于一次普通读。非公平锁在 `lock()` 入口那次 CAS 只碰 `state` 一个字段，缓存亲和性好得多。所以公平锁多出来的开销不只是"多一次上下文切换"，还有"多读几个热点字段"。

实践上的取舍：默认用非公平（`new ReentrantLock()`），吞吐优先。只有在"必须防止饥饿"的场景才用公平锁——比如任务调度器、长事务、按序处理的流水线。而且要知道，公平锁保证的是"不插队"，不是"严格 FIFO 执行"：`tryLock()` 走的是 `nonfairTryAcquire`，即使在公平锁上也是插队的；重入也不排队。

回到开头那个"公平锁反而更慢"的现场：12 个工作线程每个都要走一遍完整的唤醒链，而锁传递是串行的，任何一次调度抖动都会累加到后面所有人身上，P99 自然被拉长。非公平锁让刚跑完上一个任务的线程直接接上，省掉的是这条串行链，不是省掉一次 CAS。这也说明一件事：公平锁的成本随队列长度线性增长，非公平锁的成本几乎与队列长度无关——后者才是它吞吐高的结构性原因。

三、Condition：第二条队列

`ConditionObject` 只有两个字段：`firstWaiter`、`lastWaiter`。它跟 AQS 的 `head/tail` 没有任何关系，是**完全独立的另一条队列**。

- **同步队列**：`prev` / `next` 双向，节点在等**锁**。
- **条件队列**：`nextWaiter` 单向，节点在等**条件**。

两条队列复用同一个 `Node` 类，但用的是两组不同的指针。所以一个节点可以同时"在同步队列的某个位置"和"曾经在条件队列的某个位置"，判断依据是 `waitStatus` 和 `prev`（`isOnSyncQueue` 就是这么判的）。

为什么要两条队列？因为"等条件"和"等锁"是两件事。生产者发现队列满了，它要等的是"队列不满"这个条件，而不是"锁被释放"。如果只有一条队列，`signal` 就退化成 `notify`——随机唤醒一个，被唤醒的线程醒来发现条件不成立，又得睡回去，这叫**惊群**。`Object.wait/notify` 只有一个等待集，所以多条件场景下只能用 `notifyAll` 全唤醒再各自检查，代价是 N-1 次无效唤醒。

`Condition` 把这件事做细了：一把锁可以 `newCondition()` 多次，每次生成一个独立的 `ConditionObject`，对应一条独立的条件队列。生产者等 `notFull`，消费者等 `notEmpty`，`signal` 精确唤醒对应的那一队队头。这是 `Condition` 相对 `wait/notify` 的核心优势，也是 `ArrayBlockingQueue` 用两个 `Condition` 而不是一个的原因。

条件队列的操作天然线程安全，不需要额外加锁——因为 `await` 和 `signal` 都必须在持锁状态下调用（`signal` 开头有 `isHeldExclusively()` 检查），同一时刻只有一个线程能碰这条队列。这也解释了为什么 `unlinkCancelledWaiters` 敢用毫无保护的三指针遍历写法：它是被锁保护着的，不需要任何 CAS。

另一个设计细节：`ConditionObject` 是 AQS 的**内部类**，不是 AQS 的子类。它要调 `enq`、`isOnSyncQueue`、`acquireQueued` 这些私有方法，做成内部类最省事。代价是强绑定了 AQS，所以 `Condition` 接口虽然独立，实现却只能长在 AQS 上——`StampedLock` 就因为没继承 AQS，只能自己实现一套条件等待，没法复用这个类。

四、await 与 signal 的转移过程

`await` 是五步：

① **`addConditionWaiter`**：`new Node(thread, Node.CONDITION)`，ws 直接是 -2，挂到 `nextWaiter` 链尾。入队前会检查 `lastWaiter.waitStatus != CONDITION`，如果是，说明队尾已取消，先调 `unlinkCancelledWaiters` 清理一遍。

② **`fullyRelease`**：`savedState = getState()`，然后 `release(savedState)` 一次性把 state 减到 0。注意参数是 `savedState` 而不是 1——如果之前重入了 3 层，这里必须一次放干净，否则别的线程根本拿不到锁，而自己已经睡了，死锁。释放失败（当前线程没持锁）就抛

`IllegalMonitorStateException`，`finally` 里把节点标成 CANCELLED，留给后面的人清理。

③ 自旋 park：`while (!isOnSyncQueue(node)) park(this)`。退出只有两个口子：节点被 `signal` 搬到了同步队列，或者线程被中断（`checkInterruptWhileWaiting` 返回非 0）。`isOnSyncQueue` 的判断顺序有讲究：`ws == CONDITION` 说明还没被搬；`prev == null` 说明没进过同步队列（`prev` 只在 `enq` 里设置）；`next != null` 说明肯定在队列里；三者都不满足时，从 `tail` 反向查一遍——还是因为入队三步非原子，`prev` 有了但 `next` 还没有的那个瞬间。

④ `acquireQueued(node, savedState)`：此时节点已经在同步队列里，走的是和普通抢锁完全一样的逻辑。参数传 `savedState` 而不是 1，是为了恢复重入层数：await 前 state 是 3，唤醒后要重新变成 3，否则 `unlock` 次数就对不上了。

⑤ 处理中断：`THROW_IE`（signal 之前中断，抛 `InterruptedException`）或 `REINTERRUPT`（signal 之后中断，补标记不抛）。区分依据是 `transferAfterCancelledWait` 里那次 `CAS(CONDITION, 0)` 成不成功——成功说明 signal 还没发生，中断赢了；失败说明 signal 已经把 ws 改成 0 了，中断来晚了。

`signal` 只有一步实质动作：`transferForSignal`。CAS 把 `ws` 从 -2 改成 0（失败说明节点已取消，直接跳过，处理下一个），`enq` 搬进同步队列尾，然后看新前驱：如果前驱 `ws > 0`（已取消）或者 CAS 成 SIGNAL 失败，就当场 `unpark`；否则什么都不做。

正常路径下 signal 不唤醒任何线程。它只是把节点从一条队列搬到另一条队列。被搬的线程还在 `park`，它要等同步队列里排在它前面的人都走完、前驱释放锁时才会被 `unpark`。这一点是理解 `Condition` 的分水岭。

走一遍完整时序，把两条队列串起来：

- A 持锁（state=1），B 调 `await()`：B 进条件队列（ws=-2），`fullyRelease` 把 state 减到 0、owner 置 null，B park。
- C 也调 `await()`：条件队列变成 `B → C`，state 仍是 0。
- D 调 `lock()`：拿到锁，state=1，同步队列里只有哨兵。
- A（此时 A 已经释放了锁，但它仍持有 `Condition` 的引用，通常由持锁的 D 来调）调 `signal()`：B 被 CAS 成 ws=0，`enq` 到同步队列尾。**B 仍在 park**。
- D 调 `unlock()`：`unparkSuccessor` 唤醒 B。B 从 `await` 里的 park 返回，`isOnSyncQueue` 为 true，跳出循环，`acquireQueued(B, savedState=1)` 拿到锁，state 恢复成 1，`await` 返回。
- 此时 C 还在条件队列里，等下一次 `signal`。

关键在于倒数第二步：B 从"被 signal"到"从 await 返回"，中间隔着一次完整的抢锁。如果同步队列里本来就有人排队，B 还得排在它们后面。**所以 `await` 返回不代表条件一定成立**——这就是 `BoundedBuffer` 里必须写 `while (count == items.length) notFull.await();` 而不能写 `if` 的原因。被唤醒只是"条件可能变了"，不是"条件一定满足了"。虚假唤醒（spurious wakeup）也会让 `if` 出错，`while` 是唯一正确的写法。

顺带说下带超时的版本。`awaitNanos` 的结构和不带超时的几乎一样，差别在循环体内：剩余时间小于 `spinForTimeoutThreshold`（1000 纳秒，即 1 微秒）时不再调 `parkNanos`，改成纯自旋。原因是 `parkNanos` 要进内核、要挂定时器、要处理时钟精度，开销远大于自旋这 1 微秒。超时后调 `transferAfterCancelledWait` 自己把节点搬进同步队列——**即使超时，节点也必须进同步队列**，否则 `acquireQueued` 拿不到合法的前驱，会在 `node.predecessor()` 上直接 NPE。这个方法的返回值还顺带回答了"到底是超时还是 signal 抢先"：返回 true 是超时（CAS `CONDITION→0` 成功，说明 signal 还没动过），返回 false 是 signal 已经搬过了。

容易误解的地方：

- **以为公平锁就是严格先来后到、以为 `hasQueuedPredecessors` 在判断队列非空**。它判断的是"我前面有没有人"：队列里只有自己时返回 false，照样可以抢。公平也只保证"队列里有人时不插队"，不保证 FIFO 执行顺序——`tryLock()` 在公平锁上走的仍是 `nonfairTryAcquire`，照样插队；重入路径也不看队列；被唤醒的线程和刚到的线程在 `tryAcquire` 上的竞争本身就没有顺序保证。
- **以为 `await` 只释放一层锁**。`fullyRelease` 传的是 `getState()`，一次放干净。如果只减 1，重入 3 层的线程睡了之后 state 还剩 2，别的线程永远拿不到锁，直接死锁。唤醒后靠 `savedState` 恢复层数，这一对是配套的。
- **以为 `signal` 会让线程立刻继续执行**。`signal` 只是把节点从条件队列搬到同步队列，不改变 park 状态（除非前驱已取消）。线程真正恢复执行要等它抢到锁。把 `signal` 理解成"搬家"而不是"唤醒"，后面所有行为都能推出来。
- **以为一把锁只有一个等待队列**。每个 `Condition` 一条。`signal` 精确唤醒指定条件队列的队头，这是它比 `notify`（一个 monitor 一个等待集，只能随机唤醒）强的地方。

记住这一句：公平与非公平的差别不在队列，而在"进队列之前允不允许再试一次"；Condition 的差别不在锁，而在多了一条只等条件的队列。

第5章　共享模式：CountDownLatch 和 Semaphore 怎么复用同一套队列

一句话：AQS 只规定"负数失败、非负成功、成功后向后传播"，state 是计数还是许可，全由子类定义。

为什么需要它：假设不用共享模式，硬拿 ReentrantLock 实现"等 10 个任务跑完"：主线程 lock 10 次，每个工作线程 unlock 一次。第一个问题立刻出现——unlock 抛 `IllegalMonitorStateException`，因为 AQS 的 `tryRelease` 校验 owner，不是持有者不许释放。好，自己写个 Sync 绕过校验，第二个问题紧跟着来：主线程第二次 lock 时 state 已经是重入计数，它根本不会阻塞，"等待"变成了"什么都没等"。再狠一点，把重入也拆掉，第三个问题无解——AQS 的独占释放路径在完全释放时只 `unparkSuccessor(head)` 唤醒一个节点，主线程醒来发现自己还差 9 次 release，而 release 已经不会再来了，队列里剩下的等待者永久挂起。

这不是实现粗糙，是语义错配。独占模式的前提是"锁一次只给一个人，释放一次只对一个人有意义"；而"等 N 个事件"要求"一次状态变化对所有等待者同时生效"。硬塞进独占模式，state 语义和唤醒策略两样都得改，改完它就不叫独占模式了。

这一章要解决的是：AQS 用什么样的最小协议，把"一对多的状态变化"接进同一套 CLH 队列；以及 CountDownLatch（倒计数）和 Semaphore（许可池）如何只覆写一个方法就复用这套机制。

核心结构：

概念	作用
`state`	共享资源的唯一数字。CountDownLatch 里是剩余未完成数，Semaphore 里是剩余许可数。AQS 不解释它的含义
`tryAcquireShared(arg)`	子类唯一必须实现的方法。返回值 <0 表示失败，>=0 表示成功，且该返回值会被当作"剩余资源量"交给传播逻辑
`tryReleaseShared(arg)`	返回 true 表示"这次释放可能让等待者满足"，AQS 据此决定要不要调 `doReleaseShared`
`Node.SHARED`	一个哨兵节点，赋给 `node.nextWaiter`，标记该节点是共享节点，`isShared()` 就是拿它做恒等比较
`doAcquireShared`	共享模式的入队 + 自旋 + park 循环，和独占版只差两处
`setHeadAndPropagate`	成功后把自己设为 head，并按条件继续唤醒后继
`doReleaseShared`	释放侧与传播侧共用的唤醒循环，处理 head 的 SIGNAL / 0 / PROPAGATE 三种状态

关键代码：

```
// CountDownLatch.Sync：只回答"门开了没有"，完全不碰 state
protected int tryAcquireShared(int acquires) {
    return (getState() == 0) ? 1 : -1;
}

protected boolean tryReleaseShared(int releases) {
    for (;;) {
        int c = getState();
        if (c == 0)
            return false;                // 已经开门，再 countDown 是空操作
        if (compareAndSetState(c, c - 1))
            return c - 1 == 0;           // 只有减到 0 的那一次才触发唤醒
    }
}
```

```
// AQS：共享模式的入队循环，以及成功后的传播
private void doAcquireShared(int arg) {
    final Node node = addWaiter(Node.SHARED);   // ① 和独占版唯一的入队差异
    boolean interrupted = false;
    for (;;) {
        final Node p = node.predecessor();
        if (p == head) {
            int r = tryAcquireShared(arg);
            if (r >= 0) {
                setHeadAndPropagate(node, r);   // ② 不是 setHead，是带传播的 setHead
                p.next = null; // help GC
                return;
            }
        }
        if (shouldParkAfterFailedAcquire(p, node) && parkAndCheckInterrupt())
            interrupted = true;
    }
}

private void setHeadAndPropagate(Node node, int propagate) {
    Node h = head;              // 旧 head，后面还要用
    setHead(node);
    // propagate > 0：还有余量，别人也可能成功
    // h.waitStatus < 0：旧 head 是 SIGNAL（确实有人在等）或 PROPAGATE（有人试图唤醒但扑空）
    if (propagate > 0 || h == null || h.waitStatus < 0 ||
        (h = head) == null || h.waitStatus < 0) {   // 新 head 也可能刚被后继改成 SIGNAL
        Node s = node.next;
        if (s == null || s.isShared())              // 后继为空或也是共享节点，才继续
            doReleaseShared();
    }
}
```

拆解：

state 的两种读法：减到 0 与加减自如

CountDownLatch 的 state 从 N 单调减到 0，是倒计数。注意 `tryAcquireShared` 里一个 `setState` 都没有——await 不消耗资源，只是检查门开没开。这个细节决定了它的行为：state 归零之后，任意多个线程、任意多次 await 全部直接返回，因为检查永远通过。共享模式允许"只读不写的 tryAcquireShared"，这一点和独占模式很不一样。

Semaphore 的 state 是剩余许可，acquire 减、release 加，两个方向都动。它的 `tryAcquireShared` 就是一个 CAS 循环：算出 `remaining = available – acquires`，`remaining < 0` 直接返回负数（不做 CAS，因为注定失败），否则 CAS 提交并返回 `remaining`。

`acquire(n)` 一次要 n 个许可。remaining 不够，返回负数，进队列；remaining 够但 CAS 失败，循环重试；CAS 成功，返回剩余量 r，这个 r 会一路传到 `setHeadAndPropagate`。所以"返回值即剩余量"不是随便定的，它就是传播逻辑的输入：r > 0 说明还有余量，后面排队的共享节点可能也满足，值得继续叫。

这里有个边界值得单独看：`new Semaphore(1)` 被一个线程拿走之后，第二个线程 acquire 时 remaining 恰好等于 0。0 不是负数，按 AQS 的约定算成功——它确实拿到了最后一个许可，只是拿完之后池子空了。返回值 0 传到 `setHeadAndPropagate`，`propagate > 0` 不成立，于是不主动往下叫。这是对的：后面的人一定拿不到，叫了也是白叫，他们会在下一次 release 时被唤醒。同一个返回值，既是"成功与否"的开关，又是"要不要继续传播"的信号，AQS 用两个不同的阈值（>=0 和 >0）把它一分为二。看懂这一点，共享模式的返回值约定就彻底通了。

对比一下 CountDownLatch：它的 `tryAcquireShared` 恒返回 1，永远是"资源无限"的姿态，所以每个醒来的线程都会往下传播，一路传到底。而 Semaphore 的传播是"按余量传"，余量耗尽就停。同一个 `setHeadAndPropagate`，两种子类给出两种传播行为，这就是模板方法的价值。

由此还引出一个必须记住的点：被唤醒不等于拿到资源。Semaphore 里线程被 unpark 后重新执行 `tryAcquireShared`，如果许可在这期间被新来的线程抢走一部分，remaining 又变负，它得继续 park。共享模式的循环天然允许反复失败，这也是它比独占模式更需要传播机制的原因。

再看释放侧。Semaphore 的 `tryReleaseShared` 无条件返回 true（只做溢出检查），意味着每次 release 都触发一次 `doReleaseShared`；CountDownLatch 只有减到 0 那一次返回 true。这个差异直接决定了唤醒频率：Semaphore 是"每次归还都敲一下门"，CountDownLatch 是"最后一票才开门"。

入队之后：共享节点和独占节点只差一个标记

`addWaiter(Node.SHARED)` 把 `node.nextWaiter` 指向那个哨兵节点，队列结构本身完全一样：head 是哑节点，每个节点的 waitStatus 由后继在 park 前改成 SIGNAL(-1)。

推演 `new CountDownLatch(2)`，t3、t4 先后 await，t1、t2 各 countDown 一次：

1. t3 入队：`head(ws=0) → t3(SHARED, ws=0)`。
2. t3 自旋，`p == head` 成立但 `tryAcquireShared` 返回 -1；`shouldParkAfterFailedAcquire` 把 head.ws CAS 成 -1，返回 false，不 park；第二次循环进来才真正 park。
3. t4 入队：`head → t3 → t4`，t4 把 t3.ws 改成 -1，然后 park。
4. t1 countDown：state 2→1，`tryReleaseShared` 返回 false，**什么都不发生**。中间状态的 countDown 完全不碰队列，这是最容易被忽略的一点。
5. t2 countDown：state 1→0，返回 true，进 `doReleaseShared`。此时 head.ws == -1，CAS 成 0，`unparkSuccessor(head)` 唤醒 t3。
6. t3 从 `parkAndCheckInterrupt` 返回，重新自旋，`tryAcquireShared` 返回 1，进 `setHeadAndPropagate`。

到第 6 步，如果只做 setHead，t3 走了，t4 还在 park。而 CountDownLatch 的 state 已经是 0，不会再有 release 来唤醒 t4 了。所以必须由 t3 顺手把 t4 叫醒——这就是"向后传播"的全部动机。

顺带说清共享模式对中断的两套处理，这在读源码时经常绕晕。`doAcquireShared`（不可中断版）在 `parkAndCheckInterrupt` 返回 true 时只是把局部变量 `interrupted = true` 记下来，继续自旋，等真正拿到资源后再 `selfInterrupt()` 补一个中断标记——中断被延迟到"成功之后"。`doAcquireSharedInterruptibly` 则是在 park 返回后立刻 `throw new InterruptedException`，走 `finally` 里的 `cancelAcquire` 把节点摘掉。CountDownLatch.await 用后者，因为"门没开"是一个可以安全放弃的等待；Semaphore 两个都提供，`acquire` 可中断、`acquireUninterruptibly` 不可中断，后者在"必须拿到许可才能继续"的场景里避免调用方处理异常。两套代码 90% 相同，差异只在 park 之后那一行——AQS 没有为它做抽象，因为多一层回调的开销比重复十几行更大。

传播：为什么醒来的线程有义务叫下一个人

独占模式里"一次释放唤醒一个"是对的：锁只有一个，被唤醒的人拿走，后面的人本来就该等下一次释放。共享模式里"一次状态变化同时满足所有人"才是常态：state 到 0 是对所有 await 的终态，Semaphore 一次 release 出 3 个许可可能满足 3 个等待者。如果只叫一个，而后续没有 release 了，剩下的永久挂起。

`setHeadAndPropagate` 干的事：先把当前节点变成新的哑 head，再判断要不要继续叫 `node.next`。条件里出现了两个 h，含义不同：

- 旧的 h（setHead 之前读的）：它的 waitStatus 是 SIGNAL，说明确实有后继挂在我身上等；是 PROPAGATE(-3)，说明之前有人试图唤醒但没找到人，这个信号还没消费掉。
- 新的 h（`(h = head)` 重新赋值）：setHead 之后，后继可能刚好在自旋里把新 head 的 ws 改成 SIGNAL，说明它已经准备好被 park 了，可以放心叫。

两个条件任意一个成立就调 `doReleaseShared`。而 `doReleaseShared` 本身是个循环，看 head.ws 分支：SIGNAL 就 CAS 成 0 然后 unpark 后继；0 就 CAS 成 PROPAGATE；末尾 `if (h == head) break;`——如果刚才唤醒的线程已经把 head 抢走了（h != head），说明它正在自己往下传播，那这里再跑一轮帮着多推一把，纯粹是吞吐量考虑。CAS 失败也 `continue`，因为很可能就是那个刚被唤醒的线程在同一行代码上抢先了。

于是唤醒变成接力：t2 叫醒 t3，t3 叫醒 t4，t4 发现后面没人了，循环退出。10 个等待者就是 10 次 unpark，不是一次广播。这一点常被误认为"共享模式一次性唤醒全部"，实际不是。

PROPAGATE：一个专门用来堵丢失唤醒的值

考虑这个时序：

1. 线程 A 执行 release，进 `doReleaseShared`，读到 head.ws == 0——后继刚入队，还没走到 `shouldParkAfterFailedAcquire`，所以还没把 head.ws 改成 -1。
2. A 调 `unparkSuccessor`，但后继此刻还没 park，unpark 打空，这个许可被白白消费掉了。
3. 后继继续执行，把 head.ws 改成 -1，然后 park。
4. 没有下一次 release 了。后继永久挂起。

PROPAGATE 就是补这个洞。A 在 ws == 0 时不 unpark（叫了也没用），而是把 ws CAS 成 -3，留下一个"还有人没被通知"的记号。之后：

- 如果后继在 park 之前又自旋了一次并成功，它走 `setHeadAndPropagate`，条件里 `h.waitStatus < 0` 命中（-3 < 0），继续 `doReleaseShared`，接力传下去。
- 如果后继直接 park 了，`shouldParkAfterFailedAcquire` 会把 -3 改成 -1（规则是 ws <= 0 就 CAS 成 SIGNAL），那么下一次有人 release 时，正常的 SIGNAL 分支会把它叫醒。

所以 PROPAGATE 的价值不在它自己的那个分支，而在于它让 `waitStatus < 0` 这个宽泛判断为真，把"我可能漏了人"这件事编码进 head 的状态里，交给后来者接手。这是整个 AQS 里最像"分布式协议"的一段代码：没有全局锁，靠状态位传递责任。

容易误解的地方：

1. "共享模式就是多个线程同时持有锁"。不对。AQS 从不检查谁持有，它只看 state 和 `tryAcquireShared` 的返回值。共享模式的本质是"允许多个线程同时通过同一个检查"，检查内容完全由子类定义。CountDownLatch 的 await 连 state 都不改，谈不上"持有"。
2. "CountDownLatch 可以重置"。不能。state 减到 0 后 `tryReleaseShared` 直接 `return false`，再 countDown 无效；await 永远直接返回。要复用只能 new 一个，或者改用 Phaser。
3. "`doReleaseShared` 一次性唤醒所有等待者"。它只 `unparkSuccessor(head)`，一次一个，靠醒来的线程接力。所以如果某个等待者被中断或抛异常，传播链会断一截，这也是共享模式对中断处理格外小心的原因。
4. "Semaphore 的 release 必须由 acquire 过的线程调用"。没有这个约束。`releaseShared` 不校验 owner，任何线程都能 release，许可数会凭空增长。这是它作为"信号量"而非"锁"的体现，也是误用时最危险的坑——比如在 finally 里无条件 release，异常路径下会把许可池撑爆。
5. 顺带一条：`new Semaphore(n)` 是非公平的。公平模式才多那一行 `hasQueuedPredecessors()`。默认构造下，新来的 acquire 直接 CAS 抢，排队的老线程只能干等。

记住这一句：共享模式没有发明新队列，它只给"唤醒"加了一条规矩——谁被满足，谁就有义务把这份满足传给下一个人。

第6章　读写锁：读读不互斥是怎么做到的

一句话：把一个 int 劈成两半，读锁总数在高 16 位、写锁重入在低 16 位，一次 CAS 同时判定两者。

为什么需要它：一个本地缓存，读远多于写。用 ReentrantLock 保护，10 个读线程被串行化，吞吐掉到单线程水平。直觉方案是上两把锁：读的时候抢读锁，写的时候抢写锁。这个方案有个致命窗口——

读线程检查"写锁没被占用"，通过；此刻写线程检查"读锁计数为 0"，也通过；两边各自 CAS 自己那把锁，都成功。结果是写线程在改数据的同时，读线程正在读。读写互斥被打破，而且这种 bug 只在极窄的时间窗出现，压测压不出来，上线靠运气。

根因是：读和写的判定必须原子。两个独立的 state 变量、两次独立的 CAS，中间必然有缝。要么引入一把元锁把两次判定串起来（那读读也互斥了，回到原点），要么把两个计数塞进同一个变量，让一次 CAS 完成两次判定。

ReentrantReadWriteLock 选了后者：state 是 32 位 int，高 16 位记读锁总持有数，低 16 位记写锁重入次数。这一章讲的就是这个位布局如何支撑"读读共享、读写互斥、写独占"，以及它带来的三个副作用：65535 硬上限、读锁重入必须靠 ThreadLocal 记、升级不可行而降级可行。

核心结构：

概念	作用
`SHARED_SHIFT = 16`	分界线。读锁计数在 bit[16,31]，写锁在 bit[0,15]
`SHARED_UNIT = 1 << 16`	读锁 +1 就是 state += 65536，低位一位不动
`MAX_COUNT = 65535`	单个 16 位段的上限
`sharedCount(c) = c >>> 16`	无符号右移取高 16 位
`exclusiveCount(c) = c & 0xFFFF`	掩码取低 16 位
`HoldCounter`	每线程的读锁重入次数 + 线程 id
`readHolds`	`ThreadLocal<HoldCounter>`，重入计数的权威来源
`cachedHoldCounter`	缓存"最后一个获取读锁的线程"的计数器，省一次 ThreadLocal 查找
`firstReader` / `firstReaderHoldCount`	把读计数从 0 变 1 的线程及其层数，读无竞争时的最快路径
`readerShouldBlock()`	读锁是否该退让，公平/非公平两套实现
`writerShouldBlock()`	写锁是否该退让，非公平模式恒为 false

关键代码：

```
// 读锁获取：一次 CAS 只动高 16 位
protected final int tryAcquireShared(int unused) {
    Thread current = Thread.currentThread();
    int c = getState();
    // 有写锁且持有者不是自己 → 失败。这个"不是自己"的豁免，就是锁降级的入口
    if (exclusiveCount(c) != 0 && getExclusiveOwnerThread() != current)
        return -1;
    int r = sharedCount(c);
    if (!readerShouldBlock() && r < MAX_COUNT &&
        compareAndSetState(c, c + SHARED_UNIT)) {
        if (r == 0) {                          // 我是把读计数从 0 变 1 的人
            firstReader = current;
            firstReaderHoldCount = 1;
        } else if (firstReader == current) {
            firstReaderHoldCount++;            // 最快路径：一次自增，不碰 ThreadLocal
        } else {
            HoldCounter rh = cachedHoldCounter;
            if (rh == null || rh.tid != getThreadId(current))
                cachedHoldCounter = rh = readHolds.get();
            else if (rh.count == 0)
                readHolds.set(rh);             // 缓存命中但 count 为 0，说明 Entry 被 remove 过
            rh.count++;
        }
        return 1;
    }
    return fullTryAcquireShared(current);    // CAS 失败或有队列，还有一次机会
}
```

```
// 写锁获取：只要有读锁在，就失败
protected final boolean tryAcquire(int acquires) {
    Thread current = Thread.currentThread();
    int c = getState();
    int w = exclusiveCount(c);
    if (c != 0) {
        // c != 0 而 w == 0：state 里只有读锁（可能还是自己持有的）
        // c != 0 且 w != 0 但持有者不是自己：写锁被别人拿着
        if (w == 0 || current != getExclusiveOwnerThread())
            return false;
        if (w + exclusiveCount(acquires) > MAX_COUNT)
            throw new Error("Maximum lock count exceeded");
        setState(c + acquires);      // 能走到这里是写锁重入，独占，不需要 CAS
        return true;
    }
    if (writerShouldBlock() || !compareAndSetState(c, c + acquires))
        return false;
    setExclusiveOwnerThread(current);
    return true;
}
```

拆解：

位运算：为什么一次 CAS 能完成两次判定

读锁 +1 是 `state += 65536`，写锁 +1 是 `state += 1`。两者在二进制上不重叠，所以可以独立累加而不互相覆盖——前提是 CAS 的期望值取对了。判断"能不能拿读锁"看 `exclusiveCount(c) != 0`，判断"能不能拿写锁"看 `exclusiveCount(c) == 0 && c == 0`，两个判断读的都是同一个 c，同一次 CAS 提交，中间没有缝。这就是整个设计的全部价值。

`sharedCount(c) = c >>> 16` 必须用无符号右移。如果读锁总数超过 32767，第 31 位变成 1，用 `>>` 会得到负数，`r < MAX_COUNT` 判断直接失效，读锁逻辑崩掉。JDK 这里写的是 `>>>`，一个字符的差别就是三万并发读下的一个 bug。

`exclusiveCount(c) = c & 0xFFFF` 是掩码，天然只取低位，不怕符号位。

为什么两边各 16 位？写锁要能表达重入层数，8 位只有 255 层太容易撞；读锁要能容纳高并发读，16 位给到 65535。对称取 16 是折中。代价是硬上限：单线程写锁重入超过 65535 抛 `Error`，全部线程读锁总数超过 65535 也抛 `Error`。注意读锁的 65535 是全局总和，不是每线程——1000 个线程各重入 66 层就会撞线。虽然罕见，但这是真实事故来源。

读锁重入：为什么非得引入 ThreadLocal

state 的高 16 位是全局读锁总数。unlock 时有一个必须回答的问题：这次释放是不是当前线程的最后一层？如果无脑减 65536，一个线程 unlock 两次就会把别人的额度也减掉，state 漂移，最终该唤醒写者时不唤醒，或者提前唤醒。

所以每个线程需要自己的计数器。但绝大多数读锁是无竞争的，每次都走 `ThreadLocal.get()` 太贵（ThreadLocalMap 开放寻址、要算 hash、要处理 Entry 状态）。于是有了三级结构，从快到慢：

1. `firstReader` + `firstReaderHoldCount`：把读计数从 0 变 1 的那个线程。它的层数直接放在一个普通 int 字段里，重入就是 `++`，释放就是 `--`，完全不碰 ThreadLocal。名字有误导性——它不是"第一个线程"，而是"当前唯一持有读锁的那个线程"；它释放到 0 后 `firstReader` 置 null，下一个把计数从 0 变 1 的线程接手。
2. `cachedHoldCounter`：缓存最后一个成功获取读锁的线程的 HoldCounter。多读线程交替时，unlock 的线程往往就是上一次 lock 的线程，命中率高，省掉一次 ThreadLocal 查找。用 tid 校验，不匹配就退回 ThreadLocal。
3. `readHolds`：ThreadLocal，权威来源。释放到 0 时 `readHolds.remove()`，防止线程池里的线程长期挂着一个 HoldCounter（值不大，但 Entry 会一直留在 ThreadLocalMap 上）。

这三层是纯优化，语义等价于"一个 `ThreadLocal<HoldCounter>`"。理解这一点，读锁那 100 行就不难了。`fullTryAcquireShared` 里那段"`get()` 之后发现 count == 0 就 `remove()`"的代码，也是为了让一次失败的尝试不留下垃圾 Entry。

释放侧的收尾同样值得看一眼。`tryReleaseShared` 先把当前线程的计数减 1（firstReader 走字段、其他线程走 HoldCounter，减到 0 就 `remove()` 防泄漏），然后在 for 循环里 `nextc = c - SHARED_UNIT`，CAS 成功后 `return nextc == 0`。注意这个返回值不是"释放成功"，而是"整个 state 归零了"——高低 16 位全为 0，意味着读锁和写锁都空了。只有这一种情况才需要 `doReleaseShared`，因为队列里排着的多半是等写锁的节点：只要还剩一个读者，写者就不该被唤醒，唤醒了也会因为 `w == 0 || owner != current` 而重新 park，纯属浪费一次上下文切换。写锁释放用的是同一个思路，`tryRelease` 里 `boolean free = exclusiveCount(nextc) == 0`，只有重入层数归零才算真释放。两个模式的"释放完成"判定，都归结为"state 里对应的那段是否清零"。

还有一个细节值得单独说：`Sync` 的构造方法里有 `setState(getState())`。看着像废话，实际是写屏障——`readHolds` 是普通字段，靠这次 state 的 volatile 写，保证其他线程看到这个 Sync 时，`readHolds` 已经初始化完成。

readerShouldBlock：非公平模式并不是无脑抢

读锁获取的第一道门是 `!readerShouldBlock()`。两套实现差别很大：

- FairSync：`hasQueuedPredecessors()`。队列里有人排队就退让，标准公平语义。
- NonfairSync：`apparentlyFirstQueuedIsExclusive()`。只有当 head.next 是写者时才退让。

为什么非公平模式要给写者特殊照顾？因为读锁是共享的，读线程可以源源不断插队。完全不设限的话，一个读请求持续不断的系统里写线程永远排不到——写饥饿。判 head.next 是不是写者，是一个极低成本的方向性保护：写者已经排到队头了，新来的读线程让一让；写者不在队头，那大家就比 CAS 速度。

但检查只发生一次。CAS 失败或者被 shouldBlock 拦下，会进 `fullTryAcquireShared` 再试。这个方法里最关键的不是重试，而是那个 `else if (readerShouldBlock())` 分支：如果 `firstReader == current`，直接放行；否则去查 HoldCounter，`count > 0` 说明是重入，放行，`count == 0` 才 `return -1` 去排队。

一个已经持有读锁的线程再次 lock 读锁（重入）。如果让它去排队，而队头是等写锁的写者，写者在等所有读锁释放，这个线程在等写者让路——自己等自己，死锁。所以重入必须无条件成功，哪怕队列里有人。这是 `fullTryAcquireShared` 存在的第二个理由（第一个是给 CAS 失败一次重试机会）。

判断"是不是重入"的办法就是查 HoldCounter：`firstReader == current`，或者 ThreadLocal 里 `count > 0`。

降级可行、升级不可行

降级：持有写锁 → 获取读锁 → 释放写锁。看读锁 `tryAcquireShared` 的第一行，`exclusiveCount(c) != 0 && getExclusiveOwnerThread() != current`。写锁被自己持有时这个条件为 false，直接往下走，CAS 把高 16 位加 1，成功。降级在源码层面就是被显式允许的。

升级：持有读锁 → 获取写锁。看写锁 `tryAcquire` 的 `if (c != 0)` 分支：`w == 0` 直接 `return false`。此时 c 的高位全是读锁（包括自己那一份），低位是 0，所以 w == 0，失败，然后 `acquireQueued` 入队 park。而唤醒它的条件是"所有读锁释放"，包括它自己手里那把——它永远不会释放，因为它在 park。自锁，而且是那种日志里看不出所以然的自锁：线程栈停在 `park`，没人持锁，但就是不动。

降级的真正意义不是"提升性能"，是消除窗口。回到那个缓存例子：写完数据后如果先 unlock 写锁再 lock 读锁，中间这个空隙里另一个写线程可以抢到写锁改掉数据，你接着读到的就不是你刚写的。降级保证从写锁到读锁的交接期间没有其他写者能插入。注意它不是原子的——读锁的获取发生在写锁释放之前，只是顺序被固定了。所以写法必须是 try 块里 lock 读锁、finally 里 unlock 写锁，顺序反了就等于没降级。

StampedLock 提一句：JDK 8 引入，用版本号（stamp）代替读锁计数。`tryOptimisticRead` 读的时候完全不加锁、只取一个 stamp，读完 `validate` 一次，没被写者改过就直接用，改过再退化到悲观读。代价是不可重入、不支持 Condition、乐观读不阻塞写者（写者可能一直插队）。读多写极少的场景吞吐能高一个量级，代码复杂度也上一个量级。

容易误解的地方：

1. "读读不互斥"不等于"读锁不排队"。有写者持有写锁时，读锁一样要等；非公平模式下如果队头是写者，新来的读锁也要让。读读共享只在没有写者、或写者不在队头时成立。
2. `cachedHoldCounter` 是普通字段，被多线程并发读写，看着像数据竞争。它确实会读到过期值，但有 tid 校验兜底，读错了退回 ThreadLocal，正确性不受影响。它靠 state 的 volatile 语义间接获得可见性，本身不是同步点。
3. 65535 是全局读锁总数上限，不是每线程重入上限。写锁的 65535 才是单线程重入上限。两者共用同一个 `MAX_COUNT` 常量，容易混淆。
4. 读锁不能创建 Condition。`readLock().newCondition()` 直接抛 `UnsupportedOperationException`，因为 Condition 的等待队列要求独占语义——多个读线程同时 await，`signal` 该唤醒谁无法定义。
5. 降级不是"必须"的。如果读操作不依赖刚写入的数据（比如只打个日志），先放写锁再拿读锁也没问题，代价是多一次加锁和一个窗口。

记住这一句：读写锁的全部精巧都在那个 int 里——把两个互相排斥的判断压进一次 CAS，剩下的代码都是为这次 CAS 服务。

第7章　Phaser：比 CountDownLatch 更灵活的分阶段同步

一句话：Phaser 把"参与者数量"和"阶段编号"都做成可变状态，一个实例能跑完任意多个阶段。

为什么需要它：一个数据导入任务分三阶段——解析、校验、入库，每阶段用一批工作线程，线程数按数据量动态决定。第一版用 `CyclicBarrier(10)`：

- 某天数据量小，只起了 8 个线程。8 个线程全部 await，barrier 的 count 停在 2，永远不归零，8 个线程全挂死，主线程在 `future.get()` 上等到超时。
- 你想改成"起线程之前先算好数量再 new"，但线程池是共享的，实际并发度受池大小限制，算不准。
- 三阶段要三个 CyclicBarrier，每阶段结束还要手工把结果传给下一阶段。
- 某个线程抛异常，`breakBarrier` 把整代打破，其他线程收到 `BrokenBarrierException`；你想 `reset()` 重来，而 reset 会让已经在等的线程全部异常返回。

换 CountDownLatch 好一点（它不要求参与者互相等待），但 latch 一次性：三个阶段三个 latch，每个都要单独传引用、单独 await，"谁负责在最后一个 countDown 之后做收尾"还得调用方自己判断。

这三个工具的差别不是 API 风格，是模型。CyclicBarrier 假设"参与者集合固定、每代重来"，CountDownLatch 假设"一次性事件"。真实的分阶段任务里，参与者会中途加入、中途退出，阶段数在运行时才知道。

Phaser 就是为这个模型设计的：parties 动态增删，phase 自动递增，一个实例贯穿全流程，还带每阶段回调。

核心结构：

概念	作用
`state`（long）	四段位域：bit[0,15] 未到达数、bit[16,31] parties、bit[32,62] phase、bit63 terminated
`phase`	当前阶段号，从 0 递增，到 `Integer.MAX_VALUE` 后回绕到 0
`parties`	已注册的参与名额数。`register` / `bulkRegister` 加，deregister 减
`unarrived`	本阶段还没到达的名额数，每 arrive 一次减 1，减到 0 触发推进
`register()`	parties + 1，同时让**当前阶段**的 unarrived + 1（新人也要等本阶段）
`arrive()`	unarrived - 1，不阻塞；若是最后一个，当场推进阶段并唤醒等待者
`arriveAndDeregister()`	arrive + parties - 1，用于中途退出
`arriveAndAwaitAdvance()`	arrive，然后等本阶段结束
`awaitAdvance(phase)`	等指定阶段结束，参数是调用者自己记录的阶段号
`onAdvance(phase, parties)`	阶段推进时的回调，返回 true 则终止 Phaser
`QNode` / `evenQ` / `oddQ`	等待者节点，按阶段奇偶分两个 Treiber 栈顶

关键代码：

```
// 位域划分
private static final int  UNARRIVED_MASK = 0xffff;              // bit 0-15
private static final long PARTIES_MASK   = 0xffff0000L;         // bit 16-31
private static final long TERMINATE_BIT  = 0x8000000000000000L; // bit 63
private static final int  PARTIES_SHIFT  = 16;
private static final int  PHASE_SHIFT    = 32;

private int doArrive(int adjust) {
    final Phaser root = this.root;
    for (;;) {
        long s = (root == this) ? state : reconcileState();  // 子节点先跟 root 对齐
        int phase = (int)(s >>> PHASE_SHIFT);
        if (phase < 0)
            return phase;                                    // 已终止，返回负数
        int unarrived = ((int)s == EMPTY) ? 0 : ((int)s & UNARRIVED_MASK);
        if (unarrived <= 0)
            throw new IllegalStateException(badArrive(s));
        if (UNSAFE.compareAndSwapLong(this, stateOffset, s, s -= adjust)) {
            if (unarrived == 1) {                            // 我是最后一个到达的
                long n = s & PARTIES_MASK;                   // 保留 parties 段
                int nextUnarrived = (int)n >>> PARTIES_SHIFT;
                if (root == this) {
                    if (onAdvance(phase, nextUnarrived))       // 回调在最后一个到达者的线程上跑
                        n |= TERMINATE_BIT;
                    else if (nextUnarrived == 0)
                        n |= EMPTY;                          // parties 归零，标记空
                    else
                        n |= ((long)phase + 1) << PHASE_SHIFT; // phase + 1，unarrived 段自动等于 parties
                    UNSAFE.compareAndSwapLong(this, stateOffset, s, n);
                    releaseWaiters(phase);                   // 唤醒本阶段的等待者
                } else if (nextUnarrived == 0) {
                    phase = parent.doArrive(ONE_DEREGISTER);  // 子节点空了，向父注销
                    UNSAFE.compareAndSwapLong(this, stateOffset, s, s | EMPTY);
                } else {
                    phase = parent.doArrive(ONE_ARRIVAL);
                }
            }
            return phase;
        }
    }
}
```

```
// 等待者用 Treiber 栈存，按 phase 奇偶分两个栈顶
static final class QNode {
    final Phaser phaser;
    final int phase;
    final boolean interruptible;
    final boolean timed;
    volatile boolean wasInterrupted;
    volatile Thread thread;
    volatile QNode next;        // LIFO 指针
}

private void releaseWaiters(int phase) {
    QNode q; Thread t;
    AtomicReference<QNode> head = (phase & 1) == 0 ? evenQ : oddQ;
    // 栈顶节点的 phase 还没追上当前 phase，就说明它是上一轮遗留，摘下来唤醒
    while ((q = head.get()) != null &&
           q.phase != (int)(root.state >>> PHASE_SHIFT)) {
        if (head.compareAndSet(q, q.next) && (t = q.thread) != null) {
            q.thread = null;
            LockSupport.unpark(t);
        }
    }
}
```

拆解：

一个 long 装四个字段，arrive 就是一次减法

位域这么切的原因：parties 和 unarrived 各 16 位，所以单实例最多 65535 个参与者，超了只能上分层；phase 用 31 位，够 20 亿次推进；terminated 单独占最高位，因为它必须在任何一次读 state 时被立刻看到，不能藏在别的字段里。

arrive 的核心是一次 CAS：`s - adjust`。unarrived 在最低 16 位，只要它大于 0（前面已经检查），减 1 不会向高位借位。这是位域设计的红利——不用拆字段、不用拼回去，一个减法就完成了"到达"。

但减完之后要判断是不是最后一个，这就是 `if (unarrived == 1)` 分支：CAS 之后 unarrived 段已经是 0，需要重新构造一个 state。做法是 `n = s & PARTIES_MASK` 只保留 parties 段，然后 `n |= (phase + 1) << 32`。注意此时 unarrived 段是 0，也就是新阶段"一个人都还没到"——但新阶段需要等的人数是 parties，所以这里其实还差一步：源码靠 `nextUnarrived` 判断是否为空，实际的 unarrived 重置由后续 `n` 的构造隐式完成（新 state 的 unarrived 段置 0，等价于"新阶段所有人未到"，与 parties 相同）。这个技巧很省事，也很容易看漏。

`onAdvance` 在最后一个 arrive 的线程上同步执行，位置是 CAS 之后、`releaseWaiters` 之前。两个后果：

- 如果 onAdvance 抛异常，阶段已经推进了，但等待者没被唤醒。这批线程会在下一次有人 arrive 时被动释放，表现为"偶发卡顿"。
- 如果 onAdvance 返回 true，state 被打上 TERMINATE_BIT，之后所有 arrive/await 直接返回负数。

所以 onAdvance 里只能放轻量、且不应该抛异常的逻辑。

同一套状态还能表达三种不同的同步语义，这是 Phaser 最被低估的地方：

- 只 `arrive()` 不 await，等于 CountDownLatch——到达即走，谁在等由调用方决定。
- `arriveAndAwaitAdvance()`，等于 CyclicBarrier——到达并互相等待，而且自动开启下一轮。
- 配合 `awaitAdvance(phase)` 单独使用，等于"阶段监听器"——不参与计数，只在某个阶段结束时被放行。

三种用法的差别不在 API，而在"到达之后走不走"。这也是它比 CyclicBarrier 更适合不定数量参与者的根本原因：CyclicBarrier 的 `parties` 是 `final` 字段，构造时定死，`count` 每代从 `parties` 重置，参与者集合在语义上被冻结；Phaser 的 parties 就活在 state 的 16 位里，register 一次加一，deregister 一次减一，随阶段演进。所以"这一批 8 个线程、下一批 12 个"在 Phaser 里只是多调几次 register，在 CyclicBarrier 里只能重建实例——而重建意味着已经在等的线程全部作废。

phase 与 awaitAdvance：为什么参数要传旧 phase

`arriveAndAwaitAdvance` 的实现就是两行：`int phase = arrive();` 拿到自己到达时的阶段号，再 `return awaitAdvance(phase);`。

`arrive` 返回的是"我到达时所在的阶段号"。`awaitAdvance(phase)` 的逻辑是：如果当前阶段已经不等于这个 phase（说明我到达之后，这班车已经开走了），立刻返回；否则入栈 park。

为什么不让 `awaitAdvance` 自己去读当前 phase？因为 arrive 和 awaitAdvance 之间有窗口：arrive 之后可能瞬间就有别的线程完成最后一次 arrive，阶段已经推进。此时 `awaitAdvance` 读到的是新 phase，如果拿新 phase 去等，就会白等一整轮。

传旧 phase 的语义是"我要等的是我参与的那一班"，开走了就直接走。这是 Phaser 里最容易写错的地方：手写 `awaitAdvance(phaser.getPhase())` 几乎总是错的，因为 `getPhase()` 和 `awaitAdvance` 之间还有窗口。正确做法是在同一个线程里先把 phase 存下来再传。

返回值是"实际到达的阶段号"，正常情况下等于 phase + 1，终止时返回负数。用返回值可以判断有没有被终止。

Treiber 栈：为什么等待者用栈而不是队列

AQS 用 CLH 队列，因为锁有先后顺序，必须 FIFO。Phaser 不需要顺序：同一个 phase 上的等待者完全等价，阶段推进时全体放行，谁先被 unpark 无所谓。

既然无序，栈就是最优选：只要一个 head 指针，push/pop 都是一次 CAS，没有 tail、没有双向指针、没有前驱的 waitStatus 要维护。这是标准的 Treiber 栈——无锁 LIFO。

但 Phaser 的栈有个特殊设计：两个栈顶，`evenQ` 和 `oddQ`，按 phase 的奇偶选。原因是等待者的 phase 可能和当前 phase 不一致：

线程 A 在 phase 5 入栈等待。阶段推进到 6，A 被唤醒但还没来得及出栈（被中断、或 unpark 后还没被调度）。此时线程 B 在 phase 6 调 `awaitAdvance(6)` 入栈。如果只有一个栈，B 会被 push 到栈顶，而 `releaseWaiters(5)` 正在从栈顶往下摘——它会摘到 B。B 醒来发现 phase 还是 6，重新 park，白醒一次；更糟的是唤醒循环的条件依赖节点 phase，混在一起会让"该唤醒的没唤醒"。

用奇偶分栈，phase 5 的节点在 oddQ，phase 6 的在 evenQ，物理隔离。`releaseWaiters(5)` 只碰 oddQ，摘到 phase 已经追上来的节点就停，不会误伤。

pop 用 `head.compareAndSet(q, q.next)`：先读栈顶 q，再把栈顶换成 q.next。CAS 失败（有人同时 push 或 pop）就重新读 q 重试。这是无锁栈的标准写法。注意它不处理 ABA——这里 ABA 无害，因为节点一旦出栈就不会再入栈，指针不会回到旧值。

顺便回答一个常见疑问：Phaser 为什么不直接复用 AQS？两个原因。一是容量，AQS 的 state 是 int，装不下 parties、unarrived、phase、terminated 四段信息，硬塞就得再加一个字段，那 CAS 的原子性就没了。二是语义，AQS 的 CLH 队列是 FIFO 且每个节点挂着 waitStatus 由后继维护，这套结构是为"有先后

顺序的独占获取"设计的；而 barrier 的等待者完全无序，维护双向链和 waitStatus 纯属浪费。Phaser 干脆自己写了一个只做"批量唤醒"的无锁栈，代码量比复用 AQS 还少。这也是 JDK 里少见的"不基于 AQS 的同步器"，和 CyclicBarrier 用 Condition 拼出来是同一个思路：工具要匹配问题形状，不要为了统一而统一。

分层：当 65535 不够，或者 CAS 太挤

单个 Phaser 的所有状态在一个 long 里，所有 arrive 都在 CAS 这一个变量。parties 到几千时 CAS 失败率急剧上升，自旋成本吃掉同步收益。

Tiering 的做法是把多个 Phaser 组成一棵树：叶子各自管一小批参与者，内部 CAS 只在自己的 state 上；子节点作为一个整体，向父节点注册一个 party。

几条必须记住的规则：

- `new Phaser(parent)` 不会立刻在父节点注册，要等子节点的第一个 party 注册时才注册。
- 整棵树的 phase 由 root 统一控制。子节点本地所有人都到了之后，还要等 root 推进，因为 root 要等所有子节点。
- 子节点的 `getPhase()` 要和 root 对齐，源码里那一堆 `reconcileState()` 就是在做这件事。
- 树结构下，子节点 parties 归零不会让自己 terminated，终止由 root 决定。

实际项目里几乎用不到分层，除非 parties 稳定在几千以上。知道它存在、知道它解决什么问题，就够了。

容易误解的地方：

1. `register()` 注册的是"一个参与名额"，不是"一个线程"。同一个线程可以 register 多次、arrive 多次，Phaser 不关心调用者是谁。反过来说，一个线程 register 两次却只 arrive 一次，整个阶段就卡住。
2. `arrive()` 不阻塞，但它可能触发阶段推进。如果它是最后一个到达的，`onAdvance` 会在当前线程上同步执行，然后唤醒所有等待者。所以 arrive 也可能很慢。
3. 中途 register 会拖住当前阶段。register 让 parties +1 且 unarrived +1，意味着当前阶段多了一个必须到达的人。如果你的意图是"下一阶段再加入"，必须等当前阶段结束再 register，Phaser 没有提供"注册到下一阶段"的 API。
4. terminated 之后不抛异常，静默返回负数。arrive 返回负、awaitAdvance 立刻返回、register 返回负数且不生效。代码看起来正常跑完，实际什么都没同步。长期运行的服务里，`onAdvance` 的终止条件写错（比如 `phase >= N` 里 N 被算成 0）会让整个同步机制静默失效，这是最难排查的一类 bug。
5. phase 会回绕。phase 存在 31 位里，到 `Integer.MAX_VALUE` 之后加 1 变负数，源码用 `& 0x7fffffff` 拉回 0。所以 `onAdvance` 里比较 phase 别用 `==`，用 `>=`，或者干脆依赖 terminated 位。

记住这一句：Phaser 把"等几个人"和"走到第几步"合并成一个可变的 long，于是同步器本身也成了可以被同步的对象。

第8章　线程池：ctl 这一个 int 装了什么

一句话：把状态和人数压进同一个 int，是为了让"还能不能收活"和"现在几个人"永远读到同一时刻的快照。

为什么需要它：某网关服务滚动发布时有两个说不清的偶发现象：一是 `shutdownNow()` 之后日志里仍有一条任务打印了"处理完成"；二是极少情况下 `awaitTermination(60s)` 一直返回 false，进程被 SIGKILL，可 `getPoolSize()` 明明已经是 0。两次排查都落到同一个窗口上——提交线程"读状态"和"加人数"是两步。它先读到 RUNNING、`workerCount=7`，判断"没到 corePoolSize，可以加人"，然后被调度走了；关闭线程在这期间把状态推到 STOP、清空队列、`tryTerminate` 看到 `workerCount != 0` 直接返回；提交线程回来把人数改成 8 并启动线程，这个线程就带着一个本该被丢弃的任务跑了起来。反过来，如果加人数和减人数不是原子的，`tryTerminate` 里那句 `workerCountOf(c) == 0` 就可能读到一个从没存在过的中间值，池子要么提前终止、要么永远终止不了。

这一章要解决的是：ThreadPoolExecutor 用什么数据结构，把"五种状态"和"几百个 worker"塞进一次原子读写里；以及 Worker 为什么不是裸 Thread，而要继承 AQS。

核心结构：

概念	作用
`ctl`	唯一的 `AtomicInteger`。高 3 位 runState，低 29 位 workerCount，一次 CAS 同时改两者
`COUNT_BITS = 29`	`Integer.SIZE - 3`。低位宽度，决定 CAPACITY，也决定"为什么不是 30+2"
`CAPACITY`	`(1 << 29) - 1` = 536870911。workerCount 的物理上限，同时充当取值掩码
`runStateOf(c)`	`c & ~CAPACITY`，把低 29 位抹掉，只留状态
`workerCountOf(c)`	`c & CAPACITY`，把高 3 位抹掉，只留人数
`ctlOf(rs, wc)`	`rs | wc`。两段不重叠，或运算等价于拼装且永不进位
五个状态常量	RUNNING=-1<<29、SHUTDOWN=0、STOP、TIDYING、TERMINATED，数值单调递增
`mainLock`	`ReentrantLock`，保护 `workers` 集合、`largestPoolSize`、`completedTaskCount`，也是"提交"与"关闭"的串行化点
`workers`	`HashSet<Worker>`，只在 mainLock 下增删；遍历它做中断是关闭流程的全部依据
`Worker`	继承 AQS 的 Runnable：一个线程 + 一把不可重入的锁 + 一个 firstTask
`completedTasks`	`volatile long`，每跑完一个任务 +1，退出时汇总进 `completedTaskCount`

关键代码：

```
// 一个 int，两段互不重叠的位域
private final AtomicInteger ctl = new AtomicInteger(ctlOf(RUNNING, 0));

private static final int COUNT_BITS = Integer.SIZE - 3;        // 29
private static final int CAPACITY   = (1 << COUNT_BITS) - 1;   // 000 11111...1

// 五种状态占高 3 位。2 位只有 4 个值，装不下 5 个状态，所以必须是 3 位
private static final int RUNNING    = -1 << COUNT_BITS;  // 111 000...
private static final int SHUTDOWN   =  0 << COUNT_BITS;  // 000 000...
private static final int STOP       =  1 << COUNT_BITS;  // 001 000...
private static final int TIDYING    =  2 << COUNT_BITS;  // 010 000...
private static final int TERMINATED =  3 << COUNT_BITS;  // 011 000...

private static int runStateOf(int c)     { return c & ~CAPACITY; } // 只留高 3 位
private static int workerCountOf(int c)  { return c &  CAPACITY; } // 只留低 29 位
private static int ctlOf(int rs, int wc) { return rs | wc; }       // 用 | 不用 +：不进位
```

```
private final class Worker extends AbstractQueuedSynchronizer implements Runnable {
    final Thread thread;              // 真正的载体，由 ThreadFactory 创建
    Runnable firstTask;               // 只为"新线程别先去抢队列"服务，跑完就置 null
    volatile long completedTasks;

    Worker(Runnable firstTask) {
        setState(-1);                 // 先置 -1：此时 tryLock 必失败，谁都中断不了它
        this.firstTask = firstTask;
        this.thread = getThreadFactory().newThread(this);
    }

    protected boolean tryAcquire(int unused) {
        if (compareAndSetState(0, 1)) {          // 只认 0 -> 1，重入直接被拒
            setExclusiveOwnerThread(Thread.currentThread());
            return true;
        }
        return false;
    }
    protected boolean tryRelease(int unused) {
        setExclusiveOwnerThread(null);
        setState(0);                             // 无条件归零，-1 也一并抹掉
        return true;
    }
}

// 加锁范围 = 执行任务的时间，不含 getTask 的阻塞
w.unlock();                           // state -1 -> 0，从这一刻起允许被中断
while (task != null || (task = getTask()) != null) {
    w.lock();                         // 上锁 == 对外宣告"我正在跑任务"
    try { task.run(); }
    finally { task = null; w.completedTasks++; w.unlock(); }
}
```

拆解：

为什么是 29 + 3，以及为什么用 `|` 拼

五个状态至少需要 3 个二进制位（2 位只有 4 个组合），剩下的 29 位留给人数，上限 5.3 亿——远超任何机器的线程承载能力，也远超内核 pid 空间。这个切分的依据不是"省内存"，而是"剩下的位够不够用"：如果状态只有 4 种，Doug Lea 也完全可以切 30 + 2。

拼装用 `|` 而不是 `+` 是个容易看漏的细节。两段位域不重叠时两者结果相同，但 `|` 在语义上表达的是"覆盖"，即便将来 `wc` 用满了 29 位也不会进位污染状态位。同理，取值用掩码 `& ~CAPACITY`（`~CAPACITY` = `0xE0000000`）而不是右移，是因为右移会把 RUNNING 的符号位搞成算术右移的填充问题，而掩码对正负一视同仁。

为什么不用 `AtomicLong` 切成 32 + 32？因为 29 位已经够用（5 亿线程），而 32 位整数的 CAS 是绝大多数架构上单指令就能完成的原子操作；64 位的 `AtomicLong` 在 32 位平台上要靠 `LockCmpxchg8b` 甚至加锁实现，为了一个用不到的容量把最热的路径变慢，不划算。位宽的选择从来是被硬件约束推着走的。

五种状态：为什么 RUNNING 取 -1，为什么能直接比大小

状态机本身很简单：RUNNING 收新任务也处理队列；SHUTDOWN 不收新任务但把队列跑完；STOP 不收也不跑，还要中断正在跑的；TIDYING 是"人已经清空、马上要收尾"的过渡态，此时会调 `terminated()` 钩子；TERMINATED 是钩子执行完的终态。

真正精巧的是取值。RUNNING 定义为 `-1 << 29`，也就是 `0xE0000000`，符号位是 1——任何 RUNNING 状态的 ctl 都是负数。SHUTDOWN 是 0，STOP 及以上是正数。于是：

- `isRunning(c)` 写成 `c < SHUTDOWN`，一句整数比较，不用拆位；
- `runStateAtLeast(c, STOP)` 写成 `c >= STOP`，也不用拆位。

这依赖两个前提：状态常量必须单调递增，且 workerCount 永远非负。后者不是天然成立的——`decrementWorkerCount` 用的是 CAS 循环，一旦有人多减一次，人数变成 -1，低 29 位全 1，`c >= STOP` 会误判，整个状态机就崩了。所以那个 CAS 循环里的断言、以及 `addWorkerFailed` 的补偿路径，都不是防御性编程，是这套位技巧的前置条件。

转换条件只有五条，记住触发点即可：`shutdown()` 推 RUNNING→SHUTDOWN；`shutdownNow()` 推 RUNNING/SHUTDOWN→STOP；SHUTDOWN 下"队列空且人数为 0"→TIDYING；STOP 下"人数为 0"→TIDYING；`terminated()` 返回后 TIDYING→TERMINATED。所有推进都收口在 `tryTerminate()` 一个方法里，而 `tryTerminate()` 在每一个可能让"人数归零"或"队列变空"的地方被调用——这是后一章的主线。

把 ctl 当成一条数轴，这五个状态就变成五个刻度。一个 RUNNING、7 个 worker 的池子，ctl 是 `0xE0000007`；`shutdown()` 之后变成 `0x00000007`——注意人数那 7 位一个都没动，只有高 3 位从 `111` 翻成 `000`，所以状态迁移和人数变化在同一个值里同时可见。再 `shutdownNow()` 变成 `0x20000007`。等 worker 一个个退完，`0x20000000`，进 TIDYING 是 `0x40000000`，`terminated()` 跑完是 `0x60000000`，人数位清零。任何时刻你只要打印 `ctl` 的十六进制，就能一眼读出"什么状态、几个人"，这是这套编码额外的红利，调试线上问题时比 `toString()` 管用。

SHUTDOWN 和 STOP 的分界，不是"收不收新任务"，而是"已经进了队列的任务还算不算数"。这个区别直接决定了一次发布会不会丢任务：`shutdown()` 之后队列里剩下的几千个任务会被现有 worker 全部跑完，`shutdownNow()` 之后它们被 `drainQueue()` 捞出来变成返回值，如果调用方不看返回值、也不重新提交，这些任务就永久消失了。所以两个方法名字只差一个 Now，语义差的是"数据要不要保住"。

为什么必须是一个 AtomicInteger，而不是两个字段

假设拆成 `volatile int runState` 和 `AtomicInteger workerCount`。提交路径要做的是"确认状态允许 + 占一个名额"，这两步要么加锁，要么就会出现前面那个窗口：状态检查通过之后、名额占用之前，池子被关了。而加锁意味着每次 `execute` 都要抢 `mainLock`，线程池最热的路径直接退化成串行。

一个 int 之后，`compareAndIncrementWorkerCount(c)` 的语义变成了"状态仍是这个状态，并且人数成功 +1"，一次 CAS 完成两件事。失败时也带着信息：重新读 ctl，如果 `runStateOf(c) != rs`，说明状态被动过，之前所有的判断全部作废，必须回到外层循环重来；如果状态没变，那只是名额被别人抢了，内层重试即可。`addWorker` 那个两层循环的结构，本质上就是这个"一次 CAS 携带两个维度"的直接产物。

反过来看，两个字段还有一个更隐蔽的代价：读者会看到不存在的状态组合。监控线程读到 `runState = STOP`、再读到 `workerCount = 9`，这个组合在系统里从未出现过——关闭线程是先推状态再逐个中断的，人数在那一刻可能是 7 也可能是 8。用 ctl 之后，`getPoolSize()`、`getActiveCount()`、`isShutdown()` 全部来自同一次 `ctl.get()`，至少互相自洽。

代价是 ctl 只能表达 32 位的信息，所以"哪些 worker"这件事塞不进去，必须另开一个 `HashSet` 用 `mainLock` 保护。这就是这套设计的两层结构：**计数走无锁 CAS，集合走全局锁**。前者是每次提交都要碰的热路径，后者只在创建/销毁线程时碰。

这个分工在调用链上看得最清楚。`execute` 全程只读一次 `ctl.get()`，加上 `addWorker` 里的一次 CAS，就完成了"接不接这个任务"的决定，不碰任何锁；`addWorker` 的第二阶段才拿 `mainLock`，此时已经确定要建线程了，锁竞争的概率极低。而关闭方向刚好相反，`shutdown()` 里 `advanceState(SHUTDOWN)` 是一次 ctl 的 CAS（这一步不需要 mainLock，因为它只改状态位），随后 `interruptIdleWorkers` 必须拿 `mainLock` 才能安全遍历 `workers`——集合在并发增删，不拿锁遍历会直接 `ConcurrentModificationException`。于是"提交"和"关闭"真正串行化的地方只有一处：`addWorker` 拿锁复查状态的那几行。其余全是 CAS 和 volatile 读，这是线程池在 10 万 QPS 下仍然不成为瓶颈的原因。

Worker 继承 AQS 图什么

不是为了互斥——每个 Worker 一把锁，彼此毫不相干，没有共享资源要保护。它是把一个 boolean 标志位做成了带排队语义的锁，而这个标志位的含义是"这个线程此刻正在执行任务"。

三个地方依赖它：

第一，`runWorker` 在 `task.run()` 前后成对 `lock()/unlock()`，锁的范围精确等于一次任务执行，**不包含** `getTask()` 里阻塞在 `workQueue.take()` 的那段时间。于是"持有锁"就等于"在干活"，"不持有锁"就等于"空闲或正在等任务"。

第二，`shutdown()` 里的 `interruptIdleWorkers` 靠 `w.tryLock()` 判断空闲：抢到锁说明这个 worker 没在跑任务，可以安全中断；抢不到就跳过，正在跑的任务不能被打断。整个"优雅关闭"的语义就建立在这一个 tryLock 上。

第三，不可重入是关键。`tryAcquire` 只接受 `0 -> 1`，同一个线程再次 `lock()` 会失败。如果这里用的是 `ReentrantLock`，一个任务里调 `shutdown()` 就会走成：shutdown → interruptIdleWorkers → 对自己的 worker `tryLock()` → 因为是重入所以成功 → `t.interrupt()` 打断自己。不可重入让这条路径自然失效，任务里关自己的池子是安全的。

构造器里的 `setState(-1)` 是同一个思路的延伸：线程对象已经建好但还没 `start()`，这个窗口里 `tryLock` 必须失败，否则关闭流程会去中断一个尚未启动的线程。`runWorker` 开头那句 `w.unlock()` 把 -1 归零，注释写的是 `allow interrupts`——从这里开始，它才是一个可以被关闭流程看见的 worker。

那为什么不用一个 `volatile boolean busy` 呢？因为这里需要的是 test-and-set：`interruptIdleWorkers` 必须原子地完成"判断空闲并占住"，否则它判定为空闲、刚准备 `interrupt()`，线程恰好取到任务开始执行，中断就打在了任务身上。`tryLock()` 的 CAS 把判定和占用合成一步，`unlock()` 再把所有权释放。同时 AQS 的 acquire/release 自带内存语义：`unlock()` 之前的写（`completedTasks++`、任务对共享状态的修改）对下一个 `lock()` 的线程可见，池子不需要额外的屏障。用一把"锁"实现一个布尔标志，换来的是原子性和可见性两样东西。

顺带一提，线程池那几个监控方法几乎全部建立在这把锁上：`getActiveCount()` 遍历 workers 数 `isLocked()` 的个数，`getPoolSize()` 直接 `workers.size()`，`getLargestPoolSize()` 读 `largestPoolSize`（只在 mainLock 下更新）。所以 `getActiveCount()` 数的是"正在 `task.run()` 里"的线程，不含阻塞在 `take()` 上的空闲线程。这解释了一个常见困惑：池子明明被下游 IO 卡死了，`getActiveCount()` 却是满的、CPU 只有 15%——活跃不等于在算，锁只标记"有没有在跑任务"，不区分跑得快不快。

容易误解的地方：

1. **"低 29 位是活着的线程数"**。它是 Worker 对象的数量。`getTask()` 在返回 null 之前就已经 `decrementWorkerCount()` 了，所以存在一个短暂窗口：人数已经减掉，线程还在栈上没退完。`getPoolSize()` 和 jstack 里 `pool-1-thread-N` 的条数对不上是正常的，不是 bug。

2. **"RUNNING 就是 0"**。是 -1。写成 0 的话，`isRunning(c) = c < SHUTDOWN` 永远为 false，池子一创建就是关闭状态。这个 -1 同时承担了"符号位让所有 RUNNING 的 ctl 为负"的职责，换成任何正数都会让直接比较失效。
3. **"Worker 的锁是给多个 worker 互斥用的"**。每个 Worker 是 AQS 的独立实例，state 各管各的。它保护的不是共享数据，而是"我在跑任务"这一个事实。真正保护 `workers` 集合的是 `mainLock`，两者别混。
4. **"`allowCoreThreadTimeOut(true)` 是把 keepAliveTime 调小"**。它不动 keepAliveTime，改的是 `getTask()` 里 `timed = allowCoreThreadTimeOut || wc > corePoolSize` 这个判定，让核心线程从 `take()` 死等变成 `poll(keepAliveTime)` 限时等。同一个 keepAliveTime，语义完全不同。

记住这一句：ctl 不是省内存的位运算杂技，它是把"状态迁移"和"人数变化"压成一次 CAS 的最小临界区。

第9章　线程池：任务从提交到执行的全过程

一句话：execute 只有三条路——加人、入队、拒绝，坑全在"先入队再扩容"这个顺序上。

为什么需要它：一个报表导出接口用了 `Executors.newFixedThreadPool(8)` 。上线三个月相安无事，某天大客户一次点了 2000 个导出，每个任务要扫 30 万行。第二天堆 dump 里 40 万个 `FutureTask` 加 40 万个 `LinkedBlockingQueue$Node` 占着 3.2G，Full GC 两秒一次，最后 `OutOfMemoryError: Java heap space` 。诡异的是现场指标：线程数始终 8，CPU 15%， `getActiveCount()` 恒等于 8， `getQueue().size()` 是 39 万。没有人配错参数——`corePoolSize = maximumPoolSize = 8` 看起来天经地义，问题出在这套参数遇上无界队列之后， `maximumPoolSize` 、 `keepAliveTime` 、拒绝策略三者同时变成了死配置。

这一章走一遍从 `submit` 到 `task.run()` 再到线程退场的完整链路，重点在每一个分叉的判断依据，以及每条异常路径最后落到哪里。

核心结构：

概念	作用
`execute(Runnable)`	唯一入口。 `submit` 只是把任务包成 FutureTask 再调它
`addWorker(task, core)`	真正创建线程。 `core` 只影响上限用 corePoolSize 还是 maximumPoolSize
`Worker.firstTask`	新线程的第一个任务。避免刚建好就去队列抢，省一次上下文切换
`getTask()`	worker 的空转循环。返回 null 就是"这个线程该退出了"
`processWorkerExit`	退场收尾：补记账、移出集合、按需补人、试探终止
`tryTerminate()`	唯一把状态推进到 TIDYING/TERMINATED 的地方
`reject()` → `RejectedExecutionHandler`	四条内置策略：Abort / CallerRuns / Discard / DiscardOldest
`advanceState()`	CAS 推进状态，只许往前走， `shutdown` 与 `shutdownNow` 共用
`interruptIdleWorkers` / `interruptWorkers`	前者靠 `w.tryLock()` 只打空闲的，后者无条件全打
`drainQueue()`	`shutdownNow` 专用，把队列里没跑的任务捞出来返回

关键代码：

```
public void execute(Runnable command) {
    int c = ctl.get();
    // ① 人没招满 core，直接开线程，任务当 firstTask，不经过队列
    if (workerCountOf(c) < corePoolSize) {
        if (addWorker(command, true)) return;
        c = ctl.get();                        // 加人失败（并发抢名额或状态变了），重读再判
    }
    // ② 池子在跑，就往队列里塞。注意：能进到这里说明 core 已经满了
    if (isRunning(c) && workQueue.offer(command)) {
        int recheck = ctl.get();
        if (!isRunning(recheck) && remove(command))   // 入队后发现池子关了，捞回来拒绝
            reject(command);                          // remove 失败说明已被 worker 取走，不拒
        else if (workerCountOf(recheck) == 0)         // 队列有活但一个人都没有：补一个
            addWorker(null, false);
    }
    // ③ 队列满了（或池子已关），才尝试突破 core 往 max 扩
    else if (!addWorker(command, false))
        reject(command);                              // 扩不动了，走拒绝策略
}
```

```
private Runnable getTask() {
    boolean timedOut = false;
    for (;;) {
        int c = ctl.get();
        int rs = runStateOf(c);
        // 出口一：关闭中且（已 STOP 或队列已空）
        if (rs >= SHUTDOWN && (rs >= STOP || workQueue.isEmpty())) {
            decrementWorkerCount();          // 先减人，再退出，顺序不能反
            return null;
        }
        boolean timed = allowCoreThreadTimeOut || workerCountOf(c) > corePoolSize;
        // 出口二：setMaximumPoolSize 调小后，多出来的人自杀
        if (workerCountOf(c) > maximumPoolSize || (timedOut && timed)) {
            if (compareAndDecrementWorkerCount(c)) return null;
            continue;
        }
        try {
            // 出口三在 catch 里：被中断不算退出，重来一轮重新评估 timed
            Runnable r = timed ? workQueue.poll(keepAliveTime, NANOSECONDS)
                               : workQueue.take();
            if (r != null) return r;
            timedOut = true;                 // 这一轮超时了，下一轮直接走出口二
        } catch (InterruptedException retry) {
            timedOut = false;                // 缩容中断 / 关闭中断，都不该直接死
        }
    }
}

// 线程退场后决定要不要补人
if (runStateLessThan(c, STOP)) {
    if (!completedAbruptly) {                // 正常退出才看下限
        int min = allowCoreThreadTimeOut ? 0 : corePoolSize;
        if (min == 0 && !workQueue.isEmpty()) min = 1;   // 有活干就至少留一个
        if (workerCountOf(c) >= min) return;
    }
    addWorker(null, false);                  // 异常退出则无条件补，受 max 限制
}
```

拆解：

三段判断，以及"先入队再扩容"埋的坑

把参数设成 `core=8, max=64, queue=ArrayBlockingQueue(100)`，提交 200 个任务，实际发生的是：前 8 个直接开线程；第 9 到 108 个全部进队列；第 109 个才发现 `offer` 返回 false，此时才 `addWorker(command, false)` 把线程数从 8 往上顶。也就是说，线程数从 8 涨到 64 这件事，被推迟到队列彻底塞满之后才发生。

这个顺序不是随手写的。线程是这里最贵的资源——一个线程默认 1M 栈、一次 `clone()` 系统调用、还要参与内核调度；队列是预先分配好的内存，入队就是一次 CAS。Doug Lea 的假设是：**队列用来吸收正常波动，扩容是用来兜住异常的**。所以先试便宜的手段，再试贵的。

但它带来两个必须知道的后果。

第一个后果：**无界队列让第③段成为死代码**。`LinkedBlockingQueue()` 无参构造的容量是 `Integer.MAX_VALUE`，`offer` 几乎永远返回 true，`addWorker(command, false)` 永远执行不到。于是 `maximumPoolSize`、`keepAliveTime`、拒绝策略三者全部失效——池子的实际并发度被钉死在 `corePoolSize`，队列长度变成了唯一的缓冲，而它没有上限。`newFixedThreadPool(n)` 和 `newSingleThreadExecutor()` 都是这个形态，它们的真名应该叫"n 个线程 + 一个无限长的待办清单"。任务比线程快，清单就无限长，直到堆爆掉。这不是参数配错，是 API 的默认值把"背压"这个机制整个删掉了。

第二个后果：**有界队列下，延迟先恶化，然后才扩容**。直觉上"忙不过来就加人"，实际是"忙不过来就先排队，排到装不下才加人"。队列设 10000、max 设 64 的池子，在突发流量下的表现是：8 个线程跑，10000 个任务排队，第 10009 个任务提交时才想起可以扩到 9 个线程。用户看到的 P99 从 50ms 涨到 8 秒，而 CPU 还有 70% 空闲。想要"快速扩容"的语义，得反过来配：小队列（几十）+ 大 maxPool，让 `offer` 尽早失败，逼出扩容。

第②段里的 `recheck` 是另一个容易读漏的点。入队成功后重新读一次 ctl，是因为 `offer` 和状态检查之间池子可能被关了。此时要把任务捞回来拒绝——但只有 `remove(command)` 成功才拒。如果 remove 失败，说明任务已经被某个 worker 取走开始执行了，这时候再拒绝就是"既执行了又报错"，所以宁可放过。

addWorker：两层 CAS 循环与主锁内的二次校验

`addWorker` 的第一个循环管状态，第二个循环管名额。

外层判断是个拗口的条件：`rs >= SHUTDOWN && !(rs == SHUTDOWN && firstTask == null && !workQueue.isEmpty())` 就返回 false。翻译过来：**STOP 及以上一律不许加人；SHUTDOWN 下只有"没有新任务要带、且队列里还有活"这一种情况允许加人**。这正是 SHUTDOWN 的语义——不收新活，但得有人把存货干完。所以 `shutdown()` 之后队列里还剩 100 个任务、线程却全因超时退光了，会由 `processWorkerExit` 补人，靠的就是这个分支。

内层先比 `wc >= (core ? corePoolSize : maximumPoolSize)`，再 CAS。CAS 的对象是完整的 ctl，所以它同时校验了"状态没变"和"名额 +1 成功"。失败后的分流是关键：`if (runStateOf(c) != rs) continue retry;`——状态被动了，前面所有判断作废，回到外层重新看状态；否则只是名额被抢，内层重试。这个两层结构让"状态稳定时的竞争"只付出一次 CAS 重试的代价，不会每次都重走状态检查。

CAS 成功之后并没有结束。代码要 `new Worker(firstTask)`（一次对象分配 + 一次 ThreadFactory 调用），然后拿 `mainLock`，**在锁里再查一次状态**：`rs < SHUTDOWN || (rs == SHUTDOWN && firstTask == null)`。这次复查是必需的，因为从 CAS 到 `t.start()` 之间隔了一次线程创建，`shutdown()` 完全可能在这期间跑完整个流程。`mainLock` 是提交与关闭的公共串行化点——`shutdown()` 也要拿它才能遍历 `workers` 做中断——所以拿着它复查，就等于把"加人"和"关闭"这两件事排了序。

`t.start()` 放在 `mainLock` 外面，是因为 `ThreadFactory` 是用户代码，可能很慢甚至阻塞，不能拿着全局锁调。

任何一步失败都走 `addWorkerFailed`：从 `workers` 移除、`decrementWorkerCount()`、`tryTerminate()`。三步缺一不可，尤其第三步。如果这次 add 恰好是让 `wc` 从 0 变 1 的那一次，而关闭流程刚刚在 `wc == 0` 的判断上走开了，那么不加这一次 `tryTerminate`，池子就永远停在 SHUTDOWN——这正是第 8 章开头那个"关不掉"的现象。

getTask：一次阻塞读，三个返回 null 的出口

`getTask` 是 worker 的一生。它只有三种结局：拿到任务、超时退出、被状态赶走。

出口一是关闭路径：`rs >= SHUTDOWN && (rs >= STOP || workQueue.isEmpty())`。注意括号——STOP 及以上不看队列，SHUTDOWN 要看队列空不空。这里 `decrementWorkerCount()` 在 `return null` 之前，所以"人数已减、线程未死"的窗口是设计出来的，不是竞态。

出口二是缩容路径：`workerCountOf(c) > maximumPoolSize`。这个分支只有在有人调了 `setMaximumPoolSize` 把上限调小时才会命中，多出来的 worker 通过 CAS 减人后自杀。

出口三是超时路径：`timedOut && timed`。`timedOut` 是上一轮的遗产——上一轮 `poll` 超时返回 null，置为 true，下一轮进循环时立刻命中出口二的条件，减人退出。为什么不在超时那一轮直接退出？因为从 `poll` 返回到再次判断之间，可能刚有任务进来，多给一次机会的成本只是一次循环。

`timed` 每轮重算：`allowCoreThreadTimeOut || wc > corePoolSize`。所以同一个线程，在池子满编时是 `take()` 死等，在池子超编时是 `poll(keepAliveTime)` 限时等——行为随池子规模漂移，这是很多人没意识到的。

`catch (InterruptedException)` 里只置 `timedOut = false` 然后继续循环，不退出。这个选择对应两类中断：`setMaximumPoolSize` 缩容时会中断空闲 worker，让它们重新评估自己是否超编；`shutdown` 时中断后由循环开头的状态检查兜住。如果这里直接 `return null`，缩容就会变成"误杀"，本来该留下的核心线程也被干掉了。

退场之后：processWorkerExit、四种拒绝策略与两种关闭

`runWorker` 的 `finally` 里调 `processWorkerExit(w, completedAbruptly)`。`completedAbruptly` 为 true 表示 `task.run()` 抛出的异常穿透了 while 循环——这种情况下 `getTask` 没来得及减人，所以 `processWorkerExit` 第一件事就是补 `decrementWorkerCount()`。这就是"任务抛异常不会让池子少一个线程"的机制：人减了，但紧接着的补人逻辑又加回来一个。代价是线程 churn，以及异常本身对 `execute` 的提交者完全静默——它只会走到线程的 `UncaughtExceptionHandler`，用 `submit` 提交才会被 `Future.get()` 重新抛出。

补人的条件值得单独看：正常退出时看下限 `min`（`allowCoreThreadTimeOut` 时是 0，否则是 corePoolSize，队列非空时至少 1），人够了就不补；**异常退出时跳过下限判断，无条件补一个**（仍受 max 限制）。所以"任务抛异常"和"任务正常跑完、线程超时退出"走的是两条不同的补人路径。

`tryTerminate` 是终止的唯一入口，条件很严：状态必须 ≥ SHUTDOWN，不能是"SHUTDOWN 但队列还有货"，人数必须为 0。人数不为 0 时它不是直接返回，而是 `interruptIdleWorkers(ONLY_ONE)`——叫醒一个空闲 worker，让它自己走到 `getTask` 的出口一再调一次 `tryTerminate`。这是级联唤醒，避免所有 worker 同时醒来抢锁。理解这一点，才能理解为什么 `addWorkerFailed`、`processWorkerExit`、`shutdown` 三个地方都要调它：任何一次"人数可能归零"的动作，都得负责把终止检查接力下去。

四种拒绝策略的差别在于"任务最后落在谁手上"：

策略	行为	适用
`AbortPolicy`（默认）	抛 `RejectedExecutionException`	宁可失败也不能丢，调用方有能力处理
`CallerRunsPolicy`	池子未关闭时在**提交线程**上 `r.run()`	天然反压：提交者被占住，自然不会继续提交

策略	行为	适用
`DiscardPolicy`	空实现，静默丢弃	可丢的埋点、日志
`DiscardOldestPolicy`	`queue.poll()` 扔掉队头，再 `execute(r)`	只要最新值，如行情推送

`CallerRunsPolicy` 的反压是真的有效——它把"队列满"这个信号直接变成了"提交变慢"。但它也很危险：如果提交者是 Tomcat 的请求线程，你等于把积压从线程池搬到了 Web 容器，最终拖垮的是整个接入层。`DiscardOldestPolicy` 则有个致命细节：它先 `poll()` 再 `execute()`，配 `SynchronousQueue` 时 `poll()` 永远返回 null，`execute()` 又会走到拒绝，于是无限递归，直到 `StackOverflowError`。

最后是两种关闭。`shutdown()` 做三件事：`advanceState(SHUTDOWN)`、`interruptIdleWorkers()`（靠 `w.tryLock()` 只打空闲的）、`tryTerminate()`。`shutdownNow()` 做三件事：`advanceState(STOP)`、`interruptWorkers()`（不看锁，全打）、`drainQueue()` 把没开始的任务捞出来返回。关键差别有两个：`shutdownNow` 清空队列，返回的 `List<Runnable>` 就是那些从未执行的任务，调用方有责任处理；以及它**只是发中断**，任务是否真的停下取决于任务自己有没有检查中断标志——一个死循环里不响应中断的任务，`shutdownNow` 也拿它没办法。

容易误解的地方：

1. **"线程数到了 corePoolSize 就开始往 max 扩"**。不是。是队列满了才扩。无界队列下永远不会扩，`maximumPoolSize` 是个装饰品。这也是 `newFixedThreadPool` 把 core 和 max 设成相等的原因——反正也扩不了，不如写得诚实。
2. **"任务抛异常会让线程池少一个线程"**。不会，`processWorkerExit` 会补一个。但异常被吞掉了：`execute` 提交时没有 Future 承接，只有 `ThreadFactory` 设的 `UncaughtExceptionHandler` 能看到。生产上表现为"任务莫名消失，日志里啥也没有"。
3. **" `shutdownNow` 会停掉正在执行的任务"**。它只是 `interrupt()`，外加清空队列。正在跑的任务如果不响应中断，会一直跑到自然结束；`awaitTermination` 也就一直等不到。
4. **" `DiscardOldestPolicy` 是安全的降级方案"**。它在 `SynchronousQueue` 上会无限递归直到栈溢出，因为 `poll()` 对容量 0 的队列永远返回 null，而 `execute()` 又会再次进入拒绝分支。用之前先看队列类型。

记住这一句：线程池的真实容量不是 `maximumPoolSize`，是"队列能装多少"加上"线程跑得多快"，参数只是这两者的伪装。

第10章　BlockingQueue：七种队列差在哪

一句话：七种队列的差别只有三件事——元素存在哪、用几把锁、装不下时怎么办。

为什么需要它：一条日志采集链路，缓冲区用了 `new ArrayBlockingQueue<>(10000)`，16 个采集线程 `put`，16 个发送线程 `take`。机器 32 核，压测时 CPU 只跑到 25%，吞吐卡在 8 万条/秒上不去，加线程毫无改善。jstack 里一半线程停在 `ArrayBlockingQueue` 内部 AQS 的等待队列上——生产者和消费者在抢同一把锁，锁的临界区里还夹着 `notEmpty.signal()`。换成 `LinkedBlockingQueue`，同样参数跑到 19 万。再换成容量 1024 的 `SynchronousQueue` 配 `CallerRunsPolicy`，吞吐没涨但 P99 从 900ms 降到 40ms，因为积压被挡在了上游。

这三个结果不是调参调出来的，是三种数据结构在不同负载下的必然表现。这一章按"锁的结构"把七个实现分成几组，说清每种在什么形态的流量下会退化。

核心结构：

实现	存储	锁	边界	关键特征
`ArrayBlockingQueue`	`Object[]` 环形数组	1 把锁 + notEmpty/notFull	有界，构造必填	读写互斥；预分配无 GC；可公平
`LinkedBlockingQueue`	单向链表 + dummy head	2 把锁 + 2 个 Condition + AtomicInteger count	默认 `MAX_VALUE`	读写分离；每元素一个 Node
`PriorityBlockingQueue`	`Object[]` 二叉堆	1 把锁 + notEmpty + allocationSpinLock	无界，自动扩容	`put` 不阻塞；出队 O(log n)
`DelayQueue`	内部 `PriorityQueue<Delayed>`	1 把锁 + available + leader 字段	无界	只有到期元素可被取走
`SynchronousQueue`	无存储	无锁，CAS + park	容量 0	必须一对一配对；公平=FIFO 队列，非公平=LIFO 栈
`LinkedTransferQueue`	单一链表，节点分 data/request	无锁，CAS + park	无界	`xfer` 一个方法覆盖四种语义
`LinkedBlockingDeque`	双向链表	1 把锁 + 2 个 Condition	可选	两端都能进出，但共用一把锁

关键代码：

```
// LinkedBlockingQueue：两把锁之间靠 count 的旧值传递"该叫醒谁"
public void put(E e) throws InterruptedException {
    int c = -1;
    Node<E> node = new Node<E>(e);
    final ReentrantLock putLock = this.putLock;
    final AtomicInteger count = this.count;
    putLock.lockInterruptibly();
    try {
        while (count.get() == capacity) notFull.await();  // 满了就在 putLock 的条件上等
        enqueue(node);                                    // last = last.next = node
        c = count.getAndIncrement();                      // c 是入队前的元素数
        if (c + 1 < capacity) notFull.signal();           // 还有空位，再放一个生产者进来
    } finally { putLock.unlock(); }
    if (c == 0) signalNotEmpty();   // 入队前是空的 => 消费者全在睡，跨锁叫醒
}

public E take() throws InterruptedException {
    E x; int c = -1;
    final ReentrantLock takeLock = this.takeLock;
    final AtomicInteger count = this.count;
    takeLock.lockInterruptibly();
    try {
        while (count.get() == 0) notEmpty.await();
        x = dequeue();                                    // 摘掉 dummy head 的后继
        c = count.getAndDecrement();                      // c 是出队前的元素数
        if (c > 1) notEmpty.signal();                     // 还有货，再放一个消费者进来
    } finally { takeLock.unlock(); }
    if (c == capacity) signalNotFull();   // 出队前是满的 => 生产者全在睡
    return x;
}
```

```
// DelayQueue：leader/follower，只让一个人定时等
public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            E first = q.peek();
            if (first == null) { available.await(); continue; }   // 空队列，无限等
            long delay = first.getDelay(NANOSECONDS);
            if (delay <= 0) return q.poll();                      // 到期，直接拿走
            first = null;                                          // 别在 await 期间持有元素引用
            if (leader != null) available.await();                 // 已经有人在计时，我陪跑
            else {
                Thread thisThread = Thread.currentThread();
                leader = thisThread;
                try { available.awaitNanos(delay); }               // 只有 leader 定时醒
                finally { if (leader == thisThread) leader = null; }
            }
        }
    } finally {
        if (leader == null && q.peek() != null) available.signal(); // 交接领导权
        lock.unlock();
    }
}
```

拆解：

一把锁两个条件：Condition 就是两条独立的等待队列

`ArrayBlockingQueue` 的骨架是四个字段：`Object[] items`、`takeIndex`、`putIndex`、`count`。数组首尾相接成环，`enqueue` 里 `if (++putIndex == items.length) putIndex = 0`，`dequeue` 同理。容量固定，所以不需要扩容逻辑，也不会有 Node 分配。

并发控制是一把 `ReentrantLock` 加两个 `Condition`。这是理解所有阻塞队列的起点：`Condition.await()` 做三件事——释放锁、把当前线程挂到这条 condition 自己的等待队列上、park；`signal()` 把该 condition 队列里的头节点搬到 AQS 的同步队列，等它重新抢到锁才能返回。所以"生产者消费者"这个模型，在实现层面就是**一把锁 + 两条等待队列**：`notFull` 上排着生产者，`notEmpty` 上排着消费者，锁保证同一时刻只有一个人在改数组。

`put` 的循环是 `while (count == items.length) notFull.await()`。用 while 不用 if，是因为被唤醒不等于条件成立——可能被别的生产者抢先填满了。`enqueue` 之后紧接着 `notEmpty.signal()`，在同一次持锁内完成：单锁模型下这是最自然的写法，不需要跨锁。

代价也在"单锁"上。`put` 持锁期间 `take` 拿不到锁，反之亦然。临界区里还有一次 `signal()`，而 `signal` 要把节点从 condition 队列搬到同步队列，是一串指针操作。32 核机器上 16 生产 16 消费，全部串行通过这一个临界区，CPU 自然跑不满。这不是实现得不好，是"用一把锁保证数组下标一致"的固有成本。

四个写入方法的语义顺便对齐一下：`add` 满了抛 `IllegalStateException`，`offer` 满了返回 false（另有带超时的重载），`put` 满了阻塞。读取侧同理，`remove()` 空则抛异常，`poll()` 空则返回 null，`take()` 空则阻塞。批量场景优先用 `drainTo`——它一次拿锁搬走 N 个元素，比循环 `take` 少了 N-1 次加解锁，在消费端做批量落库时差别很明显。

两把锁一个计数：把读写拆开需要付什么

`LinkedBlockingQueue` 的核心观察是：生产者只碰 `last`，消费者只碰 `head`，两个指针不重叠。既然不重叠，就可以各用一把锁。唯一被双方共享的可变状态是 `count`，所以它必须是 `AtomicInteger`。

难点不在加锁，在**唤醒**。生产者入队后要叫醒消费者，但消费者睡在 `notEmpty` 上，而 `notEmpty` 属于 `takeLock`——`signal()` 要求持有对应的锁，否则抛 `IllegalMonitorStateException`。于是必须写成"出 `putLock` 之后再拿 `takeLock` 去 signal"，这就是 `signalNotEmpty()` 这个辅助方法存在的原因。

那什么时候该 signal？如果每次入队都 signal，就是空唤醒；如果判断错了，就是丢唤醒。这里用 `count` 的**旧值**做判据：

- `put` 拿到 `c = count.getAndIncrement()`，`c == 0` 说明入队前队列是空的，此前不可能有消费者拿到元素，它们必然都睡在 `notEmpty` 上——所以出锁后 signal 一次。
- `take` 拿到 `c = count.getAndDecrement()`，`c == capacity` 说明出队前队列是满的，生产者必然都睡在 `notFull` 上——所以 signal 一次。

为什么这样不会丢唤醒？因为消费者醒来后第一件事是 `while (count.get() == 0) notEmpty.await()`，它检查的是 `count` 这个原子量，不是某个标志位。就算生产者的 signal 来晚了、甚至没来，只要 `count > 0`，消费者要么不会进入 await，要么在下一轮循环里看到 `count > 0` 直接往下走。**正确性靠 while 里的条件判断，signal 只是减少无谓的 park**。这是整个 JUC 里反复出现的分工，值得单独记一笔。

代价有三样。一是每个元素一个 `Node` 对象（对象头 + item + next 引用，压缩指针下 24 字节），队列长的时候全是 GC 压力；二是 `size()` 读的是 `AtomicInteger`，并发下只是近似值；三是两把锁意味着不存在全局 FIFO——一个元素入队和一另一个元素出队可以同时发生，跨线程的先后顺序不成立。还有那个默认容量 `Integer.MAX_VALUE`，就是第 9 章 OOM 的直接原因。

SynchronousQueue：容量为 0，公平是队列，非公平是栈

`SynchronousQueue` 没有存储。`put(e)` 会一直阻塞到有消费者把 `e` 拿走，`take()` 会一直阻塞到有生产者送来元素。数据从一只手直接交到另一只手，中间没有任何缓冲区。

实现上只有一个抽象方法 `transfer(e, timed, nanos)`：`e != null` 表示"我是生产者，把这个交出去"，`e == null` 表示"我是消费者，来取一个"。构造时二选一：公平模式用 `TransferQueue`（FIFO 链表），非公平模式用 `TransferStack`（LIFO 栈）。

两者的差别在"和谁配对"。FIFO 模式下，新来的消费者排到队尾，匹配的是 `head.next`，也就是**等得最久**的那个生产者——严格先来后到。LIFO 模式下，新来的消费者直接和栈顶匹配，栈顶是**刚刚到达**的生产者，于是后来者插队拿走了元素，早就在等的那个继续睡。这叫 barging，吞吐更高（栈顶节点大概率还在 CPU cache 里，且省掉遍历），但理论上可以饿死。

`transfer` 里最值得看的是节点状态编码。`QNode` 的 `item` 字段身兼三职：等于 `e` 表示还在等；被 CAS 成对方的值表示配对成功；等于节点自己（`x == m`）表示已被取消。判断配对是否有效的条件是 `isData == (x != null) || x == m || !m.casItem(x, e)` 三者任一成立就重试——第一项检查对方是不是已经被别人配走了，第二项检查对方是不是已取消，第三项才是真正的 CAS 抢占。

`put` 和 `offer` 的区别就一行：`put` 用 `transfer(e, false, 0)`，无超时，返回 null 只能是中断；`offer(e)` 用 `transfer(e, true, 0)`，超时为 0，没有消费者立刻返回 false。**这个"立刻返回 false"就是 `newCachedThreadPool` 的全部机制**：`execute` 里 `workQueue.offer(command)` 返回 false，流程走到 `addWorker(command, false)` 建新线程。所谓"来一个任务开一个线程"，不是缓存池的策略选择，是容量 0 的队列逼出来的必然结果。

还有一个容易被忽略的优化：入队之后不是直接 park，而是先自旋。`awaitFulfill` 里 `spins` 的初值是 `maxUntimedSpins`（512）或 `maxTimedSpins`（32），因为配对往往在几纳秒内就完成了，自旋比 park/unpark 的内核往返快一个数量级；只有自旋耗尽才 `LockSupport.park`。这也是"短临界区用自旋、长等待用 park"这个原则在 JUC 里的标准落地方式。

剩下的三种：堆、双端链表、以及"统一节点"

`PriorityBlockingQueue` 是 `PriorityQueue` 的线程安全版，底层 `Object[]` 二叉堆：`a[i]` 的左子是 `2i+1`、右子是 `2i+2`、父是 `(i-1)>>>1`，根永远最小。`put` 直接转发给 `offer`，因为无界所以永不阻塞；入队是 `siftUp`（和父节点比，小了就上浮），出队是取 `array[0]`、把队尾元素搬到根、再 `siftDown`（和较小的子节点交换，下沉）。扩容 `tryGrow` 有一手漂亮的并发处理：先 `lock.unlock()` 释放主锁，用 CAS 抢 `allocationSpinLock` 分配新数组，分配完再 `lock()` 回来做 `System.arraycopy`。这样在分配几个 MB 新数组的那段时间里，读线程还能继续 `take`。代价是遍历无序（`iterator` 只保证遍历全部元素，不保证顺序）、同优先级元素之间不保证 FIFO、`put` 永远不阻塞所以没有任何背压。

`DelayQueue` 内部是一个按到期时间排序的 `PriorityQueue<Delayed>`，只有 `getDelay()` 返回 ≤ 0 的元素才能被取走。它的核心是 leader/follower：如果每个等待线程都用 `awaitNanos(剩余延迟)` 定时等，那么每有一个元素到期，所有线程都会在几乎同一时刻醒来抢锁，抢输的重新算一次延迟再睡——这就是惊群。leader 模式让**只有一个线程**做定时等待，其余在 `available.await()` 上无限睡；leader 取走元素后在 `finally` 里检查 `if (leader == null && q.peek() != null) available.signal()`，把领导权交给下一个。还有一处细节：算出 `delay > 0` 之后立刻 `first = null`，避免在 `awaitNanos` 期间还持有队头元素的强引用，否则一个长延迟的元素会被所有等待线程间接引用住。

`LinkedBlockingDeque` 是双向链表 + 一把锁 + `notEmpty` / `notFull`。四个入口 `linkFirst` / `linkLast` / `unlinkFirst` / `unlinkLast`，但全部共用同一把 `ReentrantLock`。所以 `putFirst` 和 `takeLast` 之间也是互斥的——它适合"两端都会用但并发不高"的场景（比如本地任务队列，正常从头部取，出问题时从尾部降级取），不适合把两端当两条独立通道用。

`LinkedTransferQueue` 是把前两者的优点合起来：一个链表，节点要么是 data（生产者留下的元素），要么是 request（消费者留下的空请求），靠 `isData` 区分。所有操作都收敛到一个 `xfer(e, haveData, how, nanos)`，`how` 有 `NOW` / `ASYNC` / `SYNC` / `TIMED` 四档：`put` / `offer` 用 ASYNC，永不阻塞，没有消费者就把元素留在链表里（这一点和 `SynchronousQueue` 相反）；`take` / `transfer` 用 SYNC，必须等到配对；`poll()` 用 NOW，立刻返回。匹配成功后它不急着摘链，而是把 `item` 指向自己标记为已匹配，靠 `forgetNext` / `forgetItem` 在后续操作中顺手清理——用延迟清理换掉每次出队的 CAS，这是它比 `SynchronousQueue` 更适合高吞吐 transfer 场景的原因。

容易误解的地方：

1. **"`LinkedBlockingQueue` 就是无界队列"**。无参构造是 `Integer.MAX_VALUE`，但它有 `LinkedBlockingQueue(int capacity)`。`newFixedThreadPool` 用的是无参版本，所以才是无界；自己写 `new LinkedBlockingQueue<>(2000)` 就是有界的，`maximumPoolSize` 立刻恢复意义。
2. **"两把锁意味着读写完全并行"**。只在队列既不空也不满时成立。队列空时消费者全部 park 在 `notEmpty` 上等生产者 signal，队列满时反过来，此时仍然是串行的；而且 `count` 的原子操作是双方共享的缓存行，高并发下本身就是热点。
3. **"`SynchronousQueue` 公平模式更好，因为公平"**。公平模式维护严格的 FIFO 队列，吞吐明显低于非公平的栈；`newCachedThreadPool` 用非公平模式是有意的。公平解决的是饥饿，不是性能。

4. **"`PriorityBlockingQueue.put` 会像 `ArrayBlockingQueue` 一样阻塞"**。它无界，`put` 就是 `offer`，永不阻塞。所以拿它当缓冲区时没有任何背压，生产多快就往堆里塞多快，直到 OOM——这是"无界队列 = 没有流控"的又一个实例。

记住这一句：选队列先问三句——要不要容量、读写要不要并行、元素要不要排序，答完七种里只剩一两种。

第11章　RateLimiter：令牌桶的两种实现

一句话：RateLimiter 不发令牌，它只维护一张"下一张票何时可用"的时间表。

为什么需要它：

先看一个线上事故的典型形状。给下游接口限流 100 QPS，代码写成 `new Semaphore(100)`。压测没问题，因为压测是匀速发请求；上线后晚高峰，100 个线程在 3 毫秒内全部拿到许可、全部把请求打出去，然后集体等一秒，再来一轮。下游看到的不是 100 QPS，而是"每秒一个 100 并发的小尖峰"，对方的令牌桶直接被削顶，返回 429。你把 Semaphore 调成 10，尖峰变成 10 并发，但一秒里能打十几轮，总量还是超。

问题在于 Semaphore 限制的是**同时在场的线程数**，不是**单位时间通过的请求数**。这两件事只在"每个请求耗时恰好等于 1/rate"时才近似相等，一旦下游变慢或者请求本身很快，就完全脱钩。

另一种常见写法是 `Thread.sleep(1000 / qps)`。这个能控制速率，但有两个毛病：sleep 的时间不含业务耗时，实际速率永远低于设定值；它也没有"欠账"概念，一次 200ms 的 GC 停顿之后，积压的请求会以全速冲出去，正好制造一个尖峰。

这一章要解决的是：怎么用**一个 long 加一个 double**，在没有后台线程、没有定时器的前提下，精确描述"第 n 个请求最早什么时候能通过"；怎么让空闲期攒下的额度被突发流量用掉，同时又不能让冷启动的系统被突发流量打死。

核心结构：

概念	作用
`RateLimiter`	抽象基类，只有 `SleepingStopwatch`（相对计时）和 `mutex`（synchronized 的锁对象）
`SmoothRateLimiter`	抽象子类，持有 `storedPermits`、`maxPermits`、`stableIntervalMicros`、`nextFreeTicketMicros` 四个状态
`SmoothBursty`	`maxBurstSeconds = 1.0`，存量令牌免费取，允许突发
`SmoothWarmingUp`	存量令牌按梯形面积收费，越冷越贵，用于预热
`storedPermits`	当前存量，double，因为令牌是按时间连续产生的，不是整数个
`maxPermits`	存量上限，超出部分直接蒸发，不是排队
`stableIntervalMicros`	稳定期生产一个令牌的间隔，微秒，等于 1e6 / permitsPerSecond
`nextFreeTicketMicros`	下一张票的最早可用时刻，相对构造时刻的微秒数，本质是一本欠账账本
`reserveEarliestAvailable`	唯一的状态机：结算存量、算欠账、推进账本、返回旧账本
`storedPermitsToWaitTime`	两种实现的唯一行为差异点：存量收不收费
`coolDownIntervalMicros`	存量补充速率，Bursty 是 stableInterval，WarmingUp 是 warmupPeriod / maxPermits

关键代码：

整个类只有一个函数真正改变状态，其余都是它的包装。

```
final long reserveEarliestAvailable(int requiredPermits, long nowMicros) {
    resync(nowMicros);                     // 先把"时间流逝产生的令牌"结算进 storedPermits

    long returnValue = nextFreeTicketMicros; // 关键：先记下旧账本，这是本次要等的时刻
    double storedPermitsToSpend = min(requiredPermits, this.storedPermits);
    double freshPermits = requiredPermits - storedPermitsToSpend;

    // 存量部分可能要收费（WarmingUp），新产生的部分按稳定间隔收费
    long waitMicros = storedPermitsToWaitTime(this.storedPermits, storedPermitsToSpend)
                    + (long) (freshPermits * stableIntervalMicros);

    this.nextFreeTicketMicros = LongMath.saturatedAdd(nextFreeTicketMicros, waitMicros);
    this.storedPermits -= storedPermitsToSpend;
    return returnValue;                    // 返回旧值，不是推进后的值
}
```

`resync` 是"惰性生产"的全部秘密：

```
void resync(long nowMicros) {
    if (nowMicros > nextFreeTicketMicros) {          // 只有账本已经过期才需要补
        double newPermits = (nowMicros - nextFreeTicketMicros) / coolDownIntervalMicros();
        storedPermits = min(maxPermits, storedPermits + newPermits);  // 封顶，多出来的蒸发
        nextFreeTicketMicros = nowMicros;            // 账本对齐到当下，欠账清零
    }
}
```

两种实现的分歧只在 `storedPermitsToWaitTime` 一个方法上：

```
// SmoothBursty：存量是白送的
long storedPermitsToWaitTime(double storedPermits, double permitsToTake) {
    return 0L;
}

// SmoothWarmingUp：存量要按梯形面积收费
long storedPermitsToWaitTime(double storedPermits, double permitsToTake) {
    double availablePermitsAboveThreshold = storedPermits - thresholdPermits;
    long micros = 0;
    if (availablePermitsAboveThreshold > 0.0) {
        double permitsAboveThresholdToTake = min(availablePermitsAboveThreshold, permitsToTake);
        // 梯形面积 =（上底 + 下底）* 高 / 2，上下底是"在这个位置取一个令牌要多久"
        double length = permitsToTime(availablePermitsAboveThreshold)
                      + permitsToTime(availablePermitsAboveThreshold - permitsAboveThresholdToTake);
        micros = (long) (permitsAboveThresholdToTake * length / 2.0);
        permitsToTake -= permitsAboveThresholdToTake;
    }
    micros += (long) (stableIntervalMicros * permitsToTake);   // 阈值以下按稳定价
    return micros;
}
```

拆解：

一、令牌桶和漏桶的区别，不在"存什么"，在"输出平不平滑"

漏桶的模型是：请求先进队列（桶），桶底有个洞，以恒定速率 r 往外漏。桶的作用是**缓冲**，输出永远是匀速的，桶满了就丢弃。这个形状决定了它天然不允许突发——不管进来多猛，出去都是 r 个每秒。

令牌桶反过来：系统以恒定速率 r 往桶里放令牌，桶容量 b。请求来了先拿令牌，拿不到就拒绝或等待。关键在于**桶里可以攒令牌**：空闲 10 秒，桶里就攒了 10r 个（封顶 b），这时突然来 10r 个请求，可以一次性全部放行。所以令牌桶的输出可以是突发的，长期平均速率仍然是 r。

一句话：漏桶做整形（shaping），令牌桶做限速（policing）但允许 burst。

Guava 的类注释写的是 "based on the token bucket algorithm"，但它继承的抽象类叫 SmoothRateLimiter，而且它**没有维护任何令牌队列**。它把"桶里有多少令牌"抽象成 `storedPermits` 这个 double，把"下一张票什么时候能拿到"抽象成 `nextFreeTicketMicros`。这两个变量加当前时间，就足以回答"我现在能不能过"。

为什么是 double 不是 int：令牌按时间连续产生，1 秒 5 个意味着每 200ms 产生 1 个。请求在 100ms 时到达，理论上桶里有 0.5 个令牌。用 double 才能不丢这"半个"，否则速率会被系统性低估。很多人读源码时看到 `storedPermits` 是 double 觉得奇怪，原因就在这里。

关于"突发"还有一个必须澄清的点，源码注释专门写过：`acquire(100)` 在 rate=5 时也能立刻返回，但这不是突发，这是**预支**——你把未来 20 秒的额度提前用掉了，代价是接下来的请求要等 20 秒。真正的突发是"桶里攒了 100 个令牌，你一次取走"，取的是过去，不是未来。这两件事在代码里的表现完全一样（都是立刻返回），但账本上的方向相反。

二、两个字段的推演：一次 acquire 到底改了什么

设 permitsPerSecond = 5，则 `stableIntervalMicros = 200_000`，`maxBurstSeconds = 1.0`，`maxPermits = 5`。构造完成时 `storedPermits = 0`、`nextFreeTicketMicros = 0`（SmoothBursty 的初始状态是"热"的，没有存货）。

调用链是 `acquire → reserve → reserveAndGetWaitLength → reserveEarliestAvailable → resync`，`reserve` 里的 synchronized 只包住计算，sleep 在锁外。

场景 A：创建后立刻 acquire(1)，t = 0。
`resync(0)`：`0 > 0` 不成立，不补。`returnValue = 0`。`storedPermitsToSpend = min(1, 0) = 0`，`fresh = 1`，`wait = 200_000`。`nextFreeTicketMicros = 200_000`。返回 0，等待 `max(0 - 0, 0) = 0`。**立刻通过，但账本已经推到 200ms 后。**

这是"预支"最纯粹的样子：第一个请求花掉的是未来的一个令牌。

场景 B：紧接着再 acquire(1)，t 仍为 0。
`resync` 不生效（0 不大于 200_000）。`returnValue = 200_000`，`fresh = 1`，`wait = 200_000`，账本推到 400_000。本次等待 `200_000 − 0 = 200ms`。

场景 C：空闲 10 秒后 acquire(1)，t = 10_000_000。
`resync`：`10_000_000 > 400_000` 成立，`newPermits = (10_000_000 − 400_000) / 200_000 = 48`，`storedPermits = min(5, 0 + 48) = 5`，账本对齐到 10_000_000。随后 `returnValue = 10_000_000`，`storedPermitsToSpend = 1`，`fresh = 0`，`wait = 0`，账本不动，`storedPermits = 4`。等待 0。

注意 `min(maxPermits, ...)` 这一刀：空闲 10 秒产生了 48 个令牌，桶只装得下 5 个，多出来的 43 个**直接蒸发**。这不是 bug，是限流器的必要行为——否则空闲一小时的系统会攒下 18000 个令牌，一次突发就能把下游打死。

场景 D：`storedPermits = 5` 时 acquire(8)。
`returnValue = now`，`storedPermitsToSpend = 5`，`fresh = 3`，`wait = 600_000`，账本推到 now + 600ms，`storedPermits = 0`。本次等待 0，**但后面 600ms 内的请求全部要排队**。

四个场景合起来说明一件事：`acquire` 返回的等待时长，永远是"我还上一笔账"的时间，而不是"我为这次请求付的钱"。本次超额的部分被记在 `nextFreeTicketMicros` 上，由**下一个**调用者承担。

顺带说 `setRate`。它不是只改速率：`doSetRate` 会把 `storedPermits` 按 `storedPermits * maxPermits / oldMaxPermits` 等比例缩放，否则调低速率后存量会超出新上限，账本和桶对不上。

三、为什么是"预支"，以及 acquire 与 tryAcquire 的分工

预支不是设计品味，是结构逼出来的。RateLimiter 的全部状态就是那两个数，它没有后台线程，不可能"每秒醒来一次往桶里加令牌"。所有令牌的生产都发生在 `resync`，而 `resync` 只在 `acquire`、`tryAcquire`、`setRate` 被调用时执行。结论很硬：**限流器只能在被调用时结算**。

如果不用预支，改成"本次要多少就等多久"，那 `acquire(1000)` 会阻塞 200 秒。虽然 sleep 不在锁内，不影响别人算账，但一个批量调用把线程挂死 200 秒，语义上不可接受。预支把这个代价摊给了后来者，让单次调用最多只等"之前欠下的账"。

代价是**无法拒绝**。`acquire(Integer.MAX_VALUE)` 会把账本推到几百年后，之后所有请求全部阻塞，而且没有任何机制能把它拉回来。生产上要么用 `tryAcquire`，要么在外面包一层信号量兜底。

两者的语义差别在源码里很直白：

```
public double acquire(int permits) {
    long microsToWait = reserve(permits);              // 无条件记账
    stopwatch.sleepMicrosUninterruptibly(microsToWait);
    return 1.0 * microsToWait / SECONDS.toMicros(1L);
}

public boolean tryAcquire(int permits, long timeout, TimeUnit unit) {
    long timeoutMicros = max(unit.toMicros(timeout), 0);
    long microsToWait;
    synchronized (mutex()) {
        long nowMicros = stopwatch.readMicros();
        if (!canAcquire(nowMicros, timeoutMicros)) {    // 先问"排得上吗"
            return false;                               // 排不上直接走，状态一点不动
        } else {
            microsToWait = reserveAndGetWaitLength(permits, nowMicros);
        }
    }
    stopwatch.sleepMicrosUninterruptibly(microsToWait);
    return true;
}

private boolean canAcquire(long nowMicros, long timeoutMicros) {
    return queryEarliestAvailable(nowMicros) - timeoutMicros <= nowMicros;
}
```

三条差异：

第一，acquire 无条件推进账本；tryAcquire 先检查"最早可用时刻减去 timeout 是否已经小于等于现在"，不满足直接 false，**账本不动**。所以 tryAcquire 失败不会污染后续请求。

第二，acquire 的等待没有上限；tryAcquire 的等待有上限，超时就是"没拿到"，不是"拿到了但晚了"。

第三，`tryAcquire()` 无参等价于 `tryAcquire(1, 0, NANOSECONDS)`，timeout 为 0，只能通过"账本已经过期"的检查，是一个纯非阻塞的 poll。

还有一个必须点出来的坑：`canAcquire` 检查的是**继承来的欠账**，不包含本次请求自己要花的钱。所以 `tryAcquire(1000, 1, SECONDS)` 在 rate=5 时会立刻返回 true——因为账本当前是空的，1 秒的 timeout 绰绰有余；然后它把账本推到 200 秒后，拍拍屁股走了。timeout 保护不了你，它只保护"排队"，不保护"预支"。

四、预热：为什么存量反而要收费

`SmoothBursty` 里 `storedPermitsToWaitTime` 返回 0，意味着系统空闲越久，桶里令牌越多，突发来了放行越爽快。这对无状态服务没问题，但对"资源需要预热"的服务是灾难：数据库连接池冷的时候，你放 1000 QPS 进来，连接建不过来，全部超时，然后雪崩。

`SmoothWarmingUp` 把逻辑反过来：**存量越多，说明系统越闲、越冷，取令牌越贵**。贵多少由一条斜线决定——`storedPermits` 在 `[0, thresholdPermits]` 区间时单价是 `stableInterval`，在 `[thresholdPermits, maxPermits]` 区间时从 `stableInterval` 线性涨到 `coldInterval = 3 * stableInterval`（coldFactor 写死为 3，用户改不了）。

参数怎么来的：`warmupPeriod` 是构造时指定的"从最冷到最热要多久"，图中梯形面积就是 warmupPeriod（横轴是令牌数、纵轴是每个令牌的时间，面积就是时间）。coldFactor 固定 3，梯形面积恰好是下面矩形面积的 2 倍，于是：

```
warmupPeriod = 2 * stableInterval * thresholdPermits
             → thresholdPermits = 0.5 * warmupPeriod / stableInterval

warmupPeriod = 0.5 * (stableInterval + coldInterval) * (maxPermits - thresholdPermits)
             → maxPermits = thresholdPermits + 2 * warmupPeriod / (stableInterval + coldInterval)
```

举个数：rate = 10，warmupPeriod = 10s。`stableInterval = 100ms`，`coldInterval = 300ms`，`thresholdPermits = 0.5 * 10 / 0.1 = 50`，`maxPermits = 50 + 2 * 10 / 0.4 = 100`。斜率 `slope = (300 - 100) / (100 - 50) = 4ms`，也就是超过阈值后，每多一个存量令牌，取一个令牌就多花 4ms。

补充速率也跟着变：`coolDownIntervalMicros() = warmupPeriodMicros / maxPermits`，保证从 0 攒到 maxPermits 恰好花 warmupPeriod。SmoothBursty 是 `stableIntervalMicros`，保证攒满 maxPermits（正好一秒的量）花一秒。初始状态也不同：Bursty 的 `storedPermits = 0`，WarmingUp 的 `storedPermits = maxPermits`（一开始最冷）。这个差异就在 `doSetRate` 的 `oldMaxPermits == 0.0` 分支里。

Sentinel 走的是另一条路。Guava 控制的是"取令牌要花多久"，Sentinel 控制的是"当前允许的 QPS 上限是多少"。`RateLimiterController` 只记一个 `latestPassedTime`，每次请求算 `expectedTime = latestPassedTime + 1000/count`，没到就 sleep（最多 maxQueueingTimeMs），这是"排队等待"模式，本质是一个匀速放行的漏桶，连桶都没有。`WarmUpController` 维护 `storedTokens`，令牌越多说明系统越冷，用 `1 / (aboveToken * slope + 1/count)` 算出当前允许的 QPS，直接和滑动窗口统计的 passQps 比。区别归结成一句：Guava 让你等，Sentinel 直接拒（除非你配了排队）。

容易误解的地方：

1. 以为 `SmoothBursty` 的"突发"是 `acquire(大数)` 一次拿很多。不是，那是预支未来，会让后续请求排队；突发指的是空闲期攒在 `storedPermits` 里的存量被一次取走，取的是过去。
2. 以为 `acquire` 返回的秒数是"这次请求的额度成本"。它返回的是本次实际睡眠时长，也就是上一笔欠账；本次自己的成本记在 `nextFreeTicketMicros` 上，由下一个调用者支付。
3. 以为 `tryAcquire(timeout)` 的 timeout 能兜住本次请求的全部成本。不能，它只检查继承的欠账是否在 timeout 内；`tryAcquire(1000, 1, SECONDS)` 在 rate=5 时立刻返回 true，然后把账本推到 200 秒后。
4. 以为 RateLimiter 有后台线程定时补令牌。没有，只有 `resync` 在被调用时结算；空闲再久，`storedPermits` 也不会超过 `maxPermits`，多出来的令牌直接蒸发。

记住这一句：RateLimiter 不生产令牌，它只记账——把这次多拿的记到 nextFreeTicketMicros 上，让下一个调用者来还。

第12章　HashMap：从 JDK 7 到 8 的那次改造

一句话：JDK 8 用尾插、高低位拆分和红黑树，把成环和长链表两个老问题一起收了口。

为什么需要它：

先看两个现场。

第一个是 2011 年前后淘宝工程师复现的那个：线上应用 CPU 持续 100%，jstack 打出来所有业务线程都停在 `HashMap.transfer` 的 `e = e.next` 上，堆 dump 出来 `table[3]` 的链表是个环——`A.next = B`，`B.next = A`。那个 HashMap 是个 static 字段，本意是本地缓存，多个线程同时 put。这就是"HashMap 死循环"，JDK 7 时代最著名的生产事故之一。

第二个现场更常见：一个接口的参数校验，把 200 个枚举值塞进 HashMap，key 是 `"1"`、`"2"`……`"200"`。上线后发现这个 map 的 get 偶尔要几百微秒。原因是 `"1".hashCode()` 到 `"200".hashCode()` 的低位分布极不均匀（连续整数的 hashCode 就是它本身），而 HashMap 只取低位做索引，200 个 key 挤在十几个桶里，最长的链表挂了三十多个节点。

这两个现场对应这一章要解决的两个问题：**并发下的结构破坏**，和**hash 分布不均时的查找退化**。前者 JDK 8 用尾插加高低位拆分把最恶劣的后果（死循环）消掉了，后者用红黑树把 O(n) 压到 O(log n)。

核心结构：

概念	作用
`table`	`Node[]`，长度恒为 2 的幂，第一次 put 时懒初始化
`Node` / `Entry`	`hash`、`key`、`value`、`next` 四个字段，链表节点
`TreeNode`	红黑树节点，多 `parent/left/right/prev/red`，约占 Node 两倍空间
`threshold`	扩容阈值 = `capacity * loadFactor`；JDK 8 里也临时用来存初始容量
`loadFactor`	默认 0.75，时间与空间的折中，也是泊松分布 λ≈0.5 的来源
`hash(key)`	扰动函数 `h ^ (h >>> 16)`，把高位折到低位
`(n - 1) & hash`	n 是 2 的幂时等价于取模，只取 hash 的低 k 位
`TREEIFY_THRESHOLD = 8`	往已有 8 个节点的桶里再加一个，就尝试树化
`UNTREEIFY_THRESHOLD = 6`	扩容拆分后节点数不超过 6，退化为链表
`MIN_TREEIFY_CAPACITY = 64`	表长不足 64 时优先扩容，而不是树化
`resize()`	初始化与扩容共用，按 `hash & oldCap` 把链表拆成 lo/hi 两条

关键代码：

先看扰动函数。它只有一行，但决定了索引质量。

```
static final int hash(Object key) {
    int h;
    // 高 16 位异或到低 16 位：让高位也参与 (n-1) & hash 的运算
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

再看扩容。JDK 8 的 rehash 优化全在这段里。

```
final Node<K,V>[] resize() {
    // ... 计算 newCap、newThr，创建 newTab ...
    if (oldTab != null) {
        for (int j = 0; j < oldCap; ++j) {
            Node<K,V> e;
            if ((e = oldTab[j]) != null) {
                oldTab[j] = null;
                if (e.next == null)
                    newTab[e.hash & (newCap - 1)] = e;          // 单节点，直接算新位置
                else if (e instanceof TreeNode)
                    ((TreeNode<K,V>)e).split(this, newTab, j, oldCap);
                else {
                    Node<K,V> loHead = null, loTail = null, hiHead = null, hiTail = null, next;
                    do {
                        next = e.next;
                        // 只看 oldCap 这一位：0 留在原位，1 挪到 j + oldCap
                        if ((e.hash & oldCap) == 0) {
                            if (loTail == null) loHead = e; else loTail.next = e;
                            loTail = e;                          // 尾插，保持相对顺序
                        } else {
                            if (hiTail == null) hiHead = e; else hiTail.next = e;
                            hiTail = e;
                        }
                    } while ((e = next) != null);
                    if (loTail != null) { loTail.next = null; newTab[j] = loHead; }
                    if (hiTail != null) { hiTail.next = null; newTab[j + oldCap] = hiHead; }
                }
            }
        }
    }
    return newTab;
}
```

拆解：

一、数组加链表：2 的幂是这个结构的地基

HashMap 的骨架是 `Node[] table`，每个槽挂一条单向链表。这个结构要成立，前提是"用 hash 算下标"必须足够快。取模 `hash % n` 是一次除法，在早期 CPU 上要十几个周期；`hash & (n-1)` 是一条与指令，一个周期。两者等价的条件是 n 为 2 的幂：n = 2^k 时，`hash % n` 的结果就是 hash 的低 k 位，而 n-1 恰好是低 k 位全 1。

所以 HashMap 强制容量是 2 的幂。`new HashMap(20)` 不会给你 20 的容量，会向上取到 32。JDK 7 用 `roundUpToPowerOf2`，JDK 8 用 `tableSizeFor`，做法都是把 cap-1 不断右移或，把最高位后面全部填 1，最后加一。

```
static final int tableSizeFor(int cap) {
    int n = cap - 1;          // 减一是为了处理 cap 本身就是 2 的幂的情况
    n |= n >>> 1;
    n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;            // 32 位最多传播 5 轮
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}
```

这个约束贯穿全篇：索引计算靠它，扩容时"新位置只可能是 i 或 i + oldCap"也靠它（因为 oldCap 是 2^k，只有一位是 1）。一旦你破坏了它（比如反射改 table 长度），整个结构立刻失效。

代价是索引只用低位。容量 16 时只用低 4 位，32 时用低 5 位。如果一批 key 的 hashCode 在高位有差异、低位全相同，它们会全部撞进一个桶。这就是扰动函数要解决的问题。

二、扰动函数：为什么是 h ^ (h >>> 16)

把高 16 位无符号右移 16 位，和低 16 位异或。效果是低 16 位变成"高 16 位与低 16 位的混合"，于是容量小于 2^16 时（几乎所有情况），参与索引的低位里包含了高位的信息。

为什么是 16 而不是 8 或 24：int 是 32 位，右移 16 正好把高半部分折到低半部分，一次操作覆盖全部 32 位。右移 8 只能让高 24 位影响低 8 位，覆盖不全；右移 24 又让高 8 位主导，丢掉中间 8 位。16 是唯一一次就能对折的值。

为什么是异或而不是与、或：异或能保留两边的差异信息。`a & b` 让结果偏向 0，`a | b` 让结果偏向 1，都损失熵；异或的可逆性保证"只要有一位不同，结果就不同"。

JDK 7 的实现要重得多：

```
final int hash(Object k) {
    int h = hashSeed;
    if (0 != h && k instanceof String) {
        return sun.misc.Hashing.stringHash32((String) k);   // 防 hash 碰撞 DoS
    }
    h ^= k.hashCode();
    h ^= (h >>> 20) ^ (h >>> 12);
    return h ^ (h >>> 7) ^ (h >>> 4);
}
```

四次移位、五次异或，还有一个可选的 hashSeed。JDK 8 砍到一次异或，因为实测这次简化对分布的影响可以忽略，而每次 get/put 都要算 hash，省下的指令很可观。

但要说清楚：扰动函数**不能消除冲突**。它对"低位相同、高位不同"的 key 有效，对"hashCode 完全相同"的 key（比如 `"Aa"` 和 `"BB"`，hashCode 都是 2112）完全无效——扰动是确定性的，输入相同输出必然相同。这类冲突只能靠链表和红黑树兜底。

三、8 和 6：树化阈值不是性能拐点

源码注释里给了依据：

> Ideally, under random hashCodes, the frequency of nodes in bins follows a Poisson distribution with a parameter of about 0.5 on average for the default resizing threshold of 0.75.

λ = 0.5 时，一个桶里有 k 个节点的概率是 `e^-0.5 * 0.5^k / k!`：k=0 是 0.6065，k=1 是 0.3033，k=2 是 0.0758，k=3 是 0.0126，k=4 是 0.00158，k=5 是 0.000158，k=6 是 0.0000132，k=7 是 0.00000094，k=8 是 0.00000006。

也就是说，正常情况下一个桶长到 8 个节点的概率是千万分之六。所以树化不是为了日常性能——链表长度 8 时线性查找也就 8 次比较，比维护红黑树的成本低得多。树化是为了**极端情况**：hash 碰撞攻击（攻击者构造大量同 hash 的 key，把 HashMap 打成 O(n)），或者 hashCode 实现很差的类。

阈值 8 的准确含义，源码注释写的是 "when adding an element to a bin with at least this many nodes"，判断语句是 `binCount >= TREEIFY_THRESHOLD - 1`。`binCount` 是从头节点走起的步数，所以桶里已有 8 个节点时还不会树化，插入第 9 个才触发。很多文章说"链表到 8 就树化"，少算了一步。

退化阈值 6 比 8 小，是为了留缓冲带。如果树化和退化都是 8，一个桶在 7、8 之间抖动时会反复重建红黑树。中间隔一个 7，避免了这种抖动。

还有一个容易忽略的阈值 `MIN_TREEIFY_CAPACITY = 64`：

```
final void treeifyBin(Node<K,V>[] tab, int hash) {
    int n, index; Node<K,V> e;
    if (tab == null || (n = tab.length) < MIN_TREEIFY_CAPACITY)
        resize();                       // 表太小，扩容比树化更划算
    else if ((e = tab[index = (n - 1) & hash]) != null) {
        // ... 真正建树 ...
    }
}
```

理由很实际：表长 16 时有 8 个节点挤在一个桶，说明冲突严重，但扩容到 32 之后这些节点大概率被分到两个桶，各 4 个，问题自然消失。扩容的代价比建树低，而且顺带缓解了其他桶的拥挤。

四、扩容：lo/hi 拆分，以及 JDK 7 为什么会成环

JDK 8 的扩容不再重算 hash，原因是一行位运算。

新容量是 oldCap 的两倍，所以 `newCap - 1` 比 `oldCap - 1` 多了一个最高位。对某个 hash 来说，`hash & (newCap-1)` 和 `hash & (oldCap-1)` 的差别只在于 newCap-1 多出来的那一位（也就是 oldCap 那一位）是 0 还是 1：

- 这一位是 0：新索引等于旧索引；

- 这一位是 1：新索引等于旧索引加 oldCap。

所以只需看 `hash & oldCap`，就能把一条链表拆成两条，一条留在原位，一条挪到 `j + oldCap`。完全不需要重新计算 hash，也不需要重新取模。这就是"扩容只花一次遍历"的全部原因。

而且 JDK 8 用的是**尾插**，lo 链和 hi 链都保持节点在旧链表里的相对顺序。JDK 7 的 transfer 是头插：

```
do {
    Entry<K,V> next = e.next;
    int i = indexFor(e.hash, newCapacity);
    e.next = newTable[i];      // 头插：新节点指向当前表头
    newTable[i] = e;           // 自己成为新表头
    e = next;
} while (e != null);
```

头插会把链表反转。单线程下无所谓，多线程下就是灾难。

成环的过程可以逐步推。假设旧桶是 `A → B → null`。线程 1 执行到 `e = A, next = B` 时被挂起。线程 2 完整跑完 transfer：第一轮 `e = A`，`newTable[i] = A`，`A.next = null`；第二轮 `e = B`，`newTable[i] = B`，`B.next = A`。结果新桶是 `B → A → null`，此时 `A.next = null`、`B.next = A`。

线程 1 恢复，手里还是 `e = A, next = B`：

- 第一轮：`A.next = newTable[i]`，而 newTable[i] 已经是 B，于是 `A.next = B`；`newTable[i] = A`；`e = next = B`。
- 第二轮：`next = B.next`，而 B.next 被线程 2 改成了 A，于是 `next = A`；`B.next = newTable[i] = A`；`newTable[i] = B`；`e = A`。
- 第三轮：`next = A.next = B`；`A.next = B`；`e = B`，回到第二轮的状态，永远出不去。

最终 `A.next = B`、`B.next = A`，成环。此后任何 get 落到这个桶，`for (Entry e = table[i]; e != null; e = e.next)` 永远走不到 null，CPU 打满，而且这个环会随着后续 put 继续被复制、扩散。

JDK 8 改尾插之后，两条链各自往尾部追加，不会出现"把别人的 next 指回头"的操作，所以不再成环。但这**不等于线程安全**：两个线程同时 put 仍然会互相覆盖，size 会算错，一个线程的 put 可能整个丢失。JDK 8 只是把"死循环"这个最恶劣的后果降级成了"丢数据"。

这一点必须说透，因为很多文章把"JDK 8 改用尾插"讲成"JDK 8 的 HashMap 线程安全了"，这是完全错误的。尾插解决的是链表反转这个特定模式，不是并发正确性。HashMap 从头到尾都不支持并发写，要并发就用 ConcurrentHashMap。

容易误解的地方：

1. 以为 JDK 8 换成尾插就线程安全了。仍然会丢更新、size 不准，只是不再成环；尾插消掉的是一个特定的事故模式，不是竞态本身。
2. 以为树化阈值 8 是"链表查询变慢的拐点"。8 次比较本来就不慢，8 是泊松分布给出的"几乎不可能自然达到"的概率阈值，树化主要是防 hash 碰撞攻击。
3. 以为扩容要重新计算每个 key 的 hash。JDK 8 只看 `hash & oldCap` 这一位，把链表拆成两条，一次遍历搞定。
4. 以为扰动函数能解决 hash 冲突。它只改善低位分布，对 hashCode 相同的 key 完全无效。

记住这一句：JDK 8 没有让 HashMap 快多少，它只是把"并发下成环"降级成"并发下丢数据"，把"长链表"降级成"红黑树"。

第13章　ConcurrentHashMap：从分段锁到 CAS + synchronized

一句话：JDK 8 把锁从 16 个 Segment 拆到每个桶的头节点，空桶写入彻底无锁。

为什么需要它：

三个现场。

第一个：JDK 7 的 ConcurrentHashMap，`concurrencyLevel` 用默认值 16。业务里有个"按用户 ID 计数"的 map，用户 ID 的 hash 恰好集中，结果 80% 的写落在同一个 Segment 上，16 核机器上 15 个核在自旋等锁，QPS 上不去。你把 concurrencyLevel 调到 256 也没用——段数在构造那一刻就定死了，而且热点 key 还是落在同一段。

第二个：压测时 `size()` 返回的数字一直在跳，两次调用差几千，甚至后一次比前一次小。因为 JDK 7 的 size 是遍历 16 个 Segment 的 count 累加，累加过程中别的线程还在写，读到的是"拼接起来的历史"。

第三个：写了个 `map.put(key, null)`，运行时报 NPE，而同样的代码在 HashMap 上跑得好好的。这不是 bug，是 API 契约差异。

这一章要解决的是：在"读多写少、写操作分散"的典型缓存场景下，怎么让并发度不再依赖一个初始化参数；怎么让 size 在并发下既快又不会错得离谱；怎么让扩容这个 O(n) 操作不阻塞整个世界。

核心结构：

概念	作用
`Segment`（JDK 7）	继承 `ReentrantLock` 的"段"，默认 16 个，一段一把锁；段数构造后不可变
`concurrencyLevel`（JDK 7）	决定段数，向上取 2 的幂；JDK 8 里退化为初始容量的提示
`HashEntry`（JDK 7）	段内链表节点，`value` 和 `next` 都是 volatile，支撑无锁读
`scanAndLockForPut`（JDK 7）	拿不到段锁时"边扫链表边重试"，超过 64 次才转阻塞
`Node`（JDK 8）	链表节点，`val`、`next` 为 volatile，`hash` 为 final
`sizeCtl`（JDK 8）	一个 int 承担四种语义：未初始化容量、-1 初始化中、负数扩容中、正数阈值
`MOVED = -1` / `ForwardingNode`	迁移标记节点，旧表某桶迁完后填入，`find` 会转向新表
`baseCount` + `CounterCell[]`	分散计数，低竞争 CAS 基值，冲突时散列到 cell 数组各自累加
`transferIndex` + `stride`	把 n 个桶切成若干任务包，各线程 CAS 领取，协作迁移
`helpTransfer`	put 撞到 MOVED 时加入迁移，而不是干等

关键代码：

JDK 7 的写入路径，锁的粒度是段：

```
final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    // 先试一次 tryLock，失败才走"边扫边等"的慢路径
    HashEntry<K,V> node = tryLock() ? null : scanAndLockForPut(key, hash, value);
    try {
        HashEntry<K,V>[] tab = table;
        int index = (tab.length - 1) & hash;
        HashEntry<K,V> first = entryAt(tab, index);
        for (HashEntry<K,V> e = first;;) {
            if (e != null) {
                // ... 命中则覆盖 ...
            } else {
                if (node != null) node.setNext(first);
                else node = new HashEntry<K,V>(hash, key, value, first);
                int c = count + 1;
                if (c > threshold && tab.length < MAXIMUM_CAPACITY)
                    rehash(node);                    // 段内扩容，仍持锁
                else
                    setEntryAt(tab, index, node);    // 头插，UNSAFE 保证可见性
                ++modCount;
                count = c;
                return null;
            }
        }
    } finally {
        unlock();
    }
}
```

JDK 8 的写入路径，锁的粒度是桶：

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode());          // 清掉符号位，负数留给特殊节点
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();                  // CAS sizeCtl 抢初始化权
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;                          // 空桶：一次 CAS，完全无锁
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);         // 正在扩容：先去帮忙，别干等
        else {
            V oldVal = null;
            synchronized (f) {                  // 锁住这个桶的头节点
                if (tabAt(tab, i) == f) {       // 双重检查：头节点没被换掉
                    if (fh >= 0) {
                        // ... 遍历链表，尾插或覆盖 ...
                    } else if (f instanceof TreeBin) {
                        // ... 红黑树插入 ...
                    }
                }
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD) treeifyBin(tab, i);
                if (oldVal != null) return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);                     // 计数，顺带检查是否需要扩容
    return null;
}
```

拆解：

一、JDK 7：16 把锁把表切成 16 份

JDK 7 的 ConcurrentHashMap 是一个 `Segment[]`，默认长度 16（`DEFAULT_CONCURRENCY_LEVEL`）。每个 Segment 继承 ReentrantLock，内部又是一个 `HashEntry[]` 加链表，结构和 HashMap 几乎一样。

定位要两次 hash：先 `(hash >>> segmentShift) & segmentMask` 选段，再 `(tab.length - 1) & hash` 选桶。默认 segmentShift = 28、segmentMask = 15，也就是取 hash 的高 4 位选段——用高位而不是低位，是因为低位要留给段内的桶索引，两级分布才独立。

段数不可扩容。`concurrencyLevel` 只是初始化时的提示，向上取到 2 的幂，之后写死。这意味着并发度的上限在构造那一刻就决定了：设 16，最多 16 个线程同时写；设 256，就要付出 256 个 Segment 加 256 个数组的内存代价，哪怕表里只有几百个元素。

段内扩容是独立的：某个 Segment 的 count 超过阈值，就把它内部的 HashEntry 数组翻倍，扩容时持该段的独占锁，其他段不受影响。这是分段锁真正的收益——扩容不再是全局停顿。

但分段锁的问题很明确：

1. 并发度与数据量脱钩。表很大、段数很小，写并发上不去；表很小、段数很大，内存浪费。
2. 热点倾斜。hash 分布不看段数，某一段被打爆时，其他段的锁是空闲的。
3. 内存开销。每个 Segment 至少有一个数组，默认 16 段就是 16 个 `HashEntry[2]`。
4. size 弱一致，遍历 16 个 count 的过程中值还在变。

还有一个值得看的细节：`scanAndLockForPut`。拿不到段锁时它不死等，而是"边等边干活"——顺着链表往下扫，扫到空位就顺便把新节点 new 出来（speculative create），每扫一轮试一次 tryLock，重试超过 `MAX_SCAN_RETRIES`（多核 64 次）才退化成 `lock()` 阻塞。这是把等待变成有用功的经典手法，JDK 8 的 `helpTransfer` 是同一思路的另一种形态。

二、JDK 8：锁头节点，为什么这样就够了

JDK 8 把 Segment 整个删了，结构退回 `Node[]`，但锁没有消失，只是粒度变了：

- 桶为空：一次 CAS 把新节点放进去，不加锁；
- 桶非空：`synchronized (f)`，f 是这个桶的头节点；
- 头节点是 ForwardingNode：说明正在扩容，当前线程去 `helpTransfer`。

为什么锁头节点就够了？因为一个桶的所有修改——链表插入、链表转树、树插入、删除——都必须先拿到头节点的 monitor。不同桶的头节点是不同对象，锁之间互不干扰。所以并发度等于桶的数量，而不是固定的 16。表长 16 时是 16，扩容到 1024 时就是 1024，跟着数据量一起长。

为什么用 synchronized 而不是 ReentrantLock，三个原因：

第一，不需要为每个桶额外创建锁对象。ReentrantLock 是一个对象，一个桶一个的话，内存开销和 GC 压力都上去了；synchronized 用的是对象头里的 mark word，零额外对象。

第二，锁的持有时间极短（一次链表遍历加一次插入），大部分情况在偏向锁、轻量级锁阶段就解决了，进不了内核。

第三，JDK 6 之后 synchronized 的性能已经和 ReentrantLock 持平，JDK 8 的 CHM 没有理由再自己管锁。

注意 `synchronized (f)` 里面还有一个 `if (tabAt(tab, i) == f)`。这是必须的：在你拿到锁之前，这个桶的头节点可能已经被别的线程换掉了（扩容迁移、或者树化把 Node 换成了 TreeBin）。如果不查，你锁的是一个已经不在表里的对象，锁了个寂寞。

还有一个细节在 `spread`：

```
static final int spread(int h) {
    return (h ^ (h >>> 16)) & HASH_BITS;   // HASH_BITS = 0x7fffffff
}
```

比 HashMap 多了一个 `& 0x7fffffff`，把符号位清掉。因为负数 hash 在这个类里是保留值：`MOVED = -1` 表示迁移中，`TREEBIN = -2` 表示红黑树，`RESERVED = -3`。普通 key 的 hash 必须非负，否则会和这些标记撞车。

三、get 为什么可以完全不加锁

```
public V get(Object key) {
    Node<K,V>[] tab; Node<K,V> e, p; int n, eh; K ek;
    int h = spread(key.hashCode());
    if ((tab = table) != null && (n = tab.length) > 0 &&
        (e = tabAt(tab, (n - 1) & h)) != null) {     // tabAt 是 Unsafe.getObjectVolatile
        if ((eh = e.hash) == h) {
            if ((ek = e.key) == key || (ek != null && key.equals(ek)))
                return e.val;
        }
        else if (eh < 0)                              // 红黑树或 ForwardingNode
            return (p = e.find(h, key)) != null ? p.val : null;
        while ((e = e.next) != null) {                // next 是 volatile
            if (e.hash == h && ((ek = e.key) == key || (ek != null && key.equals(ek))))
                return e.val;
        }
    }
    return null;
}
```

三个可见性支点：

1. `table` 是 volatile 的，数组引用本身可见。
2. 数组元素用 `Unsafe.getObjectVolatile` 读（`tabAt`），绕过普通数组访问。普通数组元素的读写不带 volatile 语义，JMM 只保证"table 指向哪个数组"可见，不保证 `table[3]` 的内容可见。这一点极容易被忽略。
3. `Node.val` 和 `Node.next` 都是 volatile 的，链表的延伸和值的替换都可见。

为什么这样就够：put 一个节点时，先 new 出完整的 Node（hash、key、val、next 都填好），再把它挂到链表尾部或作为新头节点。volatile 写 next 保证"节点构造完成"先于"节点可见"。get 要么看不到这个节点（读到旧的 next），要么看到一个完整的节点，不会看到 val 还没赋值的半成品。

代价是**弱一致**：get 可能读不到刚刚 put 的值（put 还没发布 next），也可能在扩容期间从旧表读到旧值。但它不会读到脏数据，不会阻塞。这是 ConcurrentHashMap 的核心取舍：读永远不等待，代价是读到的可能是稍旧的快照。

顺带解释 null 的问题。CHM 不允许 null key 和 null value，不是技术做不到，而是 get 返回 null 时无法区分"不存在"和"值就是 null"，而无锁的 get 没法再用 containsKey 二次确认（两次调用之间值可能变了）。HashMap 允许 null，是因为它可以在锁内保证两次调用的原子性。

四、size 与扩容：两个 O(n) 操作怎么在并发下活下来

size 的演进最能说明"并发下精确计数有多贵"。

JDK 7 的做法是遍历 16 个 Segment，每个 count 是 volatile。它先不加锁数一遍，再数一遍，两次结果一致就返回；不一致就重试，重试超过 `RETRIES_BEFORE_LOCK = 2` 次之后，把 16 个段全部锁住再数。这是典型的"乐观读加校验"，低冲突时很快，高冲突时退化成全局锁。

JDK 8 换成了 LongAdder 的思路：

```
private final void addCount(long x, int check) {
    CounterCell[] as; long b, s;
    // 先试 baseCount，失败说明有竞争
    if ((as = counterCells) != null ||
        !U.compareAndSwapLong(this, BASECOUNT, b = baseCount, s = b + x)) {
        CounterCell a; long v; int m;
        boolean uncontended = true;
        if (as == null || (m = as.length - 1) < 0 ||
            (a = as[ThreadLocalRandom.getProbe() & m]) == null ||
            !(uncontended = U.compareAndSwapLong(a, CELLVALUE, v = a.value, v + x))) {
            fullAddCount(x, uncontended);     // 新建 cell 或扩容 cells 数组
            return;
        }
    }
    // ...
}

final long sumCount() {
    CounterCell[] as = counterCells; CounterCell a;
    long sum = baseCount;
    if (as != null) {
        for (int i = 0; i < as.length; ++i) {
            if ((a = as[i]) != null) sum += a.value;
        }
    }
    return sum;
}
```

为什么要数组：所有线程 CAS 同一个 `baseCount`，在 16 核上失败率接近 100%，每次失败都要重试，缓存行在核间来回弹。改成"每个线程按 probe 散列到一个 cell，各自 CAS 自己的 cell"，冲突概率降到 1/n，代价是求和要遍历。

`CounterCell` 上还有 `@sun.misc.Contended`，把它填充到独占缓存行，避免两个 cell 落在同一行互相失效。这是 LongAdder 比 AtomicLong 快的核心原因，也是 CHM 的 size 在高并发下还能保持近似 O(1) 的原因。

代价：`sumCount` 不是原子的。遍历 cells 的过程中别的线程还在加，所以 `size()` 返回的是"某一瞬间的近似值"。JDK 8 的注释写得很清楚：并发插入删除时可能不准确。要精确值只能用 `mappingCount()`（返回 long，避免 int 溢出）配合外部同步。

扩容的协作机制是 `transfer`。它把旧数组的 n 个桶切成若干任务包，每包 `stride = max(16, (n >>> 3) / NCPU)`。`transferIndex` 从 n 开始，每个线程 CAS 把它减去 stride，领到 `[bound, i]` 这个区间，从后往前迁。迁完一个桶，就在旧表的这个位置放一个 `ForwardingNode`（hash = MOVED = -1），告诉其他线程"这里处理过了"。

其他线程 put 时撞到 MOVED，不会干等，而是调 `helpTransfer` 加入迁移——这就是"多线程协助迁移"。get 撞到 MOVED，则调 `ForwardingNode.find`，它会去 `nextTable` 里找。整个扩容没有全局停顿，只有单个桶的锁。

`sizeCtl` 在这个过程里扮演调度器：扩容开始时设为 `(resizeStamp(n) << RESIZE_STAMP_SHIFT) + 2`，一个很大的负数；每多一个线程参与就加一，每退出一个就减一；最后一个退出的线程负责把 `table` 指向 `nextTable`、把 `sizeCtl` 恢复成新容量的 0.75 倍。同一个字段在初始化时是 -1，在稳态时是阈值，在扩容时是"戳记加线程数"，这就是它难读的原因。

容易误解的地方：

1. 以为 JDK 8 完全无锁。空桶才无锁，非空桶仍然 `synchronized`，只是锁的范围从"一段"缩到"一个桶"。
2. 以为 `get` 是强一致。它是弱一致，可能读不到刚 put 的值，扩容期间可能从旧表读到旧值；但不会读到半成品。
3. 以为 `size()` 是精确值。它是 `baseCount` 加所有 cell 的和，遍历期间还在变，只是一个估计。
4. 以为 `concurrencyLevel` 在 JDK 8 还控制并发度。它只影响初始容量（`sizeCtl` 的初值），并发度现在等于桶数。

记住这一句：JDK 7 用 16 把锁把表切开，JDK 8 把锁拆到每个桶的头节点上——并发度的上限从"段数"变成了"桶数"。

第14章　字符串常量池：intern 到底做了什么

一句话：intern 是一次全局查表，返回的永远是池里那个引用，不是你手里这个对象。

为什么需要它：

两个现场。

第一个：一个日志服务，堆 8G，Full GC 后老年代还剩 5G，dump 下来 String 对象占了 62%，其中大量是重复的——从数据库读出来的 `"SUCCESS"`、`"FAIL"`、`"PENDING"`，每个订单行都 new 一个。内容完全一样，堆上是几百万个独立对象。

第二个：一段代码在 JDK 6 下跑着跑着 `java.lang.OutOfMemoryError: PermGen space`，加 `-XX:MaxPermSize=256m` 只是把崩溃时间往后推。迁到 JDK 7 之后同样的代码不报错了，但 YGC 从 20ms 涨到 200ms。

这两个现场是同一个机制的两面：字符串常量池的位置和容量。第一个说明"不用池"的代价，第二个说明"用错池"的代价。

这一章要解决三个具体问题：`intern()` 到底往池里放了什么（复制还是引用）、`new String("a")` 到底创建了几个对象、`==` 在什么情况下会骗你。

核心结构：

概念	作用
class 文件常量池	编译期产物，每个 .class 一份，`CONSTANT_String` 存字面量的符号引用
运行时常量池	类加载后每个类一份，在方法区（JDK 8 是 Metaspace），`CONSTANT_String` 解析后指向堆里的 String
StringTable（字符串常量池）	全 JVM 一份的哈希表，存字符串实例的引用，JDK 7 起在堆里
`String.intern()`	native 方法：查表，命中返回池中引用，未命中登记并返回
`-XX:StringTableSize=N`	StringTable 的 bucket 数，JDK 6 默认 1009，JDK 7u40 起和 JDK 8 默认 60013
`-XX:+PrintStringTableStatistics`	JVM 退出时打印 bucket 数、entry 数、平均链长
字面量 `"abc"`	类加载或首次执行 `ldc` 时进池，同一个类里只有一条
编译期常量折叠	`"ja" + "va"` 在 javac 阶段就变成 `"java"`，直接进池

关键代码：

`intern` 是 native 的，看不到 Java 实现，但语义可以等价地写成：

```
// 等价语义，不是真实实现
public String intern() {
    synchronized (stringTable) {
        String pooled = stringTable.get(this);   // 按内容（equals）查
        if (pooled != null) {
            return pooled;                       // 命中：返回池里那个，不是你手里这个
        }
        stringTable.put(this, this);             // 未命中：把"当前这个引用"登记进去
        return this;
    }
}
```

关键在最后两行：JDK 7 之后登记的是**当前对象的引用**，不是复制一份内容。JDK 6 是复制到 PermGen。

三个经典例子：

```
String a = "abc";                       // 字面量：进池，a 指向池中对象
String b = new String("abc");           // 堆上新对象，b 指向堆
System.out.println(a == b);             // false
System.out.println(a == b.intern());    // true，intern 返回池里那个

String c = new String("a") + new String("b");
System.out.println(c == c.intern());    // JDK 7+ true；JDK 6 false
```

拆解：

一、三个"常量池"，别混为一谈

很多人把"字符串常量池"和"运行时常量池"当成一个东西，其实它们在三层：

class 文件常量池：编译期的静态表，存在 .class 文件里。字符串字面量在里面是一条 `CONSTANT_String_info`，指向一条 `CONSTANT_Utf8_info`（真正的字符内容）。

运行时常量池：类加载后，class 文件常量池被搬进方法区，每个类一份。JDK 8 里方法区的实现是 Metaspace（本地内存），所以运行时常量池在 Metaspace。`CONSTANT_String` 在这里被解析成一个**指向堆中 String 对象的引用**。

StringTable（字符串常量池）：全 JVM 只有一份，本质是一个 Hashtable（数组加链表），存的是 String 对象的引用。**JDK 7 起它在堆里**，不在 Metaspace。

三者的关系：类 A 和类 B 都写了字面量 `"abc"`，它们的运行时常量池里各有一条 `CONSTANT_String`，但这两条最终都指向 StringTable 里同一个 String 对象。这就是"字面量相同则引用相同"的底层原因。

位置变迁：

- JDK 6：StringTable 在 PermGen，大小固定，用 `-XX:MaxPermSize` 调，满了就 `OutOfMemoryError: PermGen space`。
- JDK 7：StringTable 移到堆。字符串对象本身一直在堆里，所以 JDK 7 的 intern 不再需要复制。
- JDK 8：PermGen 被 Metaspace 取代，但 **StringTable 仍然在堆里**，没有跟着进 Metaspace。这一点网上错得很普遍。

为什么移到堆：PermGen 大小难估、默认很小、只有 Full GC 才回收、还要和类的元数据抢空间。移到堆之后，字符串受 `-Xmx` 管理，YGC 就能回收（只要池里的 String 不可达），调优只需要调堆。

代价：YGC 要扫描 StringTable。池里 entry 越多，根扫描越慢。这就是"intern 导致 YGC 变长"的由来——不是 intern 本身慢，是它把 YGC 的扫描集变大了。

StringTable 本身的结构值得单独看一眼。它是一个固定 bucket 数的哈希表，JDK 6 是 1009，JDK 7u40 起和 JDK 8 是 60013，都取质数——质数能让 hash 取模后分布更均匀，减少聚集。bucket 数在 JVM 启动时定死，运行期不能扩容，只会在探测到散列不均时 rehash。用 `-XX:+PrintStringTableStatistics` 在退出时能看到 bucket 数、entry 数、总占用、平均链长、最大链长。一个健康的应用平均链长应该远小于 1；如果平均链长到了几十，说明要么 intern 用得太滥，要么 bucket 太小。这时把 `-XX:StringTableSize` 调到四十万级别，插入耗时能降一个数量级——因为每次 intern 都要走一遍链表，链长就是比较次数。

二、intern 的行为：查表、登记、返回

`intern()` 的行为可以精确描述为三条：

1. 在 StringTable 里按**内容**查找（等价于 `equals`，不是 `==`）；
2. 找到：返回池里那个 String 的引用；
3. 没找到：把当前 String 的引用登记进池，返回当前引用。

JDK 6 和 JDK 7+ 的差别只在第 3 步：JDK 6 是把字符内容**复制**一份到 PermGen，返回复制品的引用；JDK 7+ 是把**堆上这个对象的引用**登记进去，返回原对象。

把这三步摊成一次具体的内存操作：`intern` 先算 `hash = content.hashCode()`，取 `hash % tableSize` 定位 bucket，再沿 bucket 的链表逐个 `equals` 比较。命中就返回那个节点的引用；没命中就在链表头部插入一个节点，节点里存的是"当前 String 对象的引用"，不是字符内容。所以 intern 的成本等于一次 hash 加一次桶定位加链长次 equals。字符越长、链越长，越贵。这也解释了 bucket 数为什么关键：链长是直接乘在每次 intern 成本上的。

这个差别只在"池里没有"的时候才显现：

```
String s = new String("a") + new String("b");   // 运行期拼接，堆上得到 "ab"
// 假设池里此时没有 "ab"
String t = s.intern();
// JDK 7+：t == s，因为池里登记的就是 s 这个引用
// JDK 6 ：t != s，池里是一份复制品
```

这也是 `s == s.intern()` 那道题的答案来源。但前提是**池里必须没有这个字符串**。如果代码前面已经出现过字面量 `"ab"`，池里早就有了，intern 直接返回池里那个，`s == s.intern()` 就是 false。所以这道题的答案依赖上下文，很多文章直接给 true 是不严谨的。

还有一个常被忽略的点：intern 返回的引用**不一定**是你手里这个，即使内容相同。所以正确用法是"用返回值"，而不是"调用完继续用原对象"：

```
s.intern();          // 错：返回值丢了，s 还是堆上那个
s = s.intern();      // 对：把引用换成池里的
```

另外，池里的对象是可以被回收的。JDK 7+ 的 StringTable 持有的是对堆对象的引用，当这个 String 不可达时，GC 会清理对应的 entry（类的运行时常量池还在引用的除外）。所以"intern 之后永远在池里"是错的。

三、new String("a") 到底几个对象

答案分两层。

第一层：String 对象。

- 如果池里还没有 `"a"`：类加载（或第一次执行 `ldc "a"`）时，在池里创建一个 `"a"` 的 String 对象；执行 `new` 时，在堆上再创建一个。共 2 个。
- 如果池里已经有 `"a"`：只创建堆上那 1 个。

所以严格答案是"1 个或 2 个"，"2 个"是常见但不完整的面试答案。

第二层：底层数组。

JDK 8 的 `String(String original)` 构造器是：

```
public String(String original) {
    this.value = original.value;    // 直接复用原对象的 char[]
    this.hash = original.hash;
}
```

JDK 9 之后改成 `byte[]` 加 coder，逻辑一样：**数组是共享的**。所以 `new String("a")` 不会复制字符数组，只是多了一个 String 对象头加三个字段。这也是为什么 `new String(literal)` 的开销比想象中小——真正贵的是对象头和后续的 GC，不是内容复制。

很多人答"3 个对象（String 加 char[]）"，是把数组也算进去了；但在"池里已有字面量"的情况下数组根本没有新建，这个答案不成立。

四、== 的陷阱：编译期折叠与运行期拼接

`==` 比的是引用。字符串的 `==` 之所以经常"看起来对"，是因为编译期常量折叠：

```
String a = "ja" + "va";      // javac 直接折叠成 "java"，等价于字面量
String b = "java";
System.out.println(a == b);  // true

final String x = "ja";
String c = x + "va";         // x 是编译期常量，仍然折叠
System.out.println(c == b);  // true

String y = "ja";
String d = y + "va";         // y 是变量，运行期拼接
System.out.println(d == b);  // false
```

规则只有一条：**只有编译期常量参与拼接才会折叠**。字面量、`final` 修饰的 String 常量、常量表达式都是编译期常量；普通变量不是。

JDK 9 之后 `+` 的字节码从 `StringBuilder.append` 换成了 `invokedynamic` 加 `StringConcatFactory`，但语义没变：运行期拼接产生新对象。

还要记住 StringTable 的工程属性。它用 Hashtable 实现，bucket 数启动时定死（JDK 6 是 1009，JDK 7u40 起和 JDK 8 是 60013，都取质数为了散列均匀），不能动态扩容，只会在散列不均时 rehash。bucket 少了，链表就长，intern 退化成链表遍历。大量 intern 的场景要用 `-XX:StringTableSize=N` 调大，比如调到 400031。想观察效果只能用 `-XX:+PrintStringTableStatistics`，而且它只在 JVM 退出时打印。

最后一个工程问题：什么时候该 intern，什么时候不该。

该用的场景有三个特征：值的集合有限、重复率高、比较频繁。比如协议解析里的 method 名（GET/POST/PUT）、状态机里的状态字符串、CSV 里的枚举列。这些值 intern 之后 `==` 就能替代 `equals`，省掉逐字符比较；内存上，一百万个重复对象塌缩成一个。

不该用的场景也有三个特征：值来自用户输入、集合无界、只出现一两次。对用户输入调 intern 是把不可控的数据塞进一张全局表，攻击者构造几百万个不同的字符串就能把堆打满，而且这些 entry 要等到 GC 才能清。即便 JDK 7+ 的池可以回收，bucket 数组本身不会缩小，`-XX:StringTableSize` 调大了就一直是那么大。

中间地带是"从数据库读出来的低基数字段"。正确做法不是到处 intern，而是在数据入口处归一化——读出来就 intern 一次，之后全程用同一个引用。散落在业务代码里的 intern 调用既难审计，也容易在热路径上放大成性能问题。

容易误解的地方：

1. 以为 JDK 8 把 StringTable 移到了 Metaspace。没有，还在堆里；进 Metaspace 的是运行时常量池和类元数据。
2. 以为 `intern()` 总是复制一份进池。JDK 7+ 存的是引用，不复制内容。
3. 以为 `new String("a")` 一定创建 2 个对象。池里已有 `"a"` 时只创建 1 个，而且底层数组是共享的。

4. 以为 `s == s.intern()` 永远为 true。只在"池里原本没有这个内容"时成立；有字面量提前进池就是 false，而且池里的对象还可能被 GC 回收。

记住这一句：intern 不复制字符串，它只是把"这个对象"登记进一张全局哈希表，之后所有内容相同的人都拿到同一个引用。

第15章　HotSpot 内存管理：堆、栈、方法区到底怎么分的

一句话：区域按"谁生谁死"划分，堆归 GC、栈归线程、元数据归类加载器。

为什么需要它：容器限额 4G，堆 `-Xmx2G`，监控上堆常年只用 60%，进程却被 OOM Killer 干掉；`jmap -histo` 里所有对象加起来 800M，`top` 显示 RSS 3.2G；还有更常见的一种——服务刚启动，`jstat -gc` 的 `FGC` 计数已经涨到三十几，可堆使用率一直在 10% 以下。

这三个现象背后是同一个错误：**把"JVM 内存"当成"Java 堆"**。堆只是 HotSpot 管理的一块区域，而且往往是占比最小的那块。这一章要把内存按"谁申请、谁释放、什么时候释放"重新切一遍，让每一个监控数字都能落到具体区域上——不然你连该看哪个指标都不知道。

核心结构：

概念	作用
PC 寄存器（线程私有）	记录当前线程执行到哪条字节码指令。执行 native 方法时值是 undefined。唯一不会抛 OOM 的区域
虚拟机栈（线程私有）	每个方法一个栈帧：局部变量表、操作数栈、动态链接、返回地址。生命周期严格 LIFO，方法返回即整体弹出，不需要 GC
本地方法栈（线程私有）	服务 native 方法，HotSpot 里和虚拟机栈合并实现，所以 `-Xss` 同时管两者
堆（共享）	存放对象实例和数组。唯一需要 GC 的区域，分 Eden / Survivor / Old，物理上可以不连续
方法区（共享）	规范概念，存类元数据、运行时常量池、静态变量。JDK8 后由 Metaspace 实现，位于本地内存
运行时常量池	方法区的一部分。Class 文件常量池的运行时表示，`String.intern()` 在 JDK7 后操作的是堆里的字符串池
直接内存	`DirectByteBuffer` 申请的堆外内存，受 `-XX:MaxDirectMemorySize` 约束，回收依赖 Cleaner 和 Full GC

关键代码：

对象头布局。这是理解"为什么压缩指针要求堆小于 32G"的入口：

```
64 位 HotSpot，开启压缩指针（堆 < 32G），普通对象：
┌─────────────────────────┬─────────────────────────┐
│     Mark Word  (8 bytes)   │ Klass Ptr (4 bytes)│
└─────────────────────────┴─────────────────────────┘
数组对象多 4 字节长度：
┌─────────────┬─────────────┬─────────────┬─────────┐
│ Mark Word 8B │ Klass Ptr 4B │ length 4B    │ 元素…  │
└─────────────┴─────────────┴─────────────┴─────────┘

Mark Word 不是"同时有效的字段"，是一个按锁状态切换的联合体：
┌──────────┬──────────────────┬──────┬─────┬─────────┬────────┐
│ unused   │ identity_hashcode│unused│ age │ biased  │ lock   │
│  25 bit  │      31 bit      │ 1bit │4bit │  1bit   │ 2bit   │
└──────────┴──────────────────┴──────┴─────┴─────────┴────────┘
   无锁态：hashCode + 分代年龄 + 锁标志
   偏向态：线程 ID + epoch + 分代年龄（hashCode 被挤走）
   轻量/重量锁：指向栈中锁记录或 ObjectMonitor 的指针
```

对象从 `new` 到老年代的完整链路：

```
new Object()
  │
  ├─ 逃逸分析：对象不逃逸出方法 → 标量替换，堆上根本不分配这个对象
  │
  ├─ 大对象？size > PretenureSizeThreshold → 直接进 Old（仅 Serial / ParNew 认这个参数）
  │
  ├─ 当前线程 TLAB 剩余空间够？→ bump-the-pointer，指针加 size 即可
  │       └─ 不够 → 先在 TLAB 尾部填一个 dummy 对象，再去 Eden 领新 TLAB
  │
  ├─ TLAB refill：从 Eden 原子切一块（自适应，平均约为 Eden 的 1%）
  │
  └─ Eden 满 → Young GC：Eden + From 的存活对象复制到 To
          ├─ age < MaxTenuringThreshold → 留在 Survivor，age++
          ├─ 同龄对象总和 > Survivor 的 50% → 该年龄及以上全部晋升（动态年龄判定）
          └─ To 放不下 → 直接晋升 Old（分配担保）
```

拆解：

线程私有与共享：边界到底划在哪

栈之所以不需要 GC，是因为它的分配和释放与调用栈严格同构——栈帧在方法进入时整体压入，在方法退出时整体弹出，不存在"一半是垃圾一半是活的"这种状态。GC 存在的全部理由，是对象的生命周期与分配顺序不一致；栈帧的生命周期和分配顺序完全一致（LIFO），所以最复杂的回收问题在栈上根本不成立。

由此可以推出几个日常现象。第一，递归过深抛 `StackOverflowError`，这是单个线程栈深度超限；而 `OutOfMemoryError: unable to create new native thread` 是线程数超限，两者的解法相反——前者调大 `-Xss`，后者往往要调小 `-Xss` 或收敛线程池。第二，`-Xss` 越小，能开的线程越多，但每个方法的栈帧预算越少，深递归更容易炸。第三，局部变量表里的槽位会被复用，一个已经走出作用域的引用如果槽位没被覆盖，仍然算 GC Root；这就是"方法里手动置 null"这种老技巧的由来——它在解释器下有用，JIT 编译后基本失效，因为编译器会做活跃变量分析。

PC 寄存器单独提一句：它指向当前正在执行的字节码指令地址，是线程切换后能恢复执行的依据。执行 native 方法时它没有意义（值为 undefined），这也是为什么 native 方法里的崩溃没法用 Java 栈回溯定位。

对象的一生：TLAB 才是分配的主战场

分配路径上有四个岔路口，每个岔路口都有对应的参数，也都有对应的坑。

第一道是逃逸分析。对象如果没有逃逸出方法（不被返回、不被存入全局结构、不被其他线程看到），JIT 可以做标量替换，把对象拆成几个局部变量直接放在寄存器或栈上，堆上压根没有这个对象。注意 HotSpot 并没有实现"栈上分配"这个动作本身，它做的是标量替换加锁消除，效果上等价于对象消失了。这也解释了为什么压测时 `jmap` 里的对象数比预期少——不是泄漏了，是被优化掉了。

第二道是 TLAB。多线程同时 `new`，如果都在 Eden 上做指针碰撞，那个指针就是共享状态，每次分配都得 CAS。TLAB 的做法是先把 Eden 切出一块给线程独占，线程在自己的地盘上无锁地撞指针，只有 TLAB 用尽才需要一次同步去领新的。这把"每次分配一次 CAS"降成了"每个 TLAB 一次 CAS"。代价是碎片：TLAB 尾部剩下的空间不够放新对象时，只能填一个 dummy 对象浪费掉，`-XX:TLABWasteTargetPercent`（默认 1）约束的就是这部分可容忍浪费占 Eden 的比例，JVM 反过来用它推算 TLAB 该开多大——所以 TLAB 平均只占 Eden 的 1% 左右，小到浪费可控，大到能摊销掉同步成本。这个参数调大，TLAB 变大、refill 变少但浪费变多；调小则相反，是典型的用空间换同步开销。

第三道是 Survivor。Young GC 用复制算法而不是标记清除，因为年轻代的存活率通常低于 10%，复制成本与存活对象数成正比，而标记清除的成本与整个区域大小成正比。复制还有个附带好处：复制即压缩，Survivor 里永远没有碎片。晋升年龄不是死等 `MaxTenuringThreshold`，而是动态判定——Survivor 中某个年龄及以下的所有对象加起来超过 `TargetSurvivorRatio`（默认 50%），这个年龄以上的直接晋升，避免 Survivor 被半死不活的对象塞满。

第四道是担保。Young GC 前要预估晋升量，如果老年代剩余连续空间小于历次晋升的平均大小，就提前触发 Full GC。这是"老年代明明还有 30% 空间却 Full GC"的经典来源。

顺带一个高频配置错误：`-XX:PretenureSizeThreshold` 只对 Serial 和 ParNew 生效，G1 和 Parallel 完全不认这个参数。很多人照着博客配了 1M，以为大对象都进了老年代，实际上一点作用都没有。

对象头：12 字节里的状态机

Mark Word 的设计动机很朴素：对象头越小越好。一个只有 4 字节 `int` 字段的对象，如果对象头 24 字节，内存放大 7 倍。所以 HotSpot 把 hashCode、分代年龄、锁标志、偏向线程 ID、偏向 epoch 全部塞进同一个 64 bit 里，按锁状态解释成不同的结构——它不是"字段的集合"，是一个状态机。这就是为什么一个对象一旦计算过 identity hashCode 就不能再进入偏向锁：偏向态没有位置放 hashCode。

Klass Pointer 指向 Metaspace 里的 Klass 结构，是这个对象"我是什么类"的唯一依据。开启压缩指针后它只占 4 字节，实际地址 = 基址 + (值 << 3)。左移 3 位是因为对象按 8 字节对齐，低 3 位恒为 0，可以省掉。32 位偏移最多寻址 4G × 8 = 32G，这就是"堆超过 32G 压缩指针失效"的算术来源——不是拍脑袋的经验值。

数组对象多一个 4 字节 length 字段。它同时决定了 `array.length` 的上限是 `Integer.MAX_VALUE`，以及数组对象头是 16 字节。加上 8 字节对齐要求，一个空对象最小占 16 字节——这解释了为什么 `Integer` 缓存 `-128~127` 有意义：每个 `Integer` 光对象头就 12 字节，装箱的成本远大于那个 4 字节的值。

顺带说清一个版本差异：12 字节的对象头只在"64 位 + 开启压缩指针"这个组合下成立。32 位 JVM 上是 Mark Word 4 字节加 Klass 指针 4 字节，共 8 字节；64 位关闭压缩指针时是 8 加 8，共 16 字节，一个空对象就要占 24 字节（含对齐填充）。所以同一个应用在 `-Xmx31G` 和 `-Xmx33G` 下的内存占用不是线性关系——后者跨过了 32G 这条线，压缩指针自动失效，所有引用字段从 4 字节膨胀到 8 字节，实际占用可能比前者高出四成。这是扩容时最容易踩的坑：加了内存，OOM 反而更早来。

卡表：让分代收集成立的那张作弊条

Young GC 只扫年轻代。但老年代里的对象完全可能引用年轻代的对象，如果只从年轻代的根出发，这些被老年代引用的年轻对象会被误判为垃圾——这是不可接受的正确性错误。

朴素解法是把老年代也当根扫一遍，但那样分代就白分了。卡表的思路是把堆按 512 字节切成"卡页"，用一张字节数组记录哪些卡页里存在指向年轻代的引用。Young GC 时只扫脏卡对应的内存，把里面的引用加进根集合。

维护这张表靠写屏障——注意它不是内存屏障，而是编译器在 `obj.field = ref` 这类赋值语句前后插入的一小段代码。CMS 用写后屏障（赋值后检查 ref 是否指向年轻代，是则把 obj 所在的卡弄脏），G1 的 SATB 用写前屏障（记录被覆盖的旧值）。两者语义不同，后面讲 G1 时会展开。实现上，Serial / Parallel 用的是一张朴素的全局 `CardTable`，G1 换成了 `G1CardTable` 并在此之上叠了 PRT 做二级索引，所以同一个"卡表"概念在不同收集器里的内存开销差一个数量级。

卡表是粗粒度的：一个卡页 512 字节里任何一个引用变脏，整页都要重扫。这是刻意为之——宁可多扫，不可漏扫。512 这个数字本身是个折中：卡页越小，精度越高但卡表越大、写屏障越频繁；卡页越大，卡表越省但每次要重扫的内存越多。在 4G 堆上，512 字节对应 8M 的卡表，占堆的 0.2%，这个比例是可以接受的。另外多线程同时弄脏同一张卡会造成伪共享（卡的字节落在同一缓存行），`-XX:+UseCondCardMark` 让线程先读一下卡的状态再决定要不要写，能缓解这个问题，代价是多一次读。

容易误解的地方：

- "栈上分配"在 HotSpot 里并不存在。逃逸分析之后做的是标量替换，对象被拆散成字段，堆上没有这个对象，也没有一块"栈上的对象内存"。用 `-XX:+PrintEliminateAllocations` 能看到实际被消除的分配。
- 方法区不等于永久代。方法区是 JVMS 里的规范概念，永久代只是 JDK8 之前 HotSpot 的一种实现，Metaspace 是另一种实现。说"方法区在堆里"只在永久代时期勉强成立，JDK8 之后方法区在本地内存。
- 堆只用了 60% 就 OOM，问题几乎不在堆。按概率排：元空间（类加载器泄漏）、直接内存（Netty 的 `PooledByteBufAllocator`）、线程栈（线程数 × `-Xss`）、GC 自身的 native 结构（卡表、RSet、CodeCache、符号表）。用 `-XX:NativeMemoryTracking=summary` 加 `jcmd VM.native_memory` 才能定位。
- 卡表脏了不代表一定有跨代引用。它只表示"这个卡页里可能有人改过引用"，扫描时还要逐个字段判断。脏是必要条件，不是充分条件。

记住这一句：GC 只解决堆里的对象；栈随线程消亡，元数据随类卸载，剩下的内存问题都不在 GC 的管辖范围内。

第16章　Metaspace 与堆外内存：元数据搬出堆之后

一句话：元数据搬出堆后，Metaspace 的上限由类加载器的生死决定，不由 `-Xmx` 决定。

为什么需要它：堆 dump 一切正常，老年代稳定，但进程 RSS 每天涨 200M，一周后 `java.lang.OutOfMemoryError: Metaspace`；或者发布平台每次热部署后 Metaspace 涨 20M 不回落，十几次之后启动就失败，重启才能恢复；还有一种更隐蔽的——服务启动后十秒内 `jstat -gc` 的 `FGC` 计数已经三十几次，堆使用率却始终个位数，日志里全是 `Metadata GC Threshold`。

第三种最容易被误诊成"JVM 参数有问题"，前两种最容易被误诊成"堆内存泄漏"。它们指向同一件事：**类的元数据不归堆管，它的生命周期绑在类加载器上**。这一章要讲清楚永久代为什么被废掉、Metaspace 的三层分配怎么工作、`MetaspaceSize` 到底是什么意思，以及类加载器泄漏为什么是撑爆元空间的唯一原因。

核心结构：

概念	作用
VirtualSpaceList	全局链表，每个节点是一次 `mmap` 得到的 2MB 虚拟地址区间。64 位平台固定 2MB 一块
VirtualSpaceNode	链表节点。内部维护水位线，已用内存接近水位时才向 OS commit 新页，避免频繁系统调用
Metachunk	从 Node 里切出来的块，三种规格：1K / 4K / 64K。一个 chunk 只属于一个类加载器
Metablock	类加载器本地二级分配器把 chunk 切成的最小单元，实际存放 metadata 的地方，纯指针碰撞，无锁
ChunkManager	全局空闲 chunk 列表。类加载器卸载后，它的 chunk 回到这里等待复用
ClassLoaderData	JVM 内部代表一个类加载器的结构。匿名类有自己独立的 ClassLoaderData，生命周期跟匿名类走
ClassLoaderMetaspace	挂在 ClassLoaderData 上，记录这个加载器持有 class space 和 non-class space 里的哪些 chunk
Compressed Class Space	专门存放 Klass 的连续区域，默认 1G，启动时一次性 reserved，永不归还 OS

关键代码：

`jcmd` 是唯一能把 Metaspace 拆开看的工具，先看总量再看是谁占的：

```
$ jcmd 12345 VM.metaspace
12345:
Total Usage (79 regions):
  Non-class:     27.63 MB capacity,   25.34 MB (92%) used,    1.97 MB (7%) free
      Class:     18.00 MB capacity,   17.78 MB (99%) used,  224.00 KB (1%) free
       Both:     45.63 MB capacity,   43.13 MB (94%) used,    2.19 MB (5%) free

# 关键指标解读：
#   capacity 是 committed（真正占了物理内存）
#   Class 的 capacity 逼近 CompressedClassSpaceSize → 快撞 Class Space OOM
#   free 很小但 used 涨不动 → 碎片，不是不够

$ jcmd 12345 VM.metaspace show-loaders
# 按加载器列出 class / non-class 占用，泄漏时这里会看到某个 loader
# 的 chunk 数只增不减，而它加载的类早已不再使用
```

一个最小可复现的类加载器泄漏：

```
public class MetaLeak {
    private static final ExecutorService POOL = Executors.newFixedThreadPool(4);

    public void deploy() throws Exception {
        // 模拟一次热部署：新的 WebappClassLoader
        ClassLoader app = new URLClassLoader(urls, null);
        POOL.submit(() -> {
            // 线程的 contextClassLoader 指向 app；线程活多久，app 就活多久
            Thread.currentThread().setContextClassLoader(app);
            // 线程池线程永不结束 → app 永不回收 → 它加载的类永不卸载
        });
    }
}
```

拆解：

永久代为什么必须死

永久代的问题不是"设计得不好"，而是它的三个约束在 JDK7/8 那个时间点同时到期了。

第一，大小在启动时固定。`-XX:MaxPermSize` 是个必须猜的参数：猜小了，运行一段时间后 `PermGen space` OOM，而且这个 OOM 发生在类加载路径上，往往表现为某个功能突然不可用；猜大了，永久代参与每一次 Full GC（它用的是老年代的标记-清除-压缩算法），压缩一段几百 M 但几乎全是存活对象的区域，纯粹是浪费停顿时间。更糟的是它不可动态伸缩——堆可以按需扩，永久代不行。

第二，回收效率极低。永久代的回收条件是类卸载，而类卸载要求"该类的所有实例已回收 + 类加载器已回收 + Class 对象无任何引用"。这三个条件在正常应用里几乎不会同时成立，所以每次 Full GC 扫一遍永久代，回收率接近零。它被绑在堆的 GC 周期上，却不能从堆的 GC 里获益。

第三，和 JRockit 合并。Oracle 收购 BEA 之后要把 HotSpot 和 JRockit 合成一个 JVM，而 JRockit 从来没有永久代——它的元数据就在本地内存里。两套模型必须统一，选择的是 JRockit 那一套。

实际路径是分两步走的：JDK7 先把字符串常量池和类的静态变量搬到堆（这一步很关键，因为 `intern()` 产生的字符串从此受堆大小约束，也会被 Young GC 处理），符号表搬到本地内存；JDK8 才彻底删掉 PermGen，用 Metaspace 取代。

迁移期还有个细节：JDK8 之后 `-XX:MaxPermSize` 变成了无效参数，JVM 启动时会打一行 `Ignoring option MaxPermSize` 然后忽略它。很多老服务的启动脚本里还留着这一行，看到警告就该清理——更危险的是有人以为它还在生效，把 Metaspace 的大小交给了它。

三层分配：从 mmap 到 Metablock

Metaspace 的实现分三层，每一层解决一个不同的问题。

最底层解决"向操作系统要内存"的问题。JVM 用 `mmap` 一次要 2MB 虚拟地址，挂到 `VirtualSpaceList` 上。注意这是虚拟地址映射，不是真的占了 2MB 物理内存；Node 内部维护一个水位线，用到接近水位才 commit 新页。这是为了避免每分配几十字节就进一次内核。

中间层解决"并发分配"的问题。如果每个类加载器每加载一个类都去全局链表上切一块，那把全局锁会把类加载变成串行操作。所以一次给一个 Metachunk（1K / 4K / 64K），类加载器在这个 chunk 里自己分配，用完了再来要。给多大是猜的：普通加载器第一次给 4K，连续要了 4 次之后判定"这个加载器胃口大"，之后直接给 64K；bootstrap 加载器公认会加载大量类，一开始就给 4M（`-XX:InitialBootClassLoaderMetaspaceSize` 可调）；`jdk.internal.reflect.DelegatingClassLoader` 只加载一个类，永远只给 1K。

最顶层解决"细粒度分配"的问题。类加载器本地有个二级分配器，把 chunk 切成 Metablock，纯指针碰撞，无锁。因为 metadata 的生命周期和类加载器绑定，所以不需要考虑单个 Metablock 的回收——类加载器一死，整个 chunk 一起还回去，这也是为什么这个分配器可以做得如此简陋而高效。

回收路径是反过来的：类加载器卸载 → 它的 chunk 全部回到 `ChunkManager` 空闲链表 → 其他加载器需要时优先复用这些 chunk。只有当某个 Node 上的所有 chunk 都空闲时，这个 Node 才会被 `munmap` 还给操作系统。这一点很重要：**Metaspace 释放不等于归还 OS**，RSS 不降是正常的。

这里有个反直觉的细节：metadata 严格来说不属于类加载器，而是属于 `ClassLoaderData`，两者在匿名类上是分离的。类加载器加载一个匿名类（Lambda、方法引用、`MethodHandle` 生成的适配器）时，这个类有自己独立的 `ClassLoaderData`，生命周期跟着匿名类走，而不是跟着加载器。所以一个长命的类加载器下面挂着几十万个短命的 Lambda 类，是完全可能的；反过来，一个 Lambda 类被回收时，它占的 chunk 也能在加载器还活着的时候被释放。这个设计的意义在于，不必为了几个 Lambda 把整块 metadata 的生命周期拖长到和应用同寿。

MetaspaceSize 不是初始大小

这是最容易配错的一个参数，因为它名字起得像 `-Xms`。

`-XX:MetaspaceSize` 的真实语义是：**下一次触发 Metaspace GC 的高水位线**（JVM 内部叫 `_capacity_until_GC`）。当 committed 的 Metaspace 超过这个值，JVM 会触发一次 GC 尝试卸载类加载器；如果确实回收了，它就把水位线上调（因为说明还有回收空间）；如果回收不了，它也会上调（因为再触发也没用），直到逼近 `MaxMetaspaceSize`。64 位默认值约 21MB。

所以"启动时 FGC 三十几次"是这么来的：默认水位 21MB，一个 Spring Boot 应用启动过程中类不断加载，每跨过水位就触发一次 GC，GC 回收不掉（类都还在用），水位涨一点，再跨过，再触发。这是预热行为，不是故障。但如果你显式设了 `-XX:MetaspaceSize=64m` 而应用实际需要 200M，这个过程会被拉长成十几次 Full GC，启动时间肉眼可见地变慢。

从 GC 日志的 cause 上能直接分辨这两种触发，这是排查时最有用的一个抓手：

```
# 预热期的正常触发，水位线还在爬升
[Full GC (Metadata GC Threshold) ...]

# 已经到 MaxMetaspaceSize，最后一次尝试回收，回收不到就抛 OOM
[Full GC (Last ditch collection) ...]
java.lang.OutOfMemoryError: Metaspace
```

看到 `Metadata GC Threshold` 且集中在启动阶段，可以忽略；看到 `Last ditch collection`，说明已经无路可走，这时候看 `jcmd VM.metaspace` 的 committed 是否贴着 `MaxMetaspaceSize`，是就加参数，不是就去查泄漏。

`-XX:MaxMetaspaceSize` 默认不限制。但"不限制"不等于无限——`CompressedClassSpaceSize` 默认 1G，这个上限永远存在。按经验比例 class : non-class ≈ 1 : 5，所以 1G 的 Class Space 对应的总量上限大约 6G，能装 100 万到 150 万个类。

`-XX:CompressedClassSpaceSize` 是 reserved 不是 committed：启动时占 1G 虚拟地址，实际用多少 commit 多少。它不能事后调整，因为压缩类指针要求这 1G 在虚拟地址空间里连续。

实践上的结论有点反直觉：**`MaxMetaspaceSize` 设太小比不设更危险**。设太小，一旦真实需求超过它，JVM 会进入"触发 GC → 回收不掉 → 还是不够 → 再 GC"的循环，每次都是 Full GC，CPU 打满而业务完全停摆，最后才抛 OOM。这时候堆可能还很空，但你已经在死亡螺旋里了。

类加载器泄漏：撑爆元空间的唯一原因

类卸载的三个条件必须同时满足：该类的所有实例都已被回收、加载它的 ClassLoader 已被回收、对应的 `java.lang.Class` 对象没有任何地方引用（包括反射缓存、JNI 全局引用、JVMTI）。第三条经常被忽略——一个 `Class` 对象被静态字段缓存住，整个加载器就废了。

线上高频的三个场景：

动态代理和 Lambda。`DelegatingClassLoader` 是 JVM 为反射调用生成的加载器，每个被"膨胀"（inflation）的方法生成一个类、一个加载器。默认前 15 次反射调用走 native 版本，超过阈值才生成字节码类（`-Dsun.reflect.inflationThreshold`）。所以一个高频反射的框架会稳定产生几十上百个这类加载器，每个占 1K chunk 加上一个 `ClassLoaderData`。这通常不是泄漏，但会让 `show-loaders` 的输出非常难看，也解释了为什么匿名类的 class : non-class 比例是 2:1 而不是 5:1——Klass 结构有最小尺寸，再小的类也省不掉。

热部署。Tomcat 的 `WebappClassLoader` 被谁持有，决定了它能不能卸载。常见的持有者：线程池线程的 `contextClassLoader`（最常见，见上面的代码）、`ThreadLocal` 里的值（值是应用类，类的加载器就是 webapp 加载器）、JDBC 驱动（`DriverManager` 的注册表是 bootstrap 加载器可见的静态字段，里面存着应用加载的 Driver 实例）、`java.util.logging` 的 Handler、shutdown hook。这也是 Tomcat 每次 reload 都提示"内存泄漏，已清除 X 个"的原因。

字节码生成。CGLIB 每次 `new Enhancer()` 都可能生成新类，Spring 的代理缓存如果 key 设计不当就会重复生成；Groovy、SpEL、JSP 编译、MyBatis 的 Mapper 代理、各种规则引擎都是重灾区。特征是类名带 `$$`、`$Proxy`、`EnhancerBySpringCGLIB` 前缀，数量随时间单调增长。

诊断路径是固定的：`jcmd <pid> VM.metaspace show-loaders` 看哪个加载器的 chunk 数异常 → `jmap -clstats <pid>` 看它的类列表 → `jcmd <pid> GC.class_stats`（需 `-XX:+UnlockDiagnosticVMOptions`）看类的来源 → arthas 的 `classloader -t` 看加载器的引用链。关键判断依据是**加载器数量**而不是类数量：正常的应用加载器数量是稳定的，泄漏时它单调增长。

容易误解的地方：

- `MetaspaceSize` 不是初始容量。它是 GC 触发阈值，设大一点能减少启动期的 GC 次数，但它不会预先占用内存。真正的初始行为由水位线的自适应逻辑决定。
- Metaspace "默认不限"不等于无限。`CompressedClassSpaceSize` 的 1G 是硬上限，按 1:5 比例总上限约 6G。而且 JDK8 的 Metaspace 碎片比 JDK11 严重得多，同样负载下 JDK8 需要更多余量。
- 释放不等于归还操作系统。chunk 回收后进的是 `ChunkManager` 空闲链表，Node 要整个空了才 `munmap`。所以"Full GC 后 RSS 不降"在 Metaspace 上是设计行为，不是泄漏。Class Space 更是永远不还。
- 类卸载不等于把 ClassLoader 引用置 null。还有 JNI 全局引用、JVMTI 的 class prepare 回调、反射的 `Constructor` 缓存、`ClassValue`、静态字段。判断标准是"从 GC Root 出发能不能到达这个加载器"，不是"我有没有主动清理"。

记住这一句：元空间的大小不是配置问题，是类加载器生命周期问题；调参数只能把 OOM 往后推，找不到那个还活着的加载器就永远治不好。

第17章　G1：把堆切成 Region 之后

一句话：G1 用 Region 换回收粒度，用 RSet 换独立回收，用停顿模型换可预测延迟。

为什么需要它：12G 堆跑 CMS，每天两三次 Full GC，每次停 6 到 8 秒，日志里是 `concurrent mode failure`；换成 G1 之后停顿确实降下来了，但 `-XX:MaxGCPauseMillis=200` 设了半年，GC 日志里还是能看到 `Evacuation Pause` 偶尔飙到 600ms，紧接着一行 `to-space exhausted`。

CMS 的两个死结在这里暴露得很清楚：它不压缩，堆跑几天就碎片化，大对象分配失败只能退化成单线程的标记-压缩 Full GC；它的并发标记跑不赢对象晋升速度时，就只能 concurrent mode failure。G1 的设计目标就是同时解掉这两个——用 Region 化让回收可以"挑着做"，用复制算法保证没有碎片，用停顿预测模型让延迟可预期而不是靠运气。

核心结构：

概念	作用
Region	堆被切成约 2048 个等大的块，大小 1~32MB 且为 2 的幂。每块物理连续，逻辑上动态扮演 Eden / Survivor / Old
Humongous Region	大于等于 Region 一半的对象，占用连续多个 Region。分配它需要找连续空间，是 Full GC 的常见诱因
RSet（Remembered Set）	每个 Region 一个，记录"哪些外部 Region 的哪些卡页引用了我的对象"。方向是 points-into，不是 points-out
Card	卡表的基本单位，512 字节堆内存对应 1 字节。RSet 的粗粒度实现基础，配合 PRT 做细粒度索引
CSet（Collection Set）	本次回收要处理的 Region 集合。回收成本与 CSet 里的存活对象数成正比，与 CSet 大小无关
SATB	Snapshot-At-The-Beginning，并发标记的 correctness 保证。靠写前屏障记录被删除的引用，以标记开始时的快照为准
TAMS	Top-At-Mark-Start，每个 Region 一对指针（prev / next）。标记期间新分配的对象落在 next 之上，天然视为存活
IHOP	Initiating Heap Occupancy Percent，整堆占用达到该比例启动并发标记周期，默认 45%

关键代码：

```
// 一套够用的 G1 参数。注意这里没有 -Xmn —— 设了它停顿目标就废了
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200        // 软目标，不是硬承诺。按 P90 设，别按平均值设
-XX:InitiatingHeapOccupancyPercent=45   // 调小 → 标记更早开始 → 少 CMS failure，多 CPU 开销
-XX:G1HeapRegionSize=8m         // 不设则按堆大小推导；大对象多时手动调大能减少 Humongous
-XX:G1ReservePercent=10         // 预留 10% 堆给晋升，降低 to-space exhausted 概率
-XX:ParallelGCThreads=8         // STW 阶段的并行线程数
-XX:ConcGCThreads=2             // 并发标记线程数，默认 (ParallelGCThreads+2)/4
-XX:MaxTenuringThreshold=15
```

一段真实的 Young GC 日志，重点看各阶段耗时和它揭示的成本结构：

```
[GC pause (G1 Evacuation Pause) (young), 0.0234567 secs]
   [Parallel Time: 20.1 ms, GC Workers: 8]
      [Ext Root Scanning: 2.3 ms]     ← 扫根，与线程数相关
      [Update RS:         1.8 ms]     ← 处理脏卡队列，更新 RSet
      [Scan RS:           0.9 ms]     ← 遍历 RSet 找指向 CSet 的引用
      [Object Copy:       9.4 ms]     ← 真正的复制，与存活对象数成正比
   [Eden: 512.0M(512.0M)->0.0B(480.0M) Survivors: 8192K->8192K
    Heap: 3.2G(12.0G)->2.7G(12.0G)]
```

拆解：

Region 化：为什么"切成小块"能降低停顿

传统分代收集器的堆是三个大区间，年轻代满了就得整体回收，回收成本由年轻代大小决定。G1 把堆切成 2048 个左右的等大 Region，每个 Region 独立决定自己是 Eden、Survivor 还是 Old，这个角色每轮都可能变。

真正的收益来自一个不等式：**回收一个 Region 的成本正比于它里面的存活对象数，而不正比于它的大小**。一个全是垃圾的 Region，回收成本接近零（直接标记为空闲）；一个全是存活对象的 Region，回收成本极高（要全部复制走）。既然成本可度量，就可以排序，就可以"先收最划算的那些"——这就是 Garbage-First 这个名字的来源。

有了这个不等式，停顿时间就变成了一个可计算的问题：给定目标 200ms，按历史数据估算每个 Region 的平均复制耗时，反推出这次能收多少个 Region，装进 CSet。这是 G1 和 CMS 最本质的区别——CMS 的停顿是被动承受的，G1 的停顿是主动预算的。

Region 化的代价也是实打实的。第一，RSet 要占内存，官方数据是不超过堆的 5%，但实际大堆上经常更高。第二，写屏障让每次引用赋值都多几条指令。第三，对象在物理上分散，局部性变差，复制时的 cache miss 比连续内存高。第四，大对象必须占连续 Region（Humongous），分配时可能找不到连续空间。

Region 数量有个硬上限：约 2048 个。这是"每个 Region 的管理开销（RSet、标记位图）不能超过 Region 本身"倒推出来的结论——Region 太多，簿记成本会吃掉收益。所以在 4G 堆上 Region 是 2MB，在 64G 堆上是 32MB（上限）。这意味着大堆上 Region 粒度变粗，Humongous 的阈值也随之上移：64G 堆里一个 20MB 的数组不算 Humongous，在 4G 堆里 2MB 就算。同一个应用换堆大小后行为变化，经常就是这条规则在起作用。

Humongous 值得单独说：大于等于 Region 一半的对象直接进 Humongous Region，如果一个对象超过整个 Region，就要连续多个。这类对象在回收时是"全有或全无"——只要还有一个引用，整个 Region 集合都不能回收；一旦没人引用，整片立刻释放。所以频繁创建大数组（比如一次 `byte[8MB]` 的序列化缓冲）会让 Humongous Region 快速碎片化，进而触发 Full GC。这也是"G1 下不要创建大于 Region 一半的对象"这条建议的由来。

RSet 与卡表：独立回收的前提

回收 Region A 的时候，必须先知道"谁引用了 A 里的对象"。如果不知道，就只能扫全堆找引用——那就退化成 Full GC 了。

RSet 解决的就是这个问题。每个 Region 维护一份"指向我的引用来自哪里"的记录，方向是 points-into：Region B 里的对象引用了 Region A 的对象，这条记录存在 **A 的 RSet** 里。回收 A 时，遍历 A 的 RSet，就能找到所有外部引用，把它们当作额外的根。

底层还是卡表。堆按 512 字节切卡，RSet 粗粒度地记录"哪个 Region 的哪些卡页引用了我"；细粒度部分用 PRT（Per-Region Table）做二级索引，避免每次都要扫整个卡页。维护靠写屏障：`obj.field = ref` 执行后，如果 ref 指向的 Region 和 obj 所在 Region 不同，就把 obj 所在的卡记进 ref 所属 Region 的 RSet。这里有个优化——只有跨 Region 的引用才需要记录，同 Region 内的引用回收时天然可见。

一个常见困惑：CMS 也没有 RSet，为什么它能独立回收老年代？因为 CMS 不移动对象。不移动就不需要"精确找到所有指向我的引用"，只需要从根出发标记，标记到的就是活的，没标记到的就地清除。G1 要复制，复制就必须修改所有指向被复制对象的引用，所以必须精确知道这些引用在哪。**RSet 是复制算法的必要代价，不是 G1 独有的设计**。

SATB：并发标记期间"漏标"的取舍

并发标记的根本矛盾是：标记线程在遍历对象图，应用线程在同时修改对象图。三色标记里，漏标需要两个条件同时成立——一个黑色对象新增了指向白色对象的引用，并且该白色对象的所有灰色入边都被删除了。破坏任意一个条件就能保证正确性。

CMS 选择破坏第一个：增量更新（incremental update），用写后屏障记录"黑色对象新增了哪些引用"，remark 阶段把这些黑色对象重新变灰再扫一遍。代价是 remark 的待处理集合可能很大，而 remark 是 STW 的。

G1 选择破坏第二个：SATB，用写前屏障在引用被覆盖**之前**把旧值记下来，逻辑上等价于"以标记开始那一刻的对象图为快照"。任何在快照里活着的对象，这一轮都不会被误回收。

SATB 的收益有两个。第一，remark 只需要处理 SATB 队列（被删除的引用），不需要重新扫描被修改的黑色对象，remark 更快更可预测。第二，配合 TAMS 可以天然处理新分配的对象：标记开始时记录每个 Region 的 top 指针，标记期间新分配的对象都落在 top 之上，直接视为存活，不需要任何额外记录。这让"并发标记期间的大量分配"变成零成本。

代价是浮动垃圾。快照之后才死亡的对象，这一轮不会被回收，要等下一轮。所以 G1 的实际回收率比理论值低，这也是为什么 IHOP 不能设得太高——标记期间产生的浮动垃圾加上正常晋升，很容易把老年代填满。

停顿预测模型与 Full GC

G1 的停顿预测用的是衰减平均值：记录每次 Young GC 的实际耗时和回收的 Region 数，算出"每个 Region 的平均回收成本"，再结合 RSet 大小、存活率等因素做修正，预测"这次收 N 个 Region 需要多久"。这个模型不精确，但足够指导决策——它不需要知道绝对时间，只需要知道相对排序。

`-XX:MaxGCPauseMillis` 是这个模型的输入，不是输出。G1 会通过四种手段逼近它：调整年轻代大小（Region 数量）、调整 CSet 里老年代 Region 的数量、调整晋升年龄阈值、调整堆的扩张收缩。这就是为什么**设了 `-Xmn` 会让停顿目标失效**——年轻代被钉死，G1 失去了最主要的调节手段，只能被动接受结果。

同样被这个模型影响的是 IHOP。固定 45% 是个很粗的启发式：堆大了，45% 对应的绝对空间也大，标记周期可能来不及在填满前跑完；堆小了，45% 又过于保守，白白浪费并发线程。较新的 JDK 里 IHOP 默认是自适应的，JVM 会根据"上一次并发标记期间堆增长了多少"来反推这次应该多早开始。只有在负载模式非常稳定、且你确实测过的情况下，才值得手动钉死它。

还有两个参数常被忽略但收益明显。`-XX:+ParallelRefProcEnabled` 让弱引用、软引用、finalizer 的处理并行化——如果应用里有大量 `WeakHashMap`、缓存、`Cleaner`，这个阶段的串行处理可能占掉停顿的一大半。`-XX:G1HeapRegionSize` 在大对象多的场景要调大，因为 Region 越大，Humongous 的判定阈值越高，越不容易产生碎片化的 Humongous 区。

四种情况会触发 Full GC，每一种对应不同的病：

并发模式失败（concurrent mode failure）：并发标记还没跑完，老年代已经满了。根因是标记启动太晚或跑得太慢。解法是调小 IHOP 让标记更早开始，或者加 `ConcGCThreads`。

晋升失败（promotion failure）：混合回收周期里，老年代腾出空间的速度赶不上年轻代晋升的速度。解法是让混合回收每轮处理更多老年代 Region，或者加 `G1ReservePercent`。

疏散失败（evacuation failure / to-space exhausted）：Young GC 复制存活对象时，Survivor 和 Old 都放不下。这是最危险的一种，因为此时已经几乎没有可用空间了。日志里的 `to-space exhausted` 就是它。

Humongous 分配失败：找不到连续 Region 容纳大对象。

这里有个历史包袱值得知道：JDK10 之前，G1 的 Full GC 是**单线程**的标记-清除-压缩（JEP 307 之后才并行化）。所以老版本 G1 一旦 Full GC，停顿可能比 CMS 还长——CMS 至少 Full GC 时还有并行标记。这也是"G1 比 CMS 差"这个印象的主要来源，在 JDK10 之后已经不成立了。

容易误解的地方：

- Region 不是固定分代。同一个 Region 这一轮是 Eden，下一轮可能是 Survivor，再下一轮可能是 Old。Eden / Survivor / Old 是运行时的标签，不是地址区间，所以 `jmap -heap` 看到的 Eden 大小是快照值，下一秒就变了。
- `MaxGCPauseMillis` 不是承诺。它是软目标，G1 尽力逼近但不保证。设成 20ms 的后果不是"停顿变成 20ms"，而是"年轻代被压到极小，GC 频率翻十倍，吞吐崩掉"。合理做法是按 P90 设，然后接受偶尔超标。
- G1 有年轻代。它不是"不分代收集器"，只是把年轻代做成了 Region 集合，大小动态可调。把它和 ZGC 的单代设计混为一谈是常见的概念错误。
- RSet 的方向容易记反。它记录的是"谁引用我"（points-into），不是"我引用谁"（points-out）。points-out 的话，回收任何一个 Region 都得扫全堆的所有 Region。
- `to-space exhausted` 不只是"内存不够"。它更常见的含义是"复制目标空间不够"，加堆能缓解，但根治要靠提前 IHOP 让老年代在晋升高峰前就有足够空闲 Region。

记住这一句：G1 把"回收多少"从堆结构决定变成了按预算分配，代价是每个引用赋值都要过一道写屏障。

第18章　ZGC：染色指针与读屏障

一句话：ZGC 把 GC 状态编码进指针本身，靠读屏障在访问时自愈，于是整理也能并发。

为什么需要它：64G 堆跑 G1，`MaxGCPauseMillis` 调到 200，P99 延迟还是 300ms 上下，GC 日志里 `Evacuation Pause` 稳定在 150 到 400ms 之间波动；堆再涨到 128G 之后，一次 Mixed GC 要复制几十万个对象，停顿直接破 1 秒。业务侧的 SLA 是 P99 小于 10ms，这时候再怎么调参数都没用了——**G1 的停顿下限由存活对象数量决定，而这个数量随堆增长单调上升**。

这是设计层面的天花板，不是调优能解决的。G1 的复制必须 STW，因为复制要改所有指向被移动对象的引用，而应用线程如果同时在读，就会读到已经失效的旧地址。ZGC 要解的就是这个：让"对象已经搬走"这个事实可以被应用线程在读取的瞬间发现并修正，从而把复制也变成并发操作。

核心结构：

概念	作用
染色指针（Colored Pointer）	64 位指针里用 42 位存地址、4 位存 GC 状态、18 位保留。状态位参与地址计算，因此同一块物理内存可以有多个虚拟地址视图
Marked0 / Marked1	两个标记位交替使用，用来区分相邻两轮标记，避免每轮都要清空所有标记位
Remapped	表示这个引用指向的是对象的最新地址，不需要修正
Finalizable	标记只被 finalizer 可达的对象，配合 `FinalReference` 处理
读屏障（Load Barrier）	JIT 在每次从堆加载引用后插入的检查代码。发现颜色不对就走 slow path 修正，并把修正结果写回，实现自愈
Forwarding Table	每个待搬迁的 Region 一张转发表，记录旧地址到新地址的映射；被搬走的对象在旧位置的对象头里也写入了转发指针，读屏障据此定位
Relocation Set	本轮要搬迁的 Region 集合，按垃圾比例挑选，和 G1 的 CSet 同源
多视图映射	同一块物理内存被 `mmap` 到多个虚拟地址区间，这些区间只在颜色位上不同。改颜色不需要改指针值

关键代码：

42 位地址布局。这是 ZGC 所有设计的物理基础：

```
 63                          46 45  44   43   42 41                     0
┌──────────────────────────────┬──┬──┬────┬────┬────────────────────────┐
│          unused (18)         │ F │ R  │ M1 │ M0 │    object address (42) │
└──────────────────────────────┴──┴──┴────┴────┴────────────────────────┘
                                  │   │    │    │
                                  │   │    │    └─ Marked0  ┐ 两轮标记交替使用
                                  │   │    └──── Marked1  ┘
                                  │   └─────────── Remapped（已是最新地址）
                                  └─────────────── Finalizable

42 位地址 → 2^42 = 4TB。也可以用 43 / 44 位布局支持 8TB / 16TB。

为什么不用最高 4 位：64 位是理论寻址上限，硬件实际只有 48~52 根地址线，
高位不可靠。把状态位放在 42~45，正好落在"地址线覆盖得到、但内存用不到"的区间。
```

读屏障的伪代码，以及它为什么能自愈：

```
// JIT 在每次 getfield / aaload 之后插入，不是每条指令都插
void* load_barrier(void** ref_addr) {
    uintptr_t ref = *ref_addr;
    if (ref & BAD_COLOR_MASK) {      // BAD_COLOR_MASK 随 GC 阶段变化
        ref = slow_path(ref);         // 查 forwarding table，找到新地址
        *ref_addr = ref;              // 自愈：把修好的引用写回原字段
    }
    return ref;                       // fast path 只有一条 test + 跳转
}
```

拆解：

染色指针：把"我搬走了"写进指针里

G1 的复制必须 STW，本质原因是"对象搬走了"这个信息只存在于 GC 内部，应用线程手里的引用还是旧地址，读下去就是野指针。传统解法有两种：要么停掉应用线程（G1 的做法），要么在对象头里留 forwarding pointer，让读的时候能跳转（但读路径要额外判断，而且旧地址本身还是"看起来合法"的，无法在加载时就发现）。

ZGC 的做法是把这个信息放进指针自身。指针的 42~45 位不是地址的一部分，是状态标记；判断"这个引用是不是过期"就变成了一次位运算，可以在任何一次 load 之后立即完成，成本是一条 test 指令。

关键的工程技巧是多视图映射。既然颜色位参与地址计算，那么 `0x0000_0400_0000_1000` 和 `0x0000_0000_0000_1000` 是两个不同的虚拟地址——但它们通过三次 `mmap` 映射到**同一块物理内存**。所以给指针染色不会让它指错地方，只是换了个视图访问同一份数据。这就是为什么 ZGC 需要预留大段虚拟地址空间：它要按"颜色数 × 堆大小"来预留，一个 4G 的堆在 ZGC 下会吃掉十几 G 的虚拟地址。容器里限制的是 RSS 而不是虚拟地址，所以这一点通常无害，但如果有 `ulimit -v` 之类的虚拟内存限制，ZGC 会直接起不来。

为什么是三个视图而不是两个？因为标记需要"这一轮"和"上一轮"可区分。如果只有"已标记"和"未标记"两态，每一轮开始前都得把所有对象的标记位清零，那是一次全堆遍历，等于把并发的收益吃掉了。M0 和 M1 交替，加上 Remapped 作为稳定态，三轮状态循环使用，任何时刻都不需要清零——判断"这个引用是否属于当前这一轮"只要看它的位是不是当前有效位。这是用两个 bit 换来"零成本的轮次切换"。

四个状态位的语义要连起来看。`Marked0` 和 `Marked1` 交替使用：这一轮用 M0 标记，下一轮用 M1，这样两轮之间不需要把所有指针的标记位清零——只要换一个"当前有效位"就行。`Remapped` 表示"这个引用已经指向最新地址"，是稳态。`Finalizable` 是给 finalizer 用的，标记那些只被 `FinalReference` 可达的对象。

代价很直接：**ZGC 不能用压缩指针**。压缩指针的前提是"地址可以塞进 32 位"，而 ZGC 需要 42 位地址加 4 位标记，塞不进 32 位。所以同样的对象数，ZGC 的引用字段比 G1 大一倍，堆占用更高。这是典型的空间换时间。

读屏障：只在读的时候插桩，为什么够用

读屏障不是内存屏障，是 JIT 在"从堆里加载一个引用"这个动作之后插入的检查代码。它只关心引用类型的 load，不关心基本类型，也不关心栈上的局部变量（栈是线程私有的，GC 扫描时会单独处理）。

这里有个容易被忽略的覆盖面问题：屏障必须同时存在于 JIT 编译后的机器码和解释器里，否则未编译的方法就会绕过检查，读到过期引用。所以 ZGC 在解释器里也有一套对应的慢路径实现，代价是解释执行比 G1 下更慢——这也意味着"ZGC 只适合长期运行、充分预热的服务"这个经验是有道理的，冷启动阶段大量代码走解释器，读屏障的摊销效果最差。

fast path 的成本是一条 `test` 加一条条件跳转。因为绝大多数时候引用的颜色是对的，分支预测几乎百分百命中，实测开销在 5% 以内。slow path 才做真正的工作：对象被移动过，从 forwarding table 找到新地址，把新地址写回原字段——这一步叫自愈（self-healing）。自愈的意义在于，同一个字段被访问第二次时就走 fast path 了，成本被摊薄。

为什么选读屏障而不是写屏障？因为并发整理中，绝大多数操作是"读旧引用"，而不是"写引用"。写屏障只能拦截赋值动作，对于"应用线程读到一个已经搬走的对象"这种情况完全无能为力。而读屏障覆盖了所有从堆里取引用的路径，包括字段访问、数组访问、静态字段访问。

这也是 ZGC 和 Shenandoah 的分野：Shenandoah 早期用 Brooks Pointer（每个对象前面加一个转发字），读写都要过屏障，内存开销更大；ZGC 用染色指针，只在读侧插桩。

并发整理：三个阶段怎么衔接

ZGC 的一个周期只有三次极短的 STW，其余全部并发。

Mark 阶段。初始标记是 STW 的，只扫根集合（线程栈、寄存器、静态字段、JNI 引用），把直接可达的对象标记上。这一步的耗时正比于根的数量，说白了正比于线程数——这是 ZGC 停顿与堆大小无关的原因。然后是并发标记，遍历整个对象图，读屏障在这个过程中协助处理"标记期间被修改的引用"。

Relocate 阶段。先并发地选出 relocation set（垃圾比例高的 Region），建立 forwarding table。然后一次极短的 STW，只处理"根直接引用的、且位于 relocation set 里的对象"——把这些根引用改到新地址。之后的并发搬迁由 GC 线程完成，每搬一个对象，就在旧地址留下转发信息。

Remap 阶段。搬迁完成后，堆里还有大量引用指向旧地址。这些引用不需要立刻修——应用线程读到它们时，读屏障会发现颜色不对，查转发表，改到新地址，顺便自愈。剩下的没人访问的引用，由 Remap 阶段统一修正。而 Remap 通常不独立执行，它和**下一轮的 Mark 合并**——这就是 ZGC 说的"周期重叠"：上一轮真正结束的标志，是下一轮的 Remap 跑完。

这个设计的精妙之处在于把"必须做完的工作"变成了"按需做的工作"。G1 的 STW 里必须把所有引用改完才能恢复应用线程；ZGC 只改根，其余的推迟到第一次访问时。**把同步的批量工作拆成异步的惰性工作**，这是所有并发整理算法的共同套路。

版本上值得记住三个节点：JDK11 引入时是实验特性，要加 `-XX:+UnlockExperimentalVMOptions -XX:+UseZGC`；JDK15 转正（JEP 377），参数只剩一个；JDK21 出分代版本（JEP 439），同时把停顿做到与根集合规模也基本无关。生产上如果还在 JDK11，ZGC 的可用性要打个折扣——那几年的 bug 修复量不小，而且没有分代，吞吐损失比现在明显。

和 G1 的停顿对比可以这样概括：G1 的 STW 时长 ∝ CSet 里的存活对象数，随堆增长而增长；ZGC 的 STW 时长 ∝ 根集合大小（线程数、栈深度），与堆大小基本无关。这就是为什么 128G 堆上 G1 的停顿破秒，而 ZGC 还能维持在 1~2ms。

代价落在别处。单代 ZGC 每一轮都要遍历整个堆，而绝大多数对象是短命的——它们在上一次 GC 之后分配、在下一次 GC 之前就已经死了，却仍然要在这一轮被扫描一遍。这既浪费标记带宽，也让回收率偏低：一轮下来可能只回收了 10% 的堆，但付出的扫描成本是 100%。这就是 JDK21 引入分代 ZGC 的直接动机：把年轻代独立出来，用高频、低成本的回收处理掉这批短命对象，只有晋升到老年代的对象才进入全堆级别的并发标记。分代的代价是必须维护跨代引用，于是 ZGC 从"只有读屏障"变成了"读屏障 + 写屏障"，这也是它为什么在 JDK21 里是一个独立的 JEP 而不是小版本改进。

容易误解的地方：

- ZGC 不是"无停顿"。每个周期有三次 STW：初始标记、再标记（处理并发期间的新对象和引用变更）、初始转发表扫描。只是这三次都很短。宣传里的"小于 10ms"是保守上限，实测通常在 2ms 以内，JDK21 的分代 ZGC 进一步做到了亚毫秒。
- 染色指针不是"把标记位放在最高位"。放在 42~45 位，是因为高位地址线在现有硬件上不可靠，而且高位要留给未来的地址空间扩展。放最高位会导致指针无法被硬件正确解码。
- 状态位参与地址计算，但不会指错。多视图映射保证了带颜色的地址和不带颜色的地址指向同一块物理内存。这一点如果没想通，会觉得"染色会破坏指针"。
- ZGC 的 "Z" 没有任何含义。它不是 "Zero pause" 也不是 "最后一代"，只是致敬 ZFS 的命名习惯。
- 单代 ZGC 的吞吐损失是真实存在的。官方给的上限是 15%，实际读屏障、更差的局部性、无法压缩指针都会摊薄吞吐。JDK21 的分代 ZGC（JEP 439）才把年轻代的短命对象用复制算法高效处理掉，把这块补回来——代价是引入了 store barrier 来维护跨代记忆集。

记住这一句：ZGC 没有让 GC 变快，它只是把"必须停下来做的事"改成了"访问时顺手做的事"。

第19章　NIO 三件套：Buffer、Channel、Selector

一句话：Buffer 是带三个游标的内存块，Channel 双向，Selector 让一个线程盯住上万个连接。

为什么需要它：一台 4C8G 的网关扛 8000 个长连接，用 BIO 写，`jstack` 出来 8000 多个线程，绝大多数卡在 `socketRead0` 的 native 帧上——它们全在等数据，一个字节都没读到。线程栈默认 1MB，8000 个就是 8GB 虚拟内存；`vmstat` 的 `cs` 每秒几十万。你加机器，连接数涨得比机器快。

问题的本质不是"IO 慢"，是**线程被 IO 绑架**：线程的粒度是"一个连接从头到尾"，而连接 99% 的时间在等。NIO 要做的就是把"线程等数据"翻转成"数据等线程"——用少量线程问内核"谁准备好了"，只处理准备好的那几个。这一章回答三个问题：数据放在哪、怎么进出、谁准备好了。

核心结构：

概念	作用
Buffer	一块线性内存 + position/limit/capacity 三个游标，所有读写的中转站
Channel	双向数据管道，只和 Buffer 交换；`read` 是管道→Buffer，`write` 是 Buffer→管道
Selector	内核多路复用器的 Java 包装，一个线程可监听任意多个 Channel
SelectionKey	Channel 在 Selector 上的注册凭证，持有 interestOps（关心什么）与 readyOps（发生了什么）
SelectorProvider	按平台选实现：Linux 走 epoll，macOS 走 kqueue，Windows 走 select

关键代码：

```
// 三个游标的全部语义，就这几行
public final Buffer flip() {
    limit = position;   // 写模式下 position 停在"写了多少"，它正好是数据的右边界
    position = 0;       // 读从 0 开始
    mark = -1;
    return this;
}

public final Buffer clear() {
    position = 0;
    limit = capacity;   // 恢复成"整个空间都能写"
    mark = -1;          // 注意：一个字节都没清，只是把游标拨回去
    return this;
}

public final Buffer compact() {
    int remaining = limit - position;      // 还没读走的那段
    System.arraycopy(hb, ix(position), hb, ix(0), remaining);  // 整体左移到 0
    position(remaining);                   // 新数据从剩余数据后面接着写
    limit(capacity);
    return this;
}
```

```
// Selector 的标准循环，注释处是三个必踩的坑
while (true) {
    int n = selector.select();     // 返回的是"本轮新增就绪的 key 数"，不是 selectedKeys().size()
    if (n == 0) continue;          // 可能是超时或被 wakeup，不是"没连接"

    Iterator<SelectionKey> it = selector.selectedKeys().iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next();
        it.remove();               // 必须删！selectedKeys 是累加的，不删下一轮还在
        if (!key.isValid()) continue;
        if (key.isAcceptable()) {
            SocketChannel sc = server.accept();
            sc.configureBlocking(false);          // 忘了这句，register 直接抛 IllegalBlockingModeException
            sc.register(selector, SelectionKey.OP_READ);
        } else if (key.isReadable()) {
            int r = sc.read(buf);
            if (r == -1) { key.cancel(); sc.close(); }  // -1 才是对端关闭
            else if (r == 0) { /* 非阻塞下正常现象，什么都不做 */ }
        }
    }
}
```

拆解：

一、三个游标：一次完整的写入-读出-续写

把 Buffer 想成一个 `byte[8]` 加三个整数。`capacity` 创建后永不变；`limit` 是"能碰到的右边界"；`position` 是"下一个要动的位置"。读和写的规则都是同一条：**操作 position，不许越过 limit**。

以 `ByteBuffer.allocate(8)` 为例，走一遍 `put("hello")` → `flip()` → `get()` 两次 → `compact()`：

步骤	position	limit	capacity	语义
allocate(8)	0	8	8	空，写模式
put 5 字节	5	8	8	写了 5 个，下次写第 6 个
flip()	0	5	8	读模式，可读区间 [0,5)
get() ×2	2	5	8	读了 2 个，剩 3 个没读
clear()	0	8	8	丢弃剩余 3 个，从头覆盖写
compact()（替代上一步）	3	8	8	剩余 3 个左移到 [0,3)，从第 4 位续写

关键在于 `clear()` 和 `compact()` 的分叉：`clear()` 假设"数据我全读完了"，直接丢弃；`compact()` 承认"还有 3 个字节没处理完"，把它们搬到头部保留。**半包场景必须用 compact**。一个 HTTP 请求体分两次到达，第一次读了 100 字节但只解析出 60 字节的完整消息，剩下 40 字节是下一条消息的开头——你用 `clear()`，这 40 字节就没了，表现为"偶发的 JSON 解析失败"，而且只在网络抖动时复现。

`mark()` / `reset()` 是第四个游标，`mark = position`，`reset()` 回跳。解析带长度前缀的协议时有用：先 mark，试着读长度字段，不完整就 reset 回来等下一批数据。

`limit` 可以被主动下调，这是很多协议解析器的技巧。比如你已经知道这一帧只有 12 字节，但 Buffer 里有 30 字节（两帧半），可以 `buf.limit(buf.position() + 12)` 先卡住边界，解析完再 `buf.limit(原值)` 恢复。这样同一块 Buffer 能被切成多个逻辑帧，不需要复制。

还有一组不常被提到但很好用的方法：`slice()`、`duplicate()`、`asReadOnlyBuffer()`。它们共享同一块底层内存（`slice` 共享，`duplicate` 也共享，`asReadOnlyBuffer` 共享但只读），只是各自持有一套独立的 position/limit。`slice()` 把当前位置到 limit 切出来做成一个新 Buffer，position 归零——处理"Buffer 里躺着两条消息"时，用它切出第二条，比 `arraycopy` 干净。注意它们共享内存意味着**写一个会影响另一个**，这不是拷贝，是视图。

另外 `ByteBuffer` 默认大端序（Big-Endian），和网络字节序一致，但和 x86 的本机序相反。用 `asIntBuffer()`、`asLongBuffer()` 这类视图直接读整数时，如果对端是小端，必须 `order(ByteOrder.LITTLE_ENDIAN)`，否则读出来的数字是字节翻转的——比如长度 0x00000100 会变成 0x00010000，表现为"解析出的包长莫名其妙变成 65536"。

二、堆内还是堆外：DirectByteBuffer 与它的 Cleaner

`ByteBuffer.allocate(n)` 在 Java 堆上分配 `byte[]`；`allocateDirect(n)` 走 `Unsafe.allocateMemory` 拿一块 native 内存，Java 侧只有一个 `DirectByteBuffer` 对象持有地址。

为什么要有堆外？因为**内核不认 Java 对象的地址**。堆内 Buffer 调 `channel.write()` 时，JVM 必须先把 `byte[]` 拷到一块临时的 native 内存（JDK 里叫 temporary direct buffer，有缓存但不总是命中），再交给 `write(2)`。堆外 Buffer 省掉这一次拷贝，数据原地就在能被 DMA 读的地方。大包、高吞吐的场景差距明显；小包场景省下的拷贝还不够抵消分配成本。

代价在回收。堆外内存不受 `-Xmx` 约束，只受 `-XX:MaxDirectMemorySize` 限制，而它的释放不归 GC 管——GC 只看得到那个几十字节的 `DirectByteBuffer` 对象。真正的释放靠 `Cleaner`：

```
// DirectByteBuffer 构造时注册，Deallocator 里调 Unsafe.freeMemory
 cleaner = Cleaner.create(this, new Deallocator(base, size, cap));
```

`Cleaner` 是 `PhantomReference` 的子类，挂在一条全局链表上。当 `DirectByteBuffer` 对象只被虚引用可达时，`ReferenceHandler` 线程把它入队，`Cleaner` 的清理线程执行 `Deallocator.run()` 真正 `free` 掉 native 内存。这条链路依赖"对象被 GC 判定为不可达"，所以：**堆压力小的时候，堆外可能先 OOM**。极端情况下 `Bits.reserveMemory` 累计到上限，会主动 `System.gc()` 试图触发一轮回收——这是 JDK 里少数几处显式调 GC 的地方，也是 `-XX:+DisableExplicitGC` 会让 Netty 之类框架堆外内存泄漏的原因之一（它们改用自己的引用计数 + 池化来绕开这套机制）。

三、Channel 的双向性，以及 read 的三种返回值

`InputStream` 和 `OutputStream` 是两个对象，一个连接要拿两个流。`Channel` 是一个对象，既能读又能写——`SocketChannel.read(buf)` 和 `SocketChannel.write(buf)` 是同一个实例上的两个方法。这不是设计洁癖，是因为 TCP 连接本来就是全双工的，拆成两个流反而要在中间维护"它们属于同一个 fd"的映射。

几个必须刻进肌肉记忆的点：

- `read()` 返回 `-1` 表示流结束（对端 `close` 或 `shutdownOutput`），这是唯一的关闭信号。
- `read()` 返回 `0` 在阻塞模式下几乎不会出现；在非阻塞模式下表示"内核缓冲区暂时没数据"，**不是错误、不是关闭、不是重试信号**。把它当关闭处理，就会得到"客户端一连上就被断开"的诡异现象。
- `write()` 不保证写完。返回的字节数可能小于 `remaining()`，所以标准写法是 `while (buf.hasRemaining()) channel.write(buf);`。在非阻塞模式下，写不完就得注册 `OP_WRITE` 等下次机会。
- `FileChannel` **不支持非阻塞**。`configureBlocking(false)` 会抛 `IllegalBlockingModeException`。原因很直白：普通文件没有"数据还没到"这回事，`read` 要么命中 page cache 立刻返回，要么触发一次磁盘 IO 阻塞住，内核没有为普通文件提供"就绪"这个概念。所以文件只能走阻塞或 AIO（第 20 章）。

还有一个和双向性强相关的概念：半关闭。TCP 允许一端说"我不再发了"，但仍然能收。`socketChannel.shutdownOutput()` 发一个 FIN，对端 `read` 会拿到 `-1`，但对端仍然可以往你这边写，你的 `read` 还能读到数据。HTTP/1.0 的短连接、以及一些自定义协议的"请求结束"标记，都靠这个。它和 `close()` 的区别是：`close()` 关的是整个 fd，两个方向一起断，而且会丢掉内核缓冲区里还没发完的数据（除非设了 `SO_LINGER`）。在 Reactor 里做优雅关闭，正确顺序是 `shutdownOutput()` → 等对端也发 FIN（`read` 返回 -1）→ `close()`，中间不能急。

顺带说一个高频用法：`FileChannel` 实现了 `ScatteringByteChannel` 和 `GatheringByteChannel`，可以一次传一个 Buffer 数组。

```
ByteBuffer head = ByteBuffer.allocate(8);    // 固定头
ByteBuffer body = ByteBuffer.allocate(1024); // 变长体
fileChannel.read(new ByteBuffer[]{head, body});  // 先填满 head，再填 body
```

这就是"分散读"：内核按顺序填，前一个填满才填下一个，省掉了"先读到一个大 Buffer 再切片"的步骤。`SocketChannel` 也有这两个接口，写 HTTP 响应时把响应行/头/体放在三个 Buffer 里一次 `write(ByteBuffer[])`，能减少系统调用次数，也能配合 `writev` 走 gather 优化。代价是**部分写**的处理变复杂了：返回的字节数可能落在某个 Buffer 中间，你得自己算出该从哪个 Buffer 的哪个位置续写。

四、Selector：三种事件、返回值、以及为什么必须 remove

事件是位掩码，可以叠加：`OP_READ=1`、`OP_WRITE=4`、`OP_CONNECT=8`、`OP_ACCEPT=16`。注册时传的是 **interestOps**（我关心什么），`select()` 返回后 `key.readyOps()` 是**这一轮实际发生了什么**。两者是不同的东西，`isReadable()` 判的是后者。

哪些事件对哪些通道有意义，是有硬约束的：`OP_ACCEPT` 只对 `ServerSocketChannel` 有效（对 `SocketChannel` 注册会抛 `IllegalArgumentException`）；`OP_CONNECT` 只对客户端 `SocketChannel` 有效；`OP_READ` / `OP_WRITE` 对 `SocketChannel` 有效。注册错事件不会静默失败，但会让你永远收不到通知。

`select()` 的返回值语义经常被误解。它返回的是**自上次 select 以来新进入就绪状态的通道数**，而不是 `selectedKeys().size()`。如果上一轮的 key 你没处理完，这一轮 `select()` 可能返回 0，但 `selectedKeys()` 里还躺着上一轮的 key。反过来，同一个 key 在两轮里都可能就绪，`select()` 返回 1，而 `selectedKeys()` 里始终只有它一个——因为它是个 Set。

所以 `selectedKeys()` 是**累加**的：Selector 只往里加，从不替你删。这就是 `iterator.remove()` 必须存在的原因。不删的后果分两种：轻的是同一批 key 被重复处理，比如同一个请求被读两次，读到半截数据；重的是 key 已经被 `cancel()`（对端断开），`selectedKeys()` 里还留着它，下一轮 `key.isReadable()` 直接抛 `CancelledKeyException`，整个循环被打断。注意 `cancel()` 只是把 key 加进待注销集合，真正的注销发生在下一次 `select()` 时——这也是为什么 `close()` 之后不能立刻复用。

还有一个生产事故级别的坑：**不要长期注册 `OP_WRITE`**。只要发送缓冲区有空间，`OP_WRITE` 就是就绪的，`select()` 会立刻返回，循环空转，一个核 100% CPU，而且没有任何业务流量。正确做法是：平时不注册，`write()` 返回 0 或没写完时才 `interestOps(ops | OP_WRITE)`，写完后立刻 `interestOps(ops & ~OP_WRITE)`。这个"按需注册、用完即摘"的模式，是后面 Tomcat 的 `unreg()` 和 Netty 的 `ChannelOutboundBuffer` 共同的做法。

往下钻一层看 `Selector` 的实现，很多东西就顺了。Linux 上是 `EPollSelectorImpl`，`Selector.open()` 实际做的事是 `epoll_create1()` 拿到一个 epoll fd，再建一个 loopback 的 pipe——这个 pipe 就是 `wakeup()` 的实现：往管道写一个字节，让 `epoll_wait` 因为"管道可读"而返回。所以 `wakeup()` 不是魔法，是一次真实的 fd 就绪。

`SelectionKey` 在 JDK 里是 `SelectionKeyImpl`，它持有 channel、selector、interestOps、readyOps 和一个 `attachment`。`attachment` 是**唯一一个可以挂载业务上下文的地方**，`register(selector, ops, attachment)` 的第三个参数。这个设计很关键：`select()` 返回的是 key，你要从 key 反推出"这个 fd 对应哪个连接的读缓冲、哪个半包状态、哪个会话"，全靠 attachment。生产代码里几乎不会出现不带 attachment 的注册。`key.attach()` 可以在之后替换它，`key.attachment()` 取出来。

`selectedKeys()` 返回的那个 Set，JDK 实现是 `HashSet<SelectionKey>` 加一个 `add` 的包装（`SelectedSelectionKeySet` 在 Netty 里被换成了数组，因为 `HashSet.add` 的装箱和哈希在高频下很贵）。理解它是普通集合这一点很重要：它不"属于" Selector 的内部状态机，Selector 只是往里加，什么时候清是你的责任。

`select()` 内部还有一层细节：`select` 会先把 `cancelledKeys` 集合里的 key 真正注销掉（调 `epoll_ctl(DEL)`），再去 `epoll_wait`。所以 `key.cancel()` 之后不能立刻重新 `register` 同一个 channel——旧的注册还没清掉，新注册会失败或行为未定义。正确做法是 `cancel()` 后让出一次 `select()`，或者干脆 `close()` 重建。

容易误解的地方：

1. **`flip()` 不是"模式开关"，`clear()` 也不是"清空"**。`flip` 只是 `limit = position; position = 0`，连着调两次 `limit` 就变 0，你会读到空 Buffer 而且不报错；`clear` 只拨游标，旧字节还在数组里，所以"`clear()` 之后 `array()` 转字符串"读到的是上一次的残留，正确做法是按 `limit` 截断。
2. **`select()` 返回 0 不代表没事件**。`wakeup()` 会让它立刻返回 0；`select(timeout)` 超时也返回 0；上一轮没 remove 的 key 也可能让本轮返回 0 但 `selectedKeys()` 非空。看到 0 就 `continue` 之前，先想清楚是哪种。
3. **堆外内存不是"不会被 GC"**。是"GC 看不见它的实际占用"。`DirectByteBuffer` 对象本身会被回收，回收时才连带释放 native 内存，中间有个时间差；这段时间里 `MaxDirectMemorySize` 的额度已经算上了。
4. **`Selector` 不是线程安全的**。多个线程同时 `select()` 或同时 `register()` 会死锁或丢事件（`register` 会和 `select` 抢同一把 `publicKeys` 锁）。要跨线程注册，标准做法是往队列里塞任务再 `wakeup()`——Tomcat 的 `PollerEvent` 就是这么干的。

记住这一句：Buffer 的三个游标是一台只有三个档位的机器，`flip` 拨到读、`clear` 拨到写、`compact` 拨到"带着剩余数据继续写"，拨错了不报错，只是数据悄悄没了。

第20章　非阻塞 IO 与异步 IO：差的不只是一个字母

一句话：非阻塞是"我不等你"，异步是"结果不用你管"，中间差着一整次数据拷贝。

为什么需要它：把 BIO 服务改成 NIO，最常见的两种翻车现场。第一种：改完了，`top` 显示一个核 100%，业务 QPS 是 0——有人在 `OP_WRITE` 上死循环了。第二种：压测跑 200 并发没问题，跑到 2000 就开始随机超时，抓包发现服务端 `accept` 了但从不回包——`read` 返回 0 被当成"连接关闭"处理掉了。

这两个坑都源于同一个误解：把"非阻塞"当成"异步"，把"就绪"当成"完成"。这一章要把 BIO / NIO / AIO 的线程模型摊开对比，说清 Reactor 与 Proactor 的分界，以及为什么 Java AIO 在国内基本没人用。

核心结构：

模型	谁在等	线程数	谁做数据拷贝	通知时机
BIO	业务线程	≈ 连接数	内核→用户，`read` 时同步完成	无通知，调用即阻塞
NIO（Reactor）	一个 selector 线程	≈ CPU 核数	就绪后由业务线程自己 `read`	通知"可读了"
AIO（Proactor）	内核 / 线程池	少量回调线程	由内核或线程池代劳	通知"读完了，数据在 Buffer 里"

关键代码：

```
// BIO：线程数 = 连接数，两个阻塞点都在这里
while (true) {
    SocketChannel sc = server.accept();        // 阻塞点 1：没连接就挂着
    new Thread(() -> {
        ByteBuffer buf = ByteBuffer.allocate(1024);
        while (sc.read(buf) > 0) {             // 阻塞点 2：连接建立了不代表有数据
            buf.flip();
            // ... 处理
            buf.clear();
        }
    }).start();
}
```

```
// AIO：方法立刻返回，结果在回调里
server.accept(att, new CompletionHandler<AsynchronousSocketChannel, Attachment>() {
    public void completed(AsynchronousSocketChannel client, Attachment att) {
        // 关键：必须在这里重新 accept，否则此后再也接不到新连接
        att.getServer().accept(att, this);
        client.read(att.getBuffer(), att, new ChannelHandler());
    }
    public void failed(Throwable t, Attachment att) { /* ... */ }
});
```

拆解：

一、BIO 的两个阻塞点，和它为什么死在 C10K

上面那段 BIO 代码，`accept()` 返回只说明三次握手完成了。客户端可能是先连上、隔 3 秒才发请求的浏览器（预连接），也可能是连上就不发数据的扫描器。这 3 秒里，那个新起的线程什么也没干，就卡在 `read` 上。

所以 BIO 的成本是 `连接数 × 每线程栈` 加上 `连接数 × 切换开销`。1000 连接时还能忍，10000 连接时：10GB 虚拟内存，每秒几十万次上下文切换，CPU 时间大量花在保存/恢复寄存器上，而不是业务逻辑。这就是 C10K 问题。

有人会说"用线程池限制线程数不就行了"。不行——线程池把"每连接一线程"变成"每连接排队等线程"，连接一多，排在队列里的请求全部超时。**线程池治的是并发度，治不了连接数**。

算笔账更直观。一个线程的栈默认 1MB（`-Xss`），10000 个连接就是 10GB 虚拟地址空间，虽然物理页是按需提交的，但每个线程的栈至少会真实占用几十 KB 的热页；再加上每个线程在 JVM 里的元数据、TCB、以及调度器里的运行队列节点。真正致命的是切换：Linux 的一次线程切换在微秒量级，10000 个线程如果每秒各切换两次，就是 20000 次/秒，看起来还好；但 BIO 的模式下线程是"阻塞-唤醒"式的，每个请求至少两次切换（唤醒 + 重新阻塞），QPS 5000 时就是 20000 次上下文切换，CPU 有相当比例耗在 `schedule()` 和 cache 失效上。而 NIO 用 4 个线程处理同样的 QPS，切换次数可以忽略不计。

二、Reactor：把"等"从业务线程里拿出去

NIO 的做法是把"等"这件事集中到一个地方。所有 Channel 注册到 Selector，一个线程 `select()` 阻塞着；内核告诉它"这 5 个 fd 可读了"，它把这 5 个分发给业务线程去 `read`。`read` 这时候不会阻塞，因为数据已经在内核缓冲区里了。

这就是 Reactor 模式：**事件循环 + 分发**。它的关键特征是"就绪通知"——Selector 告诉你"可以读了"，但**数据还在内核态**，真正的 `read(2)` 系统调用、数据从内核缓冲区拷到用户 Buffer，仍然是你自己做的。

演进有三代：

- **单 Reactor 单线程**：一个线程既 `select` 又处理业务。Redis 就是这个模型。优点是零锁、零切换；缺点是业务里任何一个慢操作（比如一次磁盘写）都会卡住所有连接。
- **单 Reactor 多线程**：一个线程 `select` 并分发，业务丢给 worker 池。Tomcat 的 NIO 就是这个形状（第 21 章）。适合短请求。
- **主从 Reactor**：mainReactor 只接连接，subReactor 各自 `select` 自己那批连接，业务再下一层线程池。Netty 的 `bossGroup` / `workerGroup` 就是它。这是今天所有高性能网络框架的默认形态。

非阻塞的连接建立是另一个容易漏掉的点。`SocketChannel.connect()` 在非阻塞模式下**不等三次握手完成就返回 false**，你得注册 `OP_CONNECT` 等通知，然后调 `finishConnect()`：

```
SocketChannel ch = SocketChannel.open();
ch.configureBlocking(false);
if (!ch.connect(addr)) {                       // false 不代表失败，代表"正在连"
    ch.register(selector, SelectionKey.OP_CONNECT);
}
// select 回来之后
if (key.isConnectable()) {
    if (ch.finishConnect()) { /* 连上了，改注册 OP_READ */ }
    else { /* 真失败，关掉 */ }
}
```

这段代码的价值在于**连接超时可控**。BIO 的 `connect` 阻塞，超时只能靠 `SO_TIMEOUT` 和系统参数；非阻塞下连接本身是个普通事件，超时就是你自己算的时间差。网关类应用要连几十个后端，这个能力是刚需。

然后是半包。Reactor 只告诉你"可读了"，不保证"读到一个完整请求"。一次 `read` 可能拿到半个 HTTP 头、一个半 JSON、或者三个粘在一起的小包。这是 TCP 字节流的本质，跟 NIO 无关，但 NIO 把它暴露得更彻底——BIO 里 `read` 至少会阻塞到你给的 Buffer 满了或者对端关了，看起来"一次读一个完整消息"（其实也是错觉，只是缓冲区大所以碰巧成立）。所以每个 Reactor 实现都必须自带一个"累积 + 切分"的层：累积到 Buffer 里（这就是 `compact()` 的用武之地），按协议边界切出完整帧，剩下的留着等下次。Netty 的 `ByteToMessageDecoder`、Tomcat 的 `Http11InputBuffer`，本质都是这个东西。

调用链上有个细节值得注意：`select()` 返回的 key 数量是"本轮新增就绪"，而业务线程处理是异步的。如果某个连接的数据一直没被读走，下一轮 `select()` 还会把它返回（水平触发）。所以 Reactor 实现必须在下发前先 `interestOps` 摘掉对应事件，等处理完再注册回去——否则同一个连接会被反复投递，worker 池瞬间被打爆。

三、Proactor：把"拷贝"也拿出去

AIO 往前走了一步。你调 `channel.read(buffer, attachment, handler)`，方法立刻返回；内核（Windows 的 IOCP）或 JDK 的线程池负责把数据从内核缓冲区搬到你的 `buffer`，搬完了才调 `handler.completed(n, attachment)`。

区别就在这一搬。Reactor 的通知是"可以读了"，Proactor 的通知是"读完了，`buffer` 里已经有 `n` 个字节"。前者要求你在事件回调里自己发起系统调用，后者你已经拿到了结果。

这个差别在编程模型上会放大。Reactor 的读是一个"两段式"动作：先等就绪，再读；中间这段间隔里，连接的状态可能已经变了（对端关了、超时了），所以每一段都得重新校验。Proactor 的读是一段式：回调进来时结果已经在手上，`completed` 的 `result` 是正数就是字节数，负数就是错误。代码路径短，出错面也小——这也是为什么 Proactor 模式对业务开发更友好，代价是把复杂度推给了运行时。

再看"谁拥有 Buffer"。Reactor 里 Buffer 是你的，你决定什么时候读、读到哪；Proactor 里 Buffer 被交给了内核或线程池，**在回调返回之前你不能碰它**。这条约束在 AIO 里很致命：如果你把同一个 `ByteBuffer` 同时交给两次 `read`，或者交给了 `read` 又在外面 `flip()`，结果是数据错乱或者 `BufferOverflowException`，而且不一定是必现的。生产代码里的做法是每个连接一个独立 Buffer，或者用 `Attachment` 把 Buffer 和连接绑在一起。

这也决定了两者的适用面。Proactor 少一次用户态的调度决策，理论上吞吐更好；但它要求操作系统提供"完成通知"的原语，而 Linux 长期没有（`io_uring` 是 2019 年才进主线的）。Reactor 只要求"就绪通知"，`select` / `poll` / `epoll` 都能给，所以在 Linux 上是唯一选择。

四、Java AIO 为什么在国内基本没人用

JDK 7 引入 `AsynchronousSocketChannel` / `AsynchronousServerSocketChannel`，接口很漂亮：要么返回 `Future`，要么传 `CompletionHandler`。但落到 Linux 上，实现是这样的：

`LinuxAsynchronousChannelProvider` 给每个 Channel 建一个 `EPollPort`，本质上还是一个 epoll fd + 一个 `Selector`。当你调 `read()`，它把这个 fd 注册到 epoll 上监听 `OP_READ`，然后立刻返回。等 epoll 报告可读，`EPollPort` 的轮询线程把任务丢进线程池，线程池里的线程去做真正的 `read(2)`，读完再调你的 `completed()`。

也就是说：**Linux 上的 Java AIO 是 epoll + 线程池模拟出来的 Proactor**。它没有减少任何一次系统调用，反而多了一次任务入队、一次线程唤醒、一次回调派发。而"由线程池代做 `read`"这件事，恰恰把 Reactor 的优点（数据到了就地读，不用换线程）给丢了。

具体到工程决策，还有几条：

- **没有收益**。既然底层还是 epoll，性能上限和 NIO 一样，延迟还因为多一层队列略高。
- **编程模型更难**。AIO 的回调是"一次性"的，`accept` 的 `completed` 里如果不重新 `accept`，服务端就再也不接新连接了；`read` 之后不重新 `read`，连接就静默地不再收数据。这个坑几乎每个第一次写 AIO 的人都踩过。
- **背压无处安放**。回调是内核/线程池驱动的，你没有办法对上游说"慢一点"。NIO 里 `interestOps` 摘掉 `OP_READ` 就是天然的背压阀门。
- **生态为零**。Netty 明确不支持 AIO 作为生产选项（有过 `Nio2` 实现，早已废弃）。Dubbo、gRPC-Java、Spring WebFlux 全部建在 Netty 的 Reactor 上。你用 AIO 写出来的东西，连接池、编解码、超时、心跳全得自己来。

AIO 真正有意义的地方是 **Windows + IOCP**，那里是内核级的完成通知，线程池由操作系统管，确实比 select 模拟强。但服务端部署基本是 Linux，所以这个优势用不上。`AsynchronousFileChannel` 是另一回事——文件没有非阻塞模式，异步文件 IO 至少能把阻塞从业务线程挪走，在少量大文件场景还有点用。

再补一个容易被忽略的约束：`AsynchronousChannelGroup` 的默认线程池大小是 `Runtime.getRuntime().availableProcessors()`，而且这个池**既跑 IO 完成回调，也跑你的 `CompletionHandler`**。这意味着如果你在 `completed()` 里写了阻塞代码（查一次数据库、调一次 RPC），就会把整个 group 的 IO 线程池堵死——所有连接的 `read` 完成回调都排在你后面。这是个典型的"异步框架里写同步代码"事故，和 Reactor 里在 EventLoop 上做阻塞操作是同一类错误。Netty 对此有明确的保护（`BlockHound`、`EventLoop` 的 `inEventLoop()` 检查），JDK 的 AIO 什么都没给你。

最后解释一下 Netty 为什么坚持自己写一套而不是用 JDK 的 Selector 循环。除了上面说的 AIO 问题，还有三个现实原因：JDK 的 `selectedKeys` 是 `HashSet`，高频下装箱和哈希开销可观，Netty 换成数组；JDK 的 `select` 存在著名的空轮询 bug（`epoll_wait` 返回但没有任何 key 就绪，JDK 不处理，导致 CPU 100%），Netty 用"空轮询计数超过 512 就重建 Selector"绕开；JDK 的 `Selector` 在关闭时有 fd 泄漏的历史问题。这些都不是 API 层面的缺陷，是实现的坑，只能自己重写。

容易误解的地方：

1. **"NIO 就是非阻塞 IO"**。NIO 是 New IO，指的是 `java.nio` 这套 API。`Channel` 默认就是阻塞的，`FileChannel` 永远只能阻塞。"NIO = Non-blocking IO" 是个流传很广的错译。
2. **"异步一定比非阻塞快"**。单连接、低并发下，AIO 的延迟通常更高，因为多了任务投递和回调派发的开销。异步优化的是**线程数**，不是**单次延迟**。
3. **"非阻塞 `read` 返回 0 是连接断了"**。`-1` 才是断开。返回 0 只表示"现在没数据"，在 Reactor 里这意味着你收到了一次误报，直接返回即可。
4. **"Reactor 是异步的"**。Reactor 在学术分类里叫**同步非阻塞**——事件通知是异步来的，但 `read` / `write` 是你同步做的，数据拷贝也发生在你的线程里。只有 Proactor 才算真异步。同理，"用 AIO 就不用管线程了"也是错的：`AsynchronousChannelGroup` 内部一定有线程池，默认大小是 CPU 核数，它只是不占**你的**线程。

记住这一句：Reactor 通知你"能读了"，Proactor 通知你"读完了"——前者省的是等待，后者省的是拷贝，而 Linux 从来没有给过后者。

第21章　Tomcat 的 NIO：Acceptor、Poller、Worker

一句话：一个 Acceptor 收连接、两个 Poller 等数据、两百个 Worker 干活，三层各管一段。

为什么需要它：一台 Tomcat 默认 `maxConnections=10000`。压到 10001 的时候，第 10001 个客户端不是收到拒绝，而是 TCP 连上了、然后一直没响应，客户端读超时。运维看到的现象是"服务没死，但新连接进不来"，`jstack` 里 Acceptor 线程停在 `accept0` 的 native 帧上，看起来人畜无害。

另一个常见现场：`maxThreads=200` 打满了，`jstack` 里 200 个 `http-nio-8080-exec-*` 全在 `socketRead0` 上。业务代码没慢，慢的是那些"连上来不发请求"的客户端——keep-alive 连接把 worker 线程占住了。

这两个现象都只能从 NioEndpoint 的线程分工里解释。这一章沿着 `init → start → accept → poll → dispatch` 的链路走一遍，看清每一层在等什么、被什么卡住。

核心结构：

组件	职责	默认数量
`Acceptor`	阻塞 `accept()`，拿到连接后设非阻塞、包装成 `NioChannel`、交给某个 Poller	1
`Poller`	每个持有一个 `Selector`，`select` 出就绪的 key，摘掉 interestOps 后投给 worker 池	`min(2, CPU核数)`
Worker（`ThreadPoolExecutor`）	执行 `SocketProcessor`，解析 HTTP、调 Servlet、写响应	core 10 / max 200
`LimitLatch`	基于 AQS 共享模式的计数器，卡住连接总数	10000
`NioChannel`	`SocketChannel` + 读写各 8K 的 `SocketBufferHandler`，池化复用	—
`PollerEvent`	跨线程注册的载体，把 `register` 串行化到 Poller 线程内	—
`SocketProcessor`	状态机，按 `SocketEvent` 驱动一次请求处理	—

关键代码：

```
// Acceptor.run 的核心三步
while (endpoint.isRunning()) {
    endpoint.countUpOrAwaitConnection();   // 先占额度，到 10000 就在这里 park
    // ↑ 注意顺序：是在 accept 之前卡住。所以超限时连接停在内核 backlog 里，
    //   客户端表现为"连上了但没响应"，而不是被 reset
    SocketChannel socket = endpoint.serverSocketAccept();   // 阻塞，因为 bind 时设了 blocking(true)
    if (!endpoint.setSocketOptions(socket)) {               // 设非阻塞 + 交给 Poller
        endpoint.closeSocket(socket);
    }
}
```

```
// Poller.run：注意 wakeupCounter 的作用
hasEvents = events();                                   // 先把队列里的 PollerEvent 执行掉
if (wakeupCounter.getAndSet(-1) > 0) {                  // 有人 wakeup 过 → 别睡，立刻查
    keyCount = selector.selectNow();                    // 非阻塞，返回 0 也无所谓
} else {
    keyCount = selector.select(selectorTimeout);        // 默认 1000ms
}
wakeupCounter.set(0);
// ...
while (iterator.hasNext()) {
    SelectionKey sk = iterator.next();
    NioSocketWrapper att = (NioSocketWrapper) sk.attachment();
    iterator.remove();                                  // 和上一章说的一样，必须删
    processKey(sk, att);
}
```

拆解：

一、三层线程，以及为什么 Acceptor 只有一个

`NioEndpoint.bind()` 里有一行容易被忽略：`serverSock.configureBlocking(true)`。整个 Tomcat 都在讲非阻塞，但监听套接字是**阻塞**的。这是刻意的——Acceptor 的工作就是"有连接就交出去，没连接就睡着"，阻塞 `accept` 是最省 CPU 的等待方式。真正需要非阻塞的是**已建立的连接**，它们在 `setSocketOptions` 里才被 `configureBlocking(false)`。

Acceptor 默认 1 个，源码里那行 `FIXME: Doesn't seem to work that well with multiple accept threads` 说得很直白：多个线程抢同一个 `accept()`，内核的等待队列上会有惊群和锁竞争，实际吞吐不比单线程高。accept 本身极快（只是从全连接队列取一个 fd），瓶颈从来不在这里。

Poller 默认 `min(2, CPU核数)`，不是 `= CPU核数`。原因是一个 Poller 线程一次 `select` 之后要串行处理完这一批 key 的投递，如果 key 特别多，单线程的投递就会成为瓶颈；但拆太多 Poller 又会让每个 Selector 上的 fd 变少，`epoll_wait` 的边际收益下降。2 是个经验值。

Worker 池是 Tomcat 包装过的 `ThreadPoolExecutor`，core 10 / max 200。它在创建时就预启动 core 个线程（默认 `ThreadPoolExecutor` 是懒创建的），避免流量刚进来时的建线程抖动。

三层各管一段，交接点只有两个队列：Acceptor → Poller 是 `Poller.events` 这个 `SynchronizedQueue`；Poller → Worker 是 `ThreadPoolExecutor` 的工作队列。**每个队列都是背压点**，也是故障时该看的地方。

`LimitLatch` 值得单独看一眼，因为它决定了"第 10001 个连接会怎样"。它继承 `AbstractQueuedSynchronizer`，用一个 `AtomicLong` 记当前连接数，`tryAcquireShared` 里判断"没到上限就 CAS 加一，到了就返回 -1 让 AQS 把线程挂起"。所以 `countUpOrAwaitConnection()` 在超限时是一次**真正的 park**，Acceptor 线程睡在 AQS 的等待队列里，直到有人 `countDownConnection()` 把它唤醒。这解释了为什么 `jstack` 里 Acceptor 看起来"卡住了"——它确实卡住了，而且这是正常行为。也解释了为什么超限时客户端不是被拒绝：Acceptor 根本没去 `accept`，连接躺在内核的全连接队列里，客户端以为连上了，其实对面没人接。

另一个复用设施是 `SynchronizedStack`，`nioChannels`、`processorCache`、`eventCache` 都是它。它就是个加了锁的数组栈，但配合 `ThreadPoolExecutor` 的"线程本地复用"能显著减少 GC：每个新连接要一个 `NioChannel`（内含两个 8K 的 `SocketBufferHandler`），如果每次都 `new`，GC 压力会非常大。池化之后，连接关闭时 `NioChannel` 被 `reset()` 还回栈里，下一个连接直接复用。8K × 2 × 10000 连接 = 160MB，这笔账不算池化的话是扛不住的。

缓冲区本身也是可调项。`SocketBufferHandler` 里有 socket 读缓冲（`socketBufferSize`，默认 8192）和应用读缓冲（`appReadBufSize`，默认 8192），数据先从内核读到 socket 缓冲，再交给 `Http11InputBuffer` 解析。两个 8K 乘以 `maxConnections` 就是常驻内存的下限，所以调大连接数之前得先算这笔账。Tomcat 默认用堆内 Buffer（`directBuffer=false`），因为堆内分配和回收更快；只有在 SSL 或 sendfile 场景才值得切到直接内存。

二、Acceptor 到 Poller：为什么中间要插一个事件队列

`setSocketOptions()` 里做了三件事：设非阻塞、从 `nioChannels` 这个 `SynchronizedStack` 里弹一个 `NioChannel`（弹不到就新建，带 8K 读写 buffer）、然后 `getPoller0().register(channel)`。

`getPoller0()` 是个轮询取模，把新连接均匀分给各个 Poller。`register()` 并没有直接调 `selector.register()`，而是包装成 `PollerEvent` 塞进队列，然后 `wakeup()`：

```
public void register(final NioChannel socket) {
    // ... 包装 NioSocketWrapper，设置超时、keepAliveLeft
    ka.interestOps(SelectionKey.OP_READ);
    PollerEvent r = eventCache.pop();
    if (r == null) r = new PollerEvent(socket, ka, OP_REGISTER);
    else r.reset(socket, ka, OP_REGISTER);
    addEvent(r);                    // 入队 + wakeupCounter 自增 + selector.wakeup()
}
```

为什么不直接注册？两个原因。第一，`Selector.register()` 不是线程安全的，它要和正在 `select()` 的线程抢 `publicKeys` 这把锁，跨线程调用会阻塞甚至死锁。第二，即使抢到了，正在 `epoll_wait` 的线程也不会知道有新 fd 加入，必须 `wakeup()` 打断它重新进入等待——而这个 `wakeup` 会造成一次多余的空转。

用队列把注册动作"移交"给 Poller 线程自己执行，两个问题一起解决：注册变成单线程串行，`wakeup` 也只在真有事件时才发生。`PollerEvent.run()` 里那行 `socket.getIOChannel().register(selector, OP_READ, socketWrapper)` 才是真正的注册，此时它跑在 Poller 线程里，天然安全。

`wakeupCounter` 就是这套机制的计数器：`addEvent` 时 `incrementAndGet()` 并 `wakeup()`，Poller 醒来后 `getAndSet(-1)`，看到大于 0 就知道"这轮是被叫醒的，别睡，用 `selectNow()` 快速过一遍"。这个设计避免了"事件已经入队，但 Poller 还在 `select(1000)` 里睡满一秒"的延迟。

三、processKey：先摘事件，再投递

Poller 拿到就绪的 key，进 `processKey`。这里最关键的一行是 `unreg(sk, attachment, sk.readyOps())`——**在投递之前，先把刚就绪的事件从 interestOps 里摘掉**。

```
unreg(sk, attachment, sk.readyOps());   // 比如 readyOps=OP_READ，就把 OP_READ 摘掉
if (sk.isReadable()) {
    if (!processSocket(attachment, SocketEvent.OPEN_READ, true)) closeSocket = true;
}
if (!closeSocket && sk.isWritable()) {
    if (!processSocket(attachment, SocketEvent.OPEN_WRITE, true)) closeSocket = true;
}
```

为什么必须摘？因为 epoll 默认是**水平触发**：只要缓冲区里还有数据没读走，下一轮 `select` 还会报告这个 fd 可读。如果不在投递时摘掉，Poller 会以微秒级的频率把同一个连接反复丢进 worker 队列，队列瞬间堆满，`RejectedExecutionException` 刷屏，而真正的读操作还没开始。

摘掉之后，`OP_READ` 什么时候加回来？由 `SocketProcessor` 在处理完之后自己加（通过 `PollerEvent` 走同一个队列）。这就形成了一个闭环：**就绪 → 摘事件 → 投递 → 处理 → 重新注册 → 下次就绪**。任何一个环节漏了，连接就"哑"了——表现为 TCP 还在，但再也不响应。

`processSocket(attachment, event, true)` 的第三个参数是 `dispatch`。为 `true` 时丢进 worker 池；为 `false` 时当前线程直接 `run()`。什么时候是 `false`？在 `SocketProcessor` 内部做 keep-alive 循环时，为了省一次线程切换，会让当前 worker 线程直接把下一次读也做了——但只在"数据已经到了"的情况下。

`processKey` 里还有两条容易忽略的分支。`sk.isValid()` 为 false 时直接 `cancelledKey(sk)`；key 被 cancel 通常意味着对端断开或上一轮处理出了异常。`cancelledKey` 做的事比名字多：从 `selectedKeys` 移除、`key.cancel()`、`socketWrapper.close()`、`countDownConnection()` 释放 `LimitLatch` 额度、把 `NioChannel` 还池。这一串如果在 Poller 线程里同步做完，一次 `close(2)`（要发 FIN）就可能拖慢整轮轮询，所以 Tomcat 尽量把"标记"和"真正关闭"分开。

`close` 标志位也在这里被检查。Endpoint 调 `stop()` 时置 `close=true` 并 `selector.wakeup()`，Poller 醒来走收尾路径：跑完剩余事件、`timeout(0, false)` 强制清空、`selector.close()`、`break`，最后 `getStopLatch().countDown()`。`stopLatch` 是 `CountDownLatch(pollerThreadCount)`，`AbstractEndpoint.stop()` 靠它等所有 Poller 退干净——这是优雅停机的第一道闸。

四、SocketProcessor 的状态机与 keep-alive

`SocketProcessor` 是个 `Runnable`，`run()` 里根据传入的 `SocketEvent` 分派：

- `OPEN_READ`：`handler.process(socketWrapper, status)` 走 HTTP 解析 → 调 Servlet → 生成响应。
- `OPEN_WRITE`：之前写没写完，现在可写了，继续 `flush`。
- `STOP` / `ERROR` / `TIMEOUT` / `DISCONNECT`：收尾，关连接、`countDownConnection()` 释放 `LimitLatch` 额度、把 `NioChannel` 还回 `nioChannels` 池。

它的 `run()` 骨架长这样：

```
public void run() {
    NioChannel socket = socketWrapper.getSocket();
    SelectionKey key = socket.getIOChannel().keyFor(socket.getPoller().getSelector());
    if (handshake == 0) {                       // TLS 握手未完成，先握手
        SocketState state = SocketState.OPEN;
        if (event == null) event = SocketEvent.OPEN_READ;
        state = handler.process(socketWrapper, event);
        if (state == SocketState.CLOSED) {
            socketWrapper.close();              // 里面会 countDownConnection
        } else if (state == SocketState.OPEN) {
            // 关键：把 OP_READ 注册回去，走 PollerEvent 队列而不是直接 register
            socket.getPoller().add(socket, event == SocketEvent.OPEN_READ ? OP_REGISTER : 0);
        }
    }
}
```

注意 `SocketState.OPEN` 这个分支：它不是"连接开着"，而是"这次处理完了，还需要继续监听读"。**事件是在这里被加回去的**，和 `processKey` 里的 `unreg` 正好闭合。如果业务代码抛了未捕获异常，`handler.process` 返回 `CLOSED`，这条路径就断了，连接被关——所以"Tomcat 偶发连接被断"往往能在应用日志里找到对应的异常。

内部还有个 `Handler` 状态机（`AbstractProcessorLight`），用 `SocketState` 表达"这次处理的结果"：`OPEN_READ`（读完了，等下一次读）、`OPEN_WRITE`（还没写完，等可写）、`LONG`（Servlet 3.0 异步，先释放线程）、`CLOSED`（结束）。`SocketProcessor` 拿到 `SocketState` 后决定下一步：`OPEN_READ` 且 keep-alive 还有额度，就把 `OP_READ` 重新注册回 Poller，**worker 线程归还线程池**；`CLOSED` 就销毁。

keep-alive 的账是这样算的：`register()` 时 `ka.setKeepAliveLeft(maxKeepAliveRequests)`，默认 100。每处理完一个请求递减，减到 0 就强制关闭，避免一个连接被无限复用。同时还有 `keepAliveTimeout`（默认等于 `connectionTimeout`，20 秒），由 Poller 的 `timeout()` 每轮扫描时检查。

`timeout()` 用的是"最小堆 + 时间轮"的简化版：每个 `NioSocketWrapper` 有个 `lastRead` / `lastWrite` 时间戳，Poller 每轮只处理"到期的那些"，`nextExpiration` 记住下一次最早的到期时间，没到期就跳过整个扫描。这是为了避免每轮 `select` 都遍历上万个 key——那会让 `select` 的 O(1) 优势被扫描抵消掉。代价是精度只有 `selectorTimeout`（1 秒）级别，所以 `keepAliveTimeout` 设成 500ms 是没有意义的。

这里是 NIO Tomcat 和 BIO Tomcat 最本质的差别：BIO 模式下，一个 keep-alive 连接在两次请求之间**仍然占着一个 worker 线程**，线程卡在 `read` 上等下一个请求；NIO 模式下，两次请求之间这个连接只是 Poller 的 `Selector` 上的一个 key，不占任何 worker 线程。所以 200 个 worker 能服务上万个空闲长连接——这是"连接数"和"并发处理数"第一次被真正解耦。

写响应同理。内核发送缓冲区满的时候，`write` 返回 0，`SocketState` 变成 `OPEN_WRITE`，`OP_WRITE` 被注册回去，worker 线程立刻归还线程池。这就是为什么 NIO Tomcat 的 worker 线程几乎不会阻塞在 `write` 上——慢客户端（3G 网络、故意不收数据的攻击者）不会拖死线程池，只会占着一个 key 和一份 8K 缓冲区。BIO Tomcat 里这是致命的：200 个慢客户端就能把 200 个线程全占住，这叫"慢 HTTP 攻击"，是 Tomcat 7 时代最经典的拒绝服务手法。

不过要泼一盆冷水：Tomcat 的 NIO **不是全链路非阻塞**。Servlet 规范里 `request.getInputStream().read()` 是阻塞语义，当请求体没到齐时，Tomcat 会用 `NioBlockingSelector` + `selectorPool` 把当前线程挂起，等到数据到了再唤醒。所以读 body 的时候，worker 线程仍然会被占住。真正的全异步要用 `ReadListener` / `WriteListener`（Servlet 3.1）或直接上 WebFlux。理解这一点，才不会对"Tomcat 是非阻塞服务器"这句话产生错误预期。

容易误解的地方：

1. **`maxConnections` 不是 `maxThreads`**。前者限制同时存在的连接数（`LimitLatch`，10000），后者限制同时处理的请求数（200）。一个连接在 keep-alive 空闲期只占前者不占后者。把 `maxConnections` 调到 200 以下没有意义，因为瓶颈会先出现在 worker 上。
2. **`acceptCount` 是内核 backlog，不是 Tomcat 的队列**。它传给 `bind(addr, backlog)`，默认 100。连接溢出是两级的：先撞 `maxConnections`（`LimitLatch` 卡在 `accept` 之前，连接留在内核队列里，客户端"连上了但没响应"），再撞 backlog（内核直接拒，或按 `tcp_abort_on_overflow` 回 RST）。调大 `acceptCount` 只是让排队更长，不增加处理能力。
3. **`Poller` 数量默认是 2，不是 CPU 核数**。源码是 `Math.min(2, availableProcessors())`，64 核机器上也只有 2 个，因为瓶颈在投递逻辑而不在 `epoll_wait`。
4. **`maxThreads=200` 不一定真能用满 200 个线程**。标准 `ThreadPoolExecutor` 配无界队列时，`maximumPoolSize` 永远不生效，线程数会卡在 `corePoolSize`。Tomcat 为此重写了 `TaskQueue.offer()`：入队失败且当前线程数小于 `maxThreads` 时返回 `false`，逼线程池建新线程。所以自己仿写 Tomcat 的线程池参数而不改队列，`maxThreads` 就是个摆设。

记住这一句：Acceptor 只负责把连接从内核队列里捞出来，Poller 只负责告诉你谁准备好了，Worker 只负责这一次请求——三者之间靠两个队列解耦，任何一个队列满了，故障现象都不一样。

第22章　Netty 启动：Reactor 线程模型是怎么搭起来的

一句话：一个 EventLoop 独占一个线程和一个 Selector，每个 Channel 终身只属于其中一个。

为什么需要它：

先看两个现象。

第一个：用 JDK 原生 NIO 手写服务端，通常是一个 `while (true) { selector.select(); }` 的线程包打天下——accept、read、反序列化、查一次 Redis、再 write 回去，全在这一个线程里。本地测没问题，一上压测就露馅：某个 handler 里查库慢了 200ms，这一个 select 循环就停了 200ms，注册在同一个 Selector 上的其余几百个连接全部跟着延迟；对端发来的数据堆在 socket 接收缓冲区里，接收窗口被填满，对端开始重传，延迟进一步放大。你想改，就发现改不动——因为"谁负责等事件"和"谁负责跑逻辑"被焊死在了同一个循环里。

第二个：换成人人都懂的 `new Thread(() -> handle(socket))`。连接数上到五千，默认 1MB 的线程栈就是 5G 虚拟内存，上下文切换把 CPU 吃满，而每个线程 90% 的时间在阻塞等数据。

还有一层隐形成本：Channel 注册到 Selector 要自己写、`interestOps` 什么时候从 0 改成 `OP_READ` 要自己算、一次 `write` 只写出去一半要自己挂起剩余字节、半包要自己拼缓冲区。这些活儿和业务逻辑混在一起，代码很快就没法看了。

这一章要解决的是三件事：**谁负责等事件、谁负责跑逻辑、这两者怎么绑定**。Netty 的答案就是主从 Reactor：boss 只负责 accept，worker 负责所有读写，每个 Channel 从生到死绑定在唯一一个 EventLoop 上，事件在 EventLoop 线程里串行处理，业务逻辑要么快，要么滚出去。

核心结构：

概念	作用
`ServerBootstrap` / `Bootstrap`	启动器。只做配置收集：group、channel、handler、option、attr，最后触发 bind 或 connect
`NioEventLoopGroup`	线程池。持有 `EventExecutor[] children` 和一个 chooser，负责把 Channel 分配给某个 EventLoop
`NioEventLoop`	池中的一个"线程"。内含一个 Selector、一个 taskQueue、一个定时任务队列，以及一个惰性创建的 Thread
`NioServerSocketChannel`	包装 JDK `ServerSocketChannel`，兴趣事件 `OP_ACCEPT`，一个进程通常只有一个
`NioSocketChannel`	包装 JDK `SocketChannel`，兴趣事件 `OP_READ`，每条连接一个
`ServerBootstrapAcceptor`	挂在 boss 那条 pipeline 上的 Inbound handler，accept 到连接后配置它、装 handler、注册到 worker
`ChannelPipeline`	每个 Channel 一条，`head` + 业务 handler + `tail`，事件沿链表传播
`ChannelInitializer`	一次性 handler。register 完成后把自己展开、再把自身从 pipeline 摘掉
`Channel.Unsafe`	Channel 内部接口，`doRegister` / `doBind` / `doConnect` / `doRead` 这些真正碰 JDK 的动作都在这里
`ChannelPromise`	可写的 Future，异步结果的写入口：`setSuccess` / `setFailure`

关键代码：

启动阶段真正干活的是 `register`。它要回答一个问题：Channel 和 EventLoop 是在哪一行绑死的。

```
// AbstractChannel.AbstractUnsafe
public final void register(EventLoop eventLoop, final ChannelPromise promise) {
    // 关键点一：EventLoop 被钉在 Channel 的字段上。此后这个 Channel 的每一次
    // read / write / close，都只能提交给它，这是"一个 Channel 只属于一个线程"的物理落点
    AbstractChannel.this.eventLoop = eventLoop;

    if (eventLoop.inEventLoop()) {
        // 已经在 EventLoop 线程里，直接干，省掉一次任务入队
        register0(promise);
    } else {
        // 否则入队。注意：这里是 NioEventLoop 收到的第一个任务，
        // 线程就是在这一刻被创建出来的
        eventLoop.execute(() -> register0(promise));
    }
}

// AbstractNioChannel
protected void doRegister() throws Exception {
    // 关键点二：ops 传 0。注册进 Selector，但一个事件都不监听。
    // 因为此刻还不知道这个 Channel 将来关心 ACCEPT 还是 READ，
    // 真正的 interestOps 要等到 bind/connect 成功、channelActive 事件里才挂上
    selectionKey = javaChannel().register(eventLoop().unwrappedSelector(), 0, this);
}
```

EventLoop 自己的主循环只有三段，但顺序很讲究。

```
protected void run() {
    for (;;) {
        // 有任务排队就不能阻塞：走 selectNow() 空转一圈赶紧回来跑队列
        // 没任务才敢 select(timeout) 真正睡下去，wakeup() 负责把它叫醒
        switch (selectStrategy.calculateStrategy(selectNowSupplier, hasTasks())) {
            case SelectStrategy.CONTINUE:
                continue;
            case SelectStrategy.SELECT:
                select(wakenUp.getAndSet(false));
                if (wakenUp.get()) {       // 睡下去之前有人按过门铃，补一次唤醒
                    selector.wakeup();
                }
                break;
            default:
        }

        final long ioStartTime = System.nanoTime();
        try {
            processSelectedKeys();          // 处理就绪的 Channel，这是 IO
        } finally {
            // ioRatio 默认 50：IO 花了 T 纳秒，任务队列最多只能花 T*(100-50)/50。
            // 这个配额是硬约束，防止一个刷数据的批量任务把整个 IO 线程占死
            runAllTasks((System.nanoTime() - ioStartTime) * (100 - ioRatio) / ioRatio);
        }
    }
}
```

拆解：

一、两个 group 的分工：boss 只做 accept，但 accept 不便宜

`ServerBootstrap.group(bossGroup, workerGroup)` 里的两个 group 都是 `NioEventLoopGroup`，类型完全一样，区别只在**谁把 Channel 注册给它们**。

boss 的 pipeline 上只挂一个 `NioServerSocketChannel`，它只关心 `OP_ACCEPT`。worker 的 EventLoop 上挂着所有 `NioSocketChannel`，关心 `OP_READ`。这条分界线是 `ServerBootstrapAcceptor` 划出来的：

```
// ServerBootstrapAcceptor.channelRead
public void channelRead(ChannelHandlerContext ctx, Object msg) {
    final Channel child = (Channel) msg;
    child.pipeline().addLast(childHandler);   // 装业务 handler（通常是个 ChannelInitializer）
    setChannelOptions(child, childOptions, logger);  // SO_KEEPALIVE / TCP_NODELAY 之类
    // 关键：注册到 childGroup，不是当前这个 group
    childGroup.register(child).addListener(...);
}
```

注意这段代码跑在 **boss 线程**上。也就是说 boss 线程不只调了一个 `accept()`，它还执行了 `setChannelOptions`、往 child 的 pipeline 里 `addLast`、构造 `DefaultChannelPromise`。这些都是纯 CPU 工作，串行执行。如果你的 `childHandler` 是个重活儿（建连接池、加载证书、查配置中心），boss 就成了吞吐上限。

这里顺便把 `handler(...)` 和 `childHandler(...)` 分清楚，很多人第一次写会搞反：`handler(...)` 加在 **ServerSocketChannel 自己的 pipeline** 上，处理的是"服务端 Channel 自己的事"（比如 `channelActive` 时打印一行启动日志）；`childHandler(...)` 加在 **每一个 accept 出来的 SocketChannel 的 pipeline** 上，处理的是"每条连接的事"。前者一个进程只有一份，后者每条连接一份。`ServerBootstrapAcceptor` 是 Netty 自己往 boss 的 pipeline 上追加的，和 `handler(...)` 指定的 handler 并列，都在同一条链上——顺序是先 `handler(...)` 后 `ServerBootstrapAcceptor`。

再往上追一层，`init(channel)` 里那个 `p.addLast(new ChannelInitializer<Channel>(){...})` 是在给 **boss 的 pipeline** 装初始化器，它的 `initChannel` 里做两件事：把 `config.handler()`（即 `handler(...)` 指定的那个）加进来，然后 `ch.eventLoop().execute(...)` 里把 `ServerBootstrapAcceptor` 加进来。为什么要包一层 `execute`？因为 `ServerBootstrapAcceptor` 要引用 `currentChildGroup`、`currentChildHandler` 这些字段，而这些字段必须等 `group(...)`、`childHandler(...)` 都调用完才有值；`execute` 把它推迟到 EventLoop 跑起来之后，避开了"配置还没填完就开始 accept"的时序问题。

但**调大 bossGroup 的线程数通常没用**：一个 `ServerSocketChannel` 只能注册到一个 EventLoop 上，多出来的 boss 线程全程闲置。只有你在同一个 group 上 bind 多个端口时，多线程 boss 才有意义。生产上 `new NioEventLoopGroup(1)` 是常态，真遇到 accept 瓶颈，该优化的是 `childHandler`，不是线程数。

二、NioEventLoop 的三个部件，和"线程是惰性出生的"

`new NioEventLoopGroup()` 走完构造链，实际发生的事是：算出 `nThreads`（默认 `CPU 核数 * 2`），`new EventExecutor[nThreads]`，循环调用 `newChild()` 造出 N 个 `NioEventLoop`，每个里面 `openSelector()` 开一个 Selector，最后用 `chooserFactory` 造一个选择器。**到这里一个 Thread 都还没创建**。

三个部件各自的角色：

- **Selector**。`openSelector()` 里 Netty 干了件不太规矩的事：用反射把 JDK `SelectorImpl` 里的 `selectedKeys`（一个 `HashSet<SelectionKey>`）替换成自己实现的 `SelectedSelectionKeySet`，内部是两个 `SelectionKey[]` 数组加一个 boolean 做双缓冲。原因是 `HashSet.add` 有装箱、有链表遍历，而 `processSelectedKeys` 是每轮循环都要跑的热点。替换失败就退回原生实现——这就是为什么 `NioEventLoop` 同时持有 `selector` 和 `unwrappedSelector` 两个字段：注册和 select 用包装过的，某些需要原始语义的地方用未包装的。
- **taskQueue**。`LinkedBlockingQueue`，**默认容量只有 16**（`DEFAULT_MAX_PENDING_TASKS`）。`execute()` 往里塞，塞满就走 `RejectedExecutionHandler`，默认实现是直接 `throw new RejectedExecutionException()`。这个 16 是个非常重要的信号：Netty 从设计上就没打算让你把 EventLoop 当业务线程池用，它只是个"顺手捎带一点非 IO 任务"的通道。
- **scheduledTaskQueue**。懒创建的 `PriorityQueue<ScheduledFutureTask>`。`runAllTasks` 每轮开头调 `fetchFromScheduledTaskQueue()`，把到期的任务搬进 taskQueue 再统一执行。连接超时、心跳、`IdleStateHandler` 全挂在这上面。

这三个部件不是并列的，有明确的优先级：Selector 决定"这轮有没有事干"，taskQueue 决定"干完 IO 还能干多久"，scheduledTaskQueue 决定"最多能睡多久"。`select(oldWakenUp)` 的阻塞时间不是无限长——它会取出 `scheduledTaskQueue` 里最早到期任务的剩余纳秒数，和 `selectTimeout` 取较小值作为超时。否则一个 10ms 后该执行的心跳任务，得等到有 IO 事件才能被捡起来，等于把定时器交给了流量。

`wakeup` 那段也值得单独说。`wakeup(inEventLoop)` 的逻辑是：用 CAS 把 `wakenUp` 从 false 翻到 true，只有抢到的线程才真正调 `selector.wakeup()`。为什么要这么绕？因为 `Selector.wakeup()` 是昂贵的（涉及系统调用和一次 pipe 写入），而 `execute()` 在高频写入时每毫秒都会被调好几次。CAS 相当于一个合并开关：一批并发提交只唤醒一次。`select()` 返回后第一件事是把 `wakenUp` 读出来 `getAndSet(false)` 清掉，如果清完之后发现又被置成 true，说明有人在"进入 select 到真正阻塞"这个窗口里按过门铃，必须补一次 `selector.wakeup()`，否则这次唤醒会丢，任务要等到下一个 IO 事件才被执行。这个"丢失唤醒"的窗口是自旋锁式设计的经典坑，Netty 用一次额外的 `wakeup` 把它堵上了。

线程的出生点在 `SingleThreadEventExecutor.execute()`：

```
public void execute(Runnable task) {
    boolean inEventLoop = inEventLoop();   // Thread.currentThread() == this.thread
    addTask(task);                          // 先进队列
    if (!inEventLoop) {
        startThread();                      // 再决定要不要生线程
        if (isShutdown() && removeTask(task)) reject();
    }
    if (!addTaskWakesUp && wakesUpForTask(task)) {
        wakeup(inEventLoop);                // 叫醒可能正阻塞在 select 上的它
    }
}
```

`startThread()` 用 CAS 把状态从 `ST_NOT_STARTED` 推到 `ST_STARTED`，然后 `doStartThread()` 调 `executor.execute(...)`——这个 `executor` 是构造期一路传下来的 `ThreadPerTaskExecutor`，实现就是"来一个任务 new 一个 Thread"。新线程里第一件事是 `thread = Thread.currentThread()`，把自己的引用存回 EventLoop，然后调 `SingleThreadEventExecutor.this.run()`，从此再不返回。

所以顺序是：**new group（无线程）→ register 入队（创建线程）→ run 循环开始 → 执行 register0**。第一次 register 之前，这个 EventLoop 是死的。

三、绑定关系：N 个 Channel 对一个 EventLoop，一个 EventLoop 对一个线程

完整的关系是：`N Channels : 1 EventLoop : 1 Thread : 1 Selector`。N 通常是几百到几千。

选哪个 EventLoop 由 chooser 决定：

```
// 线程数是 2 的幂时用位运算，否则取模
executors[idx.getAndIncrement() & executors.length - 1]
```

`getAndIncrement` 是全局自增，所以是**轮询**，不是"挑最闲的"。这条规则带来一个后果：如果 `nThreads` 不是 2 的幂，`Math.abs(idx % len)` 在 `idx` 溢出成负数时行为会变，Netty 专门做了 `Math.abs`，但仍然是取模，比位与慢一点点。所以默认值取 `CPU*2` 这种常见 2 的幂是有意义的。

绑定是**单向且永久**的：Channel 一旦注册到某个 EventLoop，它的所有 IO 事件都在那个线程上处理，不会迁移。想在运行时把连接挪到别的线程，只能重新 register。

`inEventLoop()` 就是 `Thread.currentThread() == thread`。注意 `thread` 可能还是 null（未启动），此时返回 false，所以启动阶段的所有调用都会老老实实走 `execute()` 入队。

为什么业务逻辑不能放在 EventLoop 里跑，不是风格问题，是四个会同时发生的物理后果：

1. 这个 EventLoop 上所有 Channel 的读写全部停摆。你阻塞 200ms，几百条连接一起延迟 200ms。
2. Selector 不能及时 `select`，就绪事件在内核里排队，接收缓冲区填满，TCP 窗口收缩，对端超时重传。
3. taskQueue 只有 16 个槽。其他线程往这个 EventLoop 提交任务，很快撞上 `RejectedExecutionException`。
4. `scheduledTaskQueue` 里的心跳和超时任务同样被推迟，连接会被对端或自己误判为超时。

正确做法是把 handler 挂到独立线程池：`pipeline.addLast(businessGroup, new BizHandler())`。
`AbstractChannelHandlerContext.executor()` 在 handler 绑定了 `EventExecutorGroup` 时返回那个 group 里的一个 executor，`invokeChannelRead` 就会 `executor.execute(...)` 切过去。代价是跨线程，消息如果还要往后传，得先 `retain`。

反过来说，**串行是这个模型给你的礼物**。同一个 Channel 的所有事件都在一个线程上按序执行，意味着你写的 handler 天然不需要 `synchronized`：解码器可以放心持有"当前拼到哪了"的累积缓冲区，业务 handler 可以把用户会话状态存在自己的字段里，`channelRead` 和 `channelInactive` 之间不存在竞态。这是 Netty 相对"每事件丢线程池"模型最大的优势——后者要求每个 handler 都是线程安全的，实际上没人写对过。Netty 只要求你遵守一条：**一个 handler 实例如果被多个 Channel 共享（`@Sharable`），那它必须是线程安全的**；不加 `@Sharable` 的 handler 只能挂到一个 Channel 上，Netty 在 `checkMultiplicity` 里会拦住你。

还有一条硬规则：**在 EventLoop 线程里调 `future.sync()` 或 `await()` 会直接抛 `BlockingOperationException`**。因为你在等的是自己，等不到。Netty 在 `DefaultPromise.checkDeadLock()` 里做了这个检查，这个检查救过很多人。

四、一次 bind 的完整调用链，以及 interestOps 的三次变化

```
ServerBootstrap.bind(port)
└─ doBind → initAndRegister()
   ├─ channelFactory.newChannel()           // new ServerSocketChannel + configureBlocking(false)
   │                                         // 同时 newUnsafe() + newChannelPipeline()（head + tail）
   ├─ init(channel)                         // p.addLast(ChannelInitializer) → head + init + tail
   └─ config().group().register(channel)    // bossGroup
      └─ next().register(channel)           // chooser 选一个 NioEventLoop
         └─ new DefaultChannelPromise(channel, this)
            └─ channel.unsafe().register(this, promise)
               ├─ this.eventLoop = eventLoop          // 绑定
               └─ eventLoop.execute(register0)        // 第一个任务，触发 startThread
                  ├─ doRegister()                     // javaChannel().register(selector, 0, this)
                  ├─ invokeHandlerAddedIfNeeded()     // ChannelInitializer 展开，随后 remove 自己
                  ├─ safeSetSuccess(promise)          // 唤醒 main 线程的 sync()
                  └─ fireChannelRegistered()          // head → 各 inbound handler
└─ doBind0 → eventLoop.execute(() -> channel.bind(...))   // 复用同一个 EventLoop
   └─ pipeline.bind → tail.bind → … → head.bind
      └─ unsafe.bind → doBind → javaChannel().bind(localAddress, backlog)
         ├─ safeSetSuccess(promise)
         └─ if (!wasActive && isActive()) pipeline.fireChannelActive()
            └─ HeadContext.channelActive → ctx.fireChannelActive() + readIfIsAutoRead()
               └─ channel.read() → unsafe.beginRead() → doBeginRead()
                  └─ selectionKey.interestOps(interestOps | OP_ACCEPT)
```

三个状态要盯住：

interestOps：register 时是 `0`（什么都不监听）→ bind 成功后由 `channelActive` 事件里的自动 read 改成 `OP_ACCEPT`。客户端是 `0` → `doConnect` 里改成 `OP_CONNECT` → 连接完成后 `finishConnect` 触发 `channelActive`，再改成 `OP_READ`。**`bind()` 本身不设 interestOps**，这一点后面还会说。

pipeline：`head + tail` → `head + ChannelInitializer + tail` → `head + 业务 handler… + tail`。`ChannelInitializer` 的展开时机是 `invokeHandlerAddedIfNeeded()`，它调 `handlerAdded`，`handlerAdded` 里判断 `ctx.channel().isRegistered()`，成立才执行 `initChannel`，`finally` 里 `remove(ctx)` 把自己摘掉。所以初始化的前置条件是"已经注册到 EventLoop"，因为 handler 可能要在自己线程里跑。

线程：`new group` 时不存在 → `register` 入队时创建 → `run()` 里死循环直到 shutdown。

`doBind0` 里那个 `eventLoop.execute` 是必要的：如果 register 是异步的（提交进了队列），bind 必须排在同一队列的后面，才能保证"先注册、后绑定"。这也是 `initAndRegister` 返回值那段注释的意思——不管 register 是否已完成，后续的 connect/bind 都排在同一个 EventLoop 上，顺序天然成立。

容易误解的地方：

1. **"bossGroup 线程越多，accept 越快"**。一个 `ServerSocketChannel` 只能注册到一个 EventLoop，其余 boss 线程全程空转。`new NioEventLoopGroup(1)` 就是正确配置，除非你在同一个 group 上 bind 多个端口。真正的 accept 瓶颈在 `ServerBootstrapAcceptor.channelRead` 里那几行同步代码，不在线程数。
2. **"register 完成后 Channel 就能收数据了"**。register 时 `ops = 0`，一个事件都不监听。要等 bind/connect 成功、`fireChannelActive()` 走到 `HeadContext.channelActive`，它调 `readIfIsAutoRead()` 触发 `read()`，才在 `doBeginRead` 里挂上 `OP_ACCEPT` / `OP_READ`。中间任何一步失败，Channel 就是个注册了但不工作的空壳。
3. **"一个 Channel 一个线程"**。是一个 EventLoop 一个线程，一个 EventLoop 服务 N 个 Channel。`nThreads` 默认 `CPU*2` 指的是 EventLoop 个数，不是连接数。反过来说，一个 EventLoop 上某个连接处理慢，会拖累同组的其他所有连接——这才是"业务不能放 EventLoop"的真正原因。
4. **"在业务线程里直接 `ctx.write()` 不安全"**。它是安全的，`AbstractChannelHandlerContext.invokeWrite` 会检查 `inEventLoop()`，不在就 `executor.execute(...)` 切过去。真正的坑是另外两个：`write` 只是把消息放进 `ChannelOutboundBuffer`，不 `flush` 就永远不出去；以及大量跨线程写会让那个 EventLoop 的 taskQueue（容量 16）打满，抛 `RejectedExecutionException`。

记住这一句：Netty 的线程模型不是"很多线程一起干活"，而是"每个 Channel 只让一个线程干活"——把并发从共享内存里拿掉，换成确定性的串行。

第23章　Netty 内核：Pipeline、ByteBuf 与内存池

一句话：Pipeline 决定事件往哪走，ByteBuf 决定内存活多久，两者一起卡住 Netty 的性能上限。

为什么需要它：

两个线上现象。

第一个：服务跑了三天，RSS 涨到 8G，堆只用了 1.2G，`jmap -histo` 看不出任何异常，最后报 `java.lang.OutOfMemoryError: Direct buffer memory`。重启就好，三天后再来一次。这不是对象泄漏——堆外内存根本不在 GC 的账本上，`DirectByteBuffer` 那个 Java 对象早就被回收了，它引用的那块 native 内存还在，要等 `Cleaner` 被触发才还回去，而 `Cleaner` 又依赖一次真正的 GC。所以堆越健康，泄漏越隐蔽。

第二个：加了 `LengthFieldBasedFrameDecoder` 之后，偶发解析错位，日志里某条消息的 body 前面多出了上一次请求的尾巴。查了半天以为是半包处理有问题，其实是某个 handler 里 `readBytes` 之后又拿去 `write`，`readerIndex` 和 `writerIndex` 的语义被搞混了；更糟的情况是消息跨线程传递没 `retain`，原线程 `release` 之后这块内存被池子收回、分配给了别人，你读到的是别人的数据。

这一章要解决的是：**事件在一条链上怎么走才不会走反、一块内存归谁所有、什么时候还**。前两个问题由 Pipeline 和 Context 回答，后一个由引用计数和内存池回答。三者是一套东西——pipeline 决定了谁在什么时机碰这块内存，引用计数决定了碰完之后谁来还。

核心结构：

概念	作用
`ChannelPipeline`	每个 Channel 一条双向链表，串起 head、业务 handler、tail。本身不做业务，只负责事件的入口和出口
`ChannelHandlerContext`	链表节点。包着 handler，还存 `prev` / `next` / `executor` / `handlerState`，是事件传播的实际载体
`HeadContext`	链表头，同时是 inbound 和 outbound。所有 outbound 动作的终点，负责把调用交给 `Unsafe` 落到 JDK
`TailContext`	链表尾，只做 inbound。inbound 事件的终点，兜底 `release` 没人处理的消息并打 WARN
`ChannelHandler`	业务逻辑。`ChannelInboundHandler` 收事件，`ChannelOutboundHandler` 发事件，`ChannelDuplexHandler` 两边都收
`ByteBuf`	字节容器。`readerIndex` / `writerIndex` 双指针，`capacity` 可增长到 `maxCapacity`
`ByteBufAllocator`	分配器。默认 `PooledByteBufAllocator`，池化，堆内/堆外可选
`AbstractReferenceCountedByteBuf`	引用计数实现。`refCnt` 归零才 `deallocate()`，多减一次抛 `IllegalReferenceCountException`
`ResourceLeakDetector`	采样追踪忘记 `release` 的 buffer，默认 SIMPLE 级别，报告里带分配点栈
`PoolArena` / `PoolChunk` / `PoolPage`	池的三级结构：线程缓存 → Arena → Chunk(16MB) → Page(8KB) → Subpage

关键代码：

方向这件事，全部藏在两个只差一个字段的方法里。

```
// DefaultChannelPipeline
public final ChannelPipeline addLast(ChannelHandler... handlers) {
    for (ChannelHandler h : handlers) {
        // 注意：链表节点是 ctx，不是 handler。handler 只是 ctx 的一个字段
        AbstractChannelHandlerContext newCtx = newContext(null, h);
        addLast0(newCtx);              // 插到 tail 之前，而不是"追加到末尾"
        callHandlerAdded0(newCtx);     // 回调 handlerAdded，ChannelInitializer 在这里展开
    }
    return this;
}

// AbstractChannelHandlerContext —— 两个方向的查找，只差 next / prev
private AbstractChannelHandlerContext findContextInbound() {
    AbstractChannelHandlerContext ctx = this;
    do { ctx = ctx.next; } while (!ctx.inbound);   // 往后找，直到 tail
    return ctx;
}

private AbstractChannelHandlerContext findContextOutbound() {
    AbstractChannelHandlerContext ctx = this;
    do { ctx = ctx.prev; } while (!ctx.outbound);  // 往前找，直到 head
    return ctx;
}
```

`ByteBuf` 的全部语义就是两个游标，加一个引用计数。

```
// AbstractByteBuf
public ByteBuf writeBytes(byte[] src) {
    ensureWritable0(src.length);        // 不够就扩容：<4MB 翻倍，>=4MB 每次加 4MB，封顶 maxCapacity
    System.arraycopy(array, 0, array, writerIndex, src.length);
    writerIndex += src.length;          // 只动写指针，读指针不动
    return this;
}

public ByteBuf readBytes(byte[] dst) {
    checkReadableBytes0(dst.length);    // writerIndex - readerIndex < len 直接抛 IndexOutOfBounds
    System.arraycopy(array, readerIndex, dst, 0, dst.length);
    readerIndex += dst.length;          // 只动读指针，写指针不动
    return this;
}

// AbstractReferenceCountedByteBuf
public boolean release() {
    for (;;) {
        int oldRef = refCntUpdater.get(this);
        // 减到 0 之后再减，或者引用计数为负，都说明有人重复释放
        if (oldRef == 0 || oldRef == Integer.MIN_VALUE) {
            throw new IllegalReferenceCountException(oldRef);
        }
        if (refCntUpdater.compareAndSet(this, oldRef, oldRef - 1)) {
            if (oldRef == 1) {
                deallocate();           // 池化时是还给 PoolArena，非池化才是真的 free
                return true;
            }
            return false;
        }
    }
}
```

拆解：

一、双向链表与传播方向：outbound 从 tail 开始的三个理由

`DefaultChannelPipeline` 的构造只有四行有效代码：造 `tail`、造 `head`、`head.next = tail`、`tail.prev = head`。此后所有 `addLast` 都是往 `tail` 前面插，`addFirst` 是往 `head` 后面插。所以"添加顺序"等于"从 head 到 tail 的物理位置顺序"。

事件传播靠两个循环，方向完全相反：

- **inbound**（`channelRegistered` / `channelActive` / `channelRead` / `channelInactive` / `exceptionCaught` / `userEventTriggered`）从 `head` 出发，`ctx.fireXxx()` 里调 `findContextInbound()` 往后走，直到 `tail`。**按添加顺序执行**。
- **outbound**（`bind` / `connect` / `disconnect` / `close` / `read` / `write` / `flush`）从 `tail` 出发，`ctx.xxx()` 里调 `findContextOutbound()` 往前走，直到 `head`。**按添加顺序的逆序执行**。

所以下面这段在无数教程里出现的代码，顺序其实是对的：

```
pipeline.addLast(new LengthFieldBasedFrameDecoder(...));  // ① inbound
pipeline.addLast(new LengthFieldPrepender(4));            // ② outbound
pipeline.addLast(new BizHandler());                       // ③ inbound + outbound
```

读进来的字节走 ① → ③，写出去的字节走 ③ → ②。写代码的人心里想的"先解码、再处理、最后编码"，和这个物理排列完全吻合——因为读是正序、写是逆序，两者在 `BizHandler` 这个位置交汇。

为什么 outbound 必须从 tail 开始，有三个层次的原因，缺一个都解释不通：

1. **终点唯一**。只有 `HeadContext` 能碰到 `Unsafe`，`unsafe.write()` 才是真正把字节塞进 `ChannelOutboundBuffer` 的那一步。outbound 事件必须终止于 head，那么起点只能在 tail——从 tail 往 head 走，才能覆盖链上所有 outbound handler。
2. **语义一致**。inbound 是"从网络侧进入业务侧"，outbound 是"从业务侧发往网络侧"。head 贴着 socket，tail 贴着业务代码。两个方向都是从一端到另一端，只不过一个是 `next`，一个是 `prev`。
3. **解耦注册顺序和执行顺序**。逆序让"我在链上的位置"和"我什么时候被执行"变成两件事：越靠近 head 越贴近网络，越靠近 tail 越贴近业务。编码器放在业务 handler 前面（物理上靠近网络），但逻辑上在业务之后执行——这个错位正是逆序带来的，它让 pipeline 的物理布局可以按"数据形态"排列（字节 → 帧 → 对象），而不是按"时间顺序"排列。

这里要区分两个容易混淆的入口：

- `channel.write(msg)` → `tail.write()`，**走完整条链**，所有 outbound handler 都会经过。这是"从外部发起"的语义。
- `ctx.write(msg)` → 从**当前节点的 prev** 开始找，跳过自己及之后的所有节点。这是"我已经处理过了，剩下的交给外面"的语义。

在 `BizHandler` 里写 `ctx.writeAndFlush(resp)`，等价于"从 BizHandler 往前找 outbound"，会经过 ② 但不会经过 ③ 自己。如果写成 `ctx.channel().writeAndFlush(resp)`，就会从 tail 重新走一遍，包括那些本该被跳过的 handler——这是很多"消息被编码两次"bug 的根因。

二、ChannelHandlerContext：为什么 API 给你的不是 handler

pipeline 里的元素是 `ChannelHandlerContext`，不是 `ChannelHandler`。`addLast(handler)` 内部会 `newContext(null, handler)` 造一个 ctx 包住它。这个设计不是为了好看，是因为 handler 需要"知道自己站在哪"。

ctx 里存了四样东西，每一样都对应一个能力：

- `handler`：真正被回调的对象。
- `next` / `prev`：决定事件往哪传。
- `executor`：决定回调在哪个线程跑。
- `handlerState`：记录 `ADDED` / `REMOVED` 状态，配合 `invokeHandler()` 判断该不该回调。

`executor` 是惰性解析的：构造时是 null，第一次 `executor()` 被调用时才去问 handler 有没有绑定 `EventExecutorGroup`，没有就用 `channel.eventLoop()`。这就是 `addLast(group, handler)` 能把 handler 挪出 IO 线程的全部机制——不需要改任何 handler 代码，只改它在哪个 executor 上跑。

`handlerState` 解决的是另一个问题：`handlerAdded` 和 `handlerRemoved` 必须成对、且只在合适的时候调用。`callHandlerAdded0` 里如果发现 pipeline 还没注册完成，就把状态置为 `ADDED_EXCLUSIVE` 之类的中间态，等 `invokeHandlerAddedIfNeeded()` 统一补上。`ChannelInitializer` 就是靠这个时机展开的——它的 `handlerAdded` 里判断 `ctx.channel().isRegistered()`，成立才执行 `initChannel`，`finally` 里 `remove(ctx)` 把自己摘掉。所以你在 `initChannel` 里 `addLast` 新 handler 是安全的：新 handler 的 `handlerAdded` 会在同一个 `callHandlerAdded0` 调用栈里立刻执行，不会丢。

两个边界要记住：

- **`TailContext` 是最后一道兜底**。inbound 消息走到 tail 还没人消费，`onUnhandledInboundMessage` 会 `ReferenceCountUtil.release(msg)` 并打一条 `Discarded inbound message` 的 WARN。这条日志出现，说明你的 pipeline 少了一个终端 handler，或者某个 handler 忘了 `ctx.fireChannelRead`。
- **`HeadContext` 是所有 outbound 的最后一站**。`head.write` 调 `unsafe.write`，`head.read` 调 `unsafe.beginRead`，`head.bind` 调 `unsafe.bind`。它同时实现了 `ChannelInboundHandler` 和 `ChannelOutboundHandler`，所以它在两个方向上都算"节点"，`findContextInbound` 和 `findContextOutbound` 的循环都能在它身上停下来。

三、ByteBuf 的双指针，和三种内存的取舍

`ByteBuf` 有三个下标：`readerIndex`（下一个可读位置）、`writerIndex`（下一个可写位置）、`capacity`（当前容量）。所有操作都只动其中一个：



- `readableBytes() = writerIndex - readerIndex`，为 0 时 `readByte()` 抛 `IndexOutOfBoundsException`。
- `writableBytes() = capacity - writerIndex`，为 0 时 `writeByte()` 触发 `ensureWritable0` 扩容。
- `discardReadBytes()` 把 `[readerIndex, writerIndex)` 搬到 0 位置，然后 `readerIndex = 0`、`writerIndex -= 旧readerIndex`。它不释放 `capacity`，只是把已读空间腾出来复用。`compact()` 做同样的搬移，但语义是"我还没读完，但我要开始写了"——搬完 `readerIndex = 0`，可读数据保留。
- 三个 `slice` / `duplicate` / `retainedSlice` 都是**共享底层内存、独立指针**。`slice` 不 retain（引用计数还是原来那个），`retainedSlice` 会 `retain()` 一次，`duplicate` 共享整个区域。用 `slice` 之后忘记 `retain` 就跨线程传，是典型的 use-after-free。

三种内存的差别不在 API，在**一次 socket write 要拷几次**：

- **堆内（`UnpooledHeapByteBuf` / `PooledHeapByteBuf`）**。`array()` 直接可用，分配快，受 GC 管理，CPU cache 友好。但 `SocketChannel.write(ByteBuffer)` 只接受直接缓冲区，JDK 内部要么用 thread-local 的临时 `DirectByteBuffer` 拷一次，要么走 `IOUtil` 的临时缓冲。**一次用户态拷贝跑不掉**。
- **堆外（`UnpooledDirectByteBuf` / `PooledDirectByteBuf`）**。`array()` 抛 `UnsupportedOperationException`，但可以直接交给 `SocketChannel`，由内核 DMA 读走，零拷贝。代价是分配走 `malloc` / `free`（非池化时），比堆内慢一个数量级；而且不受 GC 直接管理，`DirectByteBuffer` 对象被回收后靠 `Cleaner` 释放，`Cleaner` 又要等一次 GC。
- **复合（`CompositeByteBuf`）**。把多个 `ByteBuf` 逻辑拼成一块，`addComponent(true, buf)` 里的 `true` 是让 `writerIndex` 跟着往后扩。所有权上，component 一旦加进来就归 `CompositeByteBuf` 管，它 `release` 时会挨个释放子 buffer，原持有者不能再碰。典型场景是"协议头 + 消息体"分开构造，用 `CompositeByteBuf` 拼起来发，省掉一次整包拷贝。

生产上的默认选择是 `PooledByteBufAllocator.DEFAULT` 分配的**池化堆外** buffer。池化把 `malloc` 的成本摊掉了，堆外省掉了那次拷贝。这也解释了一个常见困惑：为什么 `-Dio.netty.noPreferDirect=true` 之后吞吐掉一截——不是因为堆内慢，是因为多了一次拷贝。

四、引用计数、泄漏检测，以及池的三级结构

引用计数管的是"这块内存什么时候能还"，不是"这个对象什么时候被 GC"。这两件事经常被混为一谈。`ByteBuf` 的 Java 对象可能早就不可达了，但只要 `refCnt > 0`，池里那块 Page 就一直被标记为占用，不会被分配给任何人。所以"忘记 release"在池化下的表现不是内存不释放，而是**池慢慢退化**：可用 Page 越来越少，新分配只能往更大的 Chunk 上找或者走 huge 路径直接 `malloc`，RSS 一路涨，堆却风平浪静。

所有权规则只有一条：**谁创建/谁 retain，谁负责 release**。落到 pipeline 上有三个固定点：

- 入站消息的所有者是 pipeline。每个 handler 处理完要么 `ctx.fireChannelRead(msg)` 往下传（所有权转移），要么自己 `release`。传到 `tail` 还没人管，`TailContext` 兜底 release。
- `SimpleChannelInboundHandler` 在 `channelRead` 外面包了 `try/finally { ReferenceCountUtil.release(msg); }`，所以它自动释放。**但这也意味着它不能把 `msg` 传给下一个 handler**——传了就是 use-after-free。
- 出站消息的所有者是 `ChannelOutboundBuffer`。`unsafe.write` 只是入队，`flush` 时 `doWrite` 真正写 socket，写完一批在 `remove()` 里 release。所以"写完就 release"是错的，得等 flush 完成。

跨线程传递必须 `retain()`。业务线程池里处理完要回写，先把 `msg` 交出去之前 `retain` 一次，业务线程处理完 `release` 一次，两次引用各自归零，池才回收。少一次 `retain`，原线程 release 后这块内存立刻可被复用，你手上是个悬垂指针。

泄漏检测是兜底，不是解决方案。`ResourceLeakDetector` 用 `PhantomReference` + `ReferenceQueue` 追踪每个被包装的 buffer，如果对象被 GC 时 `refCnt` 还不为 0，就判定泄漏并打印**分配点的栈**。四个级别：

- `DISABLED`：采样率 0，生产环境追求极致性能时用。
- `SIMPLE`：默认级别，约 1% 采样，能抓到大部分泄漏。
- `ADVANCED`：同样 1% 采样，但额外记录最近若干次访问位置，能定位到"谁拿过它"。
- `PARANOID`：100% 采样，只在压测和预发用，性能掉得厉害。

打开方式是 `-Dio.netty.leakDetection.level=paranoid` 或 `ResourceLeakDetector.setLevel()`。报告长这样：`LEAK: ByteBuf.release() was not called before it's garbage-collected`，后面跟着创建时的完整栈。看到这条，别急着加 `release`，先确认这块 buffer 的所有权在哪个 handler

断掉了。

内存池的分层，理解成"三级缓存 + 一个伙伴系统"就够了：

```
PooledByteBufAllocator
 ├─ HeapArena[N]      N = 2 * CPU 核数
 └─ DirectArena[N]
     └─ PoolArena
         ├─ PoolThreadCache（每线程一份，无锁）
         │   ├─ tinyCache[32]    < 512B
         │   ├─ smallCache[4]    512B ~ 8KB
         │   └─ normalCache[64]  8KB ~ 16MB
         ├─ tinySubpagePools[32]  按 16 字节步进切分
         ├─ smallSubpagePools[4]  512 / 1K / 2K / 4K
         └─ PoolChunkList: qInit → q000 → q025 → q050 → q075 → q100
             └─ PoolChunk (16MB = 8KB Page << 11)
                 └─ PoolPage (8KB)
                     └─ Subpage（Page 内再切给 tiny/small）
```

一次 `allocate(size)` 的路径：

1. 算 `normCapacity`（tiny 按 16B 对齐，small 按 2 的幂，normal 按 Page 对齐）。`> 16MB` 直接走 huge，`new UnpooledDirectByteBuf`，不池化。
2. 先查当前线程的 `PoolThreadCache` 对应 size class 的队列，命中就 `poll` 出来直接用，**全程无锁**。
3. 未命中，进 `PoolArena`。tiny/small 从 `tinySubpagePools` / `smallSubpagePools` 找有空位的 Subpage；normal 从 `PoolChunkList` 里按使用率挑一个 Chunk，用**完全二叉树 + 伙伴算法**分配：`memoryMap` 数组记录每个节点的状态，从根往下找第一个能容纳请求的节点，分配后把整条路径标记为不可用，释放时沿路径向上合并。
4. 分配到的 Page 如果还没被切过，就按需切成 Subpage 交给 subpage pool 复用。

`PoolChunkList` 那六个队列按使用率分桶（0%、1-25%、25-50%、50-75%、75-100%、100%），分配时优先从"半满"的队列找——太空的 Chunk 浪费空间，太满的 Chunk 容易分配失败。这是一个很朴素的 best-fit 近似，但避免了遍历所有 Chunk。

`PoolThreadCache` 是性能的关键。没有它，每次分配都要 CAS 抢 Arena 的锁，`2 * CPU` 个 Arena 也扛不住高并发小对象分配。有了它，绝大多数分配释放都在线程本地完成，Arena 只在缓存未命中时才被碰到。缓存容量是有限的（tiny 512 个、small 256 个、normal 64 个），超了就往 Arena 还——这个上限很重要，否则一个突发大流量的线程会把整个池子搬空。

容易误解的地方：

1. **"`addLast` 的顺序就是执行顺序"**。inbound 正序，outbound 逆序。同一份 pipeline，`channelRead` 从 head 往 tail 走，`write` 从 tail 往 head 走。看到 `addLast(decoder)` 后面紧跟 `addLast(encoder)` 不要觉得写反了，它们根本不在一趟车上。
2. **"继承了 `SimpleChannelInboundHandler` 就自动释放，所以可以随便往后传"**。它的 `finally` 里无条件 `release`，你 `fireChannelRead(msg)` 传下去，下一个 handler 拿到的就是一块已经被归还的内存。池化时会读到别人复用的脏数据——这正是"body 前面多出上一次请求尾巴"的典型成因。要往下传就用 `ChannelInboundHandlerAdapter` 自己管，或者先 `retain` 再传。
3. **"`DirectByteBuf` 一定比 `HeapByteBuf` 快"**。省的是那次用户态拷贝，付出的是分配成本和 GC 不可见。小包、短生命周期、非池化的情况下，堆外反而更慢。只有在**池化 + 大流量 + 直接对接 socket** 三个条件同时成立时，堆外才明显占优。
4. **"`retain()` 是为了防止对象被 GC"**。引用计数管的是池内内存的归还时机，和 GC 可达性无关。`refCnt` 归零只代表这块内存可以还给 Arena 了；对象本身该被 GC 还是会被 GC。反过来说，对象活着但 `refCnt = 0`，你再去读就是读到已被复用的内存。同一类误解还有 `slice()`：它共享底层数组，只新建一组独立下标，**不增加引用计数**——原 buffer `release` 之后 `slice` 跟着失效，要独立持有得用 `retainedSlice()`。

记住这一句：pipeline 用方向解决了"谁来处理"，引用计数用所有权解决了"谁来释放"——Netty 的性能不是快在某个算法，是快在它从不让同一块内存被两个人同时拥有。

第24章　IOC 容器：refresh() 的十二步

一句话：refresh() 的十二步是一条因果链，每一步都在为下一步准备它自己造不出来的前提。

为什么需要它：

先看两个现场。

第一个：你写了一个 `BeanPostProcessor`，构造函数里 `@Autowired` 了一个 `UserService`。启动不报错，只是日志里多出一行 `Bean 'userService' of type [...] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying)`。然后你发现 `UserService` 里的 `@PostConstruct` 没执行，`@Async` 方法变成了同步调用，`@Transactional` 也不回滚了。你没改任何业务代码，只是"加了个 BPP"。

第二个：三个 Service 互相 `@Autowired`，应用能启动。你把其中一个字段注入改成构造器注入，立刻 `BeanCurrentlyInCreationException`。改回去，好了。再给其中一个类加个 `@Async`，又炸了，这次报的是 `Bean with name 'a' has been injected into other beans [b] in its raw version as part of a circular reference, but has eventually been wrapped`。

这两个现场看起来毫不相干，其实是同一张时间表上的两个坐标：**谁在什么时刻被创建，被哪些处理器处理过**。前者是"创建得太早"，后者是"被别人拿走的太早"。refresh() 的十二步就是这张时间表本身，它规定了每个扩展点的执行位置，而 Spring 里几乎所有的"玄学问题"都是某个对象偏离了它该在的位置。

这一章要解决的不是"背下十二个方法名"，而是：**每一步为什么必须在这个位置，往前挪或往后挪会破坏什么**。

核心结构：

步骤	做什么	关键类
1 `prepareRefresh`	记启动时间、置 active/closed、初始化占位符来源、校验必需属性、创建 earlyApplicationEvents	`AbstractApplicationContext`
2 `obtainFreshBeanFactory`	销毁旧工厂，新建 `DefaultListableBeanFactory`，把配置解析成 `BeanDefinition` 并注册进 `beanDefinitionMap`	`AbstractRefreshableApplicationContext`、`XmlBeanDefinitionReader`
3 `prepareBeanFactory`	配类加载器与表达式解析器，加 `ApplicationContextAwareProcessor`，登记可解析依赖与忽略接口，手动注册 `environment` 等单例	`AbstractApplicationContext`
4 `postProcessBeanFactory`	子类钩子。Web 容器在这里加 `ServletContextAwareProcessor`、注册 request/session scope	各子类覆写
5 `invokeBeanFactoryPostProcessors`	调用所有 BFPP。`ConfigurationClassPostProcessor` 解析 `@Configuration/@ComponentScan/@Import/@Bean`，占位符配置器替换 `${}`	`PostProcessorRegistrationDelegate`、`ConfigurationClassPostProcessor`
6 `registerBeanPostProcessors`	实例化并按三档排序注册所有 BPP，决定 `@Autowired`、`@PostConstruct`、AOP 代理是否生效	`PostProcessorRegistrationDelegate`
7 `initMessageSource`	注册 `MessageSource`，没有就塞一个空的 `DelegatingMessageSource` 兜底	`DelegatingMessageSource`
8 `initApplicationEventMulticaster`	注册事件广播器，默认同步的 `SimpleApplicationEventMulticaster`	`SimpleApplicationEventMulticaster`
9 `onRefresh`	子类钩子，在单例初始化之前建容器自身需要的基础设施。Boot 的内嵌 Tomcat 在这里起	`ServletWebServerApplicationContext`
10 `registerListeners`	把 `ApplicationListener` 类型的 bean 加进广播器，补发第 1 步攒下的早期事件	`AbstractApplicationContext`
11 `finishBeanFactoryInitialization`	冻结配置，`preInstantiateSingletons()` 实例化所有非懒加载单例	`DefaultListableBeanFactory`
12 `finishRefresh`	启动 `Lifecycle` / `SmartLifecycle`，发布 `ContextRefreshedEvent`	`DefaultLifecycleProcessor`

关键代码：

先看骨架。十二行调用，注释写的是"为什么在这一行"。

```
public void refresh() throws BeansException, IllegalStateException {
    // 整个启动过程串行化：refresh 还没结束，不许另一个线程 close
    synchronized (this.startupShutdownMonitor) {
        prepareRefresh();                                                    // 1  环境与状态
        ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();  // 2  解析 → BeanDefinition
        prepareBeanFactory(beanFactory);                                     // 3  给工厂装基础设施
        try {
            postProcessBeanFactory(beanFactory);          // 4  子类钩子，此时还能往工厂里加 BFPP
            invokeBeanFactoryPostProcessors(beanFactory); // 5  改"定义"：@Configuration 在这一步才生效
            registerBeanPostProcessors(beanFactory);      // 6  改"实例"的能力在这一步装好
            initMessageSource();                          // 7
            initApplicationEventMulticaster();            // 8  广播器建好之后，事件才有地方去
            onRefresh();                                  // 9  子类钩子：起内嵌容器
            registerListeners();                          // 10 补发 5~8 之间攒下的早期事件
            finishBeanFactoryInitialization(beanFactory); // 11 全流程唯一的"批量创建"发生在这里
            finishRefresh();                              // 12 Lifecycle + ContextRefreshedEvent
        } catch (BeansException ex) {
            destroyBeans();    // 已建好的单例必须销毁，否则连接池、线程池全部泄漏
            cancelRefresh(ex); // 复位 active，否则这个 context 永久不可用
            throw ex;
        } finally {
            resetCommonCaches(); // 清反射元数据缓存，启动期用完就不再需要
        }
    }
}
```

`doCreateBean` 是第 11 步真正干活的地方，它的四段结构决定了所有生命周期回调的先后。

```
protected Object doCreateBean(String beanName, RootBeanDefinition mbd, Object[] args) {
    // ① 实例化：此刻对象里所有字段都是默认值，@Autowired 一个都还没注入
    BeanWrapper instanceWrapper = createBeanInstance(beanName, mbd, args);
    final Object bean = instanceWrapper.getWrappedInstance();

    // ② 提前暴露：必须在 populateBean 之前挂出去。因为 populateBean 里会 getBean(依赖)，
    //    循环依赖只有在这一行执行完之后才有解
    if (mbd.isSingleton() && this.allowCircularReferences && isSingletonCurrentlyInCreation(beanName)) {
        addSingletonFactory(beanName, () -> getEarlyBeanReference(beanName, mbd, bean));
    }

    // ③ 属性填充：@Autowired / @Value 由 AutowiredAnnotationBeanPostProcessor 在这里写入
    populateBean(beanName, mbd, instanceWrapper);

    // ④ 初始化：Aware → before-init → afterPropertiesSet/init-method → after-init（代理在这里生成）
    Object exposedObject = initializeBean(beanName, bean, mbd);

    registerDisposableBeanIfNecessary(beanName, bean, mbd); // 登记销毁回调
    return exposedObject;
}
```

三级缓存的查询逻辑只有十几行，但它解释了"为什么有的循环依赖能解、有的不能"。

```
protected Object getSingleton(String beanName, boolean allowEarlyReference) {
    Object singletonObject = this.singletonObjects.get(beanName);          // 一级：成品
    if (singletonObject == null && isSingletonCurrentlyInCreation(beanName)) {
        synchronized (this.singletonObjects) {
            singletonObject = this.earlySingletonObjects.get(beanName);    // 二级：半成品，可能已是代理
            if (singletonObject == null && allowEarlyReference) {
                ObjectFactory<?> factory = this.singletonFactories.get(beanName); // 三级：能造半成品的工厂
                if (factory != null) {
                    singletonObject = factory.getObject();                 // 到这一刻才决定要不要提前生成代理
                    this.earlySingletonObjects.put(beanName, singletonObject);
                    this.singletonFactories.remove(beanName);              // 只算一次，之后走二级
                }
            }
        }
    }
    return singletonObject;
}
```

拆解：

前四步：准备一个干净的 BeanFactory

第 1 步 `prepareRefresh` 全是小动作，但有一个字段很能说明问题：`earlyApplicationEvents = new LinkedHashSet<>()`。为什么需要一个"提前事件"集合？因为第 5 步执行 BFPP 时，代码里可能 `publishEvent`，而广播器要到第 8 步才建。这个集合就是为了跨越第 5 到第 8 步之间的空窗期，等第 10 步再补发。一个字段的存在，暴露了整条链的时序约束。

同一步里 `validateRequiredProperties()` 发生在解析之前，所以"配置文件少了一个必需项"会在启动第一秒就失败，而不是等某个 bean 用到才 NPE。

第 2 步 `obtainFreshBeanFactory` 要先澄清一个概念：**`ApplicationContext` 不是 `BeanFactory` 的实现类，它持有一个 `BeanFactory`**。所有 getBean 请求最后都委托给内部那个 `DefaultListableBeanFactory`。为什么是它？因为 `ConfigurableListableBeanFactory` 这个接口只有一个实现类，而这个实现类又把 `AutowireCapableBeanFactory` 那条分支全吃下来了，它是功能最全的一个。

`refreshBeanFactory()` 的第一件事是 `if (hasBeanFactory()) { destroyBeans(); closeBeanFactory(); }`。这就是方法名叫 refresh 而不是 init 的原因：它可以重入，重入前必须把旧容器彻底弄死，否则旧单例还挂在旧工厂里，资源不会释放。

这一步的产出物不是对象，是 `BeanDefinition`。`loadBeanDefinitions` 一路走下去，终点是 `beanDefinitionMap.put(beanName, bd)` 和 `beanDefinitionNames.add(beanName)`。跑完这一步，容器里**一个业务对象都没有**，只有一堆描述"将来怎么造对象"的元数据。这个认知很重要：后面第 5 步改的是元数据，第 11 步才按元数据造对象。

解析链本身是 `XmlBeanDefinitionReader` → `DefaultBeanDefinitionDocumentReader` → `BeanDefinitionParserDelegate`，中间没有魔法，就是 DOM 遍历。真正值得看的是它的**分叉点**：`parseBeanDefinitions` 判断每个节点的 namespace，默认 namespace（`http://www.springframework.org/schema/beans`）下的 `<bean/>`、`<import/>`、`<alias/>`、`<beans/>` 走内置解析，其余走 `parseCustomElement`，交给 `NamespaceHandler` 处理。

这个分叉解释了一件事：`<context:annotation-config/>` 不是"开个开关"，它是一个 XML 标签，由 `ContextNamespaceHandler` 派给 `AnnotationConfigBeanDefinitionParser`，后者在**解析期**就往 registry 里注册了 `AutowiredAnnotationBeanPostProcessor`、`CommonAnnotationBeanPostProcessor`、`EventListenerMethodProcessor` 等一批 `BeanDefinition`。也就是说，BPP 的定义可能在第二步就存在了，只是到第六步才被实例化。同理，`<aop:aspectj-autoproxy/>` 对应 `AopNamespaceHandler`，`<task:annotation-driven/>` 对应 `TaskNamespaceHandler` ——看到不认识的标签，先找 `XxxNamespaceHandler`，答案都在里面。

`customizeBeanFactory` 在这里设两个开关，它们会在第 11 步再次出现：`allowBeanDefinitionOverriding`（默认 null，表现为同文件重复报错、跨文件静默覆盖；Spring Boot 2.1 起改成 false，这是升级时一类典型启动失败）和 `allowCircularReferences`（默认 true）。循环依赖能不能解，一半由这个开关决定。

第 3 步 `prepareBeanFactory` 给工厂装上"运行时能力"，其中三行值得记住：

- `addBeanPostProcessor(new ApplicationContextAwareProcessor(this))` —— 你实现的 `ApplicationContextAware`、`EnvironmentAware`、`ResourceLoaderAware` 等回调，实际执行者是这个处理器，不是 Spring 内核硬编码的。
- `ignoreDependencyInterface(...)` 六个 Aware 接口 —— 不加这行，按类型装配时会去找一个 `ApplicationContext` 类型的 bean 来调 `setApplicationContext`，然后报找不到。
- `registerResolvableDependency(ApplicationContext.class, this)` —— 这是 `@Autowired ApplicationContext ctx` 能工作的原因。它根本不是 bean，是"可解析依赖"，绕过了 bean 查找。

第 4 步 `postProcessBeanFactory` 是个空方法。它的位置很讲究：在第 3 步之后（工厂已配好），在第 5 步之前（还能往工厂里加 BFPP）。Web 容器就是在这里 `addBeanPostProcessor(new ServletContextAwareProcessor(...))` 并注册 request/session scope 的——所以你在 Controller 里能

`@Autowired HttpServletRequest`，靠的是第 4 步，不是第 11 步。

第五步：BeanFactoryPostProcessor 与配置类的解析

BFPP 是唯一能在"定义期"改定义的扩展点，它的入参是 `ConfigurableListableBeanFactory`，不是 bean 实例。这是它和 BPP 最本质的区别：**BFPP 看不见对象，因为它跑的时候还没有对象**。

`invokeBeanFactoryPostProcessors` 分两轮。第一轮只跑 `BeanDefinitionRegistryPostProcessor`（`BeanFactoryPostProcessor` 的子接口，多了 `postProcessBeanDefinitionRegistry`，能往 registry 里塞新定义），内部再按 `PriorityOrdered` → `Ordered` → 无序三档排。第二轮才跑剩下的普通 BFPP，同样三档。顺序的理由很直白：registry 那批负责**生产**定义，普通 BFPP 负责**修改**定义，生产者必须先于修改者。

`ConfigurationClassPostProcessor` 是 registry 那批里的头号选手（`PriorityOrdered`，order 接近最高）。它做的事：解析 `@Configuration`、`@ComponentScan`、`@Import`、`@ImportResource`、`@Bean`、`@PropertySource`，把结果变成新的 `BeanDefinition` 注册进去，最后 `enhanceConfigurationClasses` 用 CGLIB 给 full 模式的 `@Configuration` 生成子类。为什么要生成子类？因为只有这样，`@Bean` 方法之间互相调用（`aPerson()` 里直接调 `carFactoryBean()`）才会被拦截、转成 `getBean`，而不是真的 new 出一个新对象。`proxyBeanMethods = false` 关掉的就是这个增强，代价是方法间调用会退化成普通 Java 调用。

`@Import` 有三种形态，用途完全不同：

形态	行为	典型使用者
普通类	当成配置类继续解析	`@Import(DataSourceConfig.class)`
`ImportSelector`	返回类名数组，由容器再解析	`@EnableAutoConfiguration` 的 `AutoConfigurationImportSelector`
`ImportBeanDefinitionRegistrar`	拿到 registry 自己 register	`@MapperScan`、`@EnableAspectJAutoProxy`

`DeferredImportSelector`（`ImportSelector` 的变体）会被推迟到所有配置类解析完之后再执行，这就是"用户配置优先于自动配置"的物理落点——自动配置类最后进场，`@ConditionalOnMissingBean` 才有东西可判断。

这一步结束时，容器里"该有的定义都有了"，但仍然没有对象。漏了会怎样：`@Bean` 不生效报 `NoSuchBeanDefinitionException`；`@ComponentScan` 不扫导致整包 bean 消失；`${}` 不替换导致数据源 URL 变成字面量 `jdbc:${db.url}`，直到第一次拿连接才炸。

`${}` 的替换值得单独说，因为 Spring 里有**两套完全不同的占位符解析路径**，混用时会出很怪的问题。

第一套在**定义期**：`PropertySourcesPlaceholderConfigurer` 是个 BFPP，它遍历所有 `BeanDefinition` 的 `PropertyValues`，把字符串里的 `${key}` 就地替换成真实值，然后把改好的定义写回去。所以 XML 里 `<property name="url" value="${db.url}"/>` 在第五步结束时就已经是最终字符串了，运行时看不到任何 `${}`。

第二套在**实例期**：`@Value("${db.url}")` 走的是 `AutowiredAnnotationBeanPostProcessor`，它在 `postProcessPropertyValues` 里调用 `beanFactory.resolveEmbeddedValue(...)`，靠的是 `embeddedValueResolver`——如果前面没人注册过，第 11 步 `finishBeanFactoryInitialization` 会兜底塞一个。

两条路径的交汇点就是那个 resolver 到底谁注册的。`PropertySourcesPlaceholderConfigurer` 在 `doProcessProperties` 里既替换定义中的字符串、又顺手 `addEmbeddedValueResolver`，所以两套都能用；而更老的 `PropertyPlaceholderConfigurer` 只做前半件事，于是出现那个经典现象：**XML 里的 `${}` 正常替换，`@Value("${...}")` 却报 `Could not resolve placeholder`**。`<context:property-placeholder/>` 注册的是前者，这就是它"看起来什么都能干"的原因。排查这类问题时，先确认容器里到底是哪个 configurer。

第六步：BeanPostProcessor 的注册顺序

一句话区分：BFPP 改定义，每个容器跑一次；BPP 改对象，每个 bean 都要过一遍。

`registerBeanPostProcessors` 有个容易被忽略的动作：它会把所有 BPP 类型的 bean **立刻 `getBean` 出来**，不是登记个名字了事。副作用就是开头那个现场——BPP 依赖的 bean 在这时被创建，而它们的创建发生在其余 BPP 注册完成之前，所以它们只能被"已经注册的那部分 BPP"处理。日志里那句 `not eligible for auto-proxying` 说的就是这件事。所以：**不要在 BPP 里 `@Autowired` 一个需要 `@Async` / `@Transactional` / `@PostConstruct` 的业务 bean**。它会被提前造出来，所有后置处理全部缺席，而且不报错。

排序规则是三档：`PriorityOrdered` → `Ordered` → 无序（`Ordered` 内部按 `getOrder()` 升序）。三档跑完还有一次重排：`MergedBeanDefinitionPostProcessor` 一类被挪到队尾（5.x 里 `ApplicationListenerDetector` 还会被单独再挪一次）。这个重排是给"实例化后立刻收集元数据"的处理器留的位置，属于实现细节，知道有这么一次挪动即可。

落到你会遇到的三个处理器上，它们的触发时机分属三个阶段，不要混为一谈：

- `AutowiredAnnotationBeanPostProcessor`：处理 `@Autowired` / `@Value`，在 `populateBean` 的 `postProcessPropertyValues` 阶段，也就是**属性填充**。

- `CommonAnnotationBeanPostProcessor`：处理 `@PostConstruct` / `@PreDestroy` / `@Resource`，在 `initializeBean` 的 before/after 阶段，也就是**初始化**。
- `AbstractAutoProxyCreator`：生成 AOP 代理，在 `initializeBean` 的 `postProcessAfterInitialization`，也就是**初始化之后**。

这就是 `@PostConstruct` 里能拿到注入对象、而构造函数里拿不到的原因：注入发生在属性填充，`@PostConstruct` 发生在它之后。也是 `@Async` 自调用失效的原因：代理在初始化之后才包上，类内部调用走的是 `this`，压根没经过代理。

第十一步之一：getBean 的三级缓存与循环依赖

先分清三个 `getSingleton` 重载，这是最容易看晕的地方：

- `getSingleton(String)`：纯查询，只看一级缓存。
- `getSingleton(String, boolean allowEarlyReference)`：查询，允许穿透到二、三级。**循环依赖靠它**。
- `getSingleton(String, ObjectFactory)`：创建并注册。`doGetBean` 调的是这个，它先查一级缓存，再 `beforeSingletonCreation` 打标记，然后调 `factory.getObject()`（内部就是 `createBean`），最后 `addSingleton` 放进一级缓存并清掉二、三级。

`doGetBean` 的完整路径值得走一遍，因为绝大多数"为什么又创建了一次"的疑问都在这里：

```
String beanName = transformedBeanName(name);   // 去掉 "&" 前缀、把别名解析成正式名
Object sharedInstance = getSingleton(beanName); // 一级缓存快路径，命中就返回
if (sharedInstance != null && args == null) {
    return getObjectForBeanInstance(sharedInstance, name, beanName, null); // FactoryBean 要再取一次产物
}
// 未命中：prototype 的循环、父容器、depends-on 依次处理
if (isPrototypeCurrentlyInCreation(beanName)) throw new BeanCurrentlyInCreationException(beanName);
BeanFactory parent = getParentBeanFactory();   // 当前容器没有定义就委托父容器
markBeanAsCreated(beanName);                   // 记进 alreadyCreated，防止重复走完整创建流程
RootBeanDefinition mbd = getMergedLocalBeanDefinition(beanName); // 合并 parent 里的配置
for (String dep : mbd.getDependsOn()) { getBean(dep); }  // depends-on 的先行创建，和 @Autowired 无关
```

三个 map 的分工：

缓存	内容	什么时候写入
`singletonObjects`	完整成品，可直接返回	创建完成后 `addSingleton`
`earlySingletonObjects`	早期引用，可能是代理	第一次有人穿透到三级时
`singletonFactories`	`ObjectFactory`，能产出早期引用	`doCreateBean` 里实例化之后、属性填充之前

为什么必须是三级，两级不行？ 如果只有二级、直接把裸对象存进 `earlySingletonObjects`，那"要不要提前生成代理"这个决定就必须在实例化那一刻做出——等于对所有 bean 都提前做一次 AOP 判断，破坏了"代理在 `postProcessAfterInitialization` 生成"的时序，也让所有 bean 都提前付出了代理判断的代价。第三级存的是**工厂**，把决策推迟到"真的有人来问"的那一刻；`factory.getObject()` 之后立刻 `remove` 再 `put` 到二级，保证同一个 bean 的早期引用只计算一次。所以三级缓存真正解决的不是"循环依赖"，是**"AOP 代理下的循环依赖"**——普通的循环依赖，二级就够。

为什么构造器注入的循环依赖无解？ 因为 `addSingletonFactory` 在 `createBeanInstance` **之后**。A 的构造器需要 B，B 的构造器需要 A，此时 A 连对象都没有，第三级里没东西可放，只能一路递归到栈底抛 `BeanCurrentlyInCreationException`。这就是"改成构造器注入就启动不了"的全部原因。

为什么 prototype 的循环依赖无解？ prototype 不进任何缓存，`isPrototypeCurrentlyInCreation` 一旦命中就直接抛。缓存机制天然只服务于单例。

为什么加了 `@Async` 就报 raw version？ 代理在 after-init 才生成。如果 B 在 A 初始化完成前就拿走了 A 的早期引用，B 手里是裸 A；而 A 最终被包成了代理。Spring 在 `doCreateBean` 末尾比了一次 `exposedObject == bean`，发现不一致且 A 已经被注入到别的 bean 里，直接抛那句 `has been injected into other beans in its raw version ... but has eventually been wrapped`。把 `allowRawInjectionDespiteWrapping` 设成 true 能压掉异常，但代价是 B 拿到一个 `@Async` 完全失效的 A——把错误藏起来，不叫解决。

第十一步之二：createBean 的三次机会（实例化前/初始化前/初始化后）

`createBean` 的第一件事是 `resolveBeforeInstantiation`，它给 `InstantiationAwareBeanPostProcessor.postProcessBeforeInstantiation` 一次机会：**返回非 null 就直接返回，连 `doCreateBean` 都不进**。这意味着连属性注入都不会发生，返回的对象必须自己完整。常规 AOP 不走这条（它在 after-init 才包代理），只有配了自定义 `TargetSource` 的场景才用。这条路径的存在是为了让"完全接管某个 bean 的创建"成为可能。

没被短路，就进 `doCreateBean` 的三段：

实例化。`createBeanInstance` 先问 `determineConstructorsFromBeanPostProcessors` —— `@Autowired` 标注的构造器就是在这里被 `AutowiredAnnotationBeanPostProcessor.determineCandidateConstructors` 挑出来的。有候选构造器、或者 `args` 非空、或者定义了 `constructor-arg`，走 `autowireConstructor`；否则走 `instantiateBean` → `SimpleInstantiationStrategy`：没有方法覆写就用反射调无参构造，有 `<lookup-method>` / `<replaced-method>` 就得上 CGLIB 生成子类。

属性填充。`populateBean` 里有三个口子：`postProcessAfterInstantiation`（返回 false 直接跳过整个属性填充，`@Autowired` 也不会生效）、byName/byType 自动装配、`postProcessPropertyValues`（`@Autowired` 真正写字段的地方）。最后 `applyPropertyValues` 把 XML 里配的字面量灌进去。

初始化。`initializeBean` 的顺序是固定的五拍：

1. `invokeAwareMethods` —— 只处理 `BeanNameAware`、`BeanClassLoaderAware`、`BeanFactoryAware` 三个。其余 Aware（`ApplicationContextAware` 等）是第 3 步注册的 `ApplicationContextAwareProcessor` 在下一拍处理的，这是个高频记混点。
2. `postProcessBeforeInitialization` —— `@PostConstruct` 在这里执行。
3. `afterPropertiesSet` —— `InitializingBean`。
4. `init-method` —— XML 的 `init-method` 或 `@Bean(initMethod=...)`。
5. `postProcessAfterInitialization` —— AOP 代理在这里生成。

`@PostConstruct` 排在 `afterPropertiesSet` 之前，这一点和很多人的直觉相反：它被 `CommonAnnotationBeanPostProcessor` 当作 before-init 回调处理，而 `afterPropertiesSet` 是 `invokeInitMethods` 里的第一步。

回到第 5 拍。代理在这里生成，但上一节讲过的"提前引用"可能已经生成过一次了，Spring 靠 `earlyProxyReferences` 这个 map 保证不重复包装：`getEarlyBeanReference` 里如果决定提前造代理，会把 `beanName -> 原始 bean` 记进去；等正常的 `postProcessAfterInitialization` 再来时，发现这个 key 存在，就直接返回原始 bean，不再套第二层。所以"一个 bean 被代理两次"在 Spring 里不会发生——不是因为逻辑上不可能，是因为这个 map 挡了一道。

代理方式的选择也在这条线上：类实现了接口默认用 JDK 动态代理，没有接口才用 CGLIB；`proxyTargetClass = true` 强制 CGLIB。CGLIB 靠生成子类实现，所以 `final` 类没法代理、`final` 方法没法增强、`private` 方法压根不参与重写—— `@Transactional` 加在 `private` 或 `final` 方法上静默失效，根因就在这一步，不在事务本身。

第十二步：finishRefresh 与生命周期收尾

第 12 步要发布"容器就绪"这个事件，所以第 7 到 10 步必须先把这个发布能力造出来。这四步经常被当作"和业务无关，跳过"，但它们的位置同样是有理由的。

`initMessageSource`（7）和 `initApplicationEventMulticaster`（8）都是"容器自用组件"的兜底注册：容器里没有名为 `messageSource` 的 bean，就塞一个空的 `DelegatingMessageSource`；没有 `applicationEventMulticaster`，就 new 一个 `SimpleApplicationEventMulticaster` 并 `registerSingleton`。注意这里用的是 `registerSingleton` 而不是注册 `BeanDefinition` ——它们**绕过了整个 bean 创建流程**，不会经过任何 BPP，也就不会被打上代理。所以想给事件广播器加异步，不能靠 `@Async`，要么自己定义同名的 bean，要么往里塞一个 `TaskExecutor`。

`onRefresh`（9）是留给子类"在单例初始化之前建基础设施"的钩子，Spring Boot 的内嵌 Tomcat 就是在这里起来的。为什么必须在第 11 步之前？因为后面可能有 bean 需要拿到 `ServletContext`、需要感知端口已监听，甚至需要在 `ApplicationListener<WebServerInitializedEvent>` 里读实际端口号。把 Tomcat 放到第 11 步之后，这些 bean 初始化时容器还是"没有 Web 环境"的状态。

`registerListeners`（10）做两件事：把所有 `ApplicationListener` 类型的 bean 加进广播器（此时只是登记，不创建），然后补发第 1 步攒下的 `earlyApplicationEvents`。这也是 `@EventListener` 能被识别的前提——它靠的是第 6 步注册的 `EventListenerMethodProcessor`，而真正的监听器注册发生在第 10 步。

回到第 11 步内部，`finishBeanFactoryInitialization` 在 `preInstantiateSingletons()` 之前还做了一件容易被忽略的事：`beanFactory.freezeConfiguration()`。它把 `beanDefinitionNames` 固化进 `frozenBeanDefinitionNames`，此后 `isConfigurationFrozen()` 为 true，类型匹配的结果可以安全缓存，不必担心定义再变。更实际的影响在 `preInstantiateSingletons()` 内部：它进门第一件事就是 `new ArrayList<>(this.beanDefinitionNames)` 做快照，之后再往 registry 里塞的定义根本不在这次遍历的名单里。这就是"启动到一半动态注册的 bean 没被创建"的原因——定义期在第 5 步就实质结束了，第 11 步之后想加东西，得用 `registerSingleton` 塞现成对象，或者手动 `getBean` 触发一次。

`finishRefresh` 做三件事，顺序不能换。

`initLifecycleProcessor()`：容器里没有名为 `lifecycleProcessor` 的 bean 就 new 一个 `DefaultLifecycleProcessor` 注册进去。然后 `getLifecycleProcessor().onRefresh()`，对所有 `SmartLifecycle` 按 `phase` **升序**调 `start()`。注意是 `SmartLifecycle` 而不是 `Lifecycle` ——后者必须有人显式调 `start()`，前者在容器刷新时自动启动。`phase` 小的先启动，因为小的通常是被依赖的基础设施（比如消息监听容器要等 `DataSource` 就绪）。

`publishEvent(new ContextRefreshedEvent(this))`：所有 `@EventListener(ContextRefreshedEvent.class)` 在这一刻触发。位置必须在第 11 步之后，否则监听器里 `getBean` 拿到的是半成品容器，甚至触发新的 bean 创建，把"启动完成"这个语义彻底打乱。

`LiveBeansView.registerContext(this)`：给 Actuator 的 `/beans` 端点提供数据，顺带一提。

和它对称的是 `close()`：`doClose` 依次发布 `ContextClosedEvent`、`lifecycleProcessor.onClose()`（`phase` **降序** stop，让依赖者先停）、`destroyBeans()`。销毁顺序与创建顺序相反，单个 bean 的销毁回调顺序是 `@PreDestroy` → `DisposableBean.destroy()` → `destroy-method`，和初始化那五拍正好镜像。

第 12 步不是"收尾装饰"。它是整个流程里唯一让"容器已就绪"这个事实对外可见的地方——在此之前，任何对外暴露的引用都可能指向一个还没初始化完的容器。

容易误解的地方：

1. **"三级缓存是为了解决循环依赖"**。二级缓存就能解决普通循环依赖。第三级存工厂而不是对象，是为了把"要不要提前生成 AOP 代理"这个决策推迟到真正有人需要的那一刻，避免所有 bean 都在实例化时提前走一遍代理判断。所以准确说法是：**三级缓存解决的是"AOP 代理下的循环依赖"**，而且它只对单例、只对 setter/字段注入有效。
2. **"`FactoryBean` 和 `BeanFactory` 是一回事"**。`BeanFactory` 是容器本身；`FactoryBean` 是容器里的一个 bean，它的 `getObject()` 产物才是别人拿到的对象。`getBean("car")` 返回的是 `getObject()` 的结果，`getBean("&car")` 返回 `FactoryBean` 本身——`&` 前缀就是干这个的。`preInstantiateSingletons` 里对 `FactoryBean` 先 `getBean("&" + beanName)` 造出工厂本身，只有 `SmartFactoryBean.isEagerInit()` 为 true 时才顺手把产物也造出来。
3. **"`@PostConstruct` 是 Spring 内建在 `initializeBean` 里调的"**。不是。它由 `CommonAnnotationBeanPostProcessor` 处理，走的是 `postProcessBeforeInitialization`。所以：去掉 `CommonAnnotationBeanPostProcessor`（比如漏了 `<context:annotation-config/>` 又没开注解扫描），`@PostConstruct` 就静默不执行；而它一定在 `afterPropertiesSet` 之前，也一定在属性注入之后。
4. **"`BeanPostProcessor` 对所有 bean 都生效"**。只对**在它注册之后**创建的 bean 生效。BPP 自己、BPP 的依赖、以及第 5 步里被提前创建的任何 bean，都不受后续 BPP 影响。这不是 bug，是启动顺序的必然结果——你没法用一个还没造出来的处理器去处理已经造出来的对象。
5. **"`BeanFactoryPostProcessor` 能拿到 bean 就能改 bean"**。它拿到的是 `ConfigurableListableBeanFactory`，改的是 `BeanDefinition`。这时候容器里一个业务对象都没有，你 `getBean` 一下反而会触发提前创建，把第 6 步的时序全部打乱——这也是一种经典的"启动变慢且注解失效"的成因。

记住这一句：refresh() 的十二步，本质是把"定义"和"实例"彻底分层——前五步只碰 `BeanDefinition`，第六步起才开始碰对象。记住这条分界线，循环依赖、`@PostConstruct` 失效、`@Async` 不生效，就都只是同一张时间表上的推论。

第25章　AOP 的演进：从 AspectJ 到动态代理

一句话：AOP 的本质是换掉对象：要么改字节码，要么套一层代理。

为什么需要它：

先看一个很典型的现场。`OrderService` 里有 12 个方法，产品要求每个方法都开事务、都打耗时日志、都对入参做脱敏。最朴素的写法是在每个方法里 `try { tx.begin(); ... } finally { tx.commit(); }`，再包一层计时。12 个方法写 12 遍，第 13 个方法上线时忘了加，出事的偏偏是它。

抽成模板方法？那得先有一个抽象基类，所有 Service 去继承它，接口就被基类绑死了。用装饰器？`OrderService` 一个装饰器、`UserService` 一个装饰器，每个接口配一个，类数量翻倍，而且装饰器里还是要手写一遍调用转发。

真正的翻车点不在"写起来烦"，在"写漏了不报错"。有人为了"防止子类乱改"，把 `createOrder` 声明成 `final`，事务注解还挂着，日志切面也挂着，编译通过、启动通过、单测通过——运行时这个方法就是不走事务。另一台机器上报 `ClassCastException: com.sun.proxy.$Proxy42 cannot be cast to com.xxx.OrderServiceImpl`，同一个类在两台机器上走了两种代理，一台能强转一台不能。

这一章要解决的是：横切逻辑到底应该放在哪里、它是以什么形态进入你的调用链的、以及为什么"注解加了但没反应"这件事在 AOP 里是常态而不是 bug。

核心结构：

概念	作用
`JoinPoint`	程序执行中可以插入增强的点。Spring AOP 里只有一种：方法执行
`Pointcut`	匹配 JoinPoint 的表达式，回答"增强哪些方法"
`Advice`	增强逻辑本体 + 执行时机（前/后/异常/环绕）
`Advisor`	Pointcut + Advice 的绑定。Spring 特有，是容器能管理的"切面单元"
`Aspect`	横切关注点的模块化，通常是一个 `@Aspect` 标注的 bean
`Weaving`	把 Advice 织入 JoinPoint 的过程。发生在编译期、类加载期，或运行期
`Target`	被代理的原始对象，业务逻辑真正所在
`Proxy`	织入之后对外暴露的对象，容器里 `getBean` 拿到的就是它
`AopProxy`	Spring 内部的代理抽象，两个实现：`JdkDynamicAopProxy`、`CglibAopProxy`

关键代码：

JDK 动态代理的全部秘密就是"实现同一个接口 + 把所有调用导进一个 handler"。下面这段是手写版，Spring 内部做的事和它没有本质区别。

```
OrderService target = new OrderServiceImpl();

OrderService proxy = (OrderService) Proxy.newProxyInstance(
        target.getClass().getClassLoader(),
        new Class<?>[]{OrderService.class},          // 代理类要 implements 这些接口
        (p, method, args) -> {
            long start = System.nanoTime();
            try {
                // 关键：这里必须用反射调 target，不能调 p
                // 调 p 会重新进 invoke，直接无限递归
                return method.invoke(target, args);
            } finally {
                System.out.println(method.getName() + " cost " + (System.nanoTime() - start));
            }
        });

proxy.createOrder("Tom", "book");   // 走 invoke

// proxy 的运行时类型是 com.sun.proxy.$Proxy0，它 implements OrderService，
// 但它和 OrderServiceImpl 没有任何继承关系，所以下面这行必炸
// OrderServiceImpl impl = (OrderServiceImpl) proxy;   // ClassCastException
```

CGLIB 走的是另一条路：生成子类、覆写方法。注意 `invoke` 和 `invokeSuper` 的区别，这是很多人写 CGLIB demo 时踩的第一个坑。

```
OrderServiceImpl target = new OrderServiceImpl();

Enhancer enhancer = new Enhancer();
enhancer.setSuperclass(OrderServiceImpl.class);      // 生成的是子类，不是接口实现
enhancer.setCallback((MethodInterceptor) (obj, method, args, methodProxy) -> {
    System.out.println("before " + method.getName());
    // invoke(target, args)：直接调目标对象的方法，等价于 JDK 版里的 method.invoke(target)
    // 如果写 invokeSuper(obj, args)，调的是"代理类重写后的方法"，会再次进入本拦截器
    Object ret = methodProxy.invoke(target, args);
    return ret;
});

OrderServiceImpl proxy = (OrderServiceImpl) enhancer.create();
// 目标类里如果有 final 方法，子类无法覆写，那次调用根本不会进到这里
```

拆解：

一、横切逻辑的三个解法，AOP 赢在"调用方无感"

继承、组合、AOP，本质上都在解决同一个问题：把散落在 N 个方法里的同一段逻辑收拢到一处。三者的成本完全不同。

继承要求被增强的类继承一个基类，Java 单继承，一个类只能有一个爹，业务类一旦继承了 `AbstractTxService` 就再也继承不了别的。组合要求为每个接口写一个装饰器，接口有多少个就要写多少个包装类，而且包装类里那堆转发方法是纯体力活。AOP 的做法是：**调用方代码一行不改，返回给你的对象换了一个**。

"调用方无感"是 AOP 全部价值所在，也是它全部坑的来源。因为无感，所以调用方不知道自己被增强了，也就不知道什么时候**没有**被增强。你写 `this.createOrder()` 的时候，代码看起来和 `orderService.createOrder()` 一模一样，但前者走的是原始对象，后者走的是代理——这个差别在源码里看不见，只能靠理解代理的边界来推断。

还有一层要分清：Spring AOP 只支持方法执行这一种 JoinPoint。字段访问、构造器调用、静态初始化块，它都管不了。这不是实现偷懒，是动态代理的物理上限——代理只能覆写方法，没法拦截一次 `getfield`。

顺带说清楚 AOP 和 Filter、Interceptor 的分工，这三个东西经常被拿来互相替代。Filter 在 Servlet 容器层，只能看到 `HttpServletRequest`，拿不到 bean、拿不到方法签名、拿不到参数类型，也没法在 `@Autowired` 注入的对象上生效。HandlerInterceptor 在 Spring MVC 层，能拿到 `HandlerMethod`，但只覆盖 Controller，Service 之间的调用它看不见。AOP 在 bean 方法层，能拿到方法签名、实参、返回值、目标对象，但只能作用于容器管理的 bean。选型标准很简单：要改 HTTP 报文（编码、跨域、Body 包装）用 Filter；要做登录校验、权限、Controller 级日志用 Interceptor；要在 Service 方法上做事务、缓存、重试、埋点，只有 AOP。

二、静态织入：AspectJ 在你编译之前就改了代码

AspectJ 是另一个东西，和 Spring 没关系，它是 Eclipse 基金会下的独立 AOP 框架，提供的是"AOP 的完全解决方案"：字段、构造器、静态方法、甚至另一个类里的调用点，它都能织。

它的核心是**织入发生在代码运行之前**，具体有三个时机：

- **编译期织入（CTW）**：用 `ajc` 而不是 `javac` 编译，生成的 `.class` 里已经包含了增强代码。运行时加载的就是普通类，没有任何代理、没有反射、没有额外栈帧。
- **编译后织入（post-compile）**：类已经编成 `.class` 或者打成 jar 了，再拿 `ajc` 处理一遍。典型场景是要增强第三方 jar 里的类。
- **加载期织入（LTW）**：类加载进 JVM 的那一刻改字节码。实现方式是在启动参数里挂 `-javaagent:aspectjweaver.jar`，agent 注册一个 `ClassFileTransformer`，在 `defineClass` 之前拿到字节数组改完再交回去。

这里有个容易忽略的差异：AspectJ 的切点可以写在**调用点**（`call`）上，也可以写在**方法体**（`execution`）上。写在 `call` 上，增强代码被塞进调用方那一行，所以你能拿到"谁调的我"；写在 `execution` 上，增强代码被塞进被调方法的第一行，调用方完全不知道自己被统计了。Spring AOP 只有后者，因为代理只能从"方法入口"切进去。

代价也很实在：`ajc` 是另一套编译链，Maven 要加 `aspectj-maven-plugin`，IDE 的增量编译经常绕过它（这就是为什么很多 AspectJ 教程反复强调"用 `mvn clean package` 跑，别用 IDE"）；`call` 切点会把增强代码复制到每一个调用点，代码体积随调用点数量线性膨胀。

三、JDK 代理与 CGLIB：接口委托 vs 子类覆写

两者的差别不是"性能好坏"，是**能不能代理**。

JDK 代理生成的类 `implements` 目标接口，方法体里统一转给 `InvocationHandler.invoke`，再由 `invoke` 反射调用 target。由此推出三条硬约束：

1. 目标类必须实现接口，没接口就没法生成（`Proxy.newProxyInstance` 第二个参数不能为空）。
2. 只有**接口里声明过的方法**会被增强。实现类自己加的 `public` 方法，代理类里压根没有，调不到。
3. 代理对象不是目标类的实例。`proxy instanceof OrderServiceImpl` 是 `false`，按实现类类型注入会直接失败。

CGLIB 生成的是目标类的**子类**，覆写所有非 `final`、非 `private`、非 `static` 的方法。由此推出另外三条：

1. `final` 方法不能被覆写。子类里那个方法就是父类的原版，调用时直接执行原逻辑，拦截器根本不会被触发——**不报错，静默失效**，这是最坑的一种。
2. `private` 方法同理，子类看不见父类的私有方法，谈不上覆写。`static` 方法属于类不属于实例，也拦不住。
3. 目标类不能是 `final` 类，否则连子类都生成不出来，启动时直接 `IllegalArgumentException: Cannot subclass final class`。

CGLIB 还有个构造器问题：生成子类实例总要调父类构造器，如果目标类只有带参构造器又没有默认值，就会失败。Spring 用 `Objenesis` 绕过了这一步——直接分配对象内存，不走构造器，这就是 `ObjenesisCglibAopProxy` 这个类名的由来。

性能上有个流传很广的说法是"CGLIB 比 JDK 快"。这个结论在 JDK 6 时代成立，JDK 8 之后反射经过 inflation 和字节码生成优化，两者在同一量级，CGLIB 反而因为首次生成类要跑 ASM、要在 Metaspace 里多存一份子类字节码而更贵。真正该关心的不是快慢，是**这个方法能不能被拦住**。

四、Spring 怎么选，以及五种通知的真实顺序

选择逻辑只有一处，在 `DefaultAopProxyFactory#createAopProxy`，判断顺序是：

```
optimize == true  ||  proxyTargetClass == true  ||  没有用户自定义接口
    → 目标本身是接口?  → JDK 代理
    → 否则             → CGLIB
否则
    → JDK 代理
```

翻译成人话：**有接口默认走 JDK；`proxyTargetClass=true` 时一律走 CGLIB；没接口只能走 CGLIB**。这里要特别注意 Spring Boot：`AopAutoConfiguration` 从 2.0 起默认 `spring.aop.proxy-target-class=true`，也就是说 Boot 应用里**即使有接口也用 CGLIB**。很多"我在 Boot 里按实现类注入没问题，换回传统 Spring 就报错"的疑惑，根源就在这一行默认值。

强制切换的方式：`@EnableAspectJAutoProxy(proxyTargetClass = true)`，或者 XML 里的 `<aop:config proxy-target-class="true">`。

再说通知顺序。同一个切面内，五种通知的执行次序是固定的：

```
@Around  前半段
  @Before
    target.method()
  @AfterReturning          ← 正常返回时
  @After                   ← 相当于 finally，正常/异常都走
@Around  后半段（proceed() 返回之后）
```

异常路径上是 `@AfterThrowing` 取代 `@AfterReturning`，`@After` 依然执行。注意 `@After` 在 `@AfterReturning` **之后**，不是之前——因为 `@After` 的语义是 `finally`，它包在 `try` 的最外层。

多个切面时，顺序由 `@Order` 或 `Ordered` 接口决定，值小的在外层（`@Around` 先进入、后退出）。没有 `@Order` 时顺序不确定，不要依赖它。

最后补一条演进线，因为"谁有权创建代理"这件事在 Spring 历史上换过三次主人。Spring 1.2 时代是 `ProxyFactoryBean`：你在 XML 里显式声明一个 bean，它的类型是 `ProxyFactoryBean`，`target` 指向真实实现，`interceptorNames` 挂上 Advice/Advisor，取 bean 时得写 `getBean("userServiceProxy")`。这个方案的问题是代理是"另一个 bean"，按类型注入拿不到，配置量随 bean 数量线性增长。Spring 2.0 引入 autoproxy：`BeanNameAutoProxyCreator` 按名字模式批量代理，`DefaultAdvisorAutoProxyCreator` 让容器里所有 Advisor 全局生效，代理创建权从"配置"移交给"容器"。同一个版本引入 `@AspectJ` 和 `<aop:config>`，代理创建权进一步交给注解和命名空间解析器，最终落到 `AnnotationAwareAspectJAutoProxyCreator` 这一个 `BeanPostProcessor` 上。三代配置写法的 API 完全不同，但底层产物是同一个 `ProxyFactory`——这也是为什么老项目里三种写法混着用还能正常工作。

至于 Spring 为什么不干脆用 AspectJ：因为它要解决的是"企业应用里 bean 方法上的横竖切"，不是"AOP 的完全解决方案"。用 AspectJ 意味着构建链要多一个 `ajc`、IDE 要装 AJDT 插件、运维要在启动脚本里加 `-javaagent`，换来的能力（字段织入、构造器织入、`call` 切点）在业务代码里几乎用不上。用运行期代理，代价只是多一层栈和一个反射调用，收益是零构建侵入——这笔账很好算。

容易误解的地方：

1. **"Spring AOP 就是 AspectJ"**。不是。Spring 只是从 `aspectjweaver.jar` 里借了 `@Aspect`、`@Pointcut`、`@Before` 这几个注解的**定义**，外加一个切点表达式解析器。增强逻辑的执行完全是 Spring 自己的 JDK/CGLIB 代理。你可以在依赖树里把 `aspectjweaver` 排除掉，只要不用注解式切点，纯 Spring AOP 照样跑。

2. **"实现了接口就一定用 JDK 代理"**。只在传统 Spring 的默认配置下成立。Spring Boot 2.x 起默认 `proxyTargetClass=true`，有接口也走 CGLIB。判断依据不是"有没有接口"，是 `AdvisedSupport.proxyTargetClass` 这个标志位。

3. **"`final` 方法会报错"**。不会。CGLIB 生成子类时直接跳过 `final` 方法，代理对象上那个方法就是父类原版，调用时静默走原逻辑。你只会看到"日志没打""事务没开"，不会看到任何异常。同理，`private` 方法上的 `@Transactional`、`@Async`、自定义注解，全部无效且无提示。

4. **"`@After` 比 `@AfterReturning` 先执行"**。反了。`@AfterReturning` 在 `try` 块内，`@After` 在 `finally` 块，所以先 `AfterReturning` 后 `After`。写"记录返回值并清理 MDC"这种逻辑时顺序搞反，会出现"日志里 MDC 已经清空了"的问题。

记住这一句：静态织入改的是代码，动态代理换的是对象；Spring 选了后者，于是所有"没经过代理"的调用都不存在增强。

第26章　Spring AOP 源码：代理是什么时候造出来的

一句话：代理在初始化完成后由 BeanPostProcessor 换掉，自调用拿到的永远是原对象。

为什么需要它：

`@Transactional` 加在 `createOrder` 上，`batchCreate` 里循环 `this.createOrder()`，一千条订单一千次调用，事务一个都没开。把 `batchCreate` 也加上 `@Transactional`，事务开了——但只有最外层那一个，内层还是没开，`REQUIRES_NEW` 也不生效。

换个写法想绕过去，用 `AopContext.currentProxy()`，直接抛 `IllegalStateException: Cannot find current proxy: Set 'exposeProxy' property on Advised to 'true' in order to make it available`。

再换一个现场：自定义注解切面，`@Cost` 打在 `private` 方法上，日志一条没有；打在 `protected` 方法上，也没有；换成 `public`，有了。同一个类，只是修饰符不同。

这些现象背后是同一件事：代理对象在 bean 生命周期的某一行被"换"进去，从那之后，**只有经过代理对象的调用才有增强**。这一章要回答四个问题：代理在哪一行生成、谁决定生成不生成、一次调用怎么变成 N 个拦截器、以及 `this` 为什么绕不过去。

核心结构：

组件	作用
`AbstractAutoProxyCreator`	代理创建的总调度，`BeanPostProcessor` 的实现，覆写 `postProcessAfterInitialization`
`AnnotationAwareAspectJAutoProxyCreator`	`@Aspect` 场景下的具体实现，负责把 `@Aspect` bean 解析成 Advisor
`ReflectiveAspectJAdvisorFactory`	反射扫描 `@Aspect` 类的方法，把 `@Before` 等转成 `Advice`
`BeanFactoryAdvisorRetrievalHelper`	从容器里捞出所有 `Advisor` 类型的 bean
`AopUtils#canApply`	类级粗筛 + 方法级精筛，判断一个 Advisor 是否适用于某个 bean
`ProxyFactory`	代理工厂，持有 interfaces、advisors、targetSource，最终产出代理
`AdvisorAdapterRegistry`	把 `MethodBeforeAdvice` / `AfterReturningAdvice` 适配成 `MethodInterceptor`
`ReflectiveMethodInvocation`	拦截器链的执行上下文，`proceed()` 递归推进
`SingletonTargetSource`	把原始 bean 包一层，代理通过它拿到 target
`AopContext`	基于 ThreadLocal 暴露当前代理，`exposeProxy=true` 时才可用

关键代码：

整个 AOP 的入口就这十几行。所有"代理什么时候生成"的问题，答案都在这里。

```
// AbstractAutoProxyCreator
public Object postProcessAfterInitialization(@Nullable Object bean, String beanName) {
    if (bean != null) {
        Object cacheKey = getCacheKey(bean.getClass(), beanName);
        // earlyProxyReferences 是给循环依赖准备的：如果 getEarlyBeanReference 已经
        // 提前造过代理，这里 remove 出来的就是当初那个 bean，相等就不再包第二次
        if (this.earlyProxyReferences.remove(cacheKey) != bean) {
            return wrapIfNecessary(bean, beanName, cacheKey);
        }
    }
    return bean;
}

protected Object wrapIfNecessary(Object bean, String beanName, Object cacheKey) {
    if (StringUtils.hasLength(beanName) && this.targetSourcedBeans.contains(beanName)) {
        return bean;                                    // 自定义 TargetSource 已经处理过
    }
    if (Boolean.FALSE.equals(this.advisedBeans.get(cacheKey))) {
        return bean;                                    // 上次判过"不需要代理"，直接短路
    }
    // 切面类自己（Advisor / Pointcut / Advice 的实现）不代理，
    // 否则解析切面时会再次触发代理创建，无限递归
    if (isInfrastructureClass(bean.getClass()) || shouldSkip(bean.getClass(), beanName)) {
        this.advisedBeans.put(cacheKey, Boolean.FALSE);
        return bean;
    }

    Object[] specificInterceptors = getAdvicesAndAdvisorsForBean(bean.getClass(), beanName, null);
    if (specificInterceptors != DO_NOT_PROXY) {
        this.advisedBeans.put(cacheKey, Boolean.TRUE);
        // 注意第四个参数：原始 bean 被 SingletonTargetSource 包住，
        // 从此容器里这个 beanName 对应的是 proxy，target 只被 proxy 持有
        Object proxy = createProxy(bean.getClass(), beanName, specificInterceptors,
                new SingletonTargetSource(bean));
        this.proxyTypes.put(cacheKey, proxy.getClass());
        return proxy;
    }

    this.advisedBeans.put(cacheKey, Boolean.FALSE);
    return bean;
}
```

链的执行只有一处递归，理解了这个 `index` 就理解了整条链。

```
// ReflectiveMethodInvocation
private int currentInterceptorIndex = -1;   // 游标，所有拦截器共享同一个 invocation 实例

public Object proceed() throws Throwable {
    // 走到链尾：反射调 target 的真实方法
    if (this.currentInterceptorIndex == this.interceptorsAndDynamicMethodMatchers.size() - 1) {
        return invokeJoinpoint();
    }

    Object interceptorOrInterceptionAdvice =
            this.interceptorsAndDynamicMethodMatchers.get(++this.currentInterceptorIndex);

    if (interceptorOrInterceptionAdvice instanceof InterceptorAndDynamicMethodMatcher) {
        // 运行期还要再判一次参数（@args 这类切点），不匹配就跳过，继续往下
        InterceptorAndDynamicMethodMatcher dm = (InterceptorAndDynamicMethodMatcher) interceptorOrInterceptionAdvice;
        Class<?> targetClass = (this.targetClass != null ? this.targetClass : this.method.getDeclaringClass());
        if (dm.methodMatcher.matches(this.method, targetClass, this.arguments)) {
            return dm.interceptor.invoke(this);
        }
        return proceed();
    }
    // 关键：把 this 传进去。拦截器里调 mi.proceed() 就是回到这里，
    // 此时 index 已经 +1，于是自然走到下一个拦截器。
    // 链是"递归"出来的，不是 for 循环出来的，栈深度 = 拦截器个数
    return ((MethodInterceptor) interceptorOrInterceptionAdvice).invoke(this);
}
```

拆解：

一、时机：为什么必须等到初始化之后

`doCreateBean` 的主干只有三步：`createBeanInstance`（反射调构造器，得到一个裸对象）→ `populateBean`（属性注入、`@Autowired` 生效）→ `initializeBean`。第三步内部又是三段：

```
applyBeanPostProcessorsBeforeInitialization   →  @PostConstruct、Aware 回调
invokeInitMethods                             →  InitializingBean.afterPropertiesSet、init-method
applyBeanPostProcessorsAfterInitialization    →  AOP 在这里造代理
```

选在第三段，不是随手放的。如果在前两段就把 bean 换成代理，接下来 `invokeInitMethods` 会在**代理**上调用 `afterPropertiesSet`，而代理对象自己没有那些业务字段，方法里访问 `this.xxx` 全是 null；更糟的是 `@PostConstruct` 里常见的 `this.initCache()` 这种自调用会突然走进拦截器链，此时 target 还是半初始化状态，切面拿到的是个残缺对象。放在最后一段，target 已经完整可用，代理只是给它套了层壳。

这里要澄清一个高频误解：`AbstractAutoProxyCreator` 还实现了 `InstantiationAwareBeanPostProcessor#postProcessBeforeInstantiation`，里面确实也有 `createProxy` 调用。但那条路只在配置了自定义 `TargetSourceCreator`（比如池化 target、热替换 target）时才会走，普通业务 bean 一律返回 `null`，继续走正常的创建流程。**AOP 代理的常规路径在 `postProcessAfterInitialization`，不在 `postProcessBeforeInstantiation`。**

还有循环依赖这条支线。A 依赖 B、B 依赖 A，Spring 用三级缓存解决，其中第二级 `earlySingletonObjects` 存的就是"提前暴露的引用"。`AbstractAutoProxyCreator` 实现了 `SmartInstantiationAwareBeanPostProcessor#getEarlyBeanReference`，在 B 需要注入 A 的那一刻就把 A 的代理造出来放进二级缓存。等 A 真正走完初始化，`postProcessAfterInitialization` 再进来时，`earlyProxyReferences.remove(cacheKey)` 拿到的就是当初那个 A，和当前 bean 相等，于是跳过 `wrapIfNecessary`——这就是"代理不会造两次"的机制。

这个时机也解释了一类常见困惑：`@PostConstruct` 里调用本类的 `@Async` 方法，任务永远是同步执行的。因为 `@PostConstruct` 跑在 `applyBeanPostProcessorsBeforeInitialization` 阶段，代理还没生成，`this` 就是 target 本身。同理，`InitializingBean.afterPropertiesSet`、`init-method` 里所有自调用都不会有增强。只有等 bean 完全初始化完、被 `wrapIfNecessary` 包成代理之后，外部调用才会走进拦截器链。

二、三道闸门：谁决定这个 bean 要不要代理

`wrapIfNecessary` 名字里的 "IfNecessary" 就是三道闸门：

第一道，`targetSourcedBeans`。这个集合里放的是已经由自定义 `TargetSource` 处理过的 beanName，进来说明代理已经存在，直接返回。

第二道，`advisedBeans` 缓存。这是个 `Map<Object, Boolean>`，key 是 `beanName` 或 `beanClass`。值为 `FALSE` 表示"这个类已经判定过不需要代理"，后续同类型的 bean 直接短路。这个缓存的意义在于：判断"要不要代理"需要遍历所有 Advisor、对目标类的每个方法跑一遍 Pointcut 匹配，很贵；一个

500 个 bean 的应用，如果不缓存，启动时就是 500 × N 次方法匹配。

第三道，`isInfrastructureClass` 和 `shouldSkip`。前者判断这个类是不是 `Advice`、`Pointcut`、`Advisor`、`AopInfrastructureBean` 的实现——切面类自己不能被代理，否则 `getAdvicesAndAdvisorsForBean` 去容器里捞 Advisor 时会触发切面 bean 的创建，创建又要捞 Advisor，直接栈溢出。后者是给子类留的钩子，`AspectJAwareAdvisorAutoProxyCreator` 用它跳过 `@Aspect` 标注的类本身。

三道都过了，才进 `getAdvicesAndAdvisorsForBean`。它返回一个 `Object[]`，元素是 Advisor/Advice/Interceptor 混装；如果没有任何 Advisor 匹配，返回的是哨兵值 `DO_NOT_PROXY`（就是个 `null` 数组），走 `advisedBeans.put(cacheKey, FALSE)` 并原样返回 bean。

匹配过程分两级。**类级**：`findCandidateAdvisors` 从容器捞出所有 Advisor（`@Aspect` 场景下还要用 `ReflectiveAspectJAdvisorFactory` 把 `@Aspect` bean 的每个方法拆成 Advisor）。这一步有个副作用值得知道：捞 Advisor 调的是 `beanFactory.getBean(...)`，会**提前实例化**那些还没被创建的 Advisor bean，绕过正常的创建顺序。如果你的 Advisor 依赖了某个后初始化的 bean，就可能拿到半成品或者触发循环依赖。Spring 的处理是 `BeanFactoryAdvisorRetrievalHelper` 里跳过正在创建中的 bean（`isCurrentlyInCreation`），但顺序依赖仍然是真实存在的，切面 bean 尽量别依赖业务 bean。

方法级：`AopUtils.canApply` 对目标类的每个方法跑 `pointcut.matches(method, targetClass)`，**只要有一个方法命中，这个 Advisor 就保留**。也就是说，Advisor 是挂在类上的，具体某个方法要不要执行增强，是在运行期构建拦截器链时再判一次。这个"类级粗筛 + 方法级精筛"的两段式设计，是为了让 `advisedBeans` 缓存能够按类复用：同一个类创建第二个实例时，第一步就能短路，不用再遍历所有方法。

这里还埋着一个硬限制：`canApply` 遍历方法时会跳过所有**非 public** 的方法。`protected`、包级、`private` 一律不参与匹配，所以哪怕 CGLIB 有能力覆写 `protected` 方法，Spring AOP 也不会给它生成拦截。这解释了为什么 `@Transactional` 打在 `protected` 方法上永远不生效——不是代理没生成，是这个方法从头到尾就没进过候选集。

顺序上，最终会走 `AnnotationAwareOrderComparator` 排序，`@Order` 值小的排在前面（外层）。两个切面都加了 `@Around` 时，谁包谁就看这个值。

三、一次调用怎么变成一条链

JDK 代理的 `invoke` 里有一行 `this.advised.getInterceptorsAndDynamicInterceptionAdvice(method, targetClass)`，它做的事是把 Advisor 列表翻译成 `MethodInterceptor` 列表：

- Advisor 里包的是 `Advice`，但链上跑的是 `MethodInterceptor`。中间靠 `AdvisorAdapterRegistry` 适配：`MethodBeforeAdvice` → `MethodBeforeAdviceInterceptor`，`AfterReturningAdvice` → `AfterReturningAdviceInterceptor`，`ThrowsAdvice` → `ThrowsAdviceInterceptor`。每个适配器干的事都很朴素——在 `invoke` 里调一下 advice，再调 `mi.proceed()`。
- `PointcutAdvisor` 还要过一遍 `MethodMatcher`。如果 matcher 是 `isRuntime()` 的（比如带参数匹配的切点），不能提前过滤，要包成 `InterceptorAndDynamicMethodMatcher`，等真正调用时用实参再判一次——这就是上面 `proceed()` 里那个 `instanceof` 分支的用途。
- 结果按 `Method` 缓存在 `interceptorCache` 里。同一个方法第二次调用不会再算一遍。

这里要澄清一个反直觉的点：**你看到的通知顺序，不是拦截器按时间排队的顺序，而是"链的顺序 + 递归回弹"共同决定的**。同一个 `@Aspect` 里，`ReflectiveAspectJAdvisorFactory` 生成的 Advisor 是按 `Around → Before → After → AfterReturning → AfterThrowing` 排的，链就照这个顺序建。但真正执行时，`Around` 和 `Before` 在 `proceed()` 的下降段执行，`AfterReturning`、`After`、`AfterThrowing` 在回弹段执行——回弹是逆序的，所以链尾的 `AfterReturning` 反而比链中间的 `After` 先跑。最终观察到的顺序是 `Around前 → Before → 方法 → AfterReturning → After → Around后`。想验证很简单：随便写一个五件套切面，在每条通知里打印序号，看输出就知道链长什么样了。

然后 `new ReflectiveMethodInvocation(proxy, target, method, args, targetClass, chain)`，`proceed()` 开始递归。整条链的执行形态是：

```
proceed() index=-1
  → index=0  AroundInterceptor.invoke(mi)
      → mi.proceed()  index=1  BeforeInterceptor.invoke(mi)
          → mi.proceed()  index=2  AfterReturningInterceptor.invoke(mi)
              → mi.proceed()  index=3 == size-1  → invokeJoinpoint() 反射调 target
          ← 拿到返回值，记录
      ← 返回
  ← Around 后半段
```

注意 `currentInterceptorIndex` 是 `ReflectiveMethodInvocation` 的**实例字段**，所有拦截器共享同一个 `mi`。所以"推进到下一个"这件事不需要参数传递，`++index` 就够了。这也是为什么 `@Around` 不调 `proceed()` 就能把整个方法掐断——链根本没往下走。

`invokeJoinpoint()` 最终落到 `AopUtils.invokeJoinpointUsingReflection(target, method, args)`，一次 `Method.invoke`。这一层反射是 Spring AOP 相对 AspectJ 多出来的固定开销，也是"代理比织入慢"这句话的物理来源。

排查"到底有没有被代理"，不用猜，三行代码就够：

```
Object bean = ctx.getBean("orderService");
AopUtils.isAopProxy(bean);      // 是不是代理（JDK 或 CGLIB）
AopUtils.isJdkDynamicProxy(bean);
AopUtils.isCglibProxy(bean);
bean.getClass();                // com.sun.proxy.$Proxy42 或 OrderService$$EnhancerBySpringCGLIB$$1a2b3c
```

如果 `isAopProxy` 返回 `false`，问题在 `wrapIfNecessary` 那三道闸门——大概率是 Pointcut 没匹配上，或者这个 bean 压根没被容器管理。如果返回 `true` 但方法没增强，问题在方法级：`final`、`private`、`static`、或者 Pointcut 只匹配了别的方法。想看链上到底有几个拦截器，在 `ReflectiveMethodInvocation.proceed` 上打个断点看 `interceptorsAndDynamicMethodMatchers.size()`，比读十遍源码都快。

代理对象还都实现了 `Advised`，运行时能直接把它身上的 Advisor 全捞出来：`((Advised) bean).getAdvisors()`。排查"切面顺序不对""某个切面没生效"时，打印一遍这个数组，比看配置文件直观得多——里面每一项的 `getPointcut()` 和 `getAdvice()` 就是最终生效的东西，你写的注解只是它的原材料。

四、为什么 `this` 不走代理

代理对象和 target 是两个对象。容器里存的是代理，`SingletonTargetSource` 里存的是 target。调用 `orderService.create()` 时，栈是"调用方 → 代理 → 拦截器链 → target"，target 里执行到 `this.batchCreate()`，这个 `this` 是 target 自己，不是代理——**它没有经过代理对象，自然没有拦截器链**。

```
@Service
public class OrderService {
    @Transactional
    public void create() { /* ... */ }

    public void batch(List<Item> items) {
        for (Item i : items) {
            this.create();     // this = target 实例，不是 $Proxy，事务不生效
        }
    }
}
```

三种解法，各有代价：

1. **拆类**。把 `create` 挪到另一个 bean，通过注入调用。最干净，也最推荐，但会改变领域模型的划分。
2. **`AopContext.currentProxy()`**。需要 `@EnableAspectJAutoProxy(exposeProxy = true)`，原理是 `JdkDynamicAopProxy.invoke` 里把 `proxy` 塞进 `ThreadLocal`。缺点是代码里出现了对 Spring API 的硬依赖，且 `ThreadLocal` 不跨线程——在 `CompletableFuture` 或线程池里调用 `AopContext.currentProxy()` 会抛同样的异常。另外 `exposeProxy` 是个全局开关，一开就给每次调用多两次 `ThreadLocal` 读写，量大的接口要掂量一下。
3. **注入自己**。`@Autowired @Lazy private OrderService self;`，`@Lazy` 是为了打断构造期的循环依赖。本质和方案 1 一样，只是没换类。

还有一个变体：通过 `ApplicationContext.getBean(OrderService.class)` 拿到的是代理，所以 `ctx.getBean(X.class).create()` 是有效的。很多团队因此封装了一个 `SpringUtil.getBean()` 静态方法去绕自调用——能跑，但把依赖关系藏进了静态调用，单测里没有容器就直接空指针，属于"解决了症状、加重了病"。

容易误解的地方：

1. **"代理是在 `postProcessBeforeInstantiation` 里造的"**。那条分支只在有自定义 `TargetSourceCreator` 时生效，普通 bean 一律返回 `null`。真正的位置是 `postProcessAfterInitialization`，也就是 init-method 之后。名字只差 `Instantiation` 和 `Initialization` 两个词，时机差了整整一个 bean 生命周期。
2. **"容器里有 target 和 proxy 两个 bean"**。只有一个 singleton，beanName 对应的就是 proxy。target 被 `SingletonTargetSource` 持有，`getBean` 拿不到它。`AopUtils.getTargetClass(bean)` 能拿到 target 的 Class，但拿不到实例；想拿实例只能走 `Advised#getTargetSource().getTarget()`，而且它可能是 `null`（池化 target 只在调用期间存在）。
3. **"拦截器链是循环执行的"**。是递归。栈深度等于拦截器个数，一个方法挂 8 个切面就是 8 层 `proceed`。切面多了以后栈溢出（`StackOverflowError`）不是都市传说，尤其是切面里再调别的被切方法。
4. **"`@Transactional` 加在接口方法上就行"**。JDK 代理下注解在接口方法上能被读到，但 CGLIB 代理读的是实现类的方法，接口上的注解读不到。Boot 默认 CGLIB，把注解放接口上就是典型的"本地好的、上线没事务"。

记住这一句：容器里只有代理，target 里只有 `this`——从外面调是 AOP，从里面调是普通方法调用。

第27章　AspectJ：编译期织入与 @Aspect 的真相

一句话：AspectJ 是独立编译器，Spring 只借了它的注解和切点解析器。

为什么需要它：

想统计"谁调用了 `Account.pay()`"，很自然地写：

```
@Pointcut("call(* com.x.Account.pay(..))")
```

启动直接失败：`IllegalArgumentException: Pointcut is not well-formed: expecting 'name pattern' at character 6 of 'call(* com.x.Account.pay(..))'`。换成 `execution` 就好了，但 `execution` 只能统计方法自己执行了多久，拿不到调用方是谁——而这恰恰是这次排查想要的。

另一个现场：想给第三方 jar 里的 `RedisTemplate` 某个方法加埋点，切面写了、表达式对了、bean 也扫到了，就是不生效。因为那个对象不是 Spring 容器创建的，压根没走过 `BeanPostProcessor`。

还有一个更隐蔽的：项目里引了 `aspectjweaver`，团队一直以为"我们在用 AspectJ"。压测时发现每个被切方法都多了一层反射栈，Metaspace 里堆着一批 `$Proxy` 和 `Xxx$$EnhancerBySpringCGLIB`，跟 AspectJ 没有半点关系。

这一章要解决：AspectJ 到底是个什么东西、三种织入各自改了什么、Spring 的 `@Aspect` 和它差在哪、几个高频切点表达式到底怎么选。

核心结构：

概念	作用
`ajc`	AspectJ 自己的编译器，`javac` 的超集，能编 `.java` 和 `.aj`
`.aj` 文件	AspectJ 原生语法文件，`javac` 无法编译，只能 `ajc` 处理
`aspectjrt.jar`	运行时支持，织入后的代码依赖它的少量类
`aspectjweaver.jar`	织入器，包含 `ajc` 的织入逻辑、LTW 的 agent、切点解析器
`META-INF/aop.xml`	LTW 的配置文件，声明哪些 aspect 织入哪些包
`ClassFileTransformer`	JVM 提供的字节码改写钩子，LTW 的物理入口
`AspectJExpressionPointcut`	Spring 里解析切点表达式的类，内部用 AspectJ 的 `PointcutParser`
`ReflectiveAspectJAdvisorFactory`	Spring 里把 `@Aspect` 类的方法转成 Advisor 的工厂
PCD	Pointcut Designator，切点指示符，如 `execution`、`within`、`args`

关键代码：

AspectJ 原生写法，注意 `call` 和 `args` / `target` 的组合——这是它能拿到"调用方视角"的原因。

```
// AccountAspect.aj —— 扩展名必须是 .aj，javac 编不了
public aspect AccountAspect {

    // call(...)：织入点在"调用 pay 的那一行"，不是 pay 方法体内部
    // args(amount)：把实参绑到通知方法的参数上
    // target(account)：把被调方法的接收者绑进来
    pointcut callPay(int amount, Account account):
            call(boolean com.x.Account.pay(int)) && args(amount) && target(account);

    before(int amount, Account account): callPay(amount, account) {
        System.out.println("付款前余额: " + account.balance);
    }

    // around 里可以 proceed，也可以直接 return —— 直接 return 等于取消这次调用
    boolean around(int amount, Account account): callPay(amount, account) {
        if (account.balance < amount) {
            return false;                 // 调用方拿到 false，pay() 根本没执行
        }
        return proceed(amount, account);
    }

    after(int amount, Account account): callPay(amount, account) {
        System.out.println("付款后余额: " + account.balance);
    }
}
```

Spring 里的等价物，能做的事少了一截。

```
@Aspect
@Component
public class ProfilingAspect {

    // 只能写 execution：增强点在 pay 方法体内部，
    // 所以这里拿不到"谁调的"，只能拿到"调了什么"
    @Around("execution(* com.x.Account.pay(..))")
    public Object profile(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try {
            return pjp.proceed();          // 内部走的是拦截器链 → 反射调 target
        } finally {
            System.out.println(pjp.getSignature() + " cost " + (System.nanoTime() - start));
        }
    }
}
```

拆解：

一、AspectJ 的编译模型：`ajc` 不是 `javac` 的插件

`ajc` 是基于 Eclipse 编译器（ECJ）改出来的独立编译器，它认识 `.aj` 语法，也认识 `@Aspect` 注解。编译期织入的本质是：**在生成字节码之前，把通知代码内联到切点上**。

关键在"内联到哪"。同样一个切面，`call` 和 `execution` 织出来的结果完全不同：

- `execution(* Account.pay(int))`：增强代码被插到 `Account.pay` 方法体的开头。所有调用 `pay` 的地方都会经过这段代码，但 `pay` 内部不知道是谁调的自己。
- `call(* Account.pay(int))`：增强代码被插到**每一个调用点**上。`main` 里写 `account.pay(10)`，编译出来的字节码里，这一行前面先执行 `before`，后面执行 `after`。

这解释了为什么 `call` 切点能配 `this()`（调用方）和 `within()`（调用方所在的类），而 `execution` 不行——`execution` 的织入位置在被调方，栈上根本没有调用方的信息。

织入后的 `main` 大致等价于这样（`ajc` 实际生成的是字节码，不会真有一个 `AccountAspect` 变量，但语义一致）：

```
public static void main(String[] args) {
    Account account = new Account();
    // 原本只有一行 account.pay(10)，被 call 切点展开成了三段
    AccountAspect.aspectOf().before(10, account);
    boolean ok = account.pay(10);
    AccountAspect.aspectOf().after(10, account);
}
```

注意 `AccountAspect.aspectOf()` ——切面在 AspectJ 里是**单例对象**，由 `ajc` 生成的 `aspectOf()` 静态方法返回。这也是 AspectJ 能"零运行时开销"的原因之一：没有代理、没有反射、没有 `Method` 对象查找，只有一次静态方法调用和一次普通方法调用。

代价也随之而来：`call` 切点会在每个调用点复制一份增强逻辑。一个方法被 50 处调用，就复制 50 份。所以 AspectJ 的"零运行时开销"是真的（没有反射、没有代理、没有额外对象），但"零成本"是假的——成本被提前到了编译期和代码体积上。

二、三种织入时机，以及 LTW 到底挂在哪

编译期（CTW）：Maven 配 `aspectj-maven-plugin`，`goal` 是 `compile`。`ajc` 接管编译，`.aj` 和 `.java` 一起处理，产出的 `.class` 已经织好。运行时就是普通类，`java -jar` 直接跑。

编译后（post-compile）：类已经编好了、甚至已经打成 jar 了，再拿 `ajc` 处理一遍。Maven 里的写法是给插件配 `<weaveDependencies>`，指定要增强哪个依赖。注意副作用：被增强的 jar 里的类会被**复制**到当前工程的 `target/classes` 下，等于你本地覆盖了一份。升级依赖版本时如果忘了重新编译，跑的还是旧的那份。

加载期（LTW）：这一步最容易被说成"运行时织入"，其实不是。它发生在 `ClassLoader.defineClass` 之前：

1. 启动参数加 `-javaagent:/path/aspectjweaver.jar`，JVM 在 `premain` 阶段注册一个 `ClassFileTransformer`。
2. classpath 下的 `META-INF/aop.xml` 声明要织入哪些 aspect、哪些包（`<weaver><include within="com.x..*"/></weaver>`）。
3. 每个类被加载时，transformer 拿到原始 `byte[]`，按 `aop.xml` 匹配，命中就改写，再把新字节码交回 JVM。

三个常见坑：`aop.xml` 必须能被 classpath 找到（Spring Boot 的 fat jar 里路径是 `BOOT-INF/classes/META-INF/aop.xml`，不是根目录）；`-javaagent` 必须写在 `-jar` 前面；IDE 直接 Run 不会带你配的 Maven 参数，所以"IDE 里没效果、命令行有效果"是正常现象。

LTW 还有个隐性成本：每次类加载都要跑一遍匹配和改写，应用启动会变慢；而且 agent 一旦挂上，对所有类加载都生效，排查问题时很难摘掉。

Spring 自己也有一个 LTW 入口，叫 `LoadTimeWeaver`，用 `@EnableLoadTimeWeaving` 开启。它和 AspectJ 的 agent 不是一回事：Spring 的实现是把 `ClassFileTransformer` 注册到**应用自己的 ClassLoader** 上，而不是挂 JVM 级 agent，所以不需要改启动参数。但它的用途很窄，主要是给 JPA 的 `@Configurable`、`@Entity` 这类需要在类加载期做增强的框架用。用它来织入 `@Aspect` 切面是不行的——那些切面依然走运行期代理。看到 `@EnableLoadTimeWeaving` 就以为"启用了 AspectJ 编译期织入"，是很常见的误读。

三、`@Aspect` 在 Spring 里只是语法借用

Spring 的 `@Aspect` 支持，用到的 AspectJ 组件只有两个：

1. **注解定义**。`org.aspectj.lang.annotation.Aspect`、`@Before`、`@Pointcut` 这些类物理上在 `aspectjweaver.jar` 里，Spring 直接引用，不重新定义。所以你的 pom 里必须有这个依赖，否则编译不过——但这跟"用 AspectJ 织入"是两码事。
2. **切点表达式解析器**。`AspectJExpressionPointcut` 内部持有一个 `PointcutParser`，把 `"execution(* com.x..*.*(..))"` 这种字符串解析成可匹配的对象。这是 AspectJ 真正被调用的地方，而且只在**解析和匹配**时用，执行阶段完全不碰。

其余全部是 Spring 自己的：`ReflectiveAspectJAdvisorFactory` 扫描 `@Aspect` bean 的方法，按注解类型生成 `InstantiationModelAwarePointcutAdvisor`；`AspectJExpressionPointcut` 被包进 `AspectJPointcutAdvisor`；再往后就是上一章讲的 `wrapIfNecessary` → `ProxyFactory` → JDK/CGLIB。

这里有个数量关系容易搞错：**一个 `@Aspect` 类里有几个通知方法，就会生成几个 Advisor**，不是一个切面一个 Advisor。`@Before` 一个、`@AfterReturning` 一个、`@Around` 一个，三个方法三个 Advisor，各自带自己的 Pointcut（如果写的是 `@Before("execution(...)")` 这种内联表达式，每个方法都会各自解析一遍）。用 `@Pointcut` 抽出来的命名切点，被多个通知引用时，解析结果会复用，但 Advisor 依然是每个通知一个。理解了这一点，才能理解为什么"给一个切面加 `@Order` "能控制它整体在链上的位置——排序发生在 Advisor 层面，而同一个切面的 Advisor 共享同一个 `order` 值。

至于 `InstantiationModelAwarePointcutAdvisor` 这个拗口的名字，`InstantiationModel` 指的是 `perthis` / `pertarget` 这类"每个目标对象一个切面实例"的语义。AspectJ 原生支持这种模型，Spring 只支持 `singleton`（默认）和 `perthis`，而且 `perthis` 要求作用域是 prototype 或者显式配置，实际项目里几乎见不到。看到这个类名不用深究，知道它是"切面实例化策略的包装"就够了。

想确认一个项目里到底有没有跑 AspectJ，别看 pom，看产物：

1. 反编译任意一个被切的类，方法体开头如果是 `AccountAspect.aspectOf().before(...)` 这种静态调用，是编译期织入；如果是 `((OrderService) this).xxx` 之外什么都没变、但类名带了 `$Proxy` 或 `$$EnhancerBySpringCGLIB`，是运行期代理。
2. 看启动参数有没有 `-javaagent:.*aspectjweaver`，有才是 LTW。
3. 看 `META-INF/aop.xml` 存不存在，这是 LTW 的配置，没有它 agent 挂了也不织入。

三条都对不上，那这个项目用的就是纯 Spring AOP，"AspectJ"只是出现在依赖名和注解包名里。

所以有一个判断标准很实用：**Spring 判断一个 bean 是不是切面，看的是它有没有 `@Aspect` 注解、是不是容器里的 bean，跟有没有 `.aj` 文件毫无关系。**反过来说，`@Aspect` 类必须被扫进容器——只加 `@Aspect` 不加 `@Component`，或者在 XML 里漏了 `<bean>`，这个切面就完全不存在，一条日志都不会报。反过来，`.aj` 文件放进 Spring Boot 工程也不会被处理，因为编译它需要 `ajc`，Spring 的编译器不认识这个扩展名。

这个"只借语法"的架构决定了能力边界：Spring AOP 只支持它能用代理表达的那些 PCD。`call`、`get`、`set`、`initialization`、`preinitialization`、`cflow`、`if` 全都不支持，写了就是启动期解析失败。`@Aspect` 里也不能声明 `inter-type declaration`（给别的类加字段/方法），那是字节码级别的操作，代理做不到。

四、六个切点表达式，差别在"匹配谁"

表达式	匹配对象	典型用法
`execution`	方法签名（返回值、类、方法名、参数）	`execution(* com.x.service.*.*(..))`
`within`	类型/包，不看方法	`within(com.x.service..*)`，包及子包
`this`	**代理对象**的类型	`this(com.x.OrderService)`
`target`	**目标对象**的类型	`target(com.x.OrderServiceImpl)`
`args`	运行时实参的类型，可绑定	`args(java.lang.String, ..)`
`@annotation`	**方法上**有没有某个注解	`@annotation(com.x.Cost)`
`@within`	**方法所在类上**有没有某个注解	`@within(com.x.Loggable)`
`bean`	bean 名字，Spring 独有	`bean(*Service)`

`this` 和 `target` 是最容易混的一对，区别在 JDK 代理下才显形：

- `this(com.x.OrderService)`：代理对象是不是 `OrderService` 的实例。JDK 代理 `implements OrderService`，**匹配**。
- `target(com.x.OrderServiceImpl)`：目标对象是不是 `OrderServiceImpl` 的实例，**匹配**。
- `this(com.x.OrderServiceImpl)`：JDK 代理不是 `OrderServiceImpl` 的实例，**不匹配**。

所以"给实现类写切点"这件事，在 JDK 代理下必须用 `target`，用 `this` 会静默失效。CGLIB 代理因为是子类，两者结果一样，这也是为什么这个坑在 Boot 项目里很难复现。

`@annotation` 和 `@within` 也常混。前者只看方法上的注解，后者看方法**声明所在类**上的注解。给类打 `@Loggable`、想增强类里所有方法，必须用 `@within`；用 `@annotation` 一个都匹配不到。

`args` 和 `execution` 的参数部分看着像，作用不同：`execution(* *.*(String))` 匹配"声明了单个 String 参数的方法"，`args(String)` 匹配"运行时实际传入的第一个参数是 String 的方法"。前者编译期就能定，后者要运行期判，所以 `args` 通常会让 matcher 变成 dynamic 的，每次调用多一次判断。

这些表达式可以用 `&&`、`||`、`!` 组合，也是排查"切面没生效"最有效的手段：先把表达式拆成单条逐个试，哪一条不匹配就是问题所在。比如 `@annotation(com.x.Cost)` 不生效，先换成 `within(com.x..*)` 看能不能切到类，能切到就说明是注解匹配的问题（注解没打、`@Retention` 不是 `RUNTIME`、或者切点写的是类名而不是全限定名）。`@Retention` 这条尤其常见——`@Cost` 定义时忘了写 `RetentionPolicy.RUNTIME`，编译期就丢了，运行期自然读不到。

`execution` 里的通配符语义也值得单独记一下，写错一个点就少切半个包：`*` 匹配任意字符但不跨包分隔符，`..` 在**类型位置**表示"当前包及所有子包"，在**参数位置**表示"任意个任意类型参数"。所以 `execution(* com.x.service.*.*(..))` 只切 `service` 这一层，`com.x.service.order.*` 切不到；要连子包一起切必须写 `com.x.service..*.*(..)`。`*.*(..)` 里第一个 `*` 是返回值，第二个 `*` 是方法名，漏一个就变成语法错误或者匹配到构造器。

容易误解的地方：

1. **"引了 `aspectjweaver` 就是在用 AspectJ"**。绝大多数 Spring 项目引它只是为了拿到 `@Aspect` 那几个注解类和切点解析器。真正的 AspectJ 织入需要 `ajc` 或 `-javaagent`，两者都没配的话，运行时一行 AspectJ 织入代码都不会执行。
2. **"`.aj` 文件丢进 Spring Boot 就能跑"**。编译不过。`.aj` 只有 `ajc` 认识，`javac` 会直接报语法错误。想在 Spring 项目里用原生 AspectJ，得单独配 `aspectj-maven-plugin`，而且织入发生在编译期，跟 Spring 容器没关系。

3. **"`this()` 和 `target()` 可以互换"**。JDK 代理下完全不等价。用 `this(实现类)` 写的切点，在传统 Spring 里一条都不匹配，在 Boot 里（默认 CGLIB）又正常工作——这类"换个环境就坏"的问题最难查。
4. **"`@annotation` 能匹配类上的注解"**。不能，它只看方法。类级别要用 `@within`（看声明类）或 `@target`（看运行期实际类型，子类也算）。

记住这一句：AspectJ 在编译时把切面写进你的代码，Spring 在运行时把代理套在你的对象外面——名字一样，改的东西完全不同。

第28章　Sentinel：责任链与滑动窗口

一句话：Sentinel 用一条责任链做判断，用一个环形数组做统计。

为什么需要它：

大促前在 dashboard 上把 `createOrder` 的 QPS 阈值调到 200，点了下发，页面提示成功。压测开始，QPS 冲到 600，一台机器都没拦。排查下来三个问题叠在一起：三台机器只有一台收到了规则；收到的那台重启之后规则又没了；还有一台的埋点是 `SphU.entry("createOrder", EntryType.IN)`，而规则里写的是 `createOrder`——名字一样，类型不同，在 Sentinel 里就是两个资源。

另一个现场：阈值设 100，压测时观察 QPS 在 60 到 140 之间反复跳，怀疑统计坏了。其实没坏，是秒级统计只有两个时间桶，窗口切换时必然出现的误差。

这一章要解决：一次 `entry` 到底经过哪些判断、统计数据存在哪、为什么秒级 QPS 会不准、熔断器的三种状态在这套代码里是怎么落地的、以及规则从 dashboard 到内存走了哪条路。

核心结构：

概念	作用
`Resource`	控制粒度，由 `resourceName` + `EntryType` 唯一确定
`Context`	一次调用链的入口，存在 `ThreadLocal`，跨线程要手动传
`Entry`	一次资源访问的句柄，`try-with-resources` 关闭时触发 `exit`
`ProcessorSlotChain`	责任链，每个资源一条，按资源名缓存
`Node`	统计节点。`DefaultNode` 按 context 区分，`ClusterNode` 按资源聚合，`EntranceNode` 标识入口
`StatisticNode`	真正持有统计数据，秒级/分钟级各一个 `ArrayMetric`
`LeapArray`	滑动窗口的底层结构，环形数组 + 时间分桶
`WindowWrap`	一个时间桶，含 `windowStart` 和 `MetricBucket`
`MetricBucket`	桶内的计数器：pass / block / exception / rt / occupiedPass
`FlowRule` / `DegradeRule`	流控规则 / 熔断规则，存在对应的 `RuleManager` 里
`SlotChainBuilder`	SPI 扩展点，默认实现是 `DefaultSlotChainBuilder`

关键代码：

入口方法不长，但它决定了后面所有事。

```
// CtSph#entryWithPriority
private Entry entryWithPriority(ResourceWrapper resourceWrapper, int count,
                                boolean prioritized, Object... args) throws BlockException {
    Context context = ContextUtil.getContext();
    if (context instanceof NullContext) {
        return new CtEntry(resourceWrapper, null, context);   // context 数量超 2000，降级为不统计
    }
    if (context == null) {
        context = MyContextUtil.myEnter(Constants.CONTEXT_DEFAULT_NAME, "", resourceWrapper.getType());
    }
    if (!Constants.ON) {
        return new CtEntry(resourceWrapper, null, context);   // 全局开关关掉，直接放行
    }

    // 关键：链按 resource 缓存，不是按线程、也不是按请求
    ProcessorSlot<Object> chain = lookProcessChain(resourceWrapper);
    if (chain == null) {
        return new CtEntry(resourceWrapper, null, context);   // 资源数超 6000，放弃保护
    }

    Entry e = new CtEntry(resourceWrapper, chain, context);
    try {
        chain.entry(context, resourceWrapper, null, count, prioritized, args);
    } catch (BlockException e1) {
        e.exit(count, args);      // 被拦了也要 exit，否则线程数统计会漏减
        throw e1;
    }
    return e;
}

// DefaultSlotChainBuilder：顺序是设计出来的，不是随手排的
public ProcessorSlotChain build() {
    ProcessorSlotChain chain = new DefaultProcessorSlotChain();
    chain.addLast(new NodeSelectorSlot());     // 建 DefaultNode，挂调用树
    chain.addLast(new ClusterBuilderSlot());   // 建 ClusterNode，按资源聚合
    chain.addLast(new LogSlot());              // 兜住 BlockException 记日志
    chain.addLast(new StatisticSlot());        // 统计，包住后面所有规则
    chain.addLast(new AuthoritySlot());        // 黑白名单
    chain.addLast(new SystemSlot());           // 全局自适应限流
    chain.addLast(new FlowSlot());             // 流控
    chain.addLast(new DegradeSlot());          // 熔断
    return chain;
}
```

窗口的获取是整个统计模块的核心，四个分支对应四种并发/时间状态。

```
// LeapArray#currentWindow
public WindowWrap<T> currentWindow(long timeMillis) {
    if (timeMillis < 0) return null;

    // 时间折叠进定长数组：60 个桶的分钟窗口，idx = (ms / 1000) % 60
    int idx = calculateTimeIdx(timeMillis);
    // 该桶的理论起始时间，向下对齐到窗口边界
    long windowStart = calculateWindowStart(timeMillis);

    while (true) {   // 无锁 + 重试，不是阻塞队列
        WindowWrap<T> old = array.get(idx);
        if (old == null) {
            // 首次访问：CAS 抢占这个槽位，抢不到就礼让重来
            WindowWrap<T> window = new WindowWrap<>(windowLengthInMs, windowStart, newEmptyBucket(timeMillis));
            if (array.compareAndSet(idx, null, window)) return window;
            Thread.yield();
        } else if (windowStart == old.windowStart()) {
            return old;                                  // 同一轮，直接复用
        } else if (windowStart > old.windowStart()) {
            // 上一轮的数据，必须重置。这里用 tryLock 而不是 CAS，
            // 因为 reset 要改多个字段，CAS 保证不了原子性
            if (updateLock.tryLock()) {
                try { return resetWindowTo(old, windowStart); }
                finally { updateLock.unlock(); }
            } else {
                Thread.yield();
            }
        } else {
            // windowStart < old.windowStart()：时钟回拨。
            // 返回一个临时窗口，不写回数组，避免污染已有统计
            return new WindowWrap<>(windowLengthInMs, windowStart, newEmptyBucket(timeMillis));
        }
    }
}
```

拆解：

一、SlotChain：顺序决定了语义

八个 Slot，可以分成三组：**建节点**（NodeSelector、ClusterBuilder）、**统计**（Statistic）、**规则**（Authority、System、Flow、Degrade）。LogSlot 是个旁观者。

顺序的关键在于 `StatisticSlot` 的位置。它的 `entry` 长这样：

```
public void entry(...) throws Throwable {
    try {
        fireEntry(context, resourceWrapper, node, count, prioritized, args);  // 先让后面所有规则跑
        node.increaseThreadNum();
        node.addPassRequest(count);                                            // 全过了才记 pass
        // ...
    } catch (BlockException e) {
        node.increaseBlockQps(count);                                          // 被拦了记 block
        throw e;
    } catch (Throwable e) {
        node.increaseExceptionQps(count);
        throw e;
    }
}
```

它必须"先放行、后统计"。因为一次请求到底算 pass 还是 block，取决于后面四个规则 Slot 的判断结果。如果 Statistic 放在规则前面统计，那它只能记"来了一个请求"，没法区分"通过了"和"被拦了"，QPS 和 block QPS 就全错了。

`exit` 的动作相反：先减线程数、记 RT，再 `fireExit` 往回走。RT 只有在方法真正执行完之后才有值，所以统计只能在出口做。

Node 的层级也值得说清楚。`NodeSelectorSlot` 按 `context.getName()` 建 `DefaultNode`，同一个资源被不同入口调用就有多个 `DefaultNode`，它们挂在调用树上（`EntranceNode` → `DefaultNode` → 子 `DefaultNode`）。`ClusterBuilderSlot` 再建一个 `ClusterNode`，每个资源一个，跨所有 context 聚合——**QPS 限流看的是 `ClusterNode`**。如果设置了 `origin`（调用方标识），还会在 `ClusterNode` 下挂一个 `OriginNode`，用于按来源统计。

每个 Slot 处理完自己那段，都会调 `context.setCurNode(node)` 把当前节点往下推一层，然后 `fireEntry`。这就是调用树的构建方式：`NodeSelectorSlot` 找到（或新建）本 context 下的 `DefaultNode`，把 `context.getLastNode()`（父节点）和它连起来；下一层资源再进来时，父节点就是它。所以"链路"这个词在 Sentinel 里是字面意义的树——dashboard 上看到的调用链路图，就是这棵树序列化的结果。限流规则作用于 `ClusterNode`，所以它统计的是这个资源在所有入口、所有调用路径上的总和。

链本身按资源缓存，用 `ConcurrentHashMap<ResourceWrapper, ProcessorSlotChain>`，上限 `Constants.MAX_SLOT_CHAIN_SIZE = 6000`。超了之后 `lookProcessChain` 返回 `null`，`entryWithPriority` 直接放行——**保护机制自己先降级，保证不影响业务**。这个设计值得记住：限流组件挂掉的时候，默认行为必须是"全部放行"，不能是"全部拒绝"。

二、LeapArray：时间怎么折叠进一个定长数组

滑动窗口的朴素实现是维护一个队列，每次请求入队、把窗口外的出队。问题是高 QPS 下队列操作本身就成了瓶颈，而且每次出队都要回收对象。

Sentinel 的做法是把**时间轴折叠**：`idx = (currentTimeMillis / windowLengthInMs) % sampleCount`。分钟窗口 60 个桶、每桶 1 秒，秒窗口 2 个桶、每桶 500ms。数组长度固定，槽位循环复用，永远不扩容、不分配新对象。

这样做的直接好处是：写入是 `array.get(idx)` + 一次字段自增，O(1)；读取是把整个数组扫一遍求和，O(sampleCount)，60 次读取对一次限流判断来说完全可以接受；内存恒定，没有 GC 压力。

代价是"过期"这件事要靠时间戳判断，不能靠位置。`values(timeMillis)` 遍历数组时，对每个桶调 `isWindowDeprecated`：

```
return time - windowWrap.windowStart() > intervalInMs;
```

只有落在 `[now - interval, now]` 里的桶才算有效。这也解释了为什么槽位复用时必须重置 `windowStart`——它是这个桶"属于哪一轮"的唯一标识。

`currentWindow` 的四个分支覆盖了所有并发情况：槽位为空（首次）、时间戳相同（正常复用）、时间戳更旧（跨轮，需要重置）、时间戳更新（时钟回拨，异常）。时钟回拨这一支返回的是临时对象、不写回数组，因为一旦写回，数组里就会出现一个"未来"的桶，后续所有 `values` 计算都会出错。宁可丢这一小段数据，也不能污染整个窗口。

秒级窗口用的不是 `BucketLeapArray`，而是它的子类 `OccupiableBucketLeapArray`。差别在重置：普通实现把桶清零，这个实现会先看一眼 `borrowArray` 里有没有"未来的值"，有就填进来。`borrowArray` 是 `FutureBucketLeapArray`，它的 `isWindowDeprecated` 判断是 `time >= windowStart`——也就是说，只有"未来"的桶才算有效。这套机制服务于 `prioritized` 请求：QPS 打满时，高优先级请求不直接失败，而是预占下一个窗口的额度、`sleep` 一小段再放行，占用的量记在未来的桶里，等时间走到那个窗口时再兑现。这就是类名里 `Occupiable`（可预占）的含义，也是为什么它能做到"限流但不丢请求"。普通业务用不到，但看源码时看到这个类别懵。

三、QPS 限流与线程数限流，限的不是同一个东西

`FlowRule.grade` 有两个值，对应两种完全不同的语义：

QPS 限流（`FLOW_GRADE_QPS`）：统计每秒**通过**的请求数，超阈值直接拒绝。它不关心请求执行了多久，只要放进来了就算一个。适合保护下游——下游每秒只能处理 200 个，你这边就必须卡在 200。

线程数限流（`FLOW_GRADE_THREAD`）：统计当前**在途**的请求数，`entry` 时 +1、`exit` 时 -1，用的是 `LongAdder`。超阈值拒绝新请求。适合保护自己——比如某个接口依赖的下游超时了，RT 从 50ms 涨到 5s，QPS 没变但线程池被占满，这时候 QPS 限流完全看不出问题，只有线程数能拦住。

两者的关系可以用 Little's Law 串起来：`并发数 ≈ QPS × RT`。RT 稳定时，限制 QPS 就等于限制并发；RT 抖动时，同样的 QPS 会让并发数暴涨，所以生产上常见的是"QPS 限流 + 线程数限流"双保险。

线程数统计的实现是 `StatisticNode.curThreadNum` 一个 `AtomicInteger`（`ClusterNode` 里用 `LongAdder`），`entry` 时 `increaseThreadNum()`、`exit` 时 `decreaseThreadNum()`。这就要求 `Entry` 必须被关闭——`try (Entry entry = SphU.entry(...))` 的 try-with-resources 不是语法糖，是**必须**。如果代码里 `entry` 之后抛异常又没走 `close`，线程数只增不减，跑一天之后这个资源就永久处于"线程数超限"状态，所有请求被拒，而且日志里看不出任何异常。`CtEntry.exit()` 里有 `if (this.chain == null) return;` 这类短路，但正常路径上减一是必然要执行的，别在业务代码里手动吞掉 `Entry`。

秒级 QPS 为什么不准？因为秒窗口只有 2 个 500ms 的桶。假设 QPS 稳定 100、请求均匀分布，在 `t = 1.1s` 观察"过去 1 秒"，此时第 0 个桶（`[0, 0.5)`）已经被判过期，只剩第 1 个桶（`[0.5, 1.0)`）的 50 个请求，算出来 QPS = 50。误差范围是 0~50%，实测在 50~100 之间跳就是这个原因。

那为什么不多加几个桶？因为桶越多，`values()` 的遍历成本越高，而限流判断在每一次请求的主路径上。2 个桶是精度和开销的折中。真要看准确的 QPS，dashboard 展示的是分钟窗口聚合出来的值，不是这个秒级值。

`controlBehavior` 还提供了另外三种：`WarmUp`（预热，冷启动时慢慢放量）、`RateLimiter`（匀速排队，请求进队列等待）、`WarmUpRateLimiter`。它们的判断逻辑都在对应的 `TrafficShapingController` 里，QPS 阈值本身还是从同一个 `ClusterNode` 取。

四、熔断三态，和规则是怎么被推下来的

熔断器的标准三态：

- **CLOSED**：正常放行，同时统计 RT / 异常比例 / 异常数。
- **OPEN**：达到阈值，拒绝所有请求，持续 `timeWindow` 秒。
- **HALF_OPEN**：窗口结束，放行少量探测请求。探测成功回 CLOSED，失败重新 OPEN。

Sentinel 1.7 的实现比这个状态机简单：`DegradeRule` 里只有一个 `AtomicBoolean cut` 当开关。

```
if (cut.get()) {
    throw new DegradeException(...);          // 熔断中，直接拒
}
// ... 统计判断，超过阈值时：
if (cut.compareAndSet(false, true)) {
    pool.schedule(new ResetTask(this), timeWindow, TimeUnit.SECONDS);
}
```

`ResetTask` 干的事就是 `cut.set(false)`。所以严格讲它是 **CLOSED / OPEN 两态 + 一个定时器**，没有显式的 HALF_OPEN 枚举。所谓"半开"，体现为窗口结束后 `cut` 变回 `false`、下一个请求被放进来试探——如果它还是慢，马上又会 `CAS` 回 `true`。理解这一点很重要，否则你会去找一个根本不存在的状态字段。

三种策略的判定入口都是 `passCheck`：RT 模式看平均 RT 是否超阈值（且请求数达到最小样本数 `minRequestAmount`），异常比例看 `exception / total`，异常数看绝对值。三者都受 `timeWindow` 控制熔断时长。

规则怎么到客户端？链路是：

1. 客户端启动时通过 SPI 加载 `InitFunc`，其中 `HeartbeatSenderInitFunc` 起一个定时任务（默认 10s），把 `appName / ip / port` 发给 dashboard。dashboard 收到后才知道有这个实例。
2. dashboard 下发规则时，调客户端 transport 模块暴露的 HTTP 接口（`/setRules`），服务端是 `CommandHandler` 的实现，比如 `ModifyRulesCommandHandler`。
3. Handler 里调 `FlowRuleManager.loadRules(list)`，规则写进内存里的 `AtomicReference<List<FlowRule>>`，同时通知所有注册的 `PropertyListener` 重建 `FlowSlot` 里的 checker。
4. `FlowSlot` 每次判断时从 `FlowRuleManager.getRules()` 现取，所以规则是热生效的，不需要重启。

dashboard 默认不持久化任何数据。规则只存在 dashboard 的内存和客户端的内存里，dashboard 重启、客户端重启，规则全丢。生产做法是接数据源：`sentinel-datasource-nacos` / `-zookeeper` / `-apollo`，客户端用 `FlowRuleManager.register2Property(...)` 订阅，规则变更由配置中心推过来；dashboard 那边改造一下，把规则写进同一个配置中心。这套"推"的链路和 dashboard 直连是两条路，别混着理解。

数据流是双向的，方向别搞反：**规则是 dashboard 推给客户端**（HTTP 调客户端的 `CommandHandler`），**统计数据是 dashboard 从客户端拉**（调客户端的 `/metric` 接口，客户端返回本地 `ClusterNode` 的聚合值）。客户端从头到尾只主动做一件事——发心跳，告诉 dashboard"我这个 IP:PORT 还活着、我叫什么名字"。所以 dashboard 挂了不影响限流，规则已经在客户端内存里；但客户端重启后如果没接数据源，规则就真没了，因为没有任何地方持久化过它。

最后一句关于集群流控：本章讲的都是单机限流，每台机器各自统计、各自判断，10 台机器配 100 QPS 实际可能放过 1000。要精确控制总量得用集群流控（`sentinel-cluster`），引入一个 token server 统一发令牌，客户端向它申请配额。那套代码是另一条链路，和这里的 SlotChain 并行，不冲突但也不共享统计。

容易误解的地方：

1. **"dashboard 显示下发成功，规则就生效了"**。它只证明 HTTP 调用返回了 200。规则可能只到了一台机器（其余机器心跳没注册上）、可能因为 `resourceName` 或 `EntryType` 不匹配而永远不命中、也可能客户端重启后丢了。排查顺序应该是：先确认客户端有没有收到（看 `sentinel-block.log` 或 `/getRules` 接口），再确认资源名和类型，最后才怀疑规则内容。
2. **"QPS 统计是精确的"**。秒级窗口只有 2 个桶，边界切换时必然丢一半左右的数据，误差 0~50%。用它做限流判断没问题（限流本来就允许误差），用它做容量评估会得出错误结论。
3. **"熔断器有完整的 CLOSED/OPEN/HALF_OPEN 状态机"**。Sentinel 里只有一个 `AtomicBoolean cut` 加一个定时 `ResetTask`，没有 HALF_OPEN 状态。它的"半开"是时间驱动的隐式行为，不是状态迁移。
4. **"同一个资源名就是同一个资源"**。`ResourceWrapper` 的 `equals` 同时看 `name` 和 `entryType`。`entry("order", IN)` 和 `entry("order", OUT)` 是两个独立资源，各有各的链、各有各的统计。规则里只写名字不写类型时默认匹配 `IN`，出口流量上的规则要单独配。

记住这一句：规则决定"拦不拦"，窗口决定"凭什么拦"——统计不准，规则再对也是白搭。

结语：怎么自己读源码

这本书讲的是 28 个结论。但结论会忘，方法不会。所以最后这一节不讲技术，讲怎么读。

一、别从第一行读起

新手读源码最常见的做法：打开 `AbstractQueuedSynchronizer.java`，从第 1 行开始往下读。读到 200 行就迷失了，因为大部分方法是给子类留的钩子，不是你关心的主链路。

正确的做法是**从一个公开入口进**。比如读 AQS，就从 `ReentrantLock.lock()` 开始；读 Spring IOC，就从 `refresh()` 开始；读线程池，就从 `execute()` 开始。

入口的标准是：**你能在业务代码里直接调到它**。从这里往下跟，跟到出现"看不懂的分支"就停下，先记下来，继续走主干。

二、一次只跟一条链路

源码里到处都是分支。`execute()` 里有三段判断，`getBean()` 里有七八个分支。全跟会爆炸。

做法：**先假设一条最普通的路径走完**。比如线程池，先假设"核心线程没满"，看它怎么 `addWorker`；走通了，再回来看"核心线程满了"那条路。

一条链路走完，你对整体结构就有了感觉。剩下的分支，等你真的遇到时再回来看，效率高得多。

三、画图，不要只做笔记

纯文字笔记记不住结构。至少画三种图：

图	画什么	用在哪
类图	谁持有谁、谁继承谁	搞清 AQS 的 Node、Spring 的 BeanFactory 体系
时序图	一次调用的先后顺序	搞清 `lock()` → `acquire()` → `park()` 的链路
状态图	状态怎么迁移	搞清线程池的五种状态、对象头的 Mark Word

画图的过程就是逼自己确认"这一步之后到底发生了什么"的过程。画不出来的地方，就是没读懂的地方。

四、用实验验证，别靠推理

读源码最容易犯的错，是把"我觉得应该是这样"当成结论。

三个低成本的验证手段：

```
// 1. 打断点，看真实调用栈
//    在怀疑的方法上打断点，看是谁调过来的——比读十遍调用关系快

// 2. 打日志，看状态变化
//    jstack 看线程状态，jstat 看 GC，jcmd VM.metaspace 看元空间

// 3. 写最小复现
//    把怀疑的机制抽成一个 30 行的 main 方法，跑一遍
```

第 3 条最有用。比如想确认"`HashMap` 树化阈值是 8 还是 9"，写个循环往同一个桶里塞对象，打印 `binCount` 就知道了。**一次实验胜过十次推理。**

五、读完要能讲给别人听

检验标准只有一个：**能不能用三句话讲清一个机制，而且不借助任何术语。**

比如 AQS：一个 `volatile int` 表示状态，一个双向队列存等待的线程，抢不到就入队并 `park`，释放时唤醒队首。三句话讲完，说明你抓住了骨架。

讲的时候如果发现自己在背流程（"第一步……第二步……"），说明还没抽象出来。

六、下一步读什么

这本书覆盖了 JDK 和几个主流框架的核心机制。往下走有四个方向：

方向	具体读什么	为什么
JDK 源码	`java.util.concurrent` 整包、`java.util` 集合、`sun.misc.Unsafe`	这本书只挑了重点，整包读完才知道还有哪些工具
JVM 规范	《Java 虚拟机规范》《Java 语言规范》第 17 章	这本书讲的是 HotSpot 的实现，规范讲的是"必须怎样"
框架内核	Spring 的 `spring-beans`、Netty 的 `transport`、MyBatis 的 `executor`	一个框架读透，其他框架的套路就都能猜出来
性能工程	async-profiler、JFR、Arthas	读源码是为了定位问题，工具是为了更快定位

最后一句

读源码这件事，收益不在"读完"，在"读的过程中被迫想清楚的那些事"。

你可能读完之后一个方法名都记不住，但你会永远记得"为什么 `HashMap` 扩容不用重新算 hash"——因为那个瞬间你想通了"容量是 2 的幂"和"高位参与索引"之间的关系。

那种想通的瞬间，才是读书真正的收获。 剩下的，查文档就行。